HeliOS

Kernel 0.5.0

HeliOS Developer's Guide

1 Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

2 File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

3 Data Structure Documentation

3.1 Dir_s Struct Reference

Data Fields

- struct Volume_s * **volume**
- Word_t **currentCluster**
- HalfWord_t **entryIndex**
- Base_t **isOpen**

The documentation for this struct was generated from the following file:

- HeliOS.h

3.2 DirEntry_s Struct Reference

Data Fields

- Byte_t **name** [256]
- Word_t **size**
- Word_t **firstCluster**
- Base_t **isDirectory**
- Base_t **isReadOnly**
- Base_t **isHidden**
- Base_t **isSystem**

The documentation for this struct was generated from the following file:

- HeliOS.h

3.3 File_s Struct Reference

Data Fields

- struct Volume_s * **volume**
- Word_t **firstCluster**
- Word_t **currentCluster**
- Word_t **fileSize**
- Word_t **position**
- Byte_t **mode**
- Base_t **isOpen**
- Base_t **isDirty**

The documentation for this struct was generated from the following file:

- HeliOS.h

3.4 MemoryRegionStats_s Struct Reference

Data Fields

- Word_t largestFreeEntryInBytes
- Word_t smallestFreeEntryInBytes
- Word_t numberOfFreeBlocks
- Word_t availableSpaceInBytes
- Word_t successfulAllocations
- Word_t successfulFrees
- Word_t minimumEverFreeBytesRemaining

3.4.1 Field Documentation

3.4.1.1 availableSpaceInBytes `Word_t MemoryRegionStats_s::availableSpaceInBytes`

The amount of free memory in bytes (i.e., numberOfFreeBlocks * CONFIG_MEMORY_REGION_BLOCK_SIZE).

3.4.1.2 largestFreeEntryInBytes `Word_t MemoryRegionStats_s::largestFreeEntryInBytes`

The largest free entry in bytes.

3.4.1.3 minimumEverFreeBytesRemaining `Word_t MemoryRegionStats_s::minimumEverFreeBytes↩ Remaining`

Lowest water lever since system initialization of free bytes of memory.

3.4.1.4 numberOfFreeBlocks `Word_t MemoryRegionStats_s::numberOfFreeBlocks`

The number of free blocks. See CONFIG_MEMORY_REGION_BLOCK_SIZE for block size in bytes.

3.4.1.5 smallestFreeEntryInBytes `Word_t MemoryRegionStats_s::smallestFreeEntryInBytes`

The smallest free entry in bytes.

3.4.1.6 successfulAllocations `Word_t MemoryRegionStats_s::successfulAllocations`

Number of successful memory allocations.

3.4.1.7 successfulFrees `Word_t MemoryRegionStats_s::successfulFrees`

Number of successful memory "frees".

The documentation for this struct was generated from the following file:

- HeliOS.h

3.5 QueueMessage_s Struct Reference

Data Fields

- Base_t messageBytes
- Byte_t messageValue [0x8u]

3.5.1 Field Documentation

3.5.1.1 messageBytes `Base_t QueueMessage_s::messageBytes`

The number of bytes contained in the message value which cannot exceed CONFIG_MESSAGE_VALUE_BYTES.

3.5.1.2 messageValue `Byte_t QueueMessage_s::messageValue[0x8u]`

The queue message value.

The documentation for this struct was generated from the following file:

- HeliOS.h

3.6 SystemInfo_s Struct Reference

Data Fields

- Byte_t productName [0x6u]
- Base_t majorVersion
- Base_t minorVersion
- Base_t patchVersion
- Base_t numberOfTasks
- Base_t littleEndian

3.6.1 Field Documentation

3.6.1.1 littleEndian `Base_t SystemInfo_s::littleEndian`

True if the system byte order is little endian.

3.6.1.2 majorVersion `Base_t SystemInfo_s::majorVersion`

The SemVer major version number of HeliOS.

3.6.1.3 minorVersion `Base_t SystemInfo_s::minorVersion`

The SemVer minor version number of HeliOS.

3.6.1.4 numberOfTasks `Base_t SystemInfo_s::numberOfTasks`

The number of tasks regardless of their state.

3.6.1.5 patchVersion `Base_t SystemInfo_s::patchVersion`

The SemVer patch version number of HeliOS.

3.6.1.6 productName `Byte_t SystemInfo_s::productName[0x6u]`

The product name of the operating system (always "HeliOS").

The documentation for this struct was generated from the following file:

- HeliOS.h

3.7 TaskInfo_s Struct Reference

Data Fields

- Base_t id
- Byte_t name [0x8u]
- TaskState_t state
- Ticks_t lastRunTime
- Ticks_t totalRunTime

3.7.1 Field Documentation

3.7.1.1 id `Base_t TaskInfo_s::id`

The ID of the task.

3.7.1.2 lastRunTime `Ticks_t TaskInfo_s::lastRunTime`

The duration in ticks of the task's last runtime.

3.7.1.3 name `Byte_t TaskInfo_s::name[0x8u]`

The name of the task which must be exactly CONFIG_TASK_NAME_BYTES bytes in length. Shorter task names must be padded.

3.7.1.4 state `TaskState_t TaskInfo_s::state`

The state the task is in which is one of four states specified in the TaskState_t enumerated data type.

3.7.1.5 totalRunTime `Ticks_t TaskInfo_s::totalRunTime`

The duration in ticks of the task's total runtime.

The documentation for this struct was generated from the following file:

- HeliOS.h

3.8 TaskNotification_s Struct Reference

Data Fields

- Base_t notificationBytes
- Byte_t notificationValue [0x8u]

3.8.1 Field Documentation

3.8.1.1 notificationBytes `Base_t TaskNotification_s::notificationBytes`

The length in bytes of the notification value which cannot exceed CONFIG_NOTIFICATION_VALUE_BYTES.

3.8.1.2 notificationValue `Byte_t TaskNotification_s::notificationValue[0x8u]`

The notification value whose length is specified by the notification bytes member.

The documentation for this struct was generated from the following file:

- HeliOS.h

3.9 TaskRunTimeStats_s Struct Reference

Data Fields

- Base_t id
- Ticks_t lastRunTime
- Ticks_t totalRunTime

3.9.1 Field Documentation

3.9.1.1 id `Base_t TaskRunTimeStats_s::id`

The ID of the task.

3.9.1.2 lastRunTime `Ticks_t TaskRunTimeStats_s::lastRunTime`

The duration in ticks of the task's last runtime.

3.9.1.3 totalRunTime `Ticks_t TaskRunTimeStats_s::totalRunTime`

The duration in ticks of the task's total runtime.

The documentation for this struct was generated from the following file:

- HeliOS.h

3.10 Volume_s Struct Reference

Data Fields

- HalfWord_t **blockDeviceUID**
- Word_t **fatStartSector**
- Word_t **dataStartSector**
- Word_t **rootDirCluster**
- Byte_t **sectorsPerCluster**
- HalfWord_t **bytesPerSector**
- HalfWord_t **reservedSectors**
- Byte_t **numFATs**
- Word_t **sectorsPerFAT**
- Base_t **mounted**

The documentation for this struct was generated from the following file:

- HeliOS.h

3.11 VolumeInfo_s Struct Reference

Data Fields

- Word_t **totalClusters**
- Word_t **freeClusters**
- Word_t **totalBytes**
- Word_t **freeBytes**
- HalfWord_t **bytesPerSector**
- Byte_t **sectorsPerCluster**
- Word_t **bytesPerCluster**

The documentation for this struct was generated from the following file:

- HeliOS.h

4 File Documentation

4.1 config.h File Reference

Kernel source for build configuration.

Macros

- #define CONFIG_ENABLE_ARDUINO_CPP_INTERFACE

  Define to enable the Arduino API C++ interface.
- #define CONFIG_ENABLE_SYSTEM_ASSERT

  Define to enable system assertions.
- #define CONFIG_SYSTEM_ASSERT_BEHAVIOR(f, l) __ArduinoAssert__(f, l)

  Define the system assertion behavior.
- #define CONFIG_MESSAGE_VALUE_BYTES 0x8u /* 8 */

  Define the size in bytes of the message queue message value.
- #define CONFIG_NOTIFICATION_VALUE_BYTES 0x8u /* 8 */

  Define the size in bytes of the direct to task notification value.
- #define CONFIG_TASK_NAME_BYTES 0x8u /* 8 */

  Define the size in bytes of the task name.
- #define CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS 0x10u /* 16 */

  Define the number of memory blocks available in all memory regions.
- #define CONFIG_MEMORY_REGION_BLOCK_SIZE 0x20u /* 32 */

  Define the memory block size in bytes for all memory regions.
- #define CONFIG_QUEUE_MINIMUM_LIMIT 0x5u /* 5 */

  Define the minimum value for a message queue limit.
- #define CONFIG_STREAM_BUFFER_BYTES 0x20u /* 32 */

  Define the length of the stream buffer.
- #define CONFIG_TASK_WD_TIMER_ENABLE

  Enable task watchdog timers.
- #define CONFIG_DEVICE_NAME_BYTES 0x8u /* 8 */

  Define the length of a device driver name.

4.1.1 Detailed Description

Author

Manny Peterson manny@heliosproj.org

Version

0.5.0

Date

2023-03-19

Copyright

4.1.2 Macro Definition Documentation

4.1.2.1 CONFIG_DEVICE_NAME_BYTES `#define CONFIG_DEVICE_NAME_BYTES 0x8u /* 8 */`

Setting CONFIG_DEVICE_NAME_BYTES will define the length of a device driver name. The name of device drivers should be exactly this length. There really isn't a reason to change this and doing so may break existing device drivers. The default length is 8 bytes.

Device driver names are used to identify and access device drivers within the system. This value determines the maximum length of these names including the null terminator.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x8u for 8 bytes).

Warning

Changing this value may cause compatibility issues with existing device drivers that expect the standard 8-byte name length. Only modify this setting if you are also updating all device drivers in your system.

Device driver names in the HeliOS ecosystem are standardized to 8 bytes. Changing this value may break third-party or built-in drivers.

Note

It is strongly recommended to keep this value at its default of 8 bytes unless you have a specific requirement and are maintaining all device drivers yourself.

4.1.2.2 CONFIG_ENABLE_ARDUINO_CPP_INTERFACE `#define CONFIG_ENABLE_ARDUINO_CPP_INTERFACE`

Because the HeliOS kernel is written in C, the Arduino API cannot be called directly from the kernel. For example, assertions are unable to be written to the serial bus in applications using the Arduino platform/tool-chain. The CONFIG_ENABLE_ARDUINO_CPP_INTERFACE builds the included arduino.cpp file to allow the kernel to call the Arduino API through wrapper functions such as **ArduinoAssert**(). The arduino.cpp file can be found in the /extras directory. It must be copied into the /src directory to be built.

Note

This setting is only relevant when building HeliOS for Arduino-based platforms. For other platforms, this setting should remain undefined.

When enabled, ensure arduino.cpp is present in the /src directory to avoid linker errors.

See also

CONFIG_SYSTEM_ASSERT_BEHAVIOR

4.1.2.3 CONFIG_ENABLE_SYSTEM_ASSERT `#define CONFIG_ENABLE_SYSTEM_ASSERT`

The CONFIG_ENABLE_SYSTEM_ASSERT setting allows the end-user to enable system assertions in HeliOS. Once enabled, the end-user must also define CONFIG_SYSTEM_ASSERT_BEHAVIOR for there to be an effect. By default, the CONFIG_ENABLE_SYSTEM_ASSERT setting is not defined.

When enabled, the kernel will invoke assertion checks at critical points to verify system integrity and catch programming errors. This adds runtime overhead but is invaluable for debugging.

Note

For production builds, consider disabling this setting to reduce code size and improve performance.

Requires CONFIG_SYSTEM_ASSERT_BEHAVIOR to be defined to specify the assertion handler behavior.

See also

CONFIG_SYSTEM_ASSERT_BEHAVIOR

CONFIG_ENABLE_ARDUINO_CPP_INTERFACE

4.1.2.4 CONFIG_MEMORY_REGION_BLOCK_SIZE `#define CONFIG_MEMORY_REGION_BLOCK_SIZE 0x20u /* 32 */`

Setting CONFIG_MEMORY_REGION_BLOCK_SIZE allows the end-user to define the size of a memory region block in bytes. The memory region block size should be set to achieve the best possible utilization of the available memory. The CONFIG_MEMORY_REGION_BLOCK_SIZE setting affects both the heap and kernel memory regions. The default value is 32 bytes.

This value represents the granularity of memory allocation. All memory allocations will be rounded up to the nearest multiple of this block size. Choosing an appropriate block size is crucial for minimizing internal fragmentation while maintaining efficient memory utilization.

Tuning Guidelines:

- Smaller block sizes (8-16 bytes): Better for many small allocations, less internal fragmentation, but higher overhead from block management
- Medium block sizes (32-64 bytes): Good general-purpose balance
- Larger block sizes (128+ bytes): Better for large allocations, lower management overhead, but more internal fragmentation for small allocations

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x20u for 32 bytes).

Consider using a power-of-2 value for optimal memory alignment on most architectures.

This value should be chosen based on the typical size of objects allocated by your application.

See also

xMemAlloc()

xMemFree()

CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS

4.1.2.5 CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS `#define CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS 0x10u /* 16 */`

The heap memory region is used by tasks, whereas the kernel memory region is used solely by the kernel for kernel objects. The CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS setting allows the end-user to define the size, in blocks, of all memory regions thus affecting both the heap and kernel memory regions. The size of a memory block is defined by the CONFIG_MEMORY_REGION_BLOCK_SIZE setting. The size of all memory regions needs to be adjusted to fit the memory requirements of the end-user's application. The default value is 16 blocks.

Total Memory Calculation:

Total memory per region = CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS * CONFIG_MEMORY_REGION_BLOCK_SIZE

Since HeliOS maintains two separate memory regions (heap and kernel), the total system memory usage is: Total system memory = 2 * (CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS * CONFIG_MEMORY_REGION_BLOCK_SIZE)

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x10u for 16 blocks).

Setting this value too small may result in memory allocation failures. Setting it too large may exceed the available RAM on the target platform.

This value affects both heap and kernel memory regions equally.

See also

CONFIG_MEMORY_REGION_BLOCK_SIZE

xMemAlloc()

xMemFree()

4.1.2.6 CONFIG_MESSAGE_VALUE_BYTES `#define CONFIG_MESSAGE_VALUE_BYTES 0x8u /* 8 */`

Setting the CONFIG_MESSAGE_VALUE_BYTES allows the end-user to define the size of the message queue message value. The larger the size of the message value, the greater impact there will be on system performance and memory usage. The default size is 8 bytes.

This value determines the maximum payload size that can be sent through message queues. Each message in a queue will allocate this amount of memory for its value field. Consider the trade-off between message capacity and memory efficiency when setting this value.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x8u for 8 bytes).

Smaller values conserve memory but limit the data that can be passed in a single message. Larger values increase flexibility but consume more memory per message.

See also

QueueMessage_t

xQueueSend()

xQueueReceive()

4.1.2.7 CONFIG_NOTIFICATION_VALUE_BYTES `#define CONFIG_NOTIFICATION_VALUE_BYTES 0x8u /* 8 */`

Setting the CONFIG_NOTIFICATION_VALUE_BYTES allows the end-user to define the size of the direct to task notification value. The larger the size of the notification value, the greater impact there will be on system performance and memory usage. The default size is 8 bytes.

This value determines the maximum payload size for direct-to-task notifications. Direct-to-task notifications are a lightweight alternative to message queues for simple inter-task communication. Each task that can receive notifications will allocate this amount of memory for its notification value.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x8u for 8 bytes).

Direct-to-task notifications are faster than message queues but limited to one pending notification per task.

See also

TaskNotification_t

xTaskNotify()

xTaskNotifyTake()

4.1.2.8 CONFIG_QUEUE_MINIMUM_LIMIT `#define CONFIG_QUEUE_MINIMUM_LIMIT 0x5u /* 5 */`

Setting the CONFIG_QUEUE_MINIMUM_LIMIT allows the end-user to define the MINIMUM length limit a message queue can be created with using xQueueCreate(). When a message queue length equals its limit, the message queue will be considered full and return true when xQueueIsQueueFull() is called. A full queue will also not accept messages from xQueueSend(). The default value is 5.

This setting enforces a minimum queue depth to prevent creating queues that are too small to be useful. Attempts to create queues with a limit smaller than this value will result in the queue being created with this minimum limit instead.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x5u for 5 messages).

Each message in a queue consumes CONFIG_MESSAGE_VALUE_BYTES of memory, so larger minimum limits increase the minimum memory cost per queue.

See also

xQueueIsQueueFull()

xQueueSend()

xQueueCreate()

CONFIG_MESSAGE_VALUE_BYTES

4.1.2.9 CONFIG_STREAM_BUFFER_BYTES `#define CONFIG_STREAM_BUFFER_BYTES 0x20u /* 32 */`

Setting CONFIG_STREAM_BUFFER_BYTES will define the length of stream buffers created by xStreamCreate(). When the length of the stream buffer reaches this value, it is considered full and can no longer be written to by calling xStreamSend(). The default value is 32 bytes.

Stream buffers provide a byte-oriented data transfer mechanism, suitable for streaming data between tasks or from interrupt handlers to tasks. Unlike message queues which handle discrete messages, stream buffers handle continuous byte streams.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x20u for 32 bytes).

Each stream buffer instance will allocate this amount of memory for its internal buffer storage.

Stream buffers are ideal for serial port data, sensor readings, or any continuous data flow where message boundaries are not important.

See also

xStreamCreate()

xStreamSend()

xStreamReceive()

4.1.2.10 CONFIG_SYSTEM_ASSERT_BEHAVIOR `#define CONFIG_SYSTEM_ASSERT_BEHAVIOR( f, l ) __ArduinoAssert__(f, l)`

The CONFIG_SYSTEM_ASSERT_BEHAVIOR setting allows the end-user to specify the behavior (code) of the assertion which is called when CONFIG_ENABLE_SYSTEM_ASSERT is defined. Typically some sort of output is generated over a serial or other interface. By default the CONFIG_SYSTEM_ASSERT_BEHAVIOR is not defined.

This macro-function receives two parameters:

- `f` - The filename (const char*) where the assertion occurred
- `l` - The line number (int) where the assertion occurred

Common implementations include writing to a serial port, toggling an LED, writing to a log file, or entering an infinite loop for debugger attachment.

Note

In order to use the **ArduinoAssert**() functionality, the CONFIG_ENABLE_ARDUINO_CPP_INTERFACE setting must be enabled.

This setting has no effect unless CONFIG_ENABLE_SYSTEM_ASSERT is also defined.

See also

CONFIG_ENABLE_SYSTEM_ASSERT

CONFIG_ENABLE_ARDUINO_CPP_INTERFACE

Example (Arduino):

```
#define CONFIG_SYSTEM_ASSERT_BEHAVIOR(f, l) __ArduinoAssert__(f, l)
```

4.1.2.11 CONFIG_TASK_NAME_BYTES `#define CONFIG_TASK_NAME_BYTES 0x8u /* 8 */`

Setting the CONFIG_TASK_NAME_BYTES allows the end-user to define the size of the task name. The larger the size of the task name, the greater impact there will be on system performance and memory usage. The default size is 8 bytes.

This value determines the maximum length of task names including the null terminator. Each task in the system will allocate this amount of memory for its name string. Task names are useful for debugging and identifying tasks during runtime inspection.

Note

The value should be set as a hexadecimal constant with the 'u' suffix (e.g., 0x8u for 8 bytes).

Task names longer than CONFIG_TASK_NAME_BYTES - 1 characters will be truncated to fit, with the last byte reserved for the null terminator.

Setting this value too small may make task names difficult to read in debugging output. Setting it too large wastes memory on each task.

See also

TaskInfo_t

xTaskCreate()

4.1.2.12 CONFIG_TASK_WD_TIMER_ENABLE `#define CONFIG_TASK_WD_TIMER_ENABLE`

Defining CONFIG_TASK_WD_TIMER_ENABLE will enable the task watchdog timer feature. The default is enabled.

Task watchdog timers provide a mechanism to detect and respond to tasks that have stopped executing or become unresponsive. When enabled, tasks can be monitored to ensure they execute within expected time bounds. This is particularly useful for safety-critical applications or systems that require high reliability.

Note

Enabling watchdog timers adds a small amount of runtime overhead for timer management and checking.

When disabled (by not defining this macro), watchdog timer functions will not be available, reducing code size and eliminating the associated overhead.

To disable this feature, comment out or remove the definition of CONFIG_TASK_WD_TIMER_ENABLE.

See also

xTaskStartWatchdog()

xTaskResetWatchdog()

xTaskStopWatchdog()

4.2 HeliOS.h File Reference

Public API header for HeliOS embedded operating system applications.

Data Structures

- struct Volume_s
- struct File_s
- struct Dir_s
- struct DirEntry_s
- struct VolumeInfo_s
- struct TaskNotification_s
- struct TaskRunTimeStats_s
- struct MemoryRegionStats_s
- struct TaskInfo_s
- struct QueueMessage_s
- struct SystemInfo_s

Macros

- #define **DEREF_TASKPARM**(type_, ptr_) (∗((type_ ∗) (ptr_)))
- #define TASKSTATE_T_

  Enumerated type defining the possible states of a task.
- #define SCHEDULERSTATE_T_

  Public API type alias for task states.
- #define RETURN_T_

  Enumerated type for scheduler state.
- #define TASKPARM_T_

  Enumerated type for syscall return type.
- #define BASE_T_

  Data type for the task paramater.
- #define BYTE_T_

  Data type for the base type.
- #define ADDR_T_

  Data type for an 8-bit wide byte.
- #define SIZE_T_

  Data type for a pointer to a memory address.
- #define HALFWORD_T_

  Data type for the storage requirements of an object in memory.
- #define WORD_T_

  Data type for a 16-bit half word.
- #define TICKS_T_

  Data type for a 32-bit word.
- #define TASK_T_

  Data type for system ticks.
- #define TIMER_T_

  Data type for a task.
- #define QUEUE_T_

  Data type for a timer.
- #define STREAMBUFFER_T_

  Data type for a queue.
- #define VOLUME_T_

  Data type for a stream buffer.
- #define FILE_T_

  Data type for a FAT32 volume.

- #define DIR_T_

  Data type for a file handle.
- #define DIRENTRY_T_

  Data type for a directory handle.
- #define VOLUMEINFO_T_

  Data type for a directory entry.
- #define TASKNOTIFICATION_T_

  Data type for volume information.
- #define TASKRUNTIMESTATS_T_

  Data structure for a direct to task notification.
- #define MEMORYREGIONSTATS_T_

  Data structure for task runtime statistics.
- #define TASKINFO_T_

  Data structure for memory region statistics.
- #define QUEUEMESSAGE_T_

  Data structure for information about a task.
- #define SYSTEMINFO_T_

  Data structure for a queue message.

Typedefs

- typedef enum TaskState_e **TaskState_t**
- typedef enum SchedulerState_e **SchedulerState_t**
- typedef enum Return_e **Return_t**
- typedef void **TaskParm_t**
- typedef uint8_t **Base_t**
- typedef uint8_t **Byte_t**
- typedef void **Addr_t**
- typedef size_t **Size_t**
- typedef uint16_t **HalfWord_t**
- typedef uint32_t **Word_t**
- typedef uint32_t **Ticks_t**
- typedef void **Task_t**
- typedef void **Timer_t**
- typedef void **Queue_t**
- typedef void **StreamBuffer_t**
- typedef struct Volume_s **Volume_t**
- typedef struct File_s **File_t**
- typedef struct Dir_s **Dir_t**
- typedef struct DirEntry_s **DirEntry_t**
- typedef struct VolumeInfo_s **VolumeInfo_t**
- typedef struct TaskNotification_s **TaskNotification_t**
- typedef struct TaskRunTimeStats_s **TaskRunTimeStats_t**
- typedef struct MemoryRegionStats_s **MemoryRegionStats_t**
- typedef struct TaskInfo_s **TaskInfo_t**
- typedef struct QueueMessage_s **QueueMessage_t**
- typedef struct SystemInfo_s **SystemInfo_t**

Enumerations

- enum TaskState_e { TaskStateSuspended , TaskStateRunning , TaskStateWaiting }
- enum SchedulerState_e { SchedulerStateSuspended , SchedulerStateRunning }
- enum Return_e { ReturnOK , ReturnError }

Functions

- Return_t xDeviceRegisterDevice (Return_t(*device_self_register_)())

  Data structure for information about the HeliOS system.
- Return_t xDeviceIsAvailable (const HalfWord_t uid_, Base_t *res_)

  Query device availability and readiness status.
- Return_t xDeviceSimpleWrite (const HalfWord_t uid_, Byte_t data_)

  Write a single byte to a device.
- Return_t xDeviceWrite (const HalfWord_t uid_, Size_t *size_, Addr_t *data_)

  Write data to a device.
- Return_t xDeviceSimpleRead (const HalfWord_t uid_, Byte_t *data_)

  Read a single byte from a device.
- Return_t xDeviceRead (const HalfWord_t uid_, Size_t *size_, Addr_t **data_)

  Read data from a device.
- Return_t xDeviceInitDevice (const HalfWord_t uid_)

  Initialize a device driver.
- Return_t xDeviceConfigDevice (const HalfWord_t uid_, Size_t *size_, Addr_t *config_)

  Configure a device and retrieve its current configuration.
- Return_t xMemAlloc (volatile Addr_t **addr_, const Size_t size_)

  Allocate memory from the user heap.
- Return_t xMemFree (const volatile Addr_t *addr_)

  Free memory previously allocated from the user heap.
- Return_t xMemFreeAll (void)

  Free all allocated memory from the user heap at once.
- Return_t xMemGetUsed (Size_t *size_)

  Query the total amount of allocated heap memory.
- Return_t xMemGetSize (const volatile Addr_t *addr_, Size_t *size_)

  Query the size of a specific heap allocation.
- Return_t xMemGetHeapStats (MemoryRegionStats_t **stats_)

  Retrieve detailed statistics about the user heap memory region.
- Return_t xMemGetKernelStats (MemoryRegionStats_t **stats_)

  Retrieve detailed statistics about the kernel memory region.
- Return_t xQueueCreate (Queue_t **queue_, const Base_t limit_)

  Create a message queue for inter-task communication.
- Return_t xQueueDelete (Queue_t *queue_)

  Delete a message queue and free its resources.
- Return_t xQueueGetLength (const Queue_t *queue_, Base_t *res_)

  Query the maximum message capacity of a queue.
- Return_t xQueueIsQueueEmpty (const Queue_t *queue_, Base_t *res_)

  Check if a message queue has no messages waiting.
- Return_t xQueueIsQueueFull (const Queue_t *queue_, Base_t *res_)

  Check if a message queue is at maximum capacity.
- Return_t xQueueMessagesWaiting (const Queue_t *queue_, Base_t *res_)

  Query the number of messages currently in a queue.
- Return_t xQueueSend (Queue_t *queue_, const Base_t bytes_, const Byte_t *value_)

  Send a message to a queue (producer operation)
- Return_t xQueuePeek (const Queue_t *queue_, QueueMessage_t **message_)

  Examine the next queue message without removing it.
- Return_t xQueueDropMessage (Queue_t *queue_)

  Remove the next message from queue without retrieving it.
- Return_t xQueueReceive (Queue_t *queue_, QueueMessage_t **message_)

Receive and remove a message from a queue (consumer operation)

- Return_t xQueueLockQueue (Queue_t *queue_)

  Lock a queue to prevent new messages from being sent.
- Return_t xQueueUnLockQueue (Queue_t *queue_)

  Unlock a queue to resume accepting new messages.
- Return_t xStreamCreate (StreamBuffer_t **stream_)

  Create a byte-oriented stream buffer for inter-task communication.
- Return_t xStreamDelete (const StreamBuffer_t *stream_)

  Delete a stream buffer and free its resources.
- Return_t xStreamSend (StreamBuffer_t *stream_, const Byte_t byte_)

  Send a single byte to a stream buffer (producer operation)
- Return_t xStreamReceive (const StreamBuffer_t *stream_, HalfWord_t *bytes_, Byte_t **data_)

  Receive all waiting bytes from a stream buffer (consumer operation)
- Return_t xStreamBytesAvailable (const StreamBuffer_t *stream_, HalfWord_t *bytes_)

  Query the number of bytes waiting in a stream buffer.
- Return_t xStreamReset (const StreamBuffer_t *stream_)

  Clear all bytes from a stream buffer.
- Return_t xStreamIsEmpty (const StreamBuffer_t *stream_, Base_t *res_)

  Check if a stream buffer contains no waiting bytes.
- Return_t xStreamIsFull (const StreamBuffer_t *stream_, Base_t *res_)

  Check if a stream buffer is at full capacity.
- Return_t xSystemAssert (const char *file_, const int line_)

  Trigger a system assertion failure for critical error handling.
- Return_t xSystemInit (void)

  Bootstrap HeliOS kernel and initialize all system resources.
- Return_t xSystemHalt (void)

  Immediately halt HeliOS and stop all system execution.
- Return_t xSystemGetSystemInfo (SystemInfo_t **info_)

  Retrieve comprehensive system information and runtime statistics.
- Return_t xTaskCreate (Task_t **task_, const Byte_t *name_, void(*callback_)(Task_t *task_, TaskParm_t *parm_), TaskParm_t *taskParameter_)

  Create a new task for cooperative multitasking.
- Return_t xTaskDelete (const Task_t *task_)

  Delete a task and free its resources.
- Return_t xTaskGetHandleByName (Task_t **task_, const Byte_t *name_)

  Retrieve a task handle by its name.
- Return_t xTaskGetHandleById (Task_t **task_, const Base_t id_)

  Retrieve a task handle by its numeric ID.
- Return_t xTaskGetAllRunTimeStats (TaskRunTimeStats_t **stats_, Base_t *tasks_)

  Retrieve runtime execution statistics for all tasks in the system.
- Return_t xTaskGetTaskRunTimeStats (const Task_t *task_, TaskRunTimeStats_t **stats_)

  Retrieve runtime execution statistics for a specific task.
- Return_t xTaskGetNumberOfTasks (Base_t *tasks_)

  Query the total number of tasks in the system.
- Return_t xTaskGetTaskInfo (const Task_t *task_, TaskInfo_t **info_)

  Retrieve comprehensive information about a task.
- Return_t xTaskGetAllTaskInfo (TaskInfo_t **info_, Base_t *tasks_)

  Retrieve comprehensive information about all tasks.
- Return_t xTaskGetTaskState (const Task_t *task_, TaskState_t *state_)

  Query the current execution state of a task.
- Return_t xTaskGetName (const Task_t *task_, Byte_t **name_)

Retrieve the ASCII name string of a task.

- Return_t xTaskGetId (const Task_t *task_, Base_t *id_)

  Retrieve the unique numeric identifier of a task.
- Return_t xTaskNotifyStateClear (Task_t *task_)

  Clear any pending direct-to-task notification for a task.
- Return_t xTaskNotificationIsWaiting (const Task_t *task_, Base_t *res_)

  Check if a direct-to-task notification is pending for a task.
- Return_t xTaskNotifyGive (Task_t *task_, const Base_t bytes_, const Byte_t *value_)

  Send a direct-to-task notification with optional data payload.
- Return_t xTaskNotifyTake (Task_t *task_, TaskNotification_t **notification_)

  Receive and consume a pending direct-to-task notification.
- Return_t xTaskResume (Task_t *task_)

  Activate a task for scheduler execution.
- Return_t xTaskSuspend (Task_t *task_)

  Deactivate a task to prevent scheduler execution.
- Return_t xTaskWait (Task_t *task_)

  Place a task in event-driven waiting state.
- Return_t xTaskChangePeriod (Task_t *task_, const Ticks_t period_)

  Set the execution period for periodic task scheduling.
- Return_t xTaskChangeWDPeriod (Task_t *task_, const Ticks_t period_)

  Set task watchdog timeout for detecting runaway tasks.
- Return_t xTaskGetPeriod (const Task_t *task_, Ticks_t *period_)

  Query the current execution period of a task.
- Return_t xTaskResetTimer (Task_t *task_)

  Reset a task's elapsed time counter to zero.
- Return_t xTaskStartScheduler (void)

  Start the HeliOS cooperative task scheduler.
- Return_t xTaskResumeAll (void)

  Resume the scheduler to enable task execution.
- Return_t xTaskSuspendAll (void)

  Suspend the scheduler and return control to caller.
- Return_t xTaskGetSchedulerState (SchedulerState_t *state_)

  Query the current state of the task scheduler.
- Return_t xTaskGetWDPeriod (const Task_t *task_, Ticks_t *period_)

  Query the task watchdog timeout period.
- Return_t xTimerCreate (Timer_t **timer_, const Ticks_t period_)

  Create a software timer for general-purpose timekeeping.
- Return_t xTimerDelete (const Timer_t *timer_)

  Delete an application timer and free its resources.
- Return_t xTimerChangePeriod (Timer_t *timer_, const Ticks_t period_)

  Change the period of an existing application timer.
- Return_t xTimerGetPeriod (const Timer_t *timer_, Ticks_t *period_)

  Query the current period of an application timer.
- Return_t xTimerIsTimerActive (const Timer_t *timer_, Base_t *res_)

  Check if an application timer is currently running.
- Return_t xTimerHasTimerExpired (const Timer_t *timer_, Base_t *res_)

  Check if an application timer has expired.
- Return_t xTimerReset (Timer_t *timer_)

  Reset an application timer's elapsed time to zero.
- Return_t xTimerStart (Timer_t *timer_)

  Start an application timer to begin timing.

- Return_t xTimerStop (Timer_t *timer_)

  Stop an application timer and halt timing.
- Return_t xFSFormat (const HalfWord_t blockDeviceUID_, const Byte_t *volumeLabel_)

  Format a block device with a FAT32 filesystem.
- Return_t xFSMount (Volume_t **volume_, const HalfWord_t blockDeviceUID_)

  Mount a FAT32 filesystem from a block device.
- Return_t xFSUnmount (Volume_t *volume_)

  Unmount a FAT32 filesystem volume.
- Return_t xFSGetVolumeInfo (const Volume_t *volume_, VolumeInfo_t **info_)

  Query information about a mounted FAT32 volume.
- Return_t xFileOpen (File_t **file_, Volume_t *volume_, const Byte_t *path_, const Byte_t mode_)

  Open a file on a mounted FAT32 volume.
- Return_t xFileClose (File_t *file_)

  Close an open file.
- Return_t xFileRead (File_t *file_, const Size_t size_, Byte_t **data_)

  Read data from a file.
- Return_t xFileWrite (File_t *file_, const Size_t size_, const Byte_t *data_)

  Write data to a file.
- Return_t xFileSeek (File_t *file_, const Word_t offset_, const Byte_t origin_)

  Change the file position for reading or writing.
- Return_t xFileTell (const File_t *file_, Word_t *position_)

  Get the current file position.
- Return_t xFileGetSize (const File_t *file_, Word_t *size_)

  Get the size of a file in bytes.
- Return_t xFileSync (File_t *file_)

  Flush file writes to storage.
- Return_t xFileTruncate (File_t *file_, const Word_t size_)

  Resize a file to a specified size.
- Return_t xFileEOF (const File_t *file_, Base_t *eof_)

  Check if file position is at end-of-file.
- Return_t xFileExists (Volume_t *volume_, const Byte_t *path_, Base_t *exists_)

  Check if a file exists on the volume.
- Return_t xFileUnlink (Volume_t *volume_, const Byte_t *path_)

  Delete a file from the filesystem.
- Return_t xFileRename (Volume_t *volume_, const Byte_t *oldPath_, const Byte_t *newPath_)

  Rename or move a file.
- Return_t xFileGetInfo (Volume_t *volume_, const Byte_t *path_, DirEntry_t **entry_)

  Get detailed information about a file.
- Return_t xDirOpen (Dir_t **dir_, Volume_t *volume_, const Byte_t *path_)

  Open a directory for reading.
- Return_t xDirClose (Dir_t *dir_)

  Close an open directory.
- Return_t xDirRead (Dir_t *dir_, DirEntry_t **entry_)

  Read the next entry from a directory.
- Return_t xDirRewind (Dir_t *dir_)

  Reset directory read position to beginning.
- Return_t xDirMake (Volume_t *volume_, const Byte_t *path_)

  Create a new directory.
- Return_t xDirRemove (Volume_t *volume_, const Byte_t *path_)

  Delete an empty directory.

4.2.1 Detailed Description

Author

Manny Peterson manny@heliosproj.org

Version

0.5.0

Date

2023-03-19

This header file provides the complete public API for HeliOS, a lightweight embedded operating system designed for resource-constrained microcontrollers. It includes all type definitions, enumerations, and system call (syscall) declarations needed to develop applications on HeliOS.

HeliOS provides the following subsystems:

- Task Management: Create and manage cooperative multitasking
- Memory Management: Dynamic heap allocation with safety checks
- Device I/O: Abstract device driver interface
- Timers: Software timers for periodic and one-shot events
- Queues: Inter-task message passing with FIFO semantics
- Streams: Byte-oriented data buffers for streaming I/O
- Filesystem: FAT32 filesystem support with block device abstraction

All HeliOS system calls follow the naming convention xSubsystem∗() where x is the prefix for public APIs and Subsystem identifies the functional area (e.g., Task_t ∗, xMem, Queue_t ∗).

Copyright

4.2.2 Macro Definition Documentation

4.2.2.1 ADDR_T_ `#define ADDR_T_`

See also

Byte_t

Data type for a pointer to a memory address

The Addr_t type is a pointer of type void and is used to pass addresses between the end-user application and syscalls. It is not necessary to use the Addr_t type within the end-user application as long as the type is not used to interact with the kernel through syscalls

See also

Addr_t *

4.2.2.2 BASE_T_ `#define BASE_T_`

See also

TaskParm_t

Data type for the base type

The Base_t type is a simple data type often used as an argument or result type for syscalls when the value is known not to exceed its 8-bit width and no data structure requirements exist. There are no guarantees the Base_t will always be 8-bits wide. If an 8-bit data type is needed that is guaranteed to remain 8-bits wide, the Byte_t data type should be used.

See also

Base_t

Byte_t

4.2.2.3 BYTE_T_ `#define BYTE_T_`

See also

Base_t

Data type for an 8-bit wide byte

The Byte_t type is an 8-bit wide data type and is guaranteed to always be 8-bits wide.

See also

Byte_t

4.2.2.4 DIR_T_ `#define DIR_T_`

See also

File_t
xFileOpen()
xFileClose()

Data structure for directory handle

See also

Dir_t *
xDirOpen()
xDirClose()

4.2.2.5 DIRENTRY_T_ `#define DIRENTRY_T_`

See also

Dir_t
xDirOpen()
xDirClose()

Data structure for directory entry information

See also

DirEntry_t *
xDirRead()
xFileGetInfo()

4.2.2.6 FILE_T_ `#define FILE_T_`

See also

Volume_t
xFSMount()
xFSUnmount()

Data structure for file handle

See also

File_t *
xFileOpen()
xFileClose()

4.2.2.7 HALFWORD_T_ `#define HALFWORD_T_`

See also

Size_t

Data type for a 16-bit half word

The HalfWord_t type is a 16-bit wide data type and is guaranteed to always be 16-bits wide.

See also

HalfWord_t

4.2.2.8 MEMORYREGIONSTATS_T_ `#define MEMORYREGIONSTATS_T_`

Data structure for memory region statistics

The MemoryRegionStats_t data structure is used by xMemGetHeapStats() and xMemGetKernelStats() to obtain statistics about either memory region.

See also

MemoryRegionStats_t *

xMemGetHeapStats()

xMemGetKernelStats()

xMemFree()

4.2.2.9 QUEUE_T_ `#define QUEUE_T_`

See also

Timer_t

Data type for a queue

The Queue_t data type is used as a queue The queue is created when xQueueCreate() is called. For more information about queues, see xQueueCreate().

See also

Queue_t *

xQueueCreate()

xQueueDelete()

4.2.2.10 QUEUEMESSAGE_T_ `#define QUEUEMESSAGE_T_`

Data structure for a queue message

The QueueMessage_t stucture is used to store a queue message and is returned by xQueueReceive() and xQueuePeek().

See also

QueueMessage_t *
xQueueReceive()
xQueuePeek()
CONFIG_MESSAGE_VALUE_BYTES
xMemFree()

4.2.2.11 RETURN_T_ `#define RETURN_T_`

See also

SchedulerState_t

Enumerated type for syscall return type

All HeliOS syscalls return the Return_t type which can either be ReturnOK or ReturnError. The C macros OK() and ERROR() can be used as a more concise way of checking the return value of a syscall (e.g., if(OK(xMemGet↩ Used(&size))) {} or if(ERROR(xMemGetUsed(&size))) {}).

See also

OK()
ERROR()
Return_t

4.2.2.12 SCHEDULERSTATE_T_ `#define SCHEDULERSTATE_T_`

This is the user-facing type name for task states, providing a consistent naming convention across the HeliOS public API where all types are prefixed with 'x'.

See also

TaskState_t

Enumerated type for scheduler state

The scheduler can be in one of three possible states as defined by the SchedulerState_t enumerated data type. The state the scheduler is in is changed by calling xTaskSuspendAll() and xTaskResumeAll(). The state the scheduler is in can be obtained by calling xTaskGetSchedulerState().

See also

SchedulerState_t
xTaskSuspendAll()
xTaskResumeAll()
xTaskGetSchedulerState()
xTaskStartScheduler()

4.2.2.13 SIZE_T_ `#define SIZE_T_`

See also

Addr_t

Data type for the storage requirements of an object in memory

The Size_t type is used for the storage requirements of an object in memory and is always represented in bytes.

See also

Size_t

4.2.2.14 STREAMBUFFER_T_ `#define STREAMBUFFER_T_`

See also

Queue_t

Data type for a stream buffer

The StreamBuffer_t data type is used as a stream buffer. The stream buffer is created when xStreamCreate() is called. For more information about stream buffers, see xStreamCreate(). Stream_t should be declared as xStream.

See also

xStream

xStreamCreate()

xStreamDelete()

4.2.2.15 SYSTEMINFO_T_ `#define SYSTEMINFO_T_`

Data structure for information about the HeliOS system

The SystemInfo_t data structure is used to store information about the HeliOS system and is returned by xSystemGetSystemInfo().

See also

SystemInfo_t *

xSystemGetSystemInfo()

OS_PRODUCT_NAME_SIZE

xMemFree()

4.2.2.16 TASK_T_ `#define TASK_T_`

See also

Ticks_t

Data type for a task

The Task_t data type is used as a task. The task is created when xTaskCreate() is called. For more information about tasks, see xTaskCreate().

See also

Task_t *
xTaskCreate()
xTaskDelete()

4.2.2.17 TASKINFO_T_ `#define TASKINFO_T_`

Data structure for information about a task

The TaskInfo_t structure is similar to xTaskRuntimeStats_t in that it contains runtime statistics for a task. However, TaskInfo_t also contains additional details about a task such as its name and state. The TaskInfo_t structure is returned by xTaskGetTaskInfo() and xTaskGetAllTaskInfo(). If only runtime statistics are needed, then TaskRun↩ TimeStats_t should be used because of its smaller memory footprint.

See also

TaskInfo_t *
xTaskGetTaskInfo()
xTaskGetAllTaskInfo()
CONFIG_TASK_NAME_BYTES
xMemFree()

4.2.2.18 TASKNOTIFICATION_T_ `#define TASKNOTIFICATION_T_`

See also

VolumeInfo_t
xFSGetVolumeInfo()

Data structure for a direct to task notification

The TaskNotification_t data structure is used by xTaskNotifyGive() and xTaskNotifyTake() to send and receive direct to task notifications. Direct to task notifications are part of the event-driven multitasking model. A direct to task notification may be received by event-driven and co-operative tasks alike. However, the benefit of direct to task notifications may only be realized by tasks scheduled as event-driven. In order to wait for a direct to task notification, the task must be in a "waiting" state which is set by xTaskWait().

See also

TaskNotification_t *
xMemFree()
xTaskNotifyGive()
xTaskNotifyTake()
xTaskWait()

4.2.2.19 TASKPARM_T_ `#define TASKPARM_T_`

See also

Return_t

Data type for the task paramater

The TaskParm_t type is used to pass a paramater to a task at the time of task creation using xTaskCreate(). A task paramater is a pointer of type void and can point to any number of types, arrays and/or data structures that will be passed to the task. It is up to the end-user to manage, allocate and free the memory related to these objects using xMemAlloc() and xMemFree().

See also

TaskParm_t *
xTaskCreate()
xMemAlloc()
xMemFree()

4.2.2.20 TASKRUNTIMESTATS_T_ `#define TASKRUNTIMESTATS_T_`

See also

TaskNotification_t

Data structure for task runtime statistics

The TaskRunTimeStats_t data structure is used by xTaskGetTaskRunTimeStats() and xTaskGetAllRuntimeStats() to obtain runtime statistics about a task.

See also

TaskRunTimeStats_t *
xTaskGetTaskRunTimeStats()
xTaskGetAllRunTimeStats()
xMemFree()

4.2.2.21 TASKSTATE_T_ `#define TASKSTATE_T_`

Tasks in HeliOS transition between three distinct states that control their execution behavior by the cooperative scheduler. The task state determines whether the scheduler will invoke the task's function during scheduling cycles.

State Transitions:

- New tasks begin in TaskStateSuspended
- xTaskResume() transitions a task to TaskStateRunning
- xTaskSuspend() transitions a task to TaskStateSuspended
- xTaskWait() transitions a task to TaskStateWaiting

Scheduling Behavior:

- TaskStateSuspended: Task will NOT be scheduled for execution
- TaskStateRunning: Task will be scheduled normally based on its period
- TaskStateWaiting: Task will be scheduled only after a task event occurs (e.g., timer expiration, notification)

Note

HeliOS uses cooperative multitasking. Tasks must voluntarily yield control to allow other tasks to execute.

See also

TaskState_t
xTaskResume()
xTaskSuspend()
xTaskWait()
xTaskGetTaskState()

4.2.2.22 TICKS_T_ `#define TICKS_T_`

See also

Word_t

Data type for system ticks

The Ticks_t type is used to store ticks from the system clock. Ticks is not bound to any one unit of measure for time though most systems are configured for millisecond resolution, milliseconds is not guaranteed and is dependent on the system clock frequency and prescaler.

See also

Ticks_t

4.2.2.23 TIMER_T_ `#define TIMER_T_`

See also

Task_t

Data type for a timer

The Timer_t data type is used as a timer. The timer is created when xTimerCreate() is called. For more information about timers, see xTimerCreate().

See also

Timer_t *
xTimerCreate()
xTimerDelete()

4.2.2.24 VOLUME_T_ `#define VOLUME_T_`

See also

StreamBuffer_t

Data structure for FAT32 volume metadata

See also

Volume_t *
xFSMount()
xFSUnmount()

4.2.2.25 VOLUMEINFO_T_ `#define VOLUMEINFO_T_`

See also

DirEntry_t
xDirRead()
xFileGetInfo()

Data structure for volume information

See also

VolumeInfo_t *
xFSGetVolumeInfo()

4.2.2.26 WORD_T_ `#define WORD_T_`

See also

HalfWord_t

Data type for a 32-bit word

The Word_t type is a 32-bit wide data type and is guaranteed to always be 32-bits wide.

See also

Word_t

4.2.3 Enumeration Type Documentation

4.2.3.1 Return_e `enum Return_e`

Enumerator

ReturnOK	Return value if the syscall was successful.
ReturnError	Return value if the syscall failed.

4.2.3.2 SchedulerState_e `enum SchedulerState_e`

Enumerator

SchedulerStateSuspended	State the scheduler is in after calling xTaskSuspendAll(). TaskStartScheduler() will stop scheduling tasks for execution and relinquish control when xTaskSuspendAll() is called.
SchedulerStateRunning	State the scheduler is in after calling xTaskResumeAll(). xTaskStartScheduler() will continue to schedule tasks for execution until xTaskSuspendAll() is called.

4.2.3.3 TaskState_e `enum TaskState_e`

Enumerator

TaskStateSuspended	Task is inactive and will not be scheduled. This is the initial state after task creation and the state after calling xTaskSuspend().
TaskStateRunning	Task is active and will be scheduled for execution according to its period. Set by calling xTaskResume().
TaskStateWaiting	Task is waiting for an event and will only be scheduled when that event occurs (timer, notification, etc.). Set by calling xTaskWait().

4.2.4 Function Documentation

4.2.4.1 xDeviceConfigDevice()

```
Return_t xDeviceConfigDevice (
        const HalfWord_t uid_,
        Size_t * size_,
        Addr_t * config_ )
```

Configures the specified device and retrieves its current configuration in a single bidirectional operation. This function writes configuration parameters to the device driver and reads back the effective configuration, allowing applications to both set and verify device settings atomically.

The bidirectional nature serves two purposes: writing configuration to apply new settings, and reading back the actual configuration to verify successful application or to query current device state. This is particularly useful for devices where the effective configuration may differ from requested settings (due to hardware limitations or automatic adjustments).

Configuration structure and content are driver-specific. Each device driver defines its own configuration data structure containing parameters like:

- Device operating mode (DeviceMode, DeviceState)
- Hardware settings (baud rate, clock speed, resolution)
- Feature enables (interrupts, DMA, flow control)
- Operational parameters (timeouts, buffer sizes, thresholds)

Typical configuration workflow:

1. Allocate configuration structure with xMemAlloc() 2. Populate structure with desired settings 3. Call xDeviceConfigDevice() to apply and read back
2. Verify returned configuration matches expectations 5. Free configuration structure with xMemFree()

Warning

Configuration data MUST be allocated from user heap via xMemAlloc(). Stack-allocated or static configuration structures will cause memory validation errors. HeliOS enforces this to maintain memory safety.

Example 1: Configure UART baud rate

```
#define UART0_UID 0x0100
typedef struct {
  Word_t baudRate;
  Byte_t dataBits;
  Byte_t stopBits;
  Byte_t parity;
  DeviceMode mode;
} UARTConfig_t;
Return_t configureUART(Word_t baud) {
  UARTConfig_t *config = NULL;
  Size_t configSize = sizeof(UARTConfig_t);
  // Allocate config from heap if (OK(xMemAlloc((volatile Addr_t
**)&config, configSize))) {
    // Set desired configuration config->baudRate = baud;
    config->dataBits = 8;
    config->stopBits = 1;
    config->parity = 0;  // No parity config->mode =
DeviceModeReadWrite;
    // Apply configuration and read back if
(OK(xDeviceConfigDevice(UART0_UID,
```

```
&configSize, (Addr_t *)config))) {
      // Verify effective baud rate if (config->baudRate != baud) {
        logWarning("UART baud rate adjusted to %lu", config->baudRate);
      }
    }
    xMemFree((Addr_t *)config);
    return ReturnOK;
  }
  return ReturnError;
}
```

Example 2: Query current device configuration

```
#define SENSOR_UID 0x0400
typedef struct {
  HalfWord_t sampleRate;
  Byte_t resolution;
  Byte_t powerMode;
  DeviceState state;
} SensorConfig_t;
Return_t querySensorConfig(SensorConfig_t *outConfig) {
  SensorConfig_t *config = NULL;
  Size_t configSize = sizeof(SensorConfig_t);
  if (OK(xMemAlloc((volatile Addr_t **)&config, configSize))) {
    // Don't set any fields - just query current settings if
(OK(xDeviceConfigDevice(SENSOR_UID, &configSize, (Addr_t *)config))) {
      // Copy config to output parameter *outConfig = *config;
      xMemFree((Addr_t *)config);
      return ReturnOK;
    }
    xMemFree((Addr_t *)config);
  }
  return ReturnError;
}
```

Example 3: Change device mode dynamically

```
 typedef struct {
  DeviceMode mode;
  DeviceState state;
} GenericDeviceConfig_t;
Return_t setDeviceMode(HalfWord_t deviceUID, DeviceMode newMode) {
  GenericDeviceConfig_t *config = NULL;
  Size_t configSize = sizeof(GenericDeviceConfig_t);
  if (OK(xMemAlloc((volatile Addr_t **)&config, configSize))) {
    // First query current config if (OK(xDeviceConfigDevice(deviceUID,
&configSize, (Addr_t *)config))) {
      // Change only the mode config->mode = newMode;
      // Apply updated config if (OK(xDeviceConfigDevice(deviceUID,
&configSize, (Addr_t *)config))) {
        // Verify mode change if (config->mode == newMode) {
          xMemFree((Addr_t *)config);
          return ReturnOK;
        }
      }
    }
    xMemFree((Addr_t *)config);
  }
  return ReturnError;
}
```

Parameters

in	*uid_*	Unique identifier of the device to configure. Must match a UID previously registered via xDeviceRegisterDevice().
in,out	*size_*	Pointer to size variable. On input: size in bytes of the configuration structure. On output: may be updated by driver to reflect actual configuration size returned.
in,out	*config_*	Pointer to heap-allocated configuration structure. On input: contains desired configuration parameters. On output: contains actual effective configuration as reported by driver. Must be allocated via xMemAlloc().

Returns

ReturnOK if configuration successful, ReturnError if operation failed (invalid UID, device not found, config not from heap, invalid configuration parameters, or driver config callback failed).

Warning

The config_ buffer MUST be allocated from user heap using xMemAlloc(). Stack or static allocation will cause validation failure.

After successful return, always check the returned configuration to verify the driver accepted your requested settings. Some devices may adjust parameters to supported values.

Configuration structures are driver-specific. Ensure you're using the correct structure type for the target device driver.

Note

The size_ parameter should initially contain the size of your configuration structure (e.g., sizeof(MyConfig↩ Struct)).

This function is bidirectional: it both writes and reads configuration in a single call. To query configuration without changing it, simply pass an uninitialized structure.

Not all device drivers support configuration. Drivers for simple devices may implement a no-op configuration callback that always succeeds.

See also

xDeviceInitDevice() - Initialize device before configuration

xDeviceRegisterDevice() - Register device driver

xMemAlloc() - Allocate configuration structure

xMemFree() - Free configuration structure after use

4.2.4.2 xDeviceInitDevice() `Return_t xDeviceInitDevice ( const HalfWord_t uid_ )`

Initializes the specified device by invoking its driver's initialization callback function. This performs hardware and software setup required before the device can be used for I/O operations. Initialization typically configures hardware registers, sets initial device state and mode, allocates driver-specific resources, and prepares the device for operation.

Device initialization is a critical first step after registering a device driver with xDeviceRegisterDevice(). Without proper initialization, devices may not respond to I/O requests or may behave unpredictably. The specific initialization actions depend entirely on the device driver implementation.

Common initialization tasks performed by drivers:

- **Hardware configuration**: Setting memory-mapped registers, clock speeds, baud rates
- **State initialization**: Setting device to DeviceStateRunning or initial state
- **Mode configuration**: Configuring read/write mode, interrupts, DMA
- **Buffer allocation**: Creating internal receive/transmit buffers if needed
- **Self-test**: Performing device self-checks or calibration
- **Feature enabling**: Activating device-specific features (e.g., UART flow control)

Typical device initialization sequence:

1. Register device driver with xDeviceRegisterDevice() 2. Initialize device with xDeviceInitDevice() 3. Optionally configure device with xDeviceConfigDevice() 4. Begin I/O operations with xDeviceRead() / xDeviceWrite()

Example 1: Initialize UART device

```
#define UART0_UID 0x0100
// Register UART driver (done during system startup) extern DeviceDriver_t
uart0Driver;
if (OK(xDeviceRegisterDevice(UART0_UID, &uart0Driver))) {
  // Initialize the UART hardware if (OK(xDeviceInitDevice(UART0_UID))) {
    // UART ready for use
  } else {
    logError("UART0 initialization failed");
  }
}
```

Example 2: Initialize multiple devices in sequence

```
#define UART0_UID  0x0100
#define SPI0_UID   0x0200
#define I2C0_UID   0x0300
Return_t initPeripherals(void) {
  // Initialize UART if (ERROR(xDeviceInitDevice(UART0_UID))) {
    return ReturnError;
  }
  // Initialize SPI if (ERROR(xDeviceInitDevice(SPI0_UID))) {
    return ReturnError;
  }
  // Initialize I2C if (ERROR(xDeviceInitDevice(I2C0_UID))) {
    return ReturnError;
  }
  return ReturnOK;  // All devices initialized
}
```

Example 3: Initialize with error recovery

```
#define MAX_INIT_RETRIES 3
Return_t initDeviceWithRetry(HalfWord_t deviceUID) {
  Base_t retries = 0;
  while (retries < MAX_INIT_RETRIES) {
    if (OK(xDeviceInitDevice(deviceUID))) {
      logInfo("Device 0x%04X initialized", deviceUID);
      return ReturnOK;
    }
    retries++;
    if (retries < MAX_INIT_RETRIES) {
      logWarning("Device init failed, retrying %u/%u", retries,
MAX_INIT_RETRIES);
      delayMs(100);  // Brief delay before retry
    }
  }
  logError("Device 0x%04X failed to initialize after %u attempts",
deviceUID, MAX_INIT_RETRIES);
  return ReturnError;
}
```

Parameters

in	*uid*↩ _	Unique identifier of the device to initialize. Must match a UID previously registered via xDeviceRegisterDevice().

Returns

ReturnOK if device initialized successfully, ReturnError if initialization failed (invalid UID, device not registered, driver init callback failed, or hardware initialization error).

Warning

Always check the return value. A failed initialization means the device is not ready for use, and subsequent I/O operations will likely fail.

Do not call xDeviceInitDevice() on an already-initialized device unless the driver explicitly supports re-initialization. Some drivers may leak resources or behave unpredictably if initialized multiple times.

Note

Not all device drivers require initialization. Some simple drivers may be ready immediately after registration. However, calling xDeviceInitDevice() on such devices is safe and recommended for consistency.

Initialization is typically performed during system startup, before the scheduler starts. However, it can be called at any time (e.g., for runtime device hotplug or power management).

If initialization fails, check hardware connections, power supply, clock configuration, and driver implementation for issues.

See also

xDeviceRegisterDevice() - Register device driver before initialization

xDeviceConfigDevice() - Configure device after initialization

xDeviceRead() - Read from initialized device

xDeviceWrite() - Write to initialized device

xDeviceIsAvailable() - Check if device is ready for use

4.2.4.3 xDeviceIsAvailable()

```
Return_t xDeviceIsAvailable (
    const HalfWord_t uid_,
    Base_t * res_ )
```

Checks whether a device is available and ready for I/O operations by invoking the device driver's availability callback. The meaning of "available" is driver-specific but typically indicates the device is initialized, operational, and ready to accept read/write requests without errors.

Device availability can vary based on hardware state, driver configuration, and operational conditions. For example, a UART might be unavailable if not initialized, a sensor might be unavailable during calibration, or a storage device might be unavailable if busy with another operation.

Typical availability semantics by device type:

- **Serial devices**: Ready to transmit/receive (not in error state)
- **Storage devices**: Not busy, no pending operations, hardware ready
- **Sensors**: Calibrated, powered up, and ready to sample
- **Network devices**: Link established, buffers available
- **Custom devices**: Driver-defined operational readiness

Common use cases:

- **Pre-operation checks**: Verify device is ready before I/O
- **Error recovery**: Check availability after device errors
- **Initialization validation**: Confirm device initialized successfully
- **Polling loops**: Wait for device to become ready
- **Diagnostics**: Determine which devices are operational

Example 1: Check device before write operation

```
#define UART0_UID 0x0100
Return_t sendData(const Byte_t *data, Size_t len) {
  Base_t isAvailable;
  // Check if UART is available before writing if
(OK(xDeviceIsAvailable(UART0_UID, &isAvailable))) {
    if (isAvailable) {
      // Device ready - perform write return xDeviceWrite(UART0_UID, &len,
(Addr_t *)data);
    } else {
      logWarning("UART not available for write");
      return ReturnError;
    }
  }
  logError("Failed to query UART availability");
  return ReturnError;
}
```

Example 2: Poll until device becomes available

```
#define SENSOR_UID 0x0300
#define MAX_WAIT_CYCLES 100
Return_t waitForSensorReady(void) {
  Base_t isAvailable;
  Base_t retries = 0;
  while (retries < MAX_WAIT_CYCLES) {
    if (OK(xDeviceIsAvailable(SENSOR_UID, &isAvailable))) {
      if (isAvailable) {
        logInfo("Sensor ready after %u attempts", retries);
        return ReturnOK;
      }
    }
    // Wait a bit before retrying xTaskDelayUntil(10);  // 10 ticks
retries++;
  }
  logError("Sensor failed to become available");
  return ReturnError;
}
```

Example 3: System diagnostic - check all devices

```
 typedef struct {
  HalfWord_t uid;
  const char *name;
} DeviceInfo_t;
void checkSystemDevices(void) {
  DeviceInfo_t devices[] = {
    {0x0100, "UART0"},
    {0x0200, "SPI0"},
    {0x0300, "Sensor"},
    {0x0400, "Storage"}
  };
  Base_t deviceCount = 4;
  Base_t availableCount = 0;
  printf("Device Availability Check:\n");
  for (Base_t i = 0; i < deviceCount; i++) {
    Base_t isAvailable;
    if (OK(xDeviceIsAvailable(devices[i].uid, &isAvailable))) {
      printf("  %-10s [0x%04X]: %s\n", devices[i].name, devices[i].uid,
isAvailable ? "AVAILABLE" : "UNAVAILABLE");
      if (isAvailable) {
        availableCount++;
      }
    } else {
      printf("  %-10s [0x%04X]: ERROR (not registered?)\n",
devices[i].name, devices[i].uid);
    }
  }
  printf("\nSummary: %u/%u devices available\n", availableCount,
deviceCount);
}
```

Example 4: Retry I/O with availability check

```
#define FLASH_UID 0x1000
#define MAX_RETRIES 5
Return_t writeFlashWithRetry(Byte_t *data, Size_t len) {
  Base_t retries = 0;
  while (retries < MAX_RETRIES) {
    Base_t isAvailable;
    // Check availability if (OK(xDeviceIsAvailable(FLASH_UID,
&isAvailable)) && isAvailable) {
      // Try write operation if (OK(xDeviceWrite(FLASH_UID, &len, (Addr_t
*)data))) {
        return ReturnOK;  // Success
      }
    }
```

```
    // Write failed or device unavailable - wait and retry
logWarning("Flash write failed, retry %u/%u", retries + 1, MAX_RETRIES);
    xTaskDelayUntil(100);  // Wait longer for flash retries++;
  }
  logError("Flash write failed after %u retries", MAX_RETRIES);
  return ReturnError;
}
```

Parameters

in	*uid_*	Unique identifier of the device to query. Must match a UID previously registered via xDeviceRegisterDevice().
out	*res_*	Pointer to variable that receives the availability status. Set to non-zero (true) if device is available, zero (false) if unavailable.

Returns

ReturnOK if query succeeded (res_ contains valid result), ReturnError if query failed (invalid UID, device not registered, or driver does not implement availability check).

Warning

A successful return (ReturnOK) only means the query was performed. Check the res_ value to determine actual device availability. ReturnOK with res_=0 means "successfully determined device is unavailable."

Device availability can change between checking and using. After confirming availability, the device might become unavailable before your I/O operation. Always check I/O function return values.

If the driver does not implement an availability callback, this function may return ReturnError or always report available. Check your driver documentation.

Note

The interpretation of "available" is driver-specific. Some drivers may always return available after initialization, while others perform detailed hardware checks.

This is a lightweight query that typically does not perform I/O. It's safe to call frequently in polling loops.

Combining availability checks with actual I/O operations is a common pattern for robust device communication in embedded systems.

See also

xDeviceInitDevice() - Initialize device before checking availability

xDeviceConfigDevice() - Configure device state

xDeviceWrite() - Write to device (check availability first)

xDeviceRead() - Read from device (check availability first)

xDeviceRegisterDevice() - Register device driver

4.2.4.4 xDeviceRead()

```
Return_t xDeviceRead (
        const HalfWord_t uid_,
        Size_t * size_,
        Addr_t ** data_ )
```

Reads data from the specified device through its registered driver. The device driver allocates a buffer from the user heap, fills it with data from the device, and returns it to the caller. This provides an abstract interface for device input, allowing applications to read from any registered device using a consistent API.

Unlike xDeviceWrite() where the caller allocates memory, xDeviceRead() has the DEVICE DRIVER allocate the buffer. This is necessary because the driver determines how much data is available to read. The caller receives both the data buffer pointer and the size of data read.

The read operation behavior depends on the device driver implementation:

- Serial devices typically read bytes from a receive buffer or UART
- Storage devices read from specific sectors or blocks
- Network devices receive packets
- Sensor devices read measurement data
- Custom devices implement application-specific read semantics

Data Flow:

1. Application calls xDeviceRead() with device UID 2. xDeviceRead() validates device UID, state, and permissions 3. Driver's read callback is invoked 4. Driver allocates buffer from heap (via xMemAlloc internally) 5. Driver fills buffer with data from device 6. Driver returns buffer pointer and size to HeliOS 7. HeliOS returns buffer to application 8. Application processes data then MUST free buffer with xMemFree()

Warning

The caller is RESPONSIBLE FOR FREEING the returned buffer using xMemFree(). Failing to free the buffer will cause memory leaks. The driver allocates this memory specifically for this read operation.

Note

Device read operations are synchronous by default. The function does not return until the driver completes the read operation and allocates the buffer. Drivers may implement buffering or blocking behavior internally.

The device must be in a readable state (DeviceModeReadOnly or DeviceModeReadWrite) and running state (DeviceStateRunning) for the read to succeed. Check device state via xDeviceConfigDevice() if needed.

If no data is available, driver behavior varies. Some drivers may return ReturnError, others may return an empty buffer (size=0), and others may block waiting for data. Check specific driver documentation.

Example Usage:

```
#define UART0_UID 0x0100
// Read data from UART Byte_t *rxBuffer = NULL;
Size_t rxSize = 0;
if (OK(xDeviceRead(UART0_UID, &rxSize, (Addr_t **)&rxBuffer))) {
  // Data successfully read if (rxSize > 0) {
    // Process received data processUARTData(rxBuffer, rxSize);
  }
  // IMPORTANT: Free the buffer allocated by driver xMemFree((Addr_t
*)rxBuffer);
}
// Reading from a block device
#define BLOCKDEV_UID 0x1000 Byte_t *sectorData = NULL;
Size_t sectorSize = 0;
```

```
if (OK(xDeviceRead(BLOCKDEV_UID, &sectorSize, (Addr_t **)&sectorData))) {
  // Sector data read (typically 512 or 4096 bytes) if (sectorSize > 0) {
    analyzeSectorData(sectorData, sectorSize);
  }
  // Free the buffer xMemFree((Addr_t *)sectorData);
}
// Reading in a loop (e.g., serial communication) void
serialReaderTask(Task_t *task, TaskParm_t *parm) {
  Byte_t *data = NULL;
  Size_t len = 0;
  if (OK(xDeviceRead(UART0_UID, &len, (Addr_t **)&data))) {
    if (len > 0) {
      handleSerialData(data, len);
    }
    xMemFree((Addr_t *)data);
  }
}
```

Parameters

in	*uid_*	Unique identifier of the target device. Must match a UID previously registered via xDeviceRegisterDevice().
out	*size_*	Pointer to size variable that receives the number of bytes read from the device. Set to 0 if no data available.
out	*data_*	Pointer to buffer pointer variable. Receives address of heap-allocated buffer containing the read data. Caller MUST free this buffer with xMemFree() after use.

Returns

ReturnOK if data was read successfully (size may be 0), ReturnError if the read failed (invalid UID, device not found, device in wrong state/mode, memory allocation failed, or driver read operation failed).

See also

xDeviceWrite() - Write data to a device

xDeviceSimpleRead() - Read a single byte from a device

xDeviceRegisterDevice() - Register a device driver

xDeviceInitDevice() - Initialize a device

xDeviceConfigDevice() - Configure device state or parameters

xMemFree() - Free the buffer returned by this function (REQUIRED!)

4.2.4.5 xDeviceRegisterDevice() `Return_t xDeviceRegisterDevice ( Return_t(*)() device_self_register_ )`

Register a device driver with HeliOS

Registers a device driver with the HeliOS kernel, making it available for I/O operations through the device abstraction layer. Device registration must occur before any device I/O functions (xDeviceRead(), xDeviceWrite(), etc.) can be used with that device.

HeliOS uses a self-registration pattern where each device driver provides its own registration function. This function is passed to xDeviceRegisterDevice(), which calls it to obtain the driver's callback functions, configuration, and metadata.

Device Registration Process:

1. Device driver provides a DRIVERNAME_self_register() function 2. Application calls xDeviceRegisterDevice() with this function pointer 3. HeliOS calls the registration function to obtain driver information 4. Driver is added to the internal device list with its unique ID (UID) 5. Device becomes available for I/O operations

Device Driver Requirements:

- Each driver must have a globally unique identifier (UID)
- Driver must provide callback functions for init, config, read, write
- Driver must specify its name, state, and access mode
- UID must not conflict with any other registered device

Warning

Device UIDs must be unique across all drivers in the application. Registering multiple devices with the same UID will cause undefined behavior. Common practice is to use high byte for driver type and low byte for instance (e.g., 0x0100 for first UART, 0x0200 for first SPI).

Note

Once registered, a device cannot be unregistered. However, devices can be placed in suspended state via xDeviceConfigDevice() to disable them.

Device registration should occur during system initialization, before xTaskStartScheduler() is called, to ensure devices are available when tasks begin executing.

The device driver model is extensible. Driver authors define their own state machines, configuration structures, and operational modes within the framework provided by the callback functions.

Example Usage:

```
// In device driver file (e.g., uart_driver.c) Return_t
UART0_self_register(void) {
  // Register driver with HeliOS return __RegisterDevice__(
    0x0100,              // Unique ID for UART0
    "UART0   ",          // 8-byte name DeviceStateRunning,  // Initial
state DeviceModeReadWrite, // Access mode UART0_init,          // Init
callback UART0_config,        // Config callback UART0_read,          //
Read callback UART0_write,         // Write callback UART0_simple_read,
  // Simple read callback UART0_simple_write   // Simple write callback
  );
}
// In application code (main.c) int main(void) {
  // Register UART driver if
(OK(xDeviceRegisterDevice(UART0_self_register))) {
    // Initialize the device if (OK(xDeviceInitDevice(0x0100))) {
      // Device ready for I/O xDeviceWrite(0x0100, &size, data);
    }
  }
  xTaskStartScheduler();
}
```

Parameters

in	*device_self_↩ register_*	Pointer to the device driver's self-registration function. This function must return ReturnOK on successful registration. By convention, this function is named DRIVERNAME_self_register().

Returns

ReturnOK if the device was successfully registered, ReturnError if registration failed (duplicate UID, invalid driver structure, out of memory, or driver registration function returned error).

See also

xDeviceInitDevice() - Initialize a registered device

xDeviceConfigDevice() - Configure device parameters or state

xDeviceRead() - Read data from a device

xDeviceWrite() - Write data to a device

xDeviceIsAvailable() - Check if a device is registered

CONFIG_DEVICE_NAME_BYTES - Device name length configuration

4.2.4.6 xDeviceSimpleRead()

```
Return_t xDeviceSimpleRead (
    const HalfWord_t uid_,
    Byte_t * data_ )
```

Reads a single byte of data from the specified device through its registered driver. This is a simplified read operation for single-byte transfers, ideal for character input, polling device status, or reading simple sensor values where the overhead of buffer management is unnecessary.

Unlike xDeviceRead() which allocates heap memory and returns variable-length data, xDeviceSimpleRead() reads exactly one byte directly into the provided variable. This makes it more efficient and simpler to use when you only need to read one byte at a time.

Common use cases:

- **Character input**: Reading individual characters from a UART or keyboard
- **Status polling**: Reading device status registers or flags
- **Simple sensors**: Reading 8-bit sensor values (temperature, light level)
- **Protocol parsing**: Reading header bytes, delimiters, or checksums
- **Command acknowledgment**: Reading single-byte responses from peripherals

Operational behavior:

- The byte is retrieved directly from the device driver's read callback
- No heap allocation or memory management is required
- Operation is synchronous (returns after driver completes the read)
- Device must be in readable state (DeviceModeReadOnly or DeviceModeReadWrite)
- Device must be running (DeviceStateRunning)
- If no data is available, behavior depends on driver implementation (may return error or block)

Example 1: Read character from UART

```
#define UART0_UID 0x0100
char getChar(void) {
  Byte_t data;
  if (OK(xDeviceSimpleRead(UART0_UID, &data))) {
    return (char)data;
  } else {
    return '\0';  // No data available or error
  }
}
// Read a line of text character-by-character Return_t readLine(char
buffer, Size_t maxLen) {
  Size_t idx = 0;
  Byte_t c;
  while (idx < maxLen - 1) {
    if (OK(xDeviceSimpleRead(UART0_UID, &c))) {
      if (c == '\n' || c == '\r') {
        break;
      }
      buffer[idx++] = (char)c;
    }
  }
  buffer[idx] = '\0';
  return ReturnOK;
}
```

Example 2: Poll device status register

```
#define SENSOR_UID 0x0300
#define STATUS_READY  0x01
#define STATUS_ERROR  0x80
Base_t isSensorReady(void) {
  Byte_t status;
  if (OK(xDeviceSimpleRead(SENSOR_UID, &status))) {
    return (status & STATUS_READY) != 0;
  }
  return 0;
}
Base_t checkSensorError(void) {
  Byte_t status;
  if (OK(xDeviceSimpleRead(SENSOR_UID, &status))) {
    return (status & STATUS_ERROR) != 0;
  }
  return 1;  // Assume error if can't read
}
```

Example 3: Read simple sensor value

```
#define TEMP_SENSOR_UID 0x0400
// Read temperature sensor (returns 0-255 representing 0-100°C) Base_t
readTemperature(Byte_t *tempOut) {
  if (OK(xDeviceSimpleRead(TEMP_SENSOR_UID, tempOut))) {
    // Convert to actual temperature (0-255 maps to 0-100°C)
    // Caller can do: actualTemp = (*tempOut * 100) / 255 return 1;  //
Success
  }
  return 0;  // Failed to read
}
```

Parameters

in	*uid*↩ _	Unique identifier of the target device. Must match a UID previously registered via xDeviceRegisterDevice().
out	*data*↩ _	Pointer to byte variable to receive the read data. On success, this variable is updated with the byte value read from the device.

Returns

ReturnOK if byte was read successfully, ReturnError if the read failed (invalid UID, device not found, device in wrong state/mode, no data available, or driver read operation failed).

Warning

The device must be in a readable state (DeviceModeReadOnly or DeviceModeReadWrite) for the read to succeed. Use xDeviceConfigDevice() to check or change device mode if reads are failing.

The device must be in running state (DeviceStateRunning). Devices in stopped or error states will reject read operations.

If no data is available to read, driver behavior varies. Some drivers return ReturnError immediately, while others may block waiting for data. Check your device driver documentation for specific behavior.

Note

Unlike xDeviceRead(), this function does not allocate heap memory, making it more efficient for single-byte operations.

For reading multiple bytes, consider using xDeviceRead() instead as it's more efficient for bulk transfers and reduces driver overhead.

Read behavior depends on the device driver implementation. Some drivers read from hardware registers directly, while others may maintain receive buffers.

See also

xDeviceRead() - Read multiple bytes from a device (allocates heap buffer)

xDeviceSimpleWrite() - Write a single byte to a device

xDeviceRegisterDevice() - Register a device driver

xDeviceInitDevice() - Initialize a device

xDeviceConfigDevice() - Configure device state or mode

4.2.4.7 xDeviceSimpleWrite()

```
Return_t xDeviceSimpleWrite (
    const HalfWord_t uid_,
    Byte_t data_ )
```

Writes a single byte of data to the specified device through its registered driver. This is a simplified write operation for single-byte transfers, ideal for character output, simple control commands, or low-throughput data transmission where heap allocation overhead is unnecessary.

Unlike xDeviceWrite() which requires heap-allocated buffers for bulk data, xDeviceSimpleWrite() accepts a single byte value directly. This makes it more efficient for single-character operations and simpler to use when you only need to send one byte at a time.

Common use cases:

- **Character output**: Sending individual characters to a UART or terminal
- **Control commands**: Sending single-byte commands to peripherals
- **Status updates**: Writing status bytes to LED controllers or displays
- **Protocol framing**: Sending header bytes, delimiters, or checksums
- **Low-rate data**: Simple sensors or actuators requiring occasional updates

Operational behavior:

- The byte is passed directly to the device driver's write callback
- No heap allocation or memory validation is required
- Operation is synchronous (returns after driver completes the write)
- Device must be in writable state (DeviceModeWriteOnly or DeviceModeReadWrite)
- Device must be running (DeviceStateRunning)

Example 1: Send character to UART

```
#define UART0_UID 0x0100
void sendChar(char c) {
  if (OK(xDeviceSimpleWrite(UART0_UID, (Byte_t)c))) {
    // Character sent successfully
  } else {
    // Write failed - check device state
  }
}
// Send a string character-by-character void sendString(const char *str) {
  while (*str) {
    xDeviceSimpleWrite(UART0_UID, (Byte_t)*str);
    str++;
  }
}
```

Example 2: Control LED via byte commands

```
#define LED_CTRL_UID 0x0200
#define LED_ON  0x01
#define LED_OFF 0x00
void setLED(Base_t state) {
  Byte_t command = state ? LED_ON : LED_OFF;
  if (ERROR(xDeviceSimpleWrite(LED_CTRL_UID, command))) {
    logError("Failed to control LED");
  }
}
```

Example 3: Send protocol framing bytes

```
#define START_BYTE 0xAA
#define END_BYTE   0x55
Return_t sendFrame(HalfWord_t deviceUID, Byte_t *payload, Size_t len) {
  // Send start byte if (ERROR(xDeviceSimpleWrite(deviceUID, START_BYTE)))
{
    return ReturnError;
  }
  // Send payload (using bulk write) if (ERROR(xDeviceWrite(deviceUID,
&len, (Addr_t *)payload))) {
    return ReturnError;
  }
  // Send end byte if (ERROR(xDeviceSimpleWrite(deviceUID, END_BYTE))) {
    return ReturnError;
  }
  return ReturnOK;
}
```

Parameters

in	*uid*↩ _	Unique identifier of the target device. Must match a UID previously registered via xDeviceRegisterDevice().
in	*data*↩ _	Single byte value to write to the device. Accepts any value from 0x00 to 0xFF.

Returns

ReturnOK if byte was written successfully, ReturnError if the write failed (invalid UID, device not found, device in wrong state/mode, or driver write operation failed).

Warning

The device must be in a writable state (DeviceModeWriteOnly or DeviceModeReadWrite) for the write to succeed. Use xDeviceConfigDevice() to check or change device mode if writes are failing.

The device must be in running state (DeviceStateRunning). Devices in stopped or error states will reject write operations.

Note

Unlike xDeviceWrite(), this function does not require heap-allocated memory, making it more efficient for single-byte operations.

For writing multiple bytes, consider using xDeviceWrite() instead as it's more efficient for bulk transfers and reduces driver overhead.

Write behavior depends on the device driver implementation. Some drivers may buffer writes, while others perform immediate hardware I/O.

See also

xDeviceWrite() - Write multiple bytes to a device (requires heap buffer)

xDeviceSimpleRead() - Read a single byte from a device

xDeviceRegisterDevice() - Register a device driver

xDeviceInitDevice() - Initialize a device

xDeviceConfigDevice() - Configure device state or mode

4.2.4.8 xDeviceWrite()

```
Return_t xDeviceWrite (
    const HalfWord_t uid_,
    Size_t * size_,
    Addr_t * data_ )
```

Writes a buffer of data to the specified device through its registered driver. The data is transferred from user heap memory to the device via the driver's write callback function. This provides an abstract interface for device output, allowing applications to write to any registered device using a consistent API.

The write operation behavior depends on the device driver implementation:

- Serial devices typically write bytes to a transmit buffer or UART
- Storage devices write to specific sectors or blocks
- Network devices transmit packets
- Custom devices implement application-specific write semantics

Data Flow:

1. Application prepares data in heap-allocated buffer 2. xDeviceWrite() validates device UID, state, and permissions 3. Data is copied from user heap to kernel memory 4. Driver's write callback is invoked with kernel memory 5. Driver performs hardware-specific write operations 6. Driver returns number of bytes written via size_ parameter

Warning

The data buffer MUST be allocated from the user heap using xMemAlloc(). Stack-allocated buffers or static data will cause memory validation errors. HeliOS enforces this to maintain memory safety boundaries between user space and kernel space.

Note

Device write operations are synchronous by default. The function does not return until the driver completes the write operation. Drivers may implement buffering or asynchronous operations internally.

The size_ parameter is both input and output. On input, it specifies the number of bytes to write. On output (if driver supports it), it may reflect the actual number of bytes written, which can be less than requested for devices with limited buffers.

The device must be in a writable state (DeviceModeWriteOnly or DeviceModeReadWrite) and running state (DeviceStateRunning) for the write to succeed. Check device state via xDeviceConfigDevice() if needed.

Example Usage:

```
#define UART0_UID 0x0100
// Write string to UART const char *message = "Hello, World!\n";
Size_t messageLen = strlen(message);
Byte_t *buffer = NULL;
// Allocate buffer from heap (required!) if (OK(xMemAlloc((volatile Addr_t
*
*)&buffer, messageLen))) {
  // Copy data to heap buffer memcpy(buffer, message, messageLen);
  // Write to device if (OK(xDeviceWrite(UART0_UID, &messageLen, (Addr_t
*)buffer))) {
    // Data written successfully
  }
  // Free the buffer xMemFree((Addr_t *)buffer);
}
// Writing to a block device Byte_t *sectorData = NULL;
Size_t sectorSize = 512;
if (OK(xMemAlloc((volatile Addr_t **)&sectorData, sectorSize))) {
  // Fill sector data...
  prepareSectorData(sectorData, sectorSize);
  // Write to block device if (OK(xDeviceWrite(0x1000, &sectorSize, (Addr_t
*)sectorData))) {
    // Sector written
  }
  xMemFree((Addr_t *)sectorData);
}
```

Parameters

in	*uid_*	Unique identifier of the target device. Must match a UID previously registered via xDeviceRegisterDevice().
in,out	*size_*	Pointer to size variable. On input: number of bytes to write. On output: may be updated by driver to reflect actual bytes written (driver-dependent).
in	*data_*	Pointer to data buffer allocated via xMemAlloc(). Contains the data to write to the device.

Returns

ReturnOK if data was written successfully, ReturnError if the write failed (invalid UID, device not found, device in wrong state/mode, data not from heap, or driver write operation failed).

See also

xDeviceRead() - Read data from a device

xDeviceSimpleWrite() - Write a single byte to a device

xDeviceRegisterDevice() - Register a device driver

xDeviceInitDevice() - Initialize a device

xDeviceConfigDevice() - Configure device state or parameters

xMemAlloc() - Allocate heap memory for write buffer

xMemFree() - Free the write buffer after use

4.2.4.9 xDirClose() `Return_t xDirClose ( Dir_t * dir_ )`

Closes a previously opened directory, freeing the directory handle and associated kernel resources. After closing, the directory handle becomes invalid and must not be used in any subsequent directory operations.

Always close directories when finished reading to free kernel resources and prevent resource leaks. Directory handles are limited, and failing to close them may prevent other directories from being opened.

Example 1: Basic directory listing with close

```
 Dir_t *dir;
DirEntry_t *entry;
if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
  while (OK(xDirRead(dir, &entry))) {
    printf("%s\n", entry.name);
  }
  // Always close when done xDirClose(dir);
}
```

Example 2: Error handling with guaranteed close

```
 Return_t processDirectory(Volume_t *vol, const char *path) {
  Dir_t *dir;
  DirEntry_t *entry;
  Return_t result = ReturnError;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)path))) {
    while (OK(xDirRead(dir, &entry))) {
      if (processEntry(&entry)) {
        result = ReturnOK;
      }
    }
    // Close regardless of processing success/failure xDirClose(dir);
  }
  return result;
}
```

Parameters

in	*dir_*	Handle to the open directory to close. Must be a valid directory handle from xDirOpen(). After closing, becomes invalid.

Returns

ReturnOK if close succeeded, ReturnError if close failed due to invalid directory handle or directory not open.

Warning

After calling xDirClose(), the directory handle becomes invalid and must not be used in any directory operations.

Note

It is good practice to set the directory handle to null after closing to prevent accidental use of an invalid handle.

See also

xDirOpen() - Open directory for reading

xDirRead() - Read directory entries

4.2.4.10 xDirMake()

```
Return_t xDirMake (
        Volume_t * volume_,
        const Byte_t * path_ )
```

Creates a new directory at the specified path. The parent directory must already exist. Use this function to organize files into a directory structure.

Example: Create directory and add file

```
 if (OK(xDirMake(vol, (Byte_t*)"/logs"))) {
  File_t *file;
  if (OK(xFileOpen(&file, vol, (Byte_t*)"/logs/system.log", FS_MODE_WRITE |
FS_MODE_CREATE))) {
    xFileWrite(file, 10, (Byte_t*)"Log start\n");
    xFileClose(file);
  }
}
```

Parameters

in	*volume_*	Handle to mounted volume.
in	*path_*	Path to new directory.

Returns

ReturnOK if directory created, ReturnError if failed (already exists, parent doesn't exist, or insufficient space).

Warning

Parent directory must exist. Create parent directories first if needed.

See also

xDirRemove() - Delete empty directory

xDirOpen() - Open directory

4.2.4.11 xDirOpen()

```
Return_t xDirOpen (
        Dir_t ** dir_,
        Volume_t * volume_,
        const Byte_t * path_ )
```

Opens a directory for reading its contents, creating a directory handle used to iterate through directory entries with xDirRead(). This is the first step in browsing directory contents, listing files, or searching for specific entries.

Opening a directory allocates kernel resources and positions the read pointer at the first entry. Use xDirRead() to retrieve entries sequentially, xDirRewind() to return to the beginning, and xDirClose() to free resources when finished.

Directory reading workflow:

1. Open directory with xDirOpen() 2. Read entries with xDirRead() in a loop
2. Optionally rewind with xDirRewind() to re-read 4. Close with xDirClose() when finished

Common use cases:

- **File listing**: Display all files in a directory
- **File searching**: Find specific files by name or attributes
- **Directory scanning**: Process all files in directory
- **File counting**: Count files or calculate total size
- **Filtering**: Select files matching criteria

Example 1: List all files in directory

```
 Volume_t *vol;
Dir_t *dir;
DirEntry_t *entry;
if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
  printf("Files in root directory:\n");
  while (OK(xDirRead(dir, &entry))) {
    if (entry.isDirectory) {
      printf("  [DIR]  %s\n", entry.name);
    } else {
      printf("  [FILE] %s (%lu bytes)\n", entry.name, entry.size);
    }
  }
  xDirClose(dir);
}
```

Example 2: Find specific file in directory

```
 Base_t findFileInDirectory(Volume_t *vol, const char *dirPath, const
char *filename) {
  Dir_t *dir;
  DirEntry_t *entry;
  Base_t found = 0;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)dirPath))) {
    while (OK(xDirRead(dir, &entry))) {
      if (strcmp((char*)entry.name, filename) == 0) {
        found = 1;
        break;
      }
    }
    xDirClose(dir);
  }
  return found;
}
```

Example 3: Calculate directory size

```
 Word_t calculateDirectorySize(Volume_t *vol, const char *path) {
  Dir_t *dir;
  DirEntry_t *entry;
  Word_t totalSize = 0;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)path))) {
    while (OK(xDirRead(dir, &entry))) {
      if (!entry.isDirectory) {
        totalSize += entry.size;
      }
    }
    xDirClose(dir);
  }
  return totalSize;
}
```

Example 4: Count files by type

```
 void countFileTypes(Volume_t *vol) {
  Dir_t *dir;
  DirEntry_t *entry;
  Word_t fileCount = 0, dirCount = 0;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
    while (OK(xDirRead(dir, &entry))) {
      if (entry.isDirectory) {
        dirCount++;
      } else {
        fileCount++;
      }
    }
    xDirClose(dir);
    printf("Files: %lu, Directories: %lu\n", fileCount, dirCount);
  }
}
```

Parameters

out	*dir_*	Pointer to Dir_t *handle to be initialized. On success, this handle is used for reading directory entries and must be closed with xDirClose().
in	*volume_*	Handle to mounted volume containing the directory. Must be a valid volume from xFSMount().
in	*path_*	Pointer to null-terminated string containing directory path. Use "/" for root directory, or "/dirname" for subdirectories.

Returns

ReturnOK if directory opened successfully, ReturnError if open failed due to invalid volume, directory not found, path refers to a file, or insufficient resources.

Warning

The directory handle must be closed with xDirClose() when finished. Failure to close directories causes resource leaks.

The path must refer to a directory, not a file. Opening a file path as a directory will fail.

Note

After opening, the read position is at the first entry. Use xDirRead() to retrieve entries sequentially.

Directory handles are allocated from kernel memory and are a limited resource. Always close directories promptly after use.

See also

xDirClose() - Close directory and free resources

xDirRead() - Read next directory entry

xDirRewind() - Reset to beginning of directory

xFileGetInfo() - Get info about specific path

4.2.4.12 xDirRead()

```
Return_t xDirRead (
        Dir_t * dir_,
        DirEntry_t ** entry_ )
```

Retrieves the next entry from an open directory, advancing the read position. Entries are returned sequentially including both files and subdirectories. Use the isDirectory field in the returned entry to distinguish between them.

Reading continues until all entries have been retrieved. When no more entries are available, xDirRead() returns ReturnError, indicating the end of the directory. Use xDirRewind() to return to the beginning and re-read entries if needed.

Example 1: Process all files (skip directories)

```
 Dir_t *dir;
DirEntry_t *entry;
if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
  while (OK(xDirRead(dir, &entry))) {
    // Process only files, not directories if (!entry.isDirectory) {
      processFile(vol, (char*)entry.name);
```

```
    }
  }
  xDirClose(dir);
}
```

Example 2: Filter files by extension

```
Base_t hasExtension(const char *filename, const char *ext) {
  const char *dot = strrchr(filename, '.');
  return (dot && strcmp(dot, ext) == 0);
}
void processLogFiles(Volume_t *vol) {
  Dir_t *dir;
  DirEntry_t *entry;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
    while (OK(xDirRead(dir, &entry))) {
      if (!entry.isDirectory && hasExtension((char*)entry.name, ".log")) {
        printf("Log file: %s (%lu bytes)\n", entry.name, entry.size);
      }
    }
    xDirClose(dir);
  }
}
```

Parameters

in	*dir_*	Handle to open directory from xDirOpen().
out	*entry_*	Pointer to DirEntry_t *structure to receive entry information.

Returns

ReturnOK if entry read successfully, ReturnError if no more entries (end of directory) or invalid directory handle.

Note

When xDirRead() returns ReturnError, it indicates the end of the directory, not necessarily a read error.

Use xDirRewind() to return to the beginning and re-read entries.

See also

xDirOpen() - Open directory for reading

xDirClose() - Close directory

xDirRewind() - Reset to beginning

4.2.4.13 xDirRemove()

```
Return_t xDirRemove (
        Volume_t * volume_,
        const Byte_t * path_ )
```

Removes an empty directory from the filesystem. The directory must be empty (contain no files or subdirectories) for the operation to succeed.

Example: Remove temporary directory

```
// Remove all files first Dir_t *dir;
DirEntry_t *entry;
if (OK(xDirOpen(&dir, vol, (Byte_t*)"/temp"))) {
  while (OK(xDirRead(dir, &entry))) {
    if (!entry.isDirectory) {
      Byte_t path[256];
      snprintf((char*)path, sizeof(path), "/temp/%s", entry.name);
      xFileUnlink(vol, path);
    }
  }
  xDirClose(dir);
}
// Now remove empty directory xDirRemove(vol, (Byte_t*)"/temp");
```

Parameters

in	*volume_*	Handle to mounted volume.
in	*path_*	Path to directory to remove.

Returns

ReturnOK if directory removed, ReturnError if failed (not empty, doesn't exist, or is root directory).

Warning

Directory must be empty. Remove all files and subdirectories first.

Cannot remove the root directory.

See also

xDirMake() - Create directory

xFileUnlink() - Delete file

4.2.4.14 xDirRewind() `Return_t xDirRewind ( Dir_t * dir_ )`

Resets the read position of an open directory back to the first entry, allowing the directory to be re-read from the start. This is useful when you need to make multiple passes through a directory without closing and reopening it.

Example: Count and then process files

```
 Dir_t *dir;
DirEntry_t *entry;
Word_t fileCount = 0;
if (OK(xDirOpen(&dir, vol, (Byte_t*)"/"))) {
  // First pass - count files while (OK(xDirRead(dir, &entry))) {
    if (!entry.isDirectory) {
      fileCount++;
    }
  }
  printf("Processing %lu files...\n", fileCount);
  // Rewind to beginning xDirRewind(dir);
  // Second pass - process files while (OK(xDirRead(dir, &entry))) {
    if (!entry.isDirectory) {
      processFile((char*)entry.name);
    }
  }
  xDirClose(dir);
}
```

Parameters

in	*dir_*	Handle to open directory from xDirOpen().

Returns

ReturnOK if rewind succeeded, ReturnError if invalid directory handle.

See also

xDirOpen() - Open directory

xDirRead() - Read entries

xDirClose() - Close directory

4.2.4.15 xFileClose() `Return_t xFileClose ( File_t * file_ )`

Closes a previously opened file, flushing any pending writes to storage and freeing the file handle and associated kernel resources. After closing, the file handle becomes invalid and must not be used in any subsequent file operations.

Closing a file ensures data integrity by committing all buffered writes to the filesystem and updating file metadata (size, modification time, etc.). Always close files when finished to prevent data loss and resource leaks.

Close process:

1. Flushes any buffered write data to block device 2. Updates file directory entry (size, attributes) 3. Updates File Allocation Table if file size changed 4. Frees file handle from kernel memory 5. Invalidates file handle

Common scenarios:

- **Normal completion**: Close file after successful read/write operations
- **Error handling**: Close file even if operations failed
- **Resource management**: Close files promptly to free kernel resources
- **Shutdown**: Close all open files before unmounting volume

Example 1: Basic open/close pattern

```
 File_t *file;
if (OK(xFileOpen(&file, vol, (Byte_t*)"/data.txt", FS_MODE_READ))) {
  // Perform file operations Byte_t *data;
  if (OK(xFileRead(file, 100, &data))) {
    processData(data, 100);
    xMemFree((Addr_t *)data);
  }
  // Always close when done xFileClose(file);
  // file is now invalid - do not use
}
```

Example 2: Error handling with guaranteed close

```
 Return_t writeData(Volume_t *vol, const char *path, Byte_t *data,
Size_t size) {
  File_t *file;
  Return_t result = ReturnError;
  if (OK(xFileOpen(&file, vol, (Byte_t*)path, FS_MODE_WRITE |
FS_MODE_CREATE))) {
    if (OK(xFileWrite(file, size, data))) {
      result = ReturnOK;
    }
    // Close file regardless of write success/failure xFileClose(file);
  }
  return result;
}
```

Example 3: Multiple file handling

```
 void processFiles(Volume_t *vol) {
  File_t *input, output;
  // Open input file if (OK(xFileOpen(&input, vol, (Byte_t*)"/input.dat",
FS_MODE_READ))) {
```

```
    // Open output file if (OK(xFileOpen(&output, vol,
(Byte_t*)"/output.dat", FS_MODE_WRITE | FS_MODE_CREATE))) {
      // Process data from input to output Byte_t *data;
      Word_t size;
      if (OK(xFileGetSize(input, &size)) && OK(xFileRead(input, size,
&data))) {
        xFileWrite(output, size, data);
        xMemFree((Addr_t *)data);
      }
      xFileClose(output);  // Close output first
    }
    xFileClose(input);  // Close input
  }
}
```

Example 4: Safe file handle cleanup

```
 File_t *file = null;
void useFile(Volume_t *vol) {
  if (OK(xFileOpen(&file, vol, (Byte_t*)"/temp.dat", FS_MODE_WRITE))) {
    performOperations(file);
    // Close and nullify handle xFileClose(file);
    file = null;  // Prevent use-after-close
  }
}
```

Parameters

in	*file*↩_	Handle to the open file to close. Must be a valid file handle obtained from a previous successful xFileOpen() call. After closing, this handle becomes invalid.

Returns

ReturnOK if close succeeded, ReturnError if close failed due to invalid file handle, file not open, or flush/write errors during close.

Warning

After calling xFileClose(), the file handle becomes invalid and must not be used in any file operations. Attempting to use a closed file will result in ReturnError.

If xFileClose() fails (returns ReturnError), data may not have been fully written to storage. Consider using xFileSync() before close for critical data to detect write failures earlier.

All files must be closed before unmounting the volume. Unmounting with open files may cause data loss or filesystem corruption.

Note

It is good practice to set the file handle to null after closing to prevent accidental use of an invalid handle (use-after-close bugs).

Closing a file that was opened for writing commits the final file size to the directory entry. File size is updated only on close, not during write.

File handles are a limited resource. Always close files promptly after use to make handles available for other operations.

See also

xFileOpen() - Open a file

xFileSync() - Flush writes before close

xFSUnmount() - Unmount volume (close all files first)

4.2.4.16 xFileEOF()

```
Return_t xFileEOF (
        const File_t * file_,
        Base_t * eof_ )
```

Determines whether the current file position is at or beyond the end of the file, indicating that all data has been read and no more data is available. This is essential for implementing read loops that process entire files without knowing the file size in advance.

End-of-file (EOF) occurs when the file position equals or exceeds the file size. After reading the last byte of a file, subsequent reads will not advance the position further, and xFileEOF() will return true. EOF is a normal condition, not an error.

EOF behavior:

- True when file position >= file size
- False when more data available to read
- Not an error condition - indicates normal end of data
- Can occur mid-read if reading past end
- Unaffected by write operations that extend file

Common use cases:

- **Read loops**: Continue reading until EOF reached
- **Data validation**: Verify all expected data was read
- **Sequential processing**: Process file completely
- **Stream detection**: Know when to request more data

Example 1: Read file in chunks until EOF

```
 File_t *file;
#define CHUNK_SIZE 512
void processEntireFile(File_t *file) {
  Base_t eof = 0;
  Byte_t *chunk;
  Word_t totalBytesRead = 0;
  // Read until EOF while (!eof) {
    if (OK(xFileRead(file, CHUNK_SIZE, &chunk))) {
      processChunk(chunk, CHUNK_SIZE);
      totalBytesRead += CHUNK_SIZE;
      xMemFree((Addr_t *)chunk);
      // Check for EOF xFileEOF(file, &eof);
    } else {
      break;  // Read error
    }
  }
  logInfo("Processed %lu bytes", totalBytesRead);
}
```

Example 2: Validate file completely read

```
 Return_t readAndValidate(File_t *file, Word_t expectedSize) {
  Byte_t *data;
  Base_t eof;
  // Read expected amount if (OK(xFileRead(file, expectedSize, &data))) {
    processData(data, expectedSize);
    xMemFree((Addr_t *)data);
    // Verify we're at EOF (no extra data) if (OK(xFileEOF(file, &eof))) {
      if (eof) {
        return ReturnOK;  // File size matches expected
      } else {
        logWarning("File has unexpected extra data");
        return ReturnError;
      }
    }
  }
```

```
  return ReturnError;
}
```

Example 3: Copy file with EOF detection

```
Return_t copyFile(File_t *source, File_t *dest) {
  Base_t eof = 0;
  Byte_t *buffer;
  while (!eof) {
    // Read chunk if (OK(xFileRead(source, 1024, &buffer))) {
      // Write to destination if (ERROR(xFileWrite(dest, 1024, buffer))) {
        xMemFree((Addr_t *)buffer);
        return ReturnError;
      }
      xMemFree((Addr_t *)buffer);
      // Check if source exhausted xFileEOF(source, &eof);
    } else {
      return ReturnError;
    }
  }
  return ReturnOK;  // Complete file copied
}
```

Example 4: Skip to end check

```
Base_t atEndOfFile(File_t *file) {
  Base_t eof;
  if (OK(xFileEOF(file, &eof))) {
    return eof;
  }
  return 0;
}
// Usage in parsing void parseRecords(File_t *file) {
  while (!atEndOfFile(file)) {
    Record_t record;
    if (readRecord(file, &record)) {
      processRecord(&record);
    }
  }
}
```

Parameters

in	*file*↩ _	Handle to the open file. Must be a valid file from xFileOpen().
out	*eof*↩ _	Pointer to variable receiving EOF status. Set to non-zero (true) if at end-of-file, zero (false) if more data available.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid file handle or file not open.

Note

EOF is determined by comparing file position to file size. Position equals size when all data has been read.

For files opened with FS_MODE_APPEND, EOF is initially true since position starts at end of file.

After reaching EOF, you can seek back to earlier positions with xFileSeek() to reread data, which will clear the EOF condition.

Write operations that extend the file beyond the current position will clear the EOF condition.

See also

xFileRead() - Read data (may reach EOF)

xFileGetSize() - Get file size to calculate EOF

xFileTell() - Get current position

xFileSeek() - Move position (may clear EOF)

4.2.4.17 xFileExists()

```
Return_t xFileExists (
        Volume_t * volume_,
        const Byte_t * path_,
        Base_t * exists_ )
```

Determines whether a file exists at the specified path on the mounted volume. This is useful for conditional file operations, avoiding errors when opening files, or checking for the presence of configuration or data files before attempting to access them.

The function searches the filesystem directory structure for an entry matching the specified path. It returns true if a file (not a directory) exists at that path, or false if the path does not exist or refers to a directory rather than a file.

Common use cases:

- **Conditional file access**: Open file only if it exists
- **Configuration detection**: Check for config files before loading
- **Backup verification**: Verify backup files exist before restore
- **File creation logic**: Create new file only if it doesn't exist
- **Validation**: Ensure required files are present

Example 1: Open file only if it exists

```
 Volume_t *vol;
Base_t exists;
if (OK(xFileExists(vol, (Byte_t*)"/config.txt", &exists)) && exists) {
  // File exists - safe to open File_t *file;
  if (OK(xFileOpen(&file, vol, (Byte_t*)"/config.txt", FS_MODE_READ))) {
    // Process file xFileClose(file);
  }
} else {
  // File doesn't exist - create default createDefaultConfig(vol);
}
```

Example 2: Check multiple configuration sources

```
 Return_t loadConfig(Volume_t *vol, Config_t *config) {
  Base_t exists;
  const char *configPaths[] = {
    "/config.user.txt",
    "/config.default.txt",
    "/config.txt"
  };
  // Try each config file in priority order for (int i = 0; i < 3; i++) {
    if (OK(xFileExists(vol, (Byte_t*)configPaths[i], &exists)) && exists) {
      return loadConfigFromFile(vol, configPaths[i], config);
    }
  }
  // No config file found - use defaults return loadDefaultConfig(config);
}
```

Example 3: Conditional file creation

```
 Return_t ensureLogFile(Volume_t *vol) {
  Base_t exists;
  // Check if log file exists if (OK(xFileExists(vol,
(Byte_t*)"/system.log", &exists))) {
    if (!exists) {
      // Create new log file with header File_t *logFile;
      if (OK(xFileOpen(&logFile, vol, (Byte_t*)"/system.log", FS_MODE_WRITE
| FS_MODE_CREATE))) {
        const char *header = "=== System Log ===\n";
        xFileWrite(logFile, strlen(header), (Byte_t*)header);
        xFileClose(logFile);
        return ReturnOK;
      }
    } else {
      // Log file already exists return ReturnOK;
    }
  }
  return ReturnError;
}
```

Example 4: Validate required files

```
Return_t validateRequiredFiles(Volume_t *vol) {
  const char *requiredFiles[] = {
    "/firmware.bin",
    "/config.dat",
    "/calibration.dat"
  };
  Base_t exists;
  for (int i = 0; i < 3; i++) {
    if (ERROR(xFileExists(vol, (Byte_t*)requiredFiles[i], &exists)) ||
!exists) {
      logError("Required file missing: %s", requiredFiles[i]);
      return ReturnError;
    }
  }
  // All required files present return ReturnOK;
}
```

Parameters

in	*volume_*	Handle to the mounted volume to search. Must be a valid volume from xFSMount().
in	*path_*	Pointer to null-terminated string containing file path. Use forward slashes (/) for directory separators.
out	*exists_*	Pointer to variable receiving existence status. Set to non-zero (true) if file exists, zero (false) if not found or path refers to a directory.

Returns

ReturnOK if query succeeded (regardless of whether file exists), ReturnError if query failed due to invalid volume handle or path.

Note

This function returns true only for files, not directories. Use directory operations or xFileGetInfo() to check for directory existence.

File paths are case-sensitive. "/File.txt" and "/file.txt" are different.

Checking for existence and then opening the file is not atomic. In multitasking systems, the file could be deleted between the check and open operations.

See also

xFileOpen() - Open file (fails if doesn't exist without CREATE flag)

xFileGetInfo() - Get detailed file information including type

xDirOpen() - Open directory

4.2.4.18 xFileGetInfo()

```
Return_t xFileGetInfo (
    Volume_t * volume_,
    const Byte_t * path_,
    DirEntry_t ** entry_ )
```

Retrieves comprehensive information about a file including its name, size, attributes (read-only, hidden, system), and whether it's a file or directory. This information is useful for file browsing, validation, or determining how to process a filesystem entry.

The returned DirEntry structure contains all metadata about the file as stored in the directory entry, without needing to open the file. This is more efficient than opening the file when you only need to inspect its properties.

Information provided in DirEntry_t *:

- **name**: File or directory name (up to 256 characters)
- **size**: File size in bytes (0 for directories)
- **firstCluster**: Starting cluster number on disk
- **isDirectory**: True if entry is a directory, false if file
- **isReadOnly**: True if file is marked read-only
- **isHidden**: True if file is marked hidden
- **isSystem**: True if file is marked as system file

Common use cases:

- **File browser**: Display file details in directory listings
- **Validation**: Check file attributes before processing
- **Filtering**: Select files based on attributes or size
- **Metadata inspection**: Examine file properties without opening

Example 1: Get file size without opening

```
 Volume_t *vol;
DirEntry_t *entry;
if (OK(xFileGetInfo(vol, (Byte_t*)"/data.bin", &entry))) {
  if (!entry.isDirectory) {
    logInfo("File size: %lu bytes", entry.size);
    // Check if file is too large for processing if (entry.size >
MAX_FILE_SIZE) {
      logError("File too large to process");
    }
  }
}
```

Example 2: Check file type before opening

```
 Return_t processPath(Volume_t *vol, const char *path) {
  DirEntry_t *entry;
  if (OK(xFileGetInfo(vol, (Byte_t*)path, &entry))) {
    if (entry.isDirectory) {
      // It's a directory - process directory return processDirectory(vol,
path);
    } else {
      // It's a file - process file return processFile(vol, path);
    }
  }
  return ReturnError;
}
```

Example 3: Display file attributes

```
 void displayFileInfo(Volume_t *vol, const char *path) {
  DirEntry_t *entry;
  if (OK(xFileGetInfo(vol, (Byte_t*)path, &entry))) {
    printf("Name: %s\n", entry.name);
    printf("Size: %lu bytes\n", entry.size);
    printf("Type: %s\n", entry.isDirectory ? "Directory" : "File");
    // Display attributes printf("Attributes:");
    if (entry.isReadOnly) printf(" [READ-ONLY]");
    if (entry.isHidden) printf(" [HIDDEN]");
    if (entry.isSystem) printf(" [SYSTEM]");
    printf("\n");
  }
}
```

Example 4: Filter files by size and attributes

```
 Return_t findLargeFiles(Volume_t *vol, const char *dirPath, Word_t
minSize) {
  Dir_t *dir;
  DirEntry_t *entry;
  if (OK(xDirOpen(&dir, vol, (Byte_t*)dirPath))) {
    while (OK(xDirRead(dir, &entry))) {
      // Skip directories and hidden files if (!entry.isDirectory &&
!entry.isHidden) {
```

```
        if (entry.size >= minSize) {
          printf("Large file: %s (%lu bytes)\n", entry.name, entry.size);
        }
      }
    }
    xDirClose(dir);
    return ReturnOK;
  }
  return ReturnError;
}
```

Parameters

in	*volume_*	Handle to the mounted volume containing the file. Must be a valid volume from xFSMount().
in	*path_*	Pointer to null-terminated string containing path to file or directory. Use forward slashes (/) for separators.
out	*entry_*	Pointer to DirEntry_t *structure to receive file information. Structure is populated with all available metadata.

Returns

ReturnOK if information retrieved successfully, ReturnError if operation failed due to invalid volume, file/directory not found, or path error.

Note

This function works for both files and directories. Check the isDirectory field in the returned entry to determine which type it is.

The entry_ structure is populated with a copy of the directory entry data. Changes to this structure do not affect the file on disk.

The size field is 0 for directories, as directories don't have data content in the same way files do.

See also

xFileExists() - Check if file exists (simpler boolean check)

xDirRead() - Read directory entries sequentially

xFileOpen() - Open file to access content

xFileGetSize() - Get size of open file

4.2.4.19 xFileGetSize()

```
Return_t xFileGetSize (
    const File_t * file_,
    Word_t * size_ )
```

Retrieves the total size of an open file in bytes. This is the number of bytes of actual data content in the file, not including any filesystem metadata or directory entries. File size is useful for validating file integrity, allocating read buffers, calculating progress, or checking available data.

The file size represents the current data content and may change as write operations extend the file. Size is updated dynamically during write operations but is only committed to the directory entry when the file is closed.

Size characteristics:

- Measured in bytes of actual file content
- Does not include filesystem metadata or overhead
- Updates as file is written and extended
- Committed to directory entry on xFileClose()
- Zero for newly created empty files

Common use cases:

- **Buffer allocation**: Allocate exactly enough memory for file contents
- **Validation**: Verify file has expected size
- **Progress tracking**: Calculate percentage read/written
- **Bounds checking**: Ensure reads don't exceed file bounds
- **Empty file detection**: Check if size is zero

Example 1: Read entire file into memory

```
 File_t *file;
Return_t readEntireFile(Volume_t *vol, const char *path, Byte_t **fileData,
Word_t *size) {
  File_t *file;
  if (OK(xFileOpen(&file, vol, (Byte_t*)path, FS_MODE_READ))) {
    // Get file size if (OK(xFileGetSize(file, size))) {
      // Read entire file if (OK(xFileRead(file, *size, fileData))) {
        xFileClose(file);
        return ReturnOK;
      }
    }
    xFileClose(file);
  }
  return ReturnError;
}
// Usage Byte_t *data;
Word_t dataSize;
if (OK(readEntireFile(vol, "/config.txt", &data, &dataSize))) {
  processData(data, dataSize);
  xMemFree((Addr_t *)data);
}
```

Example 2: Validate file size before processing

```
#define MIN_VALID_SIZE 100
#define MAX_VALID_SIZE 10000
Return_t processValidatedFile(File_t *file) {
  Word_t size;
  // Check file size is in valid range if (OK(xFileGetSize(file, &size))) {
    if (size < MIN_VALID_SIZE) {
      logError("File too small");
      return ReturnError;
    }
    if (size > MAX_VALID_SIZE) {
      logError("File too large");
      return ReturnError;
    }
    // Size valid - proceed with processing return processFile(file, size);
  }
  return ReturnError;
}
```

Example 3: Copy file with progress reporting

```
 void copyFileWithProgress(File_t *src, File_t *dst) {
  Word_t totalSize;
  Word_t bytesProcessed = 0;
  Byte_t *buffer;
  xFileGetSize(src, &totalSize);
  while (bytesProcessed < totalSize) {
    Size_t chunkSize = (totalSize - bytesProcessed > 512) ? 512 :
(totalSize
- bytesProcessed);
    if (OK(xFileRead(src, chunkSize, &buffer))) {
      xFileWrite(dst, chunkSize, buffer);
```

```
      xMemFree((Addr_t *)buffer);
      bytesProcessed += chunkSize;
      // Report progress Byte_t percent = (Byte_t)((bytesProcessed * 100) /
totalSize);
      updateProgress(percent);
    } else {
      break;
    }
  }
}
```

Example 4: Check for empty file

```
 Base_t isFileEmpty(File_t *file) {
  Word_t size;
  if (OK(xFileGetSize(file, &size))) {
    return (size == 0) ? 1 : 0;
  }
  return 0;
}
// Usage if (isFileEmpty(logFile)) {
  // File is empty - write header xFileWrite(logFile, strlen(HEADER),
(Byte_t*)HEADER);
}
```

Parameters

in	*file*↩ _	Handle to the open file. Must be a valid file from xFileOpen().
out	*size*↩ _	Pointer to variable receiving the file size in bytes.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid file handle or file not open.

Note

The size returned is the current file content size. For files open for writing, the size may increase as write operations extend the file.

File size is independent of file position. You can get the size at any time without affecting the current read/write position.

The size reported is the logical file size (bytes of content), not the physical storage size (which may be larger due to cluster allocation).

See also

xFileSeek() - Position within file based on size

xFileTell() - Get current position (may equal size at EOF)

xFileEOF() - Check if at end of file

xFileRead() - Read file data

xFSGetVolumeInfo() - Get total volume space

4.2.4.20 xFileOpen()

```
Return_t xFileOpen (
        File_t ** file_,
        Volume_t * volume_,
        const Byte_t * path_,
        const Byte_t mode_ )
```

Opens a file for reading, writing, or both, creating a file handle used for subsequent file operations. The file can be opened in various modes controlling read/write access, creation behavior, and positioning. File paths use forward slash notation (/path/to/file.txt) following standard POSIX conventions.

Opening a file allocates kernel resources and establishes the connection between the application and the filesystem. The returned file handle must be used in all subsequent operations on that file and must be closed with xFileClose() when finished to prevent resource leaks.

File open modes (can be combined with bitwise OR):

- **FS_MODE_READ (0x01)**: Open for reading. File must exist.
- **FS_MODE_WRITE (0x02)**: Open for writing. Truncates existing file to zero length.
- **FS_MODE_APPEND (0x04)**: Open for appending. Positions at end of file.
- **FS_MODE_CREATE (0x08)**: Create file if it doesn't exist. Combine with WRITE or APPEND.

Common mode combinations:

- Read-only: FS_MODE_READ
- Write (truncate): FS_MODE_WRITE
- Write (create if needed): FS_MODE_WRITE | FS_MODE_CREATE
- Append: FS_MODE_APPEND | FS_MODE_CREATE
- Read/Write: FS_MODE_READ | FS_MODE_WRITE

Path format:

- Use forward slashes: "/file.txt" or "/dir/subdir/file.dat"
- Root directory files: "/filename.ext"
- Subdirectory files: "/directory/filename.ext"
- Maximum path length: CONFIG_FS_MAX_PATH_LENGTH (default 256 characters)

Example 1: Read existing file

```
 Volume_t *vol;
File_t *file;
if (OK(xFSMount(&vol, blockDeviceUID))) {
  // Open file for reading if (OK(xFileOpen(&file, vol,
(Byte_t*)"/config.txt", FS_MODE_READ))) {
    Byte_t *data;
    Word_t fileSize;
    // Get file size and read all data if (OK(xFileGetSize(file,
&fileSize))) {
      if (OK(xFileRead(file, fileSize, &data))) {
        processData(data, fileSize);
        xMemFree((Addr_t *)data);
      }
    }
    xFileClose(file);
  }
  xFSUnmount(vol);
}
```

Example 2: Create and write new file

```
 Volume_t *vol;
File_t *file;
if (OK(xFSMount(&vol, deviceUID))) {
  // Open file for writing, create if doesn't exist if (OK(xFileOpen(&file,
vol, (Byte_t*)"/data.bin", FS_MODE_WRITE | FS_MODE_CREATE))) {
    Byte_t buffer[128];
    generateData(buffer, sizeof(buffer));
    // Write data if (OK(xFileWrite(file, sizeof(buffer), buffer))) {
      // Data written successfully
    }
    xFileClose(file);
  }
  xFSUnmount(vol);
}
```

Example 3: Append to log file

```
 Volume_t *vol;
void logMessage(const char *message) {
  File_t *logFile;
  // Open log file in append mode, create if doesn't exist if
(OK(xFileOpen(&logFile, vol, (Byte_t*)"/system.log", FS_MODE_APPEND |
FS_MODE_CREATE))) {
    Size_t msgLen = strlen(message);
    xFileWrite(logFile, msgLen, (Byte_t*)message);
    xFileWrite(logFile, 1, (Byte_t*)"\n");  // Add newline
    xFileClose(logFile);
  }
}
```

Example 4: Update existing file (read/write)

```
 File_t *file;
// Open for both reading and writing if (OK(xFileOpen(&file, vol,
(Byte_t*)"/counter.dat", FS_MODE_READ | FS_MODE_WRITE))) {
  Byte_t *data;
  Word_t counter;
  // Read current value if (OK(xFileRead(file, sizeof(Word_t), &data))) {
    counter = *(Word_t*)data;
    xMemFree((Addr_t *)data);
    // Increment counter counter++;
    // Seek back to start and write new value xFileSeek(file, 0,
FS_SEEK_SET);
    xFileWrite(file, sizeof(Word_t), (Byte_t*)&counter);
  }
  xFileClose(file);
}
```

Parameters

out	*file_*	Pointer to File_t *handle to be initialized. On success, this handle is used for all subsequent operations on the opened file. The handle remains valid until xFileClose() is called.
in	*volume_*	Handle to mounted volume containing the file. Must be a valid volume from xFSMount().
in	*path_*	Pointer to null-terminated string containing file path. Use forward slashes (/) for directory separators. Path is relative to volume root.
in	*mode_*	File open mode flags. Combine flags with bitwise OR (|). See FS_MODE_* constants for available modes.

Returns

ReturnOK if file opened successfully, ReturnError if open failed due to invalid volume, file not found (read mode without CREATE), insufficient space (create mode), invalid path, or too many open files.

Warning

The file handle must be closed with xFileClose() when finished. Failure to close files causes resource leaks and may prevent other files from being opened.

Opening a file with FS_MODE_WRITE (without APPEND) truncates the file to zero length immediately, destroying existing content. Use FS_MODE_READ | FS_MODE_WRITE if you need to modify existing content without truncation.

File paths are case-sensitive. "/File.txt" and "/file.txt" are different files.

Do not attempt to open the same file multiple times simultaneously. This may cause data corruption or filesystem inconsistencies.

Note

File handles are allocated from kernel memory and limited by system resources. Always close files promptly after use.

When creating files with FS_MODE_CREATE, parent directories must already exist. Use xDirMake() to create directories before creating files within them.

File position starts at 0 for READ and WRITE modes, and at end-of-file for APPEND mode. Use xFileSeek() to change position after opening.

See also

xFileClose() - Close file and free resources

xFileRead() - Read data from file

xFileWrite() - Write data to file

xFileSeek() - Change file position

xFSMount() - Mount volume before opening files

4.2.4.21 xFileRead()

```
Return_t xFileRead (
    File_t * file_,
    const Size_t size_,
    Byte_t ** data_ )
```

Reads a specified number of bytes from the current file position, allocating memory to hold the retrieved data and advancing the file position. The caller is responsible for freeing the allocated memory with xMemFree() after processing the data.

Reading retrieves data from the file starting at the current file position (initially 0, or set by xFileSeek()). After a successful read, the file position advances by the number of bytes read, positioning for the next sequential read.

Memory allocation pattern:

- xFileRead() allocates memory from user heap for the read data
- Caller receives pointer to allocated buffer via data_ parameter
- Caller must call xMemFree() after processing to prevent memory leaks
- Memory size equals the number of bytes successfully read

Read behavior:

- Reads up to size_ bytes, may read fewer if EOF reached

- Returns actual bytes read (could be less than requested)
- File position advances by number of bytes read
- Reading past EOF returns 0 bytes
- Sequential reads retrieve consecutive file data

Common scenarios:

- **Full file read**: Get file size, read entire file in one call
- **Chunked reading**: Read file in smaller blocks to conserve memory
- **Partial reads**: Read specific portions using xFileSeek()
- **Sequential processing**: Read file sequentially in fixed-size chunks

Example 1: Read entire file

```
 File_t *file;
if (OK(xFileOpen(&file, vol, (Byte_t*)"/config.txt", FS_MODE_READ))) {
  Word_t fileSize;
  Byte_t *data;
  // Get file size if (OK(xFileGetSize(file, &fileSize))) {
    // Read all data if (OK(xFileRead(file, fileSize, &data))) {
      // Process data parseConfig(data, fileSize);
      // Always free allocated memory xMemFree((Addr_t *)data);
    }
  }
  xFileClose(file);
}
```

Example 2: Read file in chunks

```
 File_t *file;
#define CHUNK_SIZE 512
void processLargeFile(File_t *file) {
  Base_t eof = 0;
  Byte_t *chunk;
  // Read until end of file while (!eof) {
    if (OK(xFileRead(file, CHUNK_SIZE, &chunk))) {
      // Process this chunk processChunk(chunk, CHUNK_SIZE);
      xMemFree((Addr_t *)chunk);
      // Check if reached EOF xFileEOF(file, &eof);
    } else {
      break;
    }
  }
}
```

Example 3: Read specific portion of file

```
 File_t *file;
// Read 100 bytes starting at offset 500 Return_t readFileSection(File_t *
file, Word_t offset, Size_t count, Byte_t **data) {
  // Seek to desired position if (ERROR(xFileSeek(file, offset,
FS_SEEK_SET))) {
    return ReturnError;
  }
  // Read data from that position if (ERROR(xFileRead(file, count, data)))
{
    return ReturnError;
  }
  return ReturnOK;
}
// Usage Byte_t *sectionData;
if (OK(readFileSection(file, 500, 100, &sectionData))) {
  processData(sectionData, 100);
  xMemFree((Addr_t *)sectionData);
}
```

Example 4: Read with EOF detection

```
 File_t *file;
void copyFileData(File_t *srcFile, File_t *dstFile) {
  Byte_t *buffer;
  Base_t eof;
  do {
    // Read chunk from source if (OK(xFileRead(srcFile, 1024, &buffer))) {
      // Write to destination xFileWrite(dstFile, 1024, buffer);
      xMemFree((Addr_t *)buffer);
    }
    // Check if source EOF reached xFileEOF(srcFile, &eof);
  } while (!eof);
}
```

Parameters

in	*file_*	Handle to the open file to read from. Must be a valid file opened with FS_MODE_READ permission.
in	*size_*	Number of bytes to read from file. Actual bytes read may be less if end-of-file is reached before size_ bytes are available.
out	*data_*	Pointer to variable receiving address of newly allocated buffer containing the read data. Memory is allocated from user heap and must be freed with xMemFree() after use.

Returns

ReturnOK if read succeeded (even if fewer than size_ bytes read), ReturnError if read failed due to invalid file handle, file not open for reading, memory allocation failure, or device read error.

Warning

The memory allocated for data_ MUST be freed by the caller using xMemFree() after processing. Failure to free this memory will cause a memory leak. Always pair xFileRead() with xMemFree() in your code.

The file must have been opened with FS_MODE_READ permission. Attempting to read from a write-only file will fail.

xFileRead() may return fewer bytes than requested if EOF is reached. Check actual data size or use xFileEOF() to detect end-of-file condition.

Note

After a successful read, the file position advances by the number of bytes read. Subsequent reads continue from the new position.

Reading 0 bytes or reading past EOF is not an error—it returns ReturnOK with zero bytes allocated. Always check the actual bytes read.

For large files, consider reading in chunks rather than loading the entire file into memory to conserve heap space.

See also

xFileWrite() - Write data to file

xFileSeek() - Change file position before reading

xFileGetSize() - Get total file size

xFileEOF() - Check if end-of-file reached

xMemFree() - Free memory allocated by xFileRead()

xFileOpen() - Open file with read permission

4.2.4.22 xFileRename() `Return_t xFileRename (`

```
            Volume_t * volume_,
            const Byte_t * oldPath_,
            const Byte_t * newPath_ )
```

Changes the name and/or location of a file within the filesystem. This operation can rename a file in the same directory or move it to a different directory with an optional new name. The file content and attributes are preserved; only the directory entry is updated.

Rename is an atomic operation that updates the directory structure without copying file data. This makes it efficient even for large files. The file must not be open during the rename operation.

Rename capabilities:

- Rename file in same directory: "/file.txt" → "/newname.txt"
- Move file to different directory: "/file.txt" → "/backup/file.txt"
- Move and rename: "/old.txt" → "/archive/new.txt"
- Preserves file content and attributes
- Efficient (no data copying required)

Common use cases:

- **Versioning**: Rename old file before creating new version
- **Organization**: Move files to appropriate directories
- **Backup**: Rename file to indicate backup or archive status
- **Atomic updates**: Write new file, rename old, rename new to final name
- **Temporary files**: Rename temp file to final name after completion

Example 1: Simple rename in same directory

```
 Volume_t *vol;
Return_t renameLogFile(void) {
  Base_t exists;
  // Check if current log exists if (OK(xFileExists(vol,
(Byte_t*)"/system.log", &exists)) && exists) {
    // Rename to backup if (OK(xFileRename(vol, (Byte_t*)"/system.log",
(Byte_t*)"/system.log.old"))) {
      logInfo("Log file renamed for archival");
      return ReturnOK;
    }
  }
  return ReturnError;
}
```

Example 2: Log rotation with multiple versions

```
 void rotateLogs(Volume_t *vol) {
  Base_t exists;
  // Rotate log.2 → log.3 (delete log.3 if exists) if (OK(xFileExists(vol,
(Byte_t*)"/log.3", &exists)) && exists) {
    xFileUnlink(vol, (Byte_t*)"/log.3");
  }
  if (OK(xFileExists(vol, (Byte_t*)"/log.2", &exists)) && exists) {
    xFileRename(vol, (Byte_t*)"/log.2", (Byte_t*)"/log.3");
  }
  // Rotate log.1 → log.2 if (OK(xFileExists(vol, (Byte_t*)"/log.1",
&exists)) && exists) {
    xFileRename(vol, (Byte_t*)"/log.1", (Byte_t*)"/log.2");
  }
  // Rotate current → log.1 if (OK(xFileExists(vol, (Byte_t*)"/system.log",
&exists)) && exists) {
    xFileRename(vol, (Byte_t*)"/system.log", (Byte_t*)"/log.1");
  }
```

```
}
```

Example 3: Atomic file replacement

```
 Return_t atomicConfigUpdate(Volume_t *vol, Byte_t *newConfig, Size_t
size)
{
  File_t *tmpFile;
  // Write to temporary file first if (OK(xFileOpen(&tmpFile, vol,
(Byte_t*)"/config.tmp", FS_MODE_WRITE | FS_MODE_CREATE))) {
    if (OK(xFileWrite(tmpFile, size, newConfig))) {
      xFileSync(tmpFile);  // Ensure written to disk xFileClose(tmpFile);
      // Rename old config to backup Base_t exists;
      if (OK(xFileExists(vol, (Byte_t*)"/config.dat", &exists)) && exists)
{
        xFileRename(vol, (Byte_t*)"/config.dat", (Byte_t*)"/config.bak");
      }
      // Rename temp to final name if (OK(xFileRename(vol,
(Byte_t*)"/config.tmp", (Byte_t*)"/config.dat"))) {
        return ReturnOK;  // Atomic update successful
      }
    } else {
      xFileClose(tmpFile);
    }
  }
  return ReturnError;
}
```

Example 4: Move file to archive directory

```
 Return_t archiveOldData(Volume_t *vol, const char *filename) {
  Byte_t oldPath[256];
  Byte_t newPath[256];
  Base_t exists;
  // Build paths snprintf((char*)oldPath, sizeof(oldPath), "/%s",
filename);
  snprintf((char*)newPath, sizeof(newPath), "/archive/%s", filename);
  // Check if file exists if (OK(xFileExists(vol, oldPath, &exists)) &&
exists) {
    // Move to archive directory if (OK(xFileRename(vol, oldPath,
newPath))) {
      logInfo("Archived: %s", filename);
      return ReturnOK;
    }
  }
  return ReturnError;
}
```

Parameters

in	*volume_*	Handle to the mounted volume containing the file. Must be a valid volume from xFSMount().
in	*oldPath_*	Pointer to null-terminated string containing current file path. The file must exist at this location.
in	*newPath_*	Pointer to null-terminated string containing new file path. If a file already exists at this path, the operation fails.

Returns

ReturnOK if file renamed successfully, ReturnError if rename failed due to invalid volume, source file not found, destination file already exists, file is open, or filesystem error.

Warning

The file must not be open during rename. Close all file handles before calling xFileRename(). Renaming an open file will fail.

If a file already exists at newPath_, the rename operation will fail. Delete the existing destination file first if you want to replace it.

Both paths must be on the same volume. You cannot use rename to move files between different volumes or storage devices.

The destination directory must exist. Create directories with xDirMake() before moving files into them.

Note

Rename is very efficient since it only updates directory entries without copying file data, regardless of file size.

File attributes and content are preserved during rename.

See also

xFileExists() - Check if source/destination exists

xFileUnlink() - Delete file instead of renaming

xFileClose() - Close file before renaming

xDirMake() - Create destination directory

4.2.4.23 xFileSeek()

```
Return_t xFileSeek (
    File_t * file_,
    const Word_t offset_,
    const Byte_t origin_ )
```

Moves the file position indicator to a new location within the file, controlling where subsequent read or write operations will occur. The file position can be set relative to the beginning of the file, the current position, or the end of the file using the origin parameter.

File positioning is essential for random access operations, allowing applications to read or write specific portions of a file without processing the entire file sequentially. The position is measured in bytes from the specified origin point.

Seek origins (defined in fs.h):

- **FS_SEEK_SET (0)**: Offset from beginning of file. offset_ is absolute position.
- **FS_SEEK_CUR (1)**: Offset from current position. offset_ is relative (can be negative).
- **FS_SEEK_END (2)**: Offset from end of file. offset_ is typically 0 or negative.

Common positioning operations:

- **Rewind to start**: xFileSeek(file, 0, FS_SEEK_SET)
- **Jump to end**: xFileSeek(file, 0, FS_SEEK_END)
- **Skip forward**: xFileSeek(file, 100, FS_SEEK_CUR)
- **Skip backward**: xFileSeek(file, -50, FS_SEEK_CUR)
- **Absolute position**: xFileSeek(file, 1000, FS_SEEK_SET)

Common scenarios:

- **Read file header then skip to data**: Seek past header to data section
- **Update specific record**: Seek to record position, write new data
- **Append detection**: Seek to end to get file size
- **Reread data**: Seek back to previous position

Example 1: Read file header and data separately

```
 File_t *file;
typedef struct {
  Word_t magic;
  Word_t version;
  Word_t dataOffset;
} FileHeader_t;
if (OK(xFileOpen(&file, vol, (Byte_t*)"/data.bin", FS_MODE_READ))) {
  Byte_t *headerData;
  FileHeader_t *header;
  // Read header at beginning if (OK(xFileRead(file, sizeof(FileHeader_t),
&headerData))) {
    header = (FileHeader_t*)headerData;
    // Validate and seek to data section if (header->magic == 0xDEADBEEF) {
      xFileSeek(file, header->dataOffset, FS_SEEK_SET);
      // Read data from new position Byte_t *data;
      xFileRead(file, 1024, &data);
      xMemFree((Addr_t *)data);
    }
    xMemFree((Addr_t *)headerData);
  }
  xFileClose(file);
}
```

Example 2: Update specific record in file

```
 typedef struct {
  Word_t id;
  Byte_t status;
  Byte_t data[64];
} Record_t;
Return_t updateRecord(File_t *file, Word_t recordIndex, Record_t newRecord)
{
  Word_t recordOffset = recordIndex * sizeof(Record_t);
  // Seek to record position if (ERROR(xFileSeek(file, recordOffset,
FS_SEEK_SET))) {
    return ReturnError;
  }
  // Write updated record return xFileWrite(file, sizeof(Record_t),
(Byte_t*)newRecord);
}
```

Example 3: Get file size using seek to end

```
 Word_t getFileSize(File_t *file) {
  Word_t size;
  Word_t currentPos;
  // Save current position xFileTell(file, &currentPos);
  // Seek to end and get position (which equals file size) xFileSeek(file,
0, FS_SEEK_END);
  xFileTell(file, &size);
  // Restore original position xFileSeek(file, currentPos, FS_SEEK_SET);
  return size;
}
```

Example 4: Skip chunks when processing

```
 File_t *file;
#define CHUNK_SIZE 512
#define CHUNK_GAP 128
void processAlternateChunks(File_t *file) {
  Byte_t *chunk;
  Base_t eof = 0;
  while (!eof) {
    // Read chunk if (OK(xFileRead(file, CHUNK_SIZE, &chunk))) {
      processChunk(chunk, CHUNK_SIZE);
      xMemFree((Addr_t *)chunk);
      // Skip gap to next chunk xFileSeek(file, CHUNK_GAP, FS_SEEK_CUR);
      xFileEOF(file, &eof);
    } else {
      break;
    }
  }
}
```

Parameters

in	*file_*	Handle to the open file. Must be a valid file from xFileOpen().
in	*offset_*	Number of bytes to offset from origin. Can be positive or negative depending on origin. For FS_SEEK_SET, must be >= 0.
in	*origin_*	Reference point for offset. Use FS_SEEK_SET for absolute position, FS_SEEK_CUR for relative to current, FS_SEEK_END for relative to end of file.

Returns

ReturnOK if seek succeeded, ReturnError if seek failed due to invalid file handle, invalid origin value, or resulting position would be before start of file or beyond reasonable file bounds.

Warning

Seeking beyond the end of the file and then writing creates a sparse file with undefined data in the gap. The gap will contain whatever data was previously in those disk sectors.

Some seek operations may fail if they would position before the start of the file (negative absolute position).

Note

After a seek operation, the next read or write occurs at the new position. The position indicator is updated immediately.

Seeking does not change the file size. Only writing beyond the current end-of-file extends the file.

For FS_SEEK_CUR, offset_ can be negative to move backwards, or positive to move forwards from the current position.

See also

xFileTell() - Get current file position

xFileGetSize() - Get file size without seeking

xFileRead() - Read from current position

xFileWrite() - Write at current position

xFileEOF() - Check if at end of file

4.2.4.24 xFileSync() `Return_t xFileSync ( File_t * file_ )`

Forces any buffered write data for the file to be written to the underlying storage device immediately, ensuring data persistence. This operation updates the file's directory entry and File Allocation Table (FAT) to reflect the current file size and cluster allocation.

Normally, write operations are buffered and only committed to storage when xFileClose() is called. For critical data that must survive system failures or power loss, call xFileSync() explicitly to guarantee the data is physically written to the storage device.

Sync ensures:

- All buffered write data is written to storage
- Directory entry is updated with current file size
- FAT is updated with current cluster allocation
- Data is persistent even if system crashes after sync
- File remains open for continued operations

When to use sync:

- **Critical data**: Financial transactions, configuration changes
- **Checkpointing**: Periodic saves during long operations
- **Error detection**: Verify writes succeeded before continuing
- **Power-loss protection**: Ensure data written before risky operations
- **Real-time logging**: Guarantee log entries are stored immediately

Example 1: Write critical configuration with sync

```
 Return_t saveConfig(Volume_t *vol, Byte_t *configData, Size_t
dataSize) {
  File_t *file;
  if (OK(xFileOpen(&file, vol, (Byte_t*)"/config.dat", FS_MODE_WRITE |
FS_MODE_CREATE))) {
    // Write configuration data if (OK(xFileWrite(file, dataSize,
configData))) {
      // Force write to storage before continuing if (OK(xFileSync(file)))
{
        // Data guaranteed on disk xFileClose(file);
        return ReturnOK;
      } else {
        // Sync failed - data may be lost xFileClose(file);
        return ReturnError;
      }
    }
    xFileClose(file);
  }
  return ReturnError;
}
```

Example 2: Periodic checkpointing during long write

```
 File_t *dataFile;
#define CHECKPOINT_INTERVAL 10
void processLargeDataset(Byte_t *dataset, Word_t recordCount) {
  Word_t i;
  for (i = 0; i < recordCount; i++) {
    // Write record xFileWrite(dataFile, RECORD_SIZE, &dataset[i *
RECORD_SIZE]);
    // Checkpoint every N records if ((i % CHECKPOINT_INTERVAL) == 0) {
      if (OK(xFileSync(dataFile))) {
        logInfo("Checkpoint at record %lu", i);
      } else {
        logError("Checkpoint failed - aborting");
        break;
      }
    }
  }
}
```

Example 3: Real-time event logging

```
 File_t *eventLog;
void logCriticalEvent(const char *eventMsg) {
  Size_t msgLen = strlen(eventMsg);
  // Write event xFileWrite(eventLog, msgLen, (Byte_t*)eventMsg);
  xFileWrite(eventLog, 1, (Byte_t*)"\n");
  // Force immediate write to storage if (ERROR(xFileSync(eventLog))) {
    // Could not guarantee persistence handleLogFailure();
  }
  // Event now safely on disk
}
```

Example 4: Transaction-style write with rollback

```
 Return_t atomicUpdate(File_t *file, Byte_t *newData, Size_t dataSize)
{
  Word_t originalPos;
  Word_t originalSize;
  // Save state for rollback xFileTell(file, &originalPos);
  xFileGetSize(file, &originalSize);
  // Attempt write if (OK(xFileWrite(file, dataSize, newData))) {
    // Try to commit if (OK(xFileSync(file))) {
      return ReturnOK;  // Success - data committed
    } else {
      // Sync failed - truncate back to original size xFileTruncate(file,
originalSize);
      xFileSeek(file, originalPos, FS_SEEK_SET);
      return ReturnError;
    }
  }
  return ReturnError;
}
```

Parameters

in	*file*↩ _	Handle to the open file to sync. Must be a valid file opened with write permission (FS_MODE_WRITE or FS_MODE_APPEND).

Returns

ReturnOK if sync succeeded, ReturnError if sync failed due to invalid file handle, file not open for writing, or device write errors during flush operation.

Warning

Sync operations involve multiple disk writes (data, FAT, directory entry) and may take significant time, especially on slow storage devices. Use sync judiciously to balance data safety with performance.

If xFileSync() returns ReturnError, some data may have been written but the filesystem may be in an inconsistent state. Close and reopen the file or check filesystem integrity.

Note

xFileSync() does NOT close the file. After sync completes, the file remains open and can be used for additional read or write operations.

Calling xFileClose() automatically performs a sync before closing, so explicit xFileSync() is not required at the end of normal file operations.

For read-only files, xFileSync() has no effect and returns ReturnOK immediately.

See also

xFileWrite() - Write data that may be buffered until sync

xFileClose() - Close file (automatically syncs)

xFileTruncate() - Change file size

4.2.4.25 xFileTell()

```
Return_t xFileTell (
    const File_t * file_,
    Word_t * position_ )
```

Retrieves the current byte offset of the file position indicator, which indicates where the next read or write operation will occur. The position is measured in bytes from the beginning of the file, with 0 representing the first byte.

The file position changes automatically after read and write operations, and can be explicitly set with xFileSeek(). Knowing the current position is useful for saving and restoring read/write locations, calculating how much data has been processed, or implementing custom file navigation.

Position characteristics:

- Measured in bytes from beginning of file (0-based)
- Advances automatically after read/write operations
- Can be set explicitly with xFileSeek()

- Equals file size when at end-of-file
- Starts at 0 for READ/WRITE modes, at EOF for APPEND mode

Common use cases:

- **Save/restore position**: Remember position to return later
- **Progress tracking**: Monitor how much of file has been processed
- **Size calculation**: Position after seek-to-end equals file size
- **Offset calculation**: Compute relative offsets for records

Example 1: Save and restore file position

```
 File_t *file;
Word_t savedPosition;
// Save current position xFileTell(file, &savedPosition);
// Perform some operation that changes position xFileSeek(file, 0,
FS_SEEK_SET);
Byte_t *header;
xFileRead(file, 64, &header);
processHeader(header);
xMemFree((Addr_t *)header);
// Restore original position xFileSeek(file, savedPosition, FS_SEEK_SET);
// Continue from where we left off Byte_t *data;
xFileRead(file, 512, &data);
xMemFree((Addr_t *)data);
```

Example 2: Track read progress

```
 File_t *file;
Word_t fileSize;
Word_t currentPos;
xFileGetSize(file, &fileSize);
while (1) {
  Byte_t *chunk;
  if (OK(xFileRead(file, 1024, &chunk))) {
    processChunk(chunk, 1024);
    xMemFree((Addr_t *)chunk);
    // Show progress xFileTell(file, &currentPos);
    Byte_t percent = (Byte_t)((currentPos * 100) / fileSize);
    updateProgressBar(percent);
    if (currentPos >= fileSize) break;
  } else {
    break;
  }
}
```

Example 3: Record file offset of data sections

```
 typedef struct {
  Word_t sectionOffset;
  Word_t sectionSize;
} SectionInfo_t;
SectionInfo_t sections[10];
Byte_t sectionCount = 0;
void indexFileSections(File_t *file) {
  Byte_t *sectionHeader;
  while (sectionCount < 10) {
    // Record where this section starts xFileTell(file,
&sections[sectionCount].sectionOffset);
    // Read section header to get size if (OK(xFileRead(file, 4,
&sectionHeader))) {
      sections[sectionCount].sectionSize = *(Word_t*)sectionHeader;
      xMemFree((Addr_t *)sectionHeader);
      // Skip to next section xFileSeek(file,
sections[sectionCount].sectionSize, FS_SEEK_CUR);
      sectionCount++;
    } else {
      break;
    }
  }
}
```

Parameters

in	*file_*	Handle to the open file. Must be a valid file from xFileOpen().
out	*position↩*	Pointer to variable receiving the current file position in bytes. Position 0 is the first byte of the file.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid file handle or file not open.

Note

The position returned is always measured from the beginning of the file, regardless of how the file was opened or what seek operations were used.

When a file is first opened with FS_MODE_APPEND, xFileTell() will return the file size (position is at end of file).

The position advances after each read/write by the number of bytes transferred.

See also

xFileSeek() - Change file position

xFileGetSize() - Get total file size

xFileEOF() - Check if at end of file

xFileRead() - Read from current position

xFileWrite() - Write at current position

4.2.4.26 xFileTruncate()

```
Return_t xFileTruncate (
    File_t * file_,
    const Word_t size_ )
```

Changes the size of an open file to the specified number of bytes, either extending the file with undefined data or truncating it by discarding data beyond the new size. This operation allows precise control over file size independent of write operations.

Truncation behavior:

- If new size $<$ current size: File is shortened, data beyond new size is lost
- If new size $>$ current size: File is extended, gap filled with undefined data
- If new size == current size: No change, operation succeeds
- File position is preserved if still valid after truncation
- Freed clusters are returned to filesystem free space

Common use cases:

- **Preallocate space**: Extend file to reserve disk space before writing
- **Remove trailing data**: Truncate to exact data size
- **Rollback writes**: Shorten file to previous checkpoint
- **Clear file**: Truncate to zero to erase all content
- **Fixed-size files**: Ensure file is exact required size

Example 1: Truncate file to zero (clear contents)

```
 File_t *file;
Return_t clearFile(Volume_t *vol, const char *path) {
  File_t *file;
  // Open for writing if (OK(xFileOpen(&file, vol, (Byte_t*)path,
FS_MODE_WRITE))) {
    // Truncate to zero bytes if (OK(xFileTruncate(file, 0))) {
      xFileClose(file);
      return ReturnOK;
    }
    xFileClose(file);
  }
  return ReturnError;
}
```

Example 2: Preallocate file space

```
#define PREALLOCATE_SIZE (100 * 1024)  // 100 KB
Return_t createPreallocatedFile(Volume_t *vol, const char *path) {
  File_t *file;
  // Create new file if (OK(xFileOpen(&file, vol, (Byte_t*)path,
FS_MODE_WRITE | FS_MODE_CREATE))) {
    // Preallocate space if (OK(xFileTruncate(file, PREALLOCATE_SIZE))) {
      // Space reserved - reset position to start xFileSeek(file, 0,
FS_SEEK_SET);
      xFileClose(file);
      return ReturnOK;
    }
    xFileClose(file);
  }
  return ReturnError;
}
```

Example 3: Trim file to actual data size

```
 File_t *file;
void trimLogFile(File_t *file, Word_t actualDataSize) {
  Word_t currentSize;
  // Get current file size if (OK(xFileGetSize(file, &currentSize))) {
    if (currentSize > actualDataSize) {
      // File is larger than needed - truncate excess xFileTruncate(file,
actualDataSize);
      logInfo("Trimmed %lu bytes from log file", currentSize -
actualDataSize);
    }
  }
}
```

Example 4: Rollback to checkpoint

```
 File_t *dataFile;
Word_t checkpointSize;
void saveCheckpoint(File_t *file) {
  // Save current size as checkpoint xFileGetSize(file, &checkpointSize);
}
void rollbackToCheckpoint(File_t *file) {
  Word_t currentSize;
  xFileGetSize(file, &currentSize);
  if (currentSize > checkpointSize) {
    // Discard data written after checkpoint if (OK(xFileTruncate(file,
checkpointSize))) {
      xFileSeek(file, checkpointSize, FS_SEEK_SET);
      logInfo("Rolled back to checkpoint");
    }
  }
}
```

Parameters

in	*file*↩ _	Handle to the open file to resize. Must be a valid file opened with write permission (FS_MODE_WRITE or FS_MODE_APPEND).
in	*size*↩ _	New file size in bytes. Can be larger (extend) or smaller (truncate) than current size.

Returns

ReturnOK if truncate succeeded, ReturnError if operation failed due to invalid file handle, file not open for writing, insufficient disk space (for extension), or FAT update errors.

Warning

Truncating a file to a smaller size permanently deletes data beyond the new size. This data cannot be recovered. Ensure the new size is correct before calling this function.

If extending a file, the gap between the old size and new size contains undefined data (whatever was previously in those disk sectors). Do not rely on gap data being zero or any specific value.

The file position is not changed by truncation unless it exceeds the new file size, in which case it is clamped to the new size.

Note

Truncating to zero is an efficient way to clear a file while keeping it open.

Extending a file with xFileTruncate() allocates disk space but does not initialize the new space with any specific values.

The new file size is committed when xFileClose() or xFileSync() is called.

See also

xFileGetSize() - Query current file size

xFileWrite() - Write data (changes size automatically)

xFileSeek() - Change file position

xFileSync() - Commit size change to storage

xFileClose() - Close file and commit size

4.2.4.27 xFileUnlink()

```
Return_t xFileUnlink (
    Volume_t * volume_,
    const Byte_t * path_ )
```

Removes a file from the filesystem, freeing its disk space and removing its directory entry. This operation is permanent—deleted files cannot be recovered. The file must not be open when deleted; close all file handles before calling xFileUnlink().

Deletion process:

- Removes file's directory entry
- Marks file's clusters as free in FAT
- Returns disk space to available pool
- Operation is permanent and irreversible

Common scenarios:

- **Temporary file cleanup**: Remove temporary or cache files
- **Old log removal**: Delete old log files to free space
- **File replacement**: Delete old file before writing new version
- **Error recovery**: Remove corrupted or incomplete files

- **Space management**: Delete unnecessary files to reclaim storage

Example 1: Delete temporary file

```
 Volume_t *vol;
void cleanupTempFile(void) {
  Base_t exists;
  // Check if temp file exists if (OK(xFileExists(vol,
(Byte_t*)"/temp.dat",
&exists)) && exists) {
    // Delete temp file if (OK(xFileUnlink(vol, (Byte_t*)"/temp.dat"))) {
      logInfo("Temp file deleted");
    }
  }
}
```

Example 2: Replace file with new version

```
 Return_t updateConfigFile(Volume_t *vol, Byte_t *newConfig, Size_t
size) {
  Base_t exists;
  // Check if old config exists if (OK(xFileExists(vol,
(Byte_t*)"/config.dat", &exists)) && exists) {
    // Delete old version if (ERROR(xFileUnlink(vol,
(Byte_t*)"/config.dat"))) {
      return ReturnError;
    }
  }
  // Write new config File_t *file;
  if (OK(xFileOpen(&file, vol, (Byte_t*)"/config.dat", FS_MODE_WRITE |
FS_MODE_CREATE))) {
    xFileWrite(file, size, newConfig);
    xFileClose(file);
    return ReturnOK;
  }
  return ReturnError;
}
```

Example 3: Delete old log files to free space

```
 void cleanOldLogs(Volume_t *vol) {
  const char *oldLogs[] = {
    "/log.old.3",
    "/log.old.2",
    "/log.old.1"
  };
  Base_t exists;
  VolumeInfo_t *volInfo;
  // Check if we need space if (OK(xFSGetVolumeInfo(vol, &volInfo))) {
    if (volInfo.freeBytes < (50 * 1024)) {  // Less than 50KB free
      // Delete old logs for (int i = 0; i < 3; i++) {
        if (OK(xFileExists(vol, (Byte_t*)oldLogs[i], &exists)) && exists) {
          xFileUnlink(vol, (Byte_t*)oldLogs[i]);
        }
      }
    }
  }
}
```

Example 4: Remove corrupted file

```
 Return_t validateAndClean(Volume_t *vol, const char *path) {
  File_t *file;
  Base_t isValid = 0;
  // Try to validate file if (OK(xFileOpen(&file, vol, (Byte_t*)path,
FS_MODE_READ))) {
    Byte_t *data;
    Word_t size;
    if (OK(xFileGetSize(file, &size)) && OK(xFileRead(file, size, &data)))
{
      isValid = validateFileData(data, size);
      xMemFree((Addr_t *)data);
    }
    xFileClose(file);
  }
  // Delete if corrupted if (!isValid) {
    logWarning("Deleting corrupted file: %s", path);
    return xFileUnlink(vol, (Byte_t*)path);
  }
  return ReturnOK;
}
```

Parameters

in	*volume_*	Handle to the mounted volume containing the file. Must be a valid volume from xFSMount().
in	*path_*	Pointer to null-terminated string containing path to file to delete. Use forward slashes (/) for directory separators.

Returns

ReturnOK if file deleted successfully, ReturnError if deletion failed due to invalid volume, file not found, file is open, or filesystem error.

Warning

File deletion is permanent and irreversible. Deleted files cannot be recovered. Ensure you have the correct path before calling this function.

The file must not be open. Close all file handles to the file before attempting to delete it. Deleting an open file will fail.

Do not delete files while other tasks may be accessing them. This may cause those tasks' operations to fail unexpectedly.

Note

Deleting a file frees its disk space immediately, making it available for new files.

You cannot delete directories with xFileUnlink(). Use xDirRemove() to delete empty directories.

See also

xFileExists() - Check if file exists before deleting

xFileRename() - Rename file instead of deleting

xFileClose() - Close file before deleting

xDirRemove() - Delete empty directory

4.2.4.28 xFileWrite()

```
Return_t xFileWrite (
    File_t * file_,
    const Size_t size_,
    const Byte_t * data_ )
```

Writes a specified number of bytes to the file at the current file position, advancing the file position and potentially extending the file size. Data is written from the provided buffer to the filesystem, and changes are committed to storage when the file is closed or explicitly synced.

Writing appends or overwrites data at the current file position, which depends on the mode the file was opened with (WRITE starts at 0, APPEND starts at EOF). After a successful write, the file position advances by the number of bytes written, and the file size is updated if the write extended beyond the previous end-of-file.

Write behavior:

- Writes size_ bytes from data_ buffer to file
- File position advances by number of bytes written
- File size updated if write extends file
- Data buffered until xFileClose() or xFileSync()
- Allocates clusters from filesystem as needed

File modes and write behavior:

- **FS_MODE_WRITE**: Position starts at 0, overwrites from beginning
- **FS_MODE_APPEND**: Position starts at EOF, extends file
- **FS_MODE_READ** | **FS_MODE_WRITE**: Can write at any position via xFileSeek()

Common scenarios:

- **Create new file**: Open with WRITE | CREATE, write data
- **Append to log**: Open with APPEND | CREATE, write entries
- **Update existing**: Open with READ | WRITE, seek and write
- **Replace content**: Open with WRITE (truncates), write new data

Example 1: Write new file

```
 File_t *file;
if (OK(xFileOpen(&file, vol, (Byte_t*)"/data.bin", FS_MODE_WRITE |
FS_MODE_CREATE))) {
  Byte_t buffer[256];
  generateData(buffer, sizeof(buffer));
  // Write data if (OK(xFileWrite(file, sizeof(buffer), buffer))) {
    // Data written successfully
  }
  xFileClose(file);  // Commits write to storage
}
```

Example 2: Append to log file

```
 void logEvent(Volume_t *vol, const char *event) {
  File_t *logFile;
  // Open in append mode if (OK(xFileOpen(&logFile, vol,
(Byte_t*)"/events.log", FS_MODE_APPEND | FS_MODE_CREATE))) {
    Size_t eventLen = strlen(event);
    // Write event message xFileWrite(logFile, eventLen, (Byte_t*)event);
    xFileWrite(logFile, 1, (Byte_t*)"\n");
    xFileClose(logFile);
  }
}
```

Example 3: Update specific file location

```
 File_t *file;
// Update bytes at specific offset Return_t updateFileBytes(File_t *file,
Word_t offset, Byte_t *newData, Size_t size) {
  // Seek to update position if (ERROR(xFileSeek(file, offset,
FS_SEEK_SET))) {
    return ReturnError;
  }
  // Write new data at that position return xFileWrite(file, size,
newData);
}
// Usage if (OK(xFileOpen(&file, vol, (Byte_t*)"/config.dat", FS_MODE_READ
|
FS_MODE_WRITE))) {
  Byte_t newValue = 0x42;
  updateFileBytes(file, 100, &newValue, 1);  // Update byte at offset 100
xFileClose(file);
}
```

Example 4: Write with explicit sync

```
 File_t *file;
if (OK(xFileOpen(&file, vol, (Byte_t*)"/critical.dat", FS_MODE_WRITE |
FS_MODE_CREATE))) {
  Byte_t importantData[512];
  prepareData(importantData, sizeof(importantData));
  // Write data if (OK(xFileWrite(file, sizeof(importantData),
importantData))) {
    // Force write to storage immediately if (OK(xFileSync(file))) {
      // Data guaranteed on disk
    } else {
      // Sync failed - write may not be persistent handleSyncError();
    }
  }
  xFileClose(file);
}
```

Parameters

in	*file_*	Handle to the open file to write to. Must be a valid file opened with FS_MODE_WRITE or FS_MODE_APPEND permission.
in	*size_*	Number of bytes to write to file. Data buffer must contain at least this many bytes.
in	*data_*	Pointer to buffer containing data to write. Buffer must be at least size_ bytes in length.

Returns

ReturnOK if write succeeded, ReturnError if write failed due to invalid file handle, file not open for writing, insufficient disk space, device write error, or FAT allocation failure.

Warning

The file must have been opened with FS_MODE_WRITE or FS_MODE_APPEND permission. Attempting to write to a read-only file will fail.

Writes are buffered and not guaranteed to be on storage until xFileClose() or xFileSync() is called. For critical data, call xFileSync() explicitly to ensure persistence before continuing.

If insufficient disk space is available, write will fail with ReturnError. Use xFSGetVolumeInfo() to check free space before writing large amounts of data.

Opening a file with FS_MODE_WRITE (without APPEND) truncates the file immediately. The first write starts at position 0, replacing all previous content.

Note

After a successful write, the file position advances by size_ bytes. Use xFileTell() to query current position or xFileSeek() to change it.

The data_ buffer is not modified by xFileWrite(). It can be a const buffer or reused after the call returns.

File size is updated as writes extend the file, but the final size is only committed to the directory entry when xFileClose() is called.

See also

xFileRead() - Read data from file

xFileOpen() - Open file with write permission

xFileClose() - Close file and commit writes

xFileSync() - Flush writes to storage

xFileSeek() - Change write position

xFSGetVolumeInfo() - Check available disk space

4.2.4.29 xFSFormat() `Return_t xFSFormat ( const HalfWord_t blockDeviceUID_, const Byte_t * volumeLabel_ )`

Initializes a block device with a FAT32 filesystem structure, creating the boot sector, file allocation tables, and root directory. This operation prepares a storage device for use with HeliOS filesystem operations by writing the complete FAT32 metadata structures.

Formatting is destructive—all existing data on the block device is lost. This operation should only be performed on new devices or when explicitly reinitializing storage. After formatting, the device must be mounted with xFSMount() before files can be created or accessed.

The formatting process:

1. Validates block device is registered and operational 2. Calculates optimal FAT32 parameters based on device size 3. Writes boot sector with filesystem parameters 4. Initializes two file allocation tables (FAT1 and FAT2 for redundancy) 5. Creates empty root directory 6. Sets volume label if provided

Common scenarios:

- **Initial setup**: Format new RAM disk or flash storage
- **Factory reset**: Erase all data and reinitialize filesystem
- **Corruption recovery**: Reformat after filesystem corruption
- **Testing**: Create clean filesystem for unit tests

Example 1: Format RAM disk with volume label

```
#include "block_driver.h"
#include "ramdisk_driver.h"
// Setup and format RAM disk HalfWord_t ramDiskUID = 0;
HalfWord_t blockDevUID = 0;
// Initialize drivers if (OK(__RamDiskRegister__(&ramDiskUID))) {
  if (OK(__BlockDeviceRegister__(&blockDevUID))) {
    // Configure block device to use RAM disk BlockDeviceConfig_t config;
    config.protocol = BLOCK_PROTOCOL_RAW;
    config.backingDeviceUID = ramDiskUID;
    if (OK(__DeviceConfigDevice__(blockDevUID, sizeof(config),
(Byte_t*)&config))) {
      // Format with volume label if (OK(xFSFormat(blockDevUID,
(Byte_t*)"HELIOS_VOL"))) {
        // Filesystem ready to mount
      }
    }
  }
}
```

Example 2: Format and mount in one operation

```
 Return_t initializeFilesystem(HalfWord_t blockDeviceUID, Volume_t *
volume) {
  // Format the device if (ERROR(xFSFormat(blockDeviceUID,
(Byte_t*)"STORAGE"))) {
    return ReturnError;
  }
  // Mount the freshly formatted filesystem if (ERROR(xFSMount(volume,
blockDeviceUID))) {
    return ReturnError;
  }
  // Filesystem formatted and ready for use return ReturnOK;
}
```

Example 3: Factory reset function

```
 Volume_t *systemVolume;
HalfWord_t storageDeviceUID;
void factoryReset(void) {
  // Unmount if currently mounted xFSUnmount(systemVolume);
  // Reformat - destroys all data if (OK(xFSFormat(storageDeviceUID,
(Byte_t*)"FACTORY"))) {
    // Remount fresh filesystem if (OK(xFSMount(&systemVolume,
storageDeviceUID))) {
      // Create default configuration files
createDefaultConfig(systemVolume);
    }
  }
}
```

Parameters

in	*blockDevice↩ UID_*	UID of the block device to format. Must be a registered block device obtained from block device driver registration. The device must support read and write operations.
in	*volumeLabel_*	Pointer to null-terminated string containing volume label (max 11 characters). Can be null for no label. Label is stored in boot sector and visible in volume information queries.

Returns

ReturnOK if format succeeded, ReturnError if format failed due to invalid block device UID, device not available, insufficient device size, or write errors during format operation.

Warning

This operation is DESTRUCTIVE. All existing data on the block device will be permanently lost. Ensure the correct device UID is specified and any important data is backed up before formatting.

The block device must be large enough to hold FAT32 structures. Very small devices may fail to format. Minimum practical size depends on sector size and cluster size calculations.

Do not format a currently mounted volume. Always call xFSUnmount() before formatting to prevent filesystem corruption.

Note

After formatting, the volume must be mounted with xFSMount() before any file operations can be performed.

HeliOS uses FAT32 filesystem format for compatibility with standard FAT32 implementations and tools.

The volume label is optional but recommended for identifying storage volumes in systems with multiple devices.

See also

xFSMount() - Mount formatted volume for file operations

xFSUnmount() - Unmount volume before reformatting

xFSGetVolumeInfo() - Query volume information including label

BlockDeviceRegister() - Register block device before formatting

4.2.4.30 xFSGetVolumeInfo()

```
Return_t xFSGetVolumeInfo (
    const Volume_t * volume_,
    VolumeInfo_t ** info_ )
```

Retrieves comprehensive information about a mounted volume including capacity, free space, and filesystem parameters. This information is useful for monitoring storage utilization, checking available space before write operations, and reporting filesystem status.

The volume information includes both logical (cluster-based) and physical (byte-based) size metrics, allowing applications to make informed decisions about storage management and space allocation.

Information provided:

- Total and free space in clusters and bytes
- Bytes per sector (typically 512)
- Sectors per cluster
- Bytes per cluster (sector size × sectors per cluster)

Common use cases:

- **Space checking**: Verify sufficient space before write operations
- **Usage monitoring**: Track filesystem utilization over time
- **Status reporting**: Display storage capacity to user/diagnostic interface
- **Quota management**: Implement storage quotas based on available space

Example 1: Check space before writing file

```
 Volume_t *vol;
VolumeInfo_t *info;
Return_t writeDataFile(Volume_t *vol, Byte_t *data, Size_t dataSize) {
  // Check available space if (OK(xFSGetVolumeInfo(vol, &info))) {
    if (info.freeBytes >= dataSize) {
      // Sufficient space - proceed with write File_t *file;
      if (OK(xFileOpen(&file, vol, (Byte_t*)"/data.bin", FS_MODE_WRITE |
FS_MODE_CREATE))) {
        xFileWrite(file, dataSize, data);
        xFileClose(file);
        return ReturnOK;
      }
    } else {
      // Insufficient space return ReturnError;
    }
  }
  return ReturnError;
}
```

Example 2: Storage utilization monitoring

```
 Volume_t *vol;
void reportStorageStatus(void) {
  VolumeInfo_t *info;
  if (OK(xFSGetVolumeInfo(vol, &info))) {
    Word_t usedBytes = info.totalBytes - info.freeBytes;
    Byte_t percentUsed = (Byte_t)((usedBytes * 100) / info.totalBytes);
    // Report to diagnostic interface printf("Storage: %lu / %lu bytes
(%u%% used)\n", usedBytes, info.totalBytes, percentUsed);

    // Warn if nearly full if (percentUsed > 90) {
      logWarning("Storage nearly full");
    }
  }
}
```

Example 3: Filesystem parameter reporting

```
 void displayVolumeDetails(Volume_t *vol) {
  VolumeInfo_t *info;
  if (OK(xFSGetVolumeInfo(vol, &info))) {
    printf("Volume Information:\n");
    printf("  Total: %lu bytes (%lu clusters)\n", info.totalBytes,
info.totalClusters);
    printf("  Free:  %lu bytes (%lu clusters)\n", info.freeBytes,
info.freeClusters);
    printf("  Cluster size: %lu bytes\n", info.bytesPerCluster);
    printf("  Sector size:  %u bytes\n", info.bytesPerSector);
    printf("  Sectors/cluster: %u\n", info.sectorsPerCluster);
  }
}
```

Parameters

in	*volume_*	Handle to the mounted volume to query. Must be a valid volume obtained from xFSMount().
out	*info_*	Pointer to VolumeInfo_t *structure to receive volume information. On success, this structure is populated with current volume statistics and parameters.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid volume handle or volume not mounted.

Note

Free space calculation requires scanning the File Allocation Table (FAT), which may take time on large volumes. Cache the result if you need to check free space frequently.

The free space reported is the total free space on the volume. Actual usable space may be less due to filesystem overhead and cluster alignment.

All byte values are in units of bytes. Cluster values represent the number of allocation units (clusters) used by the filesystem.

See also

xFSMount() - Mount volume before querying information

xFSFormat() - Format operation sets volume parameters

xFileGetSize() - Get size of individual files

4.2.4.31 xFSMount()

```
Return_t xFSMount (
    Volume_t ** volume_,
    const HalfWord_t blockDeviceUID_ )
```

Mounts a FAT32 filesystem from a formatted block device, making it available for file and directory operations. Mounting reads and validates the filesystem metadata (boot sector, FAT tables) and creates a volume handle used in all subsequent filesystem operations.

A volume must be mounted before any file operations (open, read, write, etc.) can be performed. The volume handle returned by this function is required by all file and directory functions to identify which filesystem to operate on.

Mount process:

1. Validates block device UID and availability 2. Reads boot sector from block device 3. Validates FAT32 boot sector signature and parameters 4. Allocates volume structure in kernel memory 5. Stores filesystem parameters for subsequent operations 6. Returns volume handle for use in file/directory operations

Common usage patterns:

- **System initialization**: Mount filesystem during startup
- **Removable media**: Mount when device inserted, unmount when removed
- **Multi-volume systems**: Mount multiple block devices as separate volumes
- **Filesystem switching**: Unmount and remount different filesystems

Example 1: Basic mount operation

```
 Volume_t *myVolume;
HalfWord_t blockDeviceUID = 1;  // From block device registration
// Mount the filesystem if (OK(xFSMount(&myVolume, blockDeviceUID))) {
  // Volume mounted - can now open files File_t *file;
  if (OK(xFileOpen(&file, myVolume, (Byte_t*)"/data.txt", FS_MODE_READ))) {
    // File opened successfully xFileClose(file);
  }
  // Always unmount when done xFSUnmount(myVolume);
}
```

Example 2: System initialization with mount

```
 Volume_t *systemVolume;
HalfWord_t storageUID;
void initStorage(void) {
  // Register and configure block device if
(OK(__BlockDeviceRegister__(&storageUID))) {
    BlockDeviceConfig_t config;
    config.protocol = BLOCK_PROTOCOL_RAW;
    config.backingDeviceUID = ramDiskUID;
    if (OK(__DeviceConfigDevice__(storageUID, sizeof(config),
(Byte_t*)&config))) {
      // Try to mount existing filesystem if (ERROR(xFSMount(&systemVolume,
storageUID))) {
        // Mount failed - format and mount fresh filesystem if
(OK(xFSFormat(storageUID, (Byte_t*)"SYSTEM"))) {
          xFSMount(&systemVolume, storageUID);
        }
      }
    }
  }
}
```

Example 3: Multi-volume system

```
 Volume_t *dataVolume;
Volume_t *configVolume;
HalfWord_t dataDeviceUID = 1;
HalfWord_t configDeviceUID = 2;
void mountAllVolumes(void) {
  // Mount data volume if (OK(xFSMount(&dataVolume, dataDeviceUID))) {
    // Mount config volume if (OK(xFSMount(&configVolume,
configDeviceUID))) {
      // Both volumes mounted - use different handles xFileOpen(&file1,
dataVolume, (Byte_t*)"/data.bin", FS_MODE_READ);
      xFileOpen(&file2, configVolume, (Byte_t*)"/config.txt",
FS_MODE_READ);
    }
  }
}
```

Example 4: Mount with volume information query

```
 Volume_t *vol;
HalfWord_t devUID = 1;
if (OK(xFSMount(&vol, devUID))) {
  // Query volume information VolumeInfo_t *info;
  if (OK(xFSGetVolumeInfo(vol, &info))) {
    // Check available space if (info.freeBytes > (100 * 1024)) {  // 100
KB free
      // Sufficient space for operations performFileOperations(vol);
    }
  }
  xFSUnmount(vol);
}
```

Parameters

out	*volume_*	Pointer to Volume_t *handle to be initialized. On success, this handle is used in all subsequent file and directory operations. The handle remains valid until xFSUnmount() is called.
in	*blockDevice↩ UID_*	UID of the block device containing the FAT32 filesystem. Must be a registered block device that has been formatted with xFSFormat().

Returns

ReturnOK if mount succeeded, ReturnError if mount failed due to invalid block device UID, device not available, invalid FAT32 filesystem, corrupted boot sector, or memory allocation failure.

Warning

The block device must contain a valid FAT32 filesystem created by xFSFormat() or another FAT32-compatible tool. Attempting to mount an unformatted or corrupted device will fail.

Only one volume can be mounted per block device at a time. Attempting to mount an already-mounted device will fail.

The volume handle must remain valid for the duration of all file operations. Do not unmount a volume while files are still open.

Note

The volume handle is allocated from kernel memory and persists until xFSUnmount() is called. Always unmount volumes when finished to free kernel resources.

Multiple volumes can be mounted simultaneously if you have multiple block devices, each with its own volume handle.

After mounting, use xFSGetVolumeInfo() to query volume capacity, free space, and other filesystem parameters.

See also

xFSUnmount() - Unmount volume and free resources

xFSFormat() - Format block device with FAT32 filesystem

xFSGetVolumeInfo() - Query volume information

xFileOpen() - Open file on mounted volume

xDirOpen() - Open directory on mounted volume

4.2.4.32 xFSUnmount() `Return_t xFSUnmount ( Volume_t * volume_ )`

Unmounts a previously mounted FAT32 volume, flushing any pending writes and freeing all associated kernel resources. After unmounting, the volume handle becomes invalid and must not be used in any subsequent filesystem operations.

Unmounting ensures filesystem consistency by finalizing all pending operations and is essential for proper resource management in embedded systems. Always unmount volumes before system shutdown, device removal, or when the filesystem is no longer needed.

Unmount process:

1. Validates volume handle 2. Flushes any cached filesystem data to block device 3. Updates FAT tables if necessary 4. Frees volume structure from kernel memory 5. Invalidates volume handle

Common scenarios:

- **System shutdown**: Unmount all volumes before power-off
- **Resource cleanup**: Free kernel memory when filesystem not needed
- **Device removal**: Safely unmount before removing storage media
- **Filesystem switching**: Unmount before reformatting or mounting different volume

Example 1: Basic mount/unmount lifecycle

```
 Volume_t *vol;
HalfWord_t deviceUID = 1;
// Mount volume if (OK(xFSMount(&vol, deviceUID))) {
  // Perform file operations File_t *file;
  xFileOpen(&file, vol, (Byte_t*)"/data.txt", FS_MODE_WRITE |
FS_MODE_CREATE);
  xFileWrite(file, 5, (Byte_t*)"Hello");
  xFileClose(file);
  // Always unmount when done xFSUnmount(vol);
  // vol is now invalid - do not use
}
```

Example 2: System shutdown cleanup

```
 Volume_t *dataVolume;
Volume_t *configVolume;
void shutdownFilesystems(void) {
  // Unmount all volumes before shutdown xFSUnmount(dataVolume);
  xFSUnmount(configVolume);
  // Volumes now unmounted and resources freed
}
```

Example 3: Safe volume handle cleanup

```
 Volume_t *vol = null;
HalfWord_t devUID = 1;
void useVolume(void) {
  if (OK(xFSMount(&vol, devUID))) {
    performOperations(vol);
    // Unmount and nullify handle xFSUnmount(vol);
    vol = null;  // Prevent use-after-unmount
  }
}
```

Parameters

in	*volume*↩ _	Handle to the mounted volume to unmount. Must be a valid volume obtained from a previous successful xFSMount() call. After unmounting, this handle becomes invalid.

Returns

ReturnOK if unmount succeeded, ReturnError if unmount failed due to invalid volume handle or volume not found.

Warning

All files and directories must be closed before unmounting. Unmounting a volume with open file handles may cause resource leaks or undefined behavior.

After calling xFSUnmount(), the volume handle becomes invalid and must not be used in any filesystem operations. Attempting to use an unmounted volume will result in ReturnError.

Always unmount volumes before reformatting the block device or removing storage media to prevent filesystem corruption.

Note

It is good practice to set the volume handle to null after unmounting to prevent accidental use of an invalid handle.

Unmounting frees kernel memory allocated during xFSMount(). In long-running systems, always unmount volumes when they are no longer needed.

See also

xFSMount() - Mount a FAT32 filesystem volume

xFileClose() - Close files before unmounting

xDirClose() - Close directories before unmounting

xFileSync() - Explicitly flush file data before unmount

4.2.4.33 xMemAlloc() `Return_t xMemAlloc ( volatile Addr_t ** addr_, const Size_t size_ )`

Allocates a block of memory from the HeliOS user heap and returns a pointer to the allocated memory. The allocated memory is automatically zeroed (similar to calloc() in standard C), ensuring predictable initialization.

HeliOS maintains separate user and kernel memory regions. This function allocates from the user heap, which is intended for application data structures, buffers, and general-purpose memory needs. The total heap size is determined by CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS × CONFIG_MEMORY_REGION_BLOCK_SIZE.

Memory Allocation Features:

- Automatic zero-initialization of allocated memory
- Best-fit allocation strategy to minimize fragmentation
- Memory safety checks to prevent heap corruption
- Tracking of allocation statistics (via xMemGetHeapStats())

Memory allocated by this function must be freed using xMemFree() when no longer needed to prevent memory leaks. HeliOS does not provide automatic garbage collection.

Warning

The addr_ parameter must be cast to (volatile Addr_t **) to avoid compiler warnings. This is because the function modifies the pointer variable itself (by-reference parameter passing).

Note

Allocation may fail if insufficient contiguous memory is available, even if the total free memory exceeds the requested size. This can occur due to heap fragmentation.

Several HeliOS functions allocate heap memory internally and return it to the caller (e.g., xDeviceRead(), xMemGetHeapStats()). Memory returned by these functions must also be freed with xMemFree().

Example Usage:

```
// Allocate memory for a structure typedef struct {
  int temperature;
  int humidity;
} SensorData_t;
SensorData_t *data = NULL;
if (OK(xMemAlloc((volatile Addr_t **)&data, sizeof(SensorData_t)))) {
  // Memory allocated successfully and zeroed data->temperature =
readTemperature();
  data->humidity = readHumidity();
  // Use the data...
  // Free when done xMemFree((Addr_t *)data);
} else {
  // Handle allocation failure reportError("Out of memory");
}
// Allocate an array uint8_t *buffer = NULL;
if (OK(xMemAlloc((volatile Addr_t **)&buffer, 256))) {
  // Use buffer...
  xMemFree((Addr_t *)buffer);
}
```

Parameters

out	*addr_*	Pointer to a pointer variable that will receive the address of the allocated memory. Must be cast to (volatile Addr_t **). On failure, this pointer is not modified.
in	*size_*	Number of bytes to allocate. Must be greater than zero.

Returns

ReturnOK if memory was successfully allocated, ReturnError if allocation failed (insufficient memory, invalid parameters, or memory system error).

See also

xMemFree() - Free allocated memory

xMemFreeAll() - Free all allocated memory

xMemGetUsed() - Get total allocated memory

xMemGetSize() - Get size of an allocation

xMemGetHeapStats() - Get detailed heap statistics

CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS - Heap size configuration

CONFIG_MEMORY_REGION_BLOCK_SIZE - Memory block size configuration

4.2.4.34 xMemFree()

```
Return_t xMemFree (
    const volatile Addr_t * addr_ )
```

Deallocates a block of memory that was previously allocated by xMemAlloc() or returned by HeliOS functions that allocate memory (such as xDeviceRead(), xMemGetHeapStats(), or xTaskGetAllRunTimeStats()). The freed memory becomes available for future allocations.

After freeing memory, the pointer should be considered invalid and must not be dereferenced. HeliOS does not automatically NULL the pointer; the caller is responsible for proper pointer management.

Memory Management Best Practices:

- Always free memory when it is no longer needed to prevent leaks
- Set pointers to NULL after freeing to avoid use-after-free bugs
- Never free the same memory twice (double-free)
- Never free memory that was not allocated by xMemAlloc() or HeliOS
- Never free stack-allocated variables or static memory

Warning

Freeing invalid memory addresses or freeing the same memory twice will cause heap corruption and undefined behavior. HeliOS performs validation checks, but cannot detect all misuse scenarios.

After calling xMemFree(), do not access the freed memory. Doing so results in undefined behavior and may cause data corruption or system crashes.

Note

Some HeliOS functions allocate memory and return it to the caller. The caller is responsible for freeing this memory. Check function documentation to determine if memory management is required.

Example Usage:

```
// Proper memory management uint8_t *buffer = NULL;
if (OK(xMemAlloc((volatile Addr_t **)&buffer, 128))) {
  // Use the buffer processData(buffer, 128);
  // Free when done if (OK(xMemFree((Addr_t *)buffer))) {
    buffer = NULL;  // Good practice: NULL the pointer
  }
}
// Freeing memory returned by HeliOS functions MemoryRegionStats_t **stats
=
NULL;
if (OK(xMemGetHeapStats(&stats))) {
  // Use stats...
  printf("Free bytes: %lu\n", stats->availableSpaceInBytes);
  // Must free the stats structure xMemFree((Addr_t *)stats);
  stats = NULL;
}
```

Parameters

in	*addr*↩_	Pointer to the memory block to free. Must be a valid pointer previously returned by xMemAlloc() or a HeliOS allocation function. Passing NULL is safe and results in no operation.

Returns

ReturnOK if memory was successfully freed, ReturnError if the operation failed (invalid address, double-free attempt, or memory system error).

See also

xMemAlloc() - Allocate memory from the heap

xMemFreeAll() - Free all allocated memory at once

xMemGetUsed() - Get total allocated memory

xMemGetSize() - Get size of an allocation

xMemGetHeapStats() - Get detailed heap statistics

4.2.4.35 xMemFreeAll() `Return_t xMemFreeAll ( void )`

Deallocates all memory blocks currently allocated from the user heap, effectively resetting the heap to its initial empty state. This is a nuclear option that invalidates ALL heap pointers in one operation, making it useful for complete system resets, test cleanup, or transitioning between major application modes.

After calling this function, every pointer previously returned by xMemAlloc() or HeliOS allocation functions becomes invalid. Accessing any of these pointers results in undefined behavior. This makes xMemFreeAll() powerful but dangerous—use it only when you're certain all heap references can be safely discarded.

Common use cases:

- **System reset**: Preparing for a complete application restart
- **Mode transitions**: Switching between major operating modes that use different data structures
- **Test cleanup**: Resetting memory state between unit tests
- **Error recovery**: Clearing all allocations after detecting heap corruption
- **Initialization**: Ensuring clean slate during system startup

Typical scenarios:

- Application has completed a major operation and needs to free all associated data
- System is entering a low-power mode and needs to minimize memory usage
- Test harness needs to reset state between test cases
- Fatal error occurred and system is preparing to restart

Example 1: Reset between application modes

```
 typedef enum {
  MODE_INITIALIZATION, MODE_NORMAL_OPERATION, MODE_DIAGNOSTICS
} AppMode_t;
void transitionToMode(AppMode_t newMode) {
  // Free all current mode's allocations xMemFreeAll();
  // Initialize new mode switch (newMode) {
    case MODE_INITIALIZATION:
      initializeSystem();
      break;
    case MODE_NORMAL_OPERATION:
      startNormalOperation();
      break;
    case MODE_DIAGNOSTICS:
      startDiagnostics();
      break;
  }
}
```

Example 2: Unit test cleanup

```
 void setUp(void) {
  // Start each test with clean heap xMemFreeAll();
}
void tearDown(void) {
  // Clean up after test xMemFreeAll();
}
void testMemoryAllocation(void) {
  Byte_t *buffer1 = NULL;
  Byte_t *buffer2 = NULL;
  // Allocate test data xMemAlloc((volatile Addr_t **)&buffer1, 128);
  xMemAlloc((volatile Addr_t **)&buffer2, 256);
  // Perform test...
  // Cleanup happens automatically in tearDown()
}
```

Example 3: Error recovery

```
 void handleFatalError(const char *errorMsg) {
  // Log the error logError("Fatal error: %s", errorMsg);
  // Free all memory before restart xMemFreeAll();
  // Reset system state resetSystemState();
  // Restart application restartApplication();
}
```

Example 4: Periodic memory defragmentation

```
 void performMaintenanceCycle(void) {
  // Save critical state to non-volatile storage saveCriticalState();
  // Free all heap memory xMemFreeAll();
  // Restore state with fresh allocations (reduces fragmentation)
restoreCriticalState();
}
```

Returns

ReturnOK if all memory successfully freed, ReturnError if the operation failed (rare, typically indicates heap corruption).

Warning

After calling xMemFreeAll(), ALL heap pointers become invalid. This includes pointers to task parameters, queue data, stream buffers, device configurations, and any application data structures. Accessing these pointers will cause undefined behavior.

Do not call xMemFreeAll() while tasks are running that depend on heap-allocated data. Suspend all tasks or ensure they can handle their data being freed unexpectedly.

This function does NOT affect kernel memory allocations. Only user heap allocations are freed. Kernel structures (tasks, timers, queues) are NOT affected.

Be aware that some HeliOS internal structures may hold pointers to heap memory (e.g., task parameters, queue messages). Freeing all heap memory may cause these structures to contain dangling pointers.

Note

This is the fastest way to free large numbers of allocations, as it doesn't need to process each allocation individually.

After xMemFreeAll(), the heap is fully defragmented with no fragmentation overhead, making it ideal for periodic memory maintenance.

Consider suspending the scheduler with xTaskSuspendAll() before calling xMemFreeAll() to prevent tasks from attempting to use freed memory.

See also

xMemFree() - Free individual memory blocks

xMemAlloc() - Allocate memory from heap

xMemGetUsed() - Check memory usage before/after freeing

xMemGetHeapStats() - Get detailed heap statistics

xTaskSuspendAll() - Suspend scheduler during memory reset

4.2.4.36 xMemGetHeapStats() `Return_t xMemGetHeapStats ( MemoryRegionStats_t ** stats_ )`

Returns comprehensive statistics about the user heap including total size, used/free space, allocation counts, fragmentation metrics, and largest available block. This provides deep insight into memory utilization, allocation patterns, and fragmentation for optimization and diagnostics.

The function allocates a MemoryRegionStats_t ∗structure from the heap and populates it with current heap metrics. The caller MUST free this structure with xMemFree() after use. The statistics represent a snapshot at the time of the call and may become outdated as allocations/frees occur.

Statistics provided in MemoryRegionStats_t ∗:

- **totalSize**: Total heap capacity in bytes
- **usedSpace**: Bytes currently allocated
- **freeSpace**: Bytes available for allocation
- **allocationCount**: Number of active allocations
- **largestFreeBlock**: Size of largest contiguous free block
- **fragmentationPercent**: Degree of memory fragmentation (0-100%)

Common use cases:

- **Memory profiling**: Understanding application memory footprint
- **Fragmentation analysis**: Detecting and addressing heap fragmentation
- **Capacity planning**: Determining if heap size is adequate
- **Performance tuning**: Optimizing allocation patterns
- **Diagnostic logging**: Capturing memory state for troubleshooting

- **Runtime monitoring**: Tracking memory trends over time

Example 1: Display heap utilization

```
 void displayHeapStatus(void) {
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetHeapStats(&stats))) {
    Byte_t usagePercent = (Byte_t)((stats->usedSpace * 100) /
stats->totalSize);
    printf("Heap Memory Status:\n");
    printf("  Total:       %lu bytes\n", stats->totalSize);
    printf("  Used:        %lu bytes (%u%%)\n", stats->usedSpace,
usagePercent);
    printf("  Free:        %lu bytes\n", stats->freeSpace);
    printf("  Allocations: %lu\n", stats->allocationCount);
    printf("  Largest free block: %lu bytes\n", stats->largestFreeBlock);
    printf("  Fragmentation: %u%%\n", stats->fragmentationPercent);
    // Always free the stats structure xMemFree((Addr_t *)stats);
  }
}
```

Example 2: Monitor for fragmentation issues

```
#define FRAG_WARNING_THRESHOLD 30
#define FRAG_CRITICAL_THRESHOLD 50
void checkHeapFragmentation(void) {
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetHeapStats(&stats))) {
    if (stats->fragmentationPercent >= FRAG_CRITICAL_THRESHOLD) {
      logCritical("Heap fragmentation critical: %u%%",
stats->fragmentationPercent);
      // Consider xMemFreeAll() or defragmentation strategy
    } else if (stats->fragmentationPercent >= FRAG_WARNING_THRESHOLD) {
      logWarning("Heap fragmentation high: %u%%",
stats->fragmentationPercent);
    }
    xMemFree((Addr_t *)stats);
  }
}
```

Example 3: Check if allocation will succeed

```
 Return_t canAllocate(Size_t requestedSize) {
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetHeapStats(&stats))) {
    // Check if largest free block can satisfy request if
(stats->largestFreeBlock >= requestedSize) {
      xMemFree((Addr_t *)stats);
      return ReturnOK;  // Allocation likely to succeed
    }
    logWarning("Insufficient contiguous space: need %lu, have %lu",
requestedSize, stats->largestFreeBlock);
    xMemFree((Addr_t *)stats);
  }
  return ReturnError;
}
```

Example 4: Periodic heap health monitoring

```
#define HEAP_WARNING_THRESHOLD 85
void heapMonitorTask(Task_t *task, TaskParm_t *parm) {
  static Word_t lastUsed = 0;
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetHeapStats(&stats))) {
    Byte_t usagePercent = (Byte_t)((stats->usedSpace * 100) /
stats->totalSize);
    // Check for high usage if (usagePercent >= HEAP_WARNING_THRESHOLD) {
      logWarning("Heap usage high: %u%% (%lu/%lu bytes)", usagePercent,
stats->usedSpace, stats->totalSize);
    }
    // Check for memory leaks (continually increasing usage) if
(stats->usedSpace > lastUsed) {
      Word_t increase = stats->usedSpace - lastUsed;
      logInfo("Heap usage increased by %lu bytes", increase);
    }
    lastUsed = stats->usedSpace;
    xMemFree((Addr_t *)stats);
  }
}
```

Parameters

out	*stats*↩ _	Pointer to MemoryRegionStats_t *pointer that will receive the allocated statistics structure. Caller MUST free with xMemFree() after use.

Returns

ReturnOK if statistics retrieved successfully, ReturnError if operation failed (memory allocation error or invalid parameter).

Warning

The caller MUST free the returned statistics structure using xMemFree(). Failing to do so will cause a memory leak.

The statistics are a snapshot at call time. Memory state changes continuously, so values may be outdated immediately after return.

Calling this function allocates memory from the heap it's measuring, slightly affecting the reported statistics. The stats structure itself consumes heap space.

Note

For a quick usage check without detailed stats, use xMemGetUsed() instead, which is faster and doesn't allocate memory.

High fragmentation (>40%) indicates many small free blocks scattered throughout the heap, which can prevent large allocations even when total free space is adequate.

The largestFreeBlock value indicates the maximum allocation size that can succeed, regardless of total free space available.

This function provides heap statistics only. For kernel memory stats, use xMemGetKernelStats().

See also

xMemGetKernelStats() - Get kernel memory region statistics

xMemGetUsed() - Quick check of heap usage (no allocation)

xMemFree() - Free the statistics structure

xMemFreeAll() - Free all heap memory (may help with fragmentation)

xMemAlloc() - Allocate heap memory

4.2.4.37 xMemGetKernelStats() `Return_t xMemGetKernelStats ( MemoryRegionStats_t ** stats_ )`

Returns comprehensive statistics about the kernel memory region including total size, used/free space, and allocation metrics. Kernel memory is used exclusively by HeliOS for internal data structures like tasks, timers, queues, and streams, separate from user heap memory.

The function allocates a MemoryRegionStats_t *structure from the user heap (not kernel memory) and populates it with kernel memory metrics. The caller MUST free this structure with xMemFree() after use. The statistics represent a snapshot at call time.

Kernel memory is consumed by:

- **Task control blocks**: One per task created
- **Timers**: Task timers and watchdog timers
- **Queues**: Message queue structures and buffers

- **Streams**: Stream buffer control structures
- **Notifications**: Task notification state
- **Internal structures**: Scheduler and system data

Common use cases:

- **System capacity monitoring**: Tracking kernel resource usage
- **Leak detection**: Identifying kernel memory leaks
- **Capacity planning**: Determining if kernel memory is sufficient
- **Performance analysis**: Understanding kernel memory overhead
- **Diagnostics**: Troubleshooting kernel memory exhaustion

Example 1: Display kernel memory status

```
 void displayKernelMemoryStatus(void) {
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetKernelStats(&stats))) {
    Byte_t usagePercent = (Byte_t)((stats->usedSpace * 100) /
stats->totalSize);
    printf("Kernel Memory Status:\n");
    printf("  Total:       %lu bytes\n", stats->totalSize);
    printf("  Used:        %lu bytes (%u%%)\n", stats->usedSpace,
usagePercent);
    printf("  Free:        %lu bytes\n", stats->freeSpace);
    printf("  Allocations: %lu\n", stats->allocationCount);
    // Always free (from user heap, not kernel) xMemFree((Addr_t *)stats);
  }
}
```

Example 2: Compare heap vs kernel memory usage

```
 void compareMemoryRegions(void) {
  MemoryRegionStats_t **heapStats = NULL;
  MemoryRegionStats_t **kernelStats = NULL;
  if (OK(xMemGetHeapStats(&heapStats)) &&
      OK(xMemGetKernelStats(&kernelStats))) {
    printf("Memory Comparison:\n");
    printf("  Heap   - Used: %lu / Total: %lu (%u%%)\n",
heapStats->usedSpace, heapStats->totalSize, (Byte_t)((heapStats->usedSpace
*
100) / heapStats->totalSize));
    printf("  Kernel - Used: %lu / Total: %lu (%u%%)\n",
kernelStats->usedSpace, kernelStats->totalSize,
(Byte_t)((kernelStats->usedSpace * 100) / kernelStats->totalSize));
    xMemFree((Addr_t *)heapStats);
    xMemFree((Addr_t *)kernelStats);
  }
}
```

Example 3: Monitor kernel memory for leaks

```
 void monitorKernelMemory(void) {
  static Word_t lastUsed = 0;
  static Base_t lastAllocCount = 0;
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetKernelStats(&stats))) {
    // Check for increasing usage if (stats->usedSpace > lastUsed) {
      Word_t increase = stats->usedSpace - lastUsed;
      Base_t newAllocs = stats->allocationCount - lastAllocCount;
      logInfo("Kernel memory increased: +%lu bytes, +%u allocations",
increase, newAllocs);
      // Unexpected growth might indicate a leak if (newAllocs == 0 &&
increase > 0) {
        logWarning("Kernel memory grew without new allocations!");
      }
    }
    lastUsed = stats->usedSpace;
    lastAllocCount = stats->allocationCount;
    xMemFree((Addr_t *)stats);
  }
}
```

Example 4: Check if kernel memory available for new resources

```
#define KERNEL_MIN_FREE_BYTES 512
Return_t canCreateKernelResource(void) {
  MemoryRegionStats_t **stats = NULL;
  if (OK(xMemGetKernelStats(&stats))) {
    if (stats->freeSpace < KERNEL_MIN_FREE_BYTES) {
      logWarning("Kernel memory low: %lu bytes free (min: %u)",
stats->freeSpace, KERNEL_MIN_FREE_BYTES);
      xMemFree((Addr_t *)stats);
      return ReturnError;
    }
    xMemFree((Addr_t *)stats);
    return ReturnOK;
  }
  return ReturnError;
}
```

Parameters

out	*stats*↩ _	Pointer to MemoryRegionStats_t *pointer that will receive the allocated statistics structure. Allocated from user heap. Caller MUST free with xMemFree() after use.

Returns

ReturnOK if statistics retrieved successfully, ReturnError if operation failed (memory allocation error or invalid parameter).

Warning

The caller MUST free the returned statistics structure using xMemFree(). The structure is allocated from user heap, not kernel memory.

Kernel memory statistics are a snapshot. Values change as tasks, queues, timers, and other kernel objects are created or destroyed.

Running out of kernel memory prevents creating new tasks, queues, timers, and other kernel objects. Monitor kernel usage to avoid exhaustion.

Note

The statistics structure itself is allocated from user heap, NOT kernel memory. Only the statistics content describes kernel memory.

Kernel memory size is configured at compile time via CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS. It cannot be changed at runtime.

Unlike user heap, kernel memory is typically not fragmented because kernel objects are usually longer-lived and not freed frequently.

Kernel memory usage increases primarily when creating tasks, queues, timers, or streams. Deleting these objects frees kernel memory.

See also

xMemGetHeapStats() - Get user heap memory statistics

xMemFree() - Free the statistics structure (from user heap)

xTaskCreate() - Creates task (uses kernel memory)

xQueueCreate() - Creates queue (uses kernel memory)

CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS - Kernel memory size configuration

4.2.4.38 xMemGetSize() `Return_t xMemGetSize (`

```
const volatile Addr_t * addr_,
Size_t * size_ )
```

Returns the size in bytes of a memory block at the specified address. The address must be a valid pointer previously returned by xMemAlloc() or a HeliOS function that allocates memory. This allows applications to determine allocation sizes at runtime, useful for dynamic buffer management, serialization, and memory accounting.

The returned size is the exact number of bytes originally requested during allocation, not including any internal heap management overhead. This matches the size parameter originally passed to xMemAlloc().

Common use cases:

- **Dynamic buffer handling**: Determining buffer capacity without tracking size separately
- **Serialization**: Knowing data structure size for network transmission or storage
- **Memory accounting**: Calculating per-object memory usage
- **Validation**: Verifying allocation size before operations
- **Debugging**: Inspecting allocation sizes during development

Example 1: Generic buffer processing without size tracking

```
 void processBuffer(Byte_t *buffer) {
  Size_t bufferSize;
  // Discover buffer size dynamically if (OK(xMemGetSize((Addr_t *)buffer,
&bufferSize))) {
    // Process up to bufferSize bytes for (Size_t i = 0; i < bufferSize;
i++) {
      processData(buffer[i]);
    }
  }
}
```

Example 2: Safe buffer copy with size verification

```
 Return_t safeCopy(Byte_t *dest, const Byte_t *src, Size_t copySize) {
  Size_t destSize, srcSize;
  // Verify destination has enough space if (ERROR(xMemGetSize((Addr_t
*)dest,
&destSize))) {
    return ReturnError;
  }
  // Verify source has enough data if (ERROR(xMemGetSize((Addr_t *)src,
&srcSize))) {
    return ReturnError;
  }
  // Check bounds if (copySize > destSize || copySize > srcSize) {
    logError("Copy size exceeds buffer capacity");
    return ReturnError;
  }
  // Safe to copy memcpy(dest, src, copySize);
  return ReturnOK;
}
```

Example 3: Serialization with automatic size detection

```
 Return_t serializeToStream(Byte_t *data, StreamBuffer_t *stream) {
  Size_t dataSize;
  // Get actual data size if (ERROR(xMemGetSize((Addr_t *)data,
&dataSize))) {
    return ReturnError;
  }
  // Send size header first xStreamSend(stream, (Byte_t)(dataSize » 8));
  // High byte xStreamSend(stream, (Byte_t)(dataSize & 0xFF)); // Low byte
  // Send data for (Size_t i = 0; i < dataSize; i++) {
    if (ERROR(xStreamSend(stream, data[i]))) {
      return ReturnError;
    }
  }
  return ReturnOK;
}
```

Example 4: Memory usage reporting for debug

```
 typedef struct {
```

```
  Byte_t *rxBuffer;
  Byte_t *txBuffer;
  Byte_t *workBuffer;
} CommBuffers_t;
void reportBufferUsage(CommBuffers_t *buffers) {
  Size_t rxSize, txSize, workSize;
  xMemGetSize((Addr_t *)buffers->rxBuffer, &rxSize);
  xMemGetSize((Addr_t *)buffers->txBuffer, &txSize);
  xMemGetSize((Addr_t *)buffers->workBuffer, &workSize);
  Size_t totalBufferMemory = rxSize + txSize + workSize;
  printf("Communication Buffer Memory:\n");
  printf("  RX Buffer:   %lu bytes\n", rxSize);
  printf("  TX Buffer:   %lu bytes\n", txSize);
  printf("  Work Buffer: %lu bytes\n", workSize);
  printf("  Total:       %lu bytes\n", totalBufferMemory);
}
```

Parameters

in	*addr_*	Pointer to the memory block. Must be a valid address previously returned by xMemAlloc() or a HeliOS allocation function.
out	*size_*	Pointer to variable that receives the size in bytes of the allocation at addr_.

Returns

ReturnOK if size query succeeded, ReturnError if the operation failed (invalid address, address not from heap, or memory system error).

Warning

The address must be a valid heap pointer returned by xMemAlloc() or a HeliOS allocation function. Passing stack addresses, static addresses, or already-freed addresses will result in ReturnError.

Do not call xMemGetSize() on addresses that have been freed with xMemFree() or invalidated by xMemFreeAll(). The result is undefined and will likely return ReturnError.

Note

The returned size is the user-requested allocation size, not the total memory consumed including heap overhead. Actual heap usage may be slightly larger due to alignment and metadata.

Passing NULL for addr_ is safe and will return ReturnError with size_ unmodified.

This function is useful when working with buffers returned by HeliOS functions (like xDeviceRead()) where you receive a pointer but need to know its size.

See also

xMemAlloc() - Allocate memory with known size

xMemGetUsed() - Get total heap usage

xMemGetHeapStats() - Get comprehensive heap statistics

xMemFree() - Free memory block

4.2.4.39 xMemGetUsed() `Return_t xMemGetUsed (`

`Size_t * size_ )`

Returns the total number of bytes currently allocated from the user heap across all active allocations. This provides a quick snapshot of heap utilization without the overhead of detailed statistics, making it ideal for runtime memory monitoring, leak detection, and capacity planning.

The returned value represents the sum of all memory blocks allocated via xMemAlloc() or HeliOS functions that allocate memory, excluding any internal heap management overhead. This gives an accurate picture of actual application memory consumption.

Common use cases:

- **Memory monitoring**: Tracking heap usage over time to detect trends
- **Leak detection**: Comparing usage before and after operations to find leaks
- **Capacity planning**: Determining if more heap space is needed
- **Threshold alerts**: Triggering warnings when usage exceeds limits
- **Performance tuning**: Identifying memory-intensive operations

Example 1: Monitor heap usage in diagnostic task

```
 void memoryMonitorTask(Task_t *task, TaskParm_t *parm) {
  Size_t usedBytes;
  Size_t totalHeap = CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS *
                     CONFIG_MEMORY_REGION_BLOCK_SIZE;
  if (OK(xMemGetUsed(&usedBytes))) {
    Byte_t percentUsed = (Byte_t)((usedBytes * 100) / totalHeap);
    if (percentUsed > 90) {
      logWarning("Heap usage critical: %u%% (%lu / %lu bytes)",
percentUsed, usedBytes, totalHeap);
    } else if (percentUsed > 75) {
      logInfo("Heap usage high: %u%%", percentUsed);
    }
  }
}
```

Example 2: Detect memory leaks

```
 Return_t checkForLeaks(void) {
  Size_t beforeSize, afterSize;
  // Record usage before operation if (ERROR(xMemGetUsed(&beforeSize))) {
    return ReturnError;
  }
  // Perform operation that should clean up after itself
performOperation();
  // Check usage after operation if (OK(xMemGetUsed(&afterSize))) {
    if (afterSize > beforeSize) {
      Size_t leaked = afterSize - beforeSize;
      logError("Memory leak detected: %lu bytes not freed", leaked);
      return ReturnError;
    }
  }
  return ReturnOK;
}
```

Example 3: Pre-allocation size check

```
 Return_t allocateBuffer(Byte_t **buffer, Size_t requestedSize) {
  Size_t currentUsage;
  Size_t totalHeap = CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS *
                     CONFIG_MEMORY_REGION_BLOCK_SIZE;
  // Check if allocation would exceed safe threshold if
(OK(xMemGetUsed(&currentUsage))) {
    if (currentUsage + requestedSize > (totalHeap * 85 / 100)) {
      logWarning("Allocation would exceed 85%% heap capacity");
      return ReturnError;
    }
  }
  // Proceed with allocation return xMemAlloc((volatile Addr_t **)buffer,
requestedSize);
}
```

Example 4: Runtime memory statistics reporting

```
void reportMemoryStatus(void) {
  Size_t usedBytes;
  Size_t totalBytes = CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS *
                      CONFIG_MEMORY_REGION_BLOCK_SIZE;
  if (OK(xMemGetUsed(&usedBytes))) {
    Size_t freeBytes = totalBytes - usedBytes;
    Byte_t percentFree = (Byte_t)((freeBytes * 100) / totalBytes);
    printf("Memory Status:\n");
    printf("  Total:     %lu bytes\n", totalBytes);
    printf("  Used:      %lu bytes\n", usedBytes);
    printf("  Free:      %lu bytes (%u%%)\n", freeBytes, percentFree);
  }
}
```

Parameters

out	*size*↩ _	Pointer to variable that receives the total number of bytes currently allocated from the heap.

Returns

ReturnOK if usage query succeeded, ReturnError if the operation failed (invalid parameter or memory system error).

Note

The returned value represents allocated memory only. It does not include heap management overhead (block headers, alignment padding, etc.).

This function is very fast as it simply returns a tracked counter. Call it frequently for monitoring without performance concerns.

For more detailed information including fragmentation statistics, free space breakdown, and allocation counts, use xMemGetHeapStats() instead.

The value returned is a snapshot at the moment of the call. Concurrent allocations or frees by other tasks may change the value immediately after this function returns.

See also

xMemGetHeapStats() - Get comprehensive heap statistics

xMemGetSize() - Get size of a specific allocation

xMemAlloc() - Allocate memory (increases used bytes)

xMemFree() - Free memory (decreases used bytes)

xMemFreeAll() - Free all memory (resets used bytes to zero)

4.2.4.40 xQueueCreate()

```
Return_t xQueueCreate (
        Queue_t ** queue_,
        const Base_t limit_ )
```

Creates a message queue that enables tasks to exchange data safely in a producer-consumer pattern. Queues provide a First-In-First-Out (FIFO) mechanism for passing messages between tasks, making them ideal for decoupling tasks and implementing asynchronous communication.

Message queues in HeliOS store variable-length byte arrays, allowing flexible message formats. Each queue has a configurable maximum capacity (limit), and operations fail gracefully when the queue is full or empty.

Queue Characteristics:

- FIFO ordering: Messages are retrieved in the order they were sent
- Bounded capacity: Queue has a maximum message limit
- Non-blocking: Send and receive operations return immediately
- Thread-safe: Can be safely accessed from multiple tasks
- Variable message size: Each message can be a different size

Common Use Cases:

- Passing sensor data from interrupt handlers to processing tasks
- Command queues for serial communication
- Event queues for state machines
- Data buffering between producer and consumer tasks
- Decoupling hardware drivers from application logic

Queue Operation Flow:

1. Producer task sends message with xQueueSend() 2. Message is stored in queue if space available 3. Consumer task receives message with xQueueReceive() 4. Message is removed from queue

Note

The limit parameter must be at least CONFIG_QUEUE_MINIMUM_LIMIT (default 5). This ensures reasonable queue capacity and prevents overly small queues that could cause frequent overflow conditions.

Messages are copied into the queue, so the original data buffer can be reused or freed after calling xQueueSend(). Similarly, xQueueReceive() provides a copy of the message.

Queues can be locked with xQueueLockQueue() to prevent modifications during critical operations, then unlocked with xQueueUnLockQueue().

Example Usage:

```
 Queue_t *sensorDataQueue;
Queue_t *commandQueue;
// Create a queue that can hold up to 10 messages if
(OK(xQueueCreate(&sensorDataQueue, 10))) {
  // Queue created successfully
  // Producer task: Send sensor readings void sensorTask(Task_t *task,
TaskParm_t *parm) {
    uint8_t sensorData[4];
    readSensor(sensorData, sizeof(sensorData));
    // Send to queue Size_t dataSize = sizeof(sensorData);
    if (OK(xQueueSend(sensorDataQueue, dataSize, sensorData))) {
      // Data queued successfully
    } else {
      // Queue full - handle overflow
    }
  }
  // Consumer task: Process sensor readings void processingTask(Task_t
task, TaskParm_t *parm) {
    QueueMessage_t *message;
    // Check if messages available Base_t messagesWaiting;
    if (OK(xQueueMessagesWaiting(sensorDataQueue, &messagesWaiting))) {
      if (messagesWaiting > 0) {
        // Receive message if (OK(xQueueReceive(sensorDataQueue,
&message))) {
          // Process the data processSensorData(message.message,
message.size);
        }
      }
    }
  }
}
// Create a command queue if (OK(xQueueCreate(&commandQueue, 5))) {
  // Use queue for command processing...
  // Clean up when done xQueueDelete(commandQueue);
}
```

Parameters

out	*queue_*	Pointer to Queue_t *variable that will receive the queue handle. This handle is used in subsequent queue operations.
in	*limit_*	Maximum number of messages the queue can hold. Must be at least CONFIG_QUEUE_MINIMUM_LIMIT (default 5). When this limit is reached, the queue is full and xQueueSend() will fail.

Returns

ReturnOK if the queue was successfully created, ReturnError if creation failed (out of memory, invalid limit, or system error).

See also

xQueueDelete() - Delete a queue and free its resources

xQueueSend() - Send a message to a queue

xQueueReceive() - Receive a message from a queue

xQueuePeek() - Examine next message without removing it

xQueueMessagesWaiting() - Get number of messages in queue

xQueueIsQueueFull() - Check if queue is at capacity

xQueueIsQueueEmpty() - Check if queue has no messages

xQueueLockQueue() - Lock queue to prevent modifications

CONFIG_QUEUE_MINIMUM_LIMIT - Minimum queue capacity configuration

4.2.4.41 xQueueDelete() `Return_t xQueueDelete ( Queue_t * queue_ )`

Permanently removes a message queue and releases all associated kernel resources, including all queued messages. After deletion, the queue handle becomes invalid and must not be used in any subsequent queue operations. This operation is typically performed during cleanup, reconfiguration, or when a communication channel is no longer needed.

xQueueDelete() immediately removes the queue and all its messages from the kernel, freeing both the queue structure and all message memory. Any messages that were waiting in the queue are lost—there is no mechanism to retrieve them after deletion. Tasks attempting to send or receive on a deleted queue will receive ReturnError.

Key characteristics:

- **Immediate deletion**: Queue and all messages removed immediately
- **Message loss**: All pending messages are discarded
- **Resource cleanup**: All kernel memory associated with queue is freed
- **Handle invalidation**: Queue handle cannot be reused after deletion
- **Non-blocking**: Returns immediately after cleanup completes

Common deletion scenarios:

- **Task cleanup**: Remove queues during task shutdown
- **Dynamic reconfiguration**: Delete and recreate queues with different limits
- **Resource reclamation**: Free unused queues to reduce memory usage
- **Error recovery**: Clean up queues after initialization failures

Example 1: Queue lifecycle management

```
 Queue_t *commandQueue;
void initCommandQueue(void) {
  if (OK(xQueueCreate(&commandQueue, 10))) {
    // Use queue for communication
  }
}
void shutdownCommandQueue(void) {
  // Delete queue when no longer needed if (OK(xQueueDelete(commandQueue)))
{
    // Queue and all messages freed
  }
}
```

Example 2: Dynamic queue reconfiguration

```
 Queue_t *dataQueue;
void resizeQueue(Base_t newLimit) {
  // Delete existing queue if (dataQueue != null) {
    xQueueDelete(dataQueue);
  }
  // Create new queue with different limit if (OK(xQueueCreate(&dataQueue,
newLimit))) {
    // New queue ready
  }
}
```

Example 3: Multiple queue cleanup

```
 Queue_t *queues[MAX_CHANNELS];
Base_t queueCount = 0;
void initQueues(void) {
  for (Base_t i = 0; i < MAX_CHANNELS; i++) {
    if (OK(xQueueCreate(&queues[i], 5))) {
      queueCount++;
    }
  }
}
void shutdownQueues(void) {
  for (Base_t i = 0; i < queueCount; i++) {
    xQueueDelete(queues[i]);
  }
  queueCount = 0;
}
```

Example 4: Conditional queue cleanup

```
 Queue_t *eventQueue;
Base_t queueActive = 0;
void disableEvents(void) {
  if (queueActive) {
    xQueueDelete(eventQueue);
    queueActive = 0;
  }
}
void enableEvents(void) {
  if (!queueActive) {
    if (OK(xQueueCreate(&eventQueue, 8))) {
      queueActive = 1;
    }
  }
}
```

Parameters

in	*queue*↩_	Handle to the queue to delete. Must be a valid queue created with xQueueCreate(). After deletion, this handle becomes invalid.

Returns

ReturnOK if queue deleted successfully, ReturnError if deletion failed due to invalid queue handle or queue not found.

Warning

All messages pending in the queue are permanently lost when the queue is deleted. If you need to preserve messages, drain the queue with xQueueReceive() before calling xQueueDelete().

After xQueueDelete() returns successfully, the queue handle is invalid and must not be used in any subsequent operations. Using a deleted queue handle will result in ReturnError.

Deleting a queue that is actively used by multiple tasks can lead to errors if those tasks attempt to access the queue after deletion. Coordinate queue deletion across all tasks using the queue.

Note

xQueueDelete() frees all memory associated with the queue, including the queue structure and all message memory. This is the only way to reclaim kernel memory allocated for a queue.

Unlike stopping a timer, which preserves the timer for reuse, deleting a queue permanently removes it. To reuse queue functionality, create a new queue with xQueueCreate().

See also

xQueueCreate() - Create a message queue

xQueueReceive() - Receive messages before deletion

xQueueMessagesWaiting() - Check for pending messages

xTaskDelete() - Delete a task (similar resource cleanup pattern)

xTimerDelete() - Delete a timer (similar resource cleanup pattern)

4.2.4.42 xQueueDropMessage() `Return_t xQueueDropMessage ( Queue_t * queue_ )`

Removes and discards the oldest message from the queue without retrieving its contents. This operation decrements the message count and frees the message memory, but does not return the message data to the caller. Use this when you want to skip or discard messages without processing them.

Unlike xQueueReceive() which allocates and returns the message, xQueueDropMessage() simply removes the message and frees its memory immediately. This is more efficient when message content is not needed—for example, when clearing stale messages, handling overflow conditions, or implementing message filtering.

Common use cases:

- **Queue clearing**: Discard all messages when resetting state
- **Message filtering**: Skip unwanted messages after peeking
- **Overflow handling**: Drop old messages when queue backs up
- **Selective processing**: Discard messages based on peek inspection

Example 1: Clear all messages from queue

```
 Queue_t *eventQueue;
void clearQueue(void) {
  Base_t isEmpty;
  if (OK(xQueueIsQueueEmpty(eventQueue, &isEmpty))) {
    while (!isEmpty) {
      xQueueDropMessage(eventQueue);
      xQueueIsQueueEmpty(eventQueue, &isEmpty);
    }
  }
}
```

Example 2: Selective message processing with peek and drop

```
 Queue_t *dataQueue;
void processOnlyValidMessages(void) {
  Base_t waiting;
  if (OK(xQueueMessagesWaiting(dataQueue, &waiting))) {
    for (Base_t i = 0; i < waiting; i++) {
      QueueMessage_t *msg;
      if (OK(xQueuePeek(dataQueue, &msg))) {
        Byte_t msgType = msg.value[0];
        xMemFree((Addr_t *)msg.value);  // Free peek copy
        if (isValidMessageType(msgType)) {
          // Valid - receive and process if (OK(xQueueReceive(dataQueue,
&msg))) {
            processMessage(&msg);
            xMemFree((Addr_t *)msg.value);
          }
        } else {
          // Invalid - drop without retrieving
xQueueDropMessage(dataQueue);
        }
      }
    }
  }
}
```

Example 3: Drop stale messages on timeout

```
 Queue_t *timeStampedQueue;
Timer_t *timeoutTimer;
void dropStaleMessages(void) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(timeoutTimer, &expired)) && expired) {
    // Timeout elapsed - drop oldest message
xQueueDropMessage(timeStampedQueue);
    xTimerReset(timeoutTimer);
  }
}
```

Parameters

in	*queue*↩ _	Handle to the queue from which to drop a message. Must be a valid queue created with xQueueCreate().

Returns

ReturnOK if message dropped successfully, ReturnError if operation failed due to empty queue or invalid queue handle.

Warning

The dropped message is permanently lost—there is no way to retrieve it after calling xQueueDropMessage(). If you need the message content, use xQueueReceive() instead.

Note

xQueueDropMessage() is more efficient than xQueueReceive() followed by xMemFree() because it avoids allocating memory for the message copy.

If the queue is empty, xQueueDropMessage() returns ReturnError immediately.

See also

xQueueReceive() - Receive and retrieve message

xQueuePeek() - Examine message without removing it

xQueueMessagesWaiting() - Check message count before dropping

xQueueIsQueueEmpty() - Check if queue has messages

4.2.4.43 xQueueGetLength() `Return_t xQueueGetLength (`

```
const Queue_t * queue_,
Base_t * res_ )
```

Retrieves the maximum number of messages (limit) that a queue can hold, as configured during queue creation with xQueueCreate(). This non-destructive query returns the queue's capacity, not the number of messages currently waiting. The limit represents the upper bound on how many messages can be queued before xQueueSend() returns ReturnError due to queue full.

Note: Despite the name "GetLength", this function returns the queue's maximum capacity (limit), not the current message count. To get the number of messages currently waiting, use xQueueMessagesWaiting().

Common use cases:

- **Capacity validation**: Verify queue created with expected limit
- **Flow control**: Calculate available space before bulk sends
- **Diagnostics**: Report queue configuration for debugging
- **Dynamic adjustment**: Determine if queue needs resizing

Example 1: Validate queue configuration

```
 Queue_t *dataQueue;
Base_t expectedLimit = 10;
if (OK(xQueueCreate(&dataQueue, expectedLimit))) {
  Base_t actualLimit;
  if (OK(xQueueGetLength(dataQueue, &actualLimit))) {
    if (actualLimit == expectedLimit) {
      // Queue configured correctly
    }
  }
}
```

Example 2: Calculate available queue space

```
 Queue_t *commandQueue;
Base_t getAvailableSpace(void) {
  Base_t limit, waiting;
  if (OK(xQueueGetLength(commandQueue, &limit)) &&
      OK(xQueueMessagesWaiting(commandQueue, &waiting))) {
    return limit - waiting;
  }
  return 0;
}
```

Example 3: Queue utilization monitoring

```
 Queue_t *eventQueue;
void reportQueueStatus(void) {
  Base_t limit, waiting;
  if (OK(xQueueGetLength(eventQueue, &limit)) &&
      OK(xQueueMessagesWaiting(eventQueue, &waiting))) {
    Base_t utilization = (waiting * 100) / limit;
    printf("Queue: %d/%d messages (%d%% full)\n", waiting, limit,
utilization);
  }
}
```

Parameters

in	*queue_*	Handle to the queue to query. Must be a valid queue created with xQueueCreate().
out	*res_*	Pointer to variable receiving the queue limit (maximum capacity). On success, contains the limit value specified during xQueueCreate().

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid queue handle or queue not found.

Warning

This function returns the queue's maximum capacity (limit), NOT the current number of messages. Use xQueueMessagesWaiting() to get the current message count.

Note

The returned limit is constant for the lifetime of the queue—it cannot be changed after creation. To change capacity, delete and recreate the queue.

See also

xQueueMessagesWaiting() - Get current message count

xQueueIsQueueFull() - Check if queue is at capacity

xQueueIsQueueEmpty() - Check if queue has no messages

xQueueCreate() - Create queue with specified limit

4.2.4.44 xQueueIsQueueEmpty()

```
Return_t xQueueIsQueueEmpty (
    const Queue_t * queue_,
    Base_t * res_ )
```

Queries whether a message queue is empty (contains zero messages). This non-destructive check allows tasks to determine if messages are available before attempting to receive, enabling conditional receive logic and avoiding unnecessary xQueueReceive() calls on empty queues. A queue is considered empty when it contains no messages, regardless of its maximum capacity.

Common use cases:

- **Conditional receive**: Only call xQueueReceive() if messages available
- **Flow control**: Check for messages before processing
- **Diagnostics**: Monitor queue activity
- **Polling loops**: Detect when queue has been drained

Example 1: Conditional message processing

```
 Queue_t *eventQueue;
void processEvents(void) {
  Base_t isEmpty;
  if (OK(xQueueIsQueueEmpty(eventQueue, &isEmpty)) && !isEmpty) {
    // Queue has messages - process them QueueMessage_t *msg;
    while (OK(xQueueReceive(eventQueue, &msg))) {
      handleEvent(&msg);
      xMemFree((Addr_t *)msg.value);
    }
  }
}
```

Example 2: Wait for messages

```
 Queue_t *dataQueue;
void waitForData(void) {
  Base_t isEmpty = 1;
  // Poll until message arrives while (isEmpty) {
    if (OK(xQueueIsQueueEmpty(dataQueue, &isEmpty))) {
      if (isEmpty) {
        xTaskWait(10);  // Wait before checking again
      }
    }
  }
  // Message available - process it QueueMessage_t *msg;
  if (OK(xQueueReceive(dataQueue, &msg))) {
    processData(&msg);
    xMemFree((Addr_t *)msg.value);
  }
}
```

Example 3: Drain queue completely

```
 Queue_t *commandQueue;
void drainQueue(void) {
  Base_t isEmpty;
  if (OK(xQueueIsQueueEmpty(commandQueue, &isEmpty))) {
    while (!isEmpty) {
      QueueMessage_t *msg;
      if (OK(xQueueReceive(commandQueue, &msg))) {
        xMemFree((Addr_t *)msg.value);
      }
      xQueueIsQueueEmpty(commandQueue, &isEmpty);
    }
  }
}
```

Parameters

in	*queue_*	Handle to the queue to query. Must be a valid queue created with xQueueCreate().
out	*res_*	Pointer to variable receiving the empty status. Set to non-zero (true) if queue is empty, zero (false) if queue contains one or more messages.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid queue handle or queue not found.

Note

This is a snapshot query—in multitasking environments, the queue state may change immediately after the check if other tasks send messages.

xQueueIsQueueEmpty() is equivalent to checking if xQueueMessagesWaiting() returns 0, but provides a more readable API for empty checks.

See also

xQueueMessagesWaiting() - Get exact message count

xQueueIsQueueFull() - Check if queue is at capacity

xQueueReceive() - Receive message from queue

xQueuePeek() - Check message without removing it

4.2.4.45 xQueueIsQueueFull() `Return_t xQueueIsQueueFull (`

```
const Queue_t * queue_,
Base_t * res_ )
```

Queries whether a message queue is full (at maximum capacity). This check allows tasks to determine if there is space available before attempting to send, enabling flow control and preventing message loss. A queue is considered full when the number of waiting messages equals the limit specified during xQueueCreate(). When full, xQueueSend() will return ReturnError.

Common use cases:

- **Conditional send**: Only send if space available
- **Flow control**: Back off when queue approaches capacity
- **Overflow prevention**: Detect and handle full queue conditions
- **Diagnostics**: Monitor queue utilization and congestion

Example 1: Conditional message send

```
 Queue_t *commandQueue;
Return_t sendCommand(Byte_t *cmd, Base_t size) {
  Base_t isFull;
  if (OK(xQueueIsQueueFull(commandQueue, &isFull))) {
    if (isFull) {
      logWarning("Command queue full - dropping command");
      return ReturnError;
    }
    // Space available - send message return xQueueSend(commandQueue, size,
cmd);
  }
  return ReturnError;
}
```

Example 2: Flow control with backoff

```
 Queue_t *dataQueue;
void sendDataWithBackoff(Byte_t *data, Base_t size) {
  Base_t isFull;
  Base_t retries = 0;
  while (retries < 10) {
    if (OK(xQueueIsQueueFull(dataQueue, &isFull))) {
      if (!isFull) {
        // Space available - send now if (OK(xQueueSend(dataQueue, size,
data))) {
          return;
        }
      }
    }
    // Queue full - wait and retry xTaskWait(10);
    retries++;
  }
  logError("Failed to send data - queue full");
}
```

Example 3: Monitor queue pressure

```
 Queue_t *eventQueue;
void monitorQueuePressure(void) {
  Base_t isFull, waiting;
  if (OK(xQueueIsQueueFull(eventQueue, &isFull)) && isFull) {
    logWarning("Event queue at capacity");
  } else if (OK(xQueueMessagesWaiting(eventQueue, &waiting))) {
    Base_t limit;
    if (OK(xQueueGetLength(eventQueue, &limit))) {
      if (waiting > (limit * 80) / 100) {
        logWarning("Event queue nearly full: %d/%d", waiting, limit);
      }
    }
  }
}
```

Parameters

in	*queue_*	Handle to the queue to query. Must be a valid queue created with xQueueCreate().
out	*res_*	Pointer to variable receiving the full status. Set to non-zero (true) if queue is full, zero (false) if queue has space for more messages.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid queue handle or queue not found.

Note

This is a snapshot query—in multitasking environments, the queue state may change immediately after the check if other tasks receive messages.

xQueueIsQueueFull() checks if message count equals the limit. To check for nearly-full conditions, use xQueueMessagesWaiting() and compare to xQueueGetLength().

See also

xQueueMessagesWaiting() - Get exact message count

xQueueIsQueueEmpty() - Check if queue has no messages

xQueueGetLength() - Get queue capacity

xQueueSend() - Send message (fails when queue is full)

4.2.4.46 xQueueLockQueue() `Return_t xQueueLockQueue ( Queue_t * queue_ )`

Places a queue in locked state, preventing tasks from sending new messages with xQueueSend(). While locked, send operations return ReturnError immediately. However, locked queues still allow receiving, peeking, and dropping messages— only send operations are blocked. This enables controlled queue drainage and prevents queue overflow during critical processing.

Queue locking is useful for implementing flow control, preventing message accumulation during batch processing, or ensuring a queue is drained before reconfiguration. Locks must be explicitly released with xQueueUnLockQueue() to resume normal operation—they do not timeout or automatically unlock.

Common use cases:

- **Batch processing**: Lock queue while processing all pending messages
- **Flow control**: Prevent producer overrun during high processing load
- **Queue drainage**: Ensure queue is empty before shutdown or reconfiguration
- **Critical sections**: Prevent message accumulation during critical operations

Example 1: Batch processing with lock

```
 Queue_t *eventQueue;
void processBatch(void) {
  // Lock to prevent new messages during batch xQueueLockQueue(eventQueue);
  // Process all current messages Base_t isEmpty;
  if (OK(xQueueIsQueueEmpty(eventQueue, &isEmpty))) {
    while (!isEmpty) {
      QueueMessage_t *msg;
      if (OK(xQueueReceive(eventQueue, &msg))) {
        handleEvent(&msg);
        xMemFree((Addr_t *)msg.value);
      }
      xQueueIsQueueEmpty(eventQueue, &isEmpty);
    }
  }
  // Unlock to resume message sending xQueueUnLockQueue(eventQueue);
}
```

Example 2: Controlled shutdown

```
 Queue_t *commandQueue;
void shutdownCommandProcessor(void) {
  // Lock queue to prevent new commands xQueueLockQueue(commandQueue);
  // Process remaining commands QueueMessage_t *msg;
  while (OK(xQueueReceive(commandQueue, &msg))) {
    executeCommand(&msg);
    xMemFree((Addr_t *)msg.value);
  }
  // Delete queue (no need to unlock) xQueueDelete(commandQueue);
}
```

Parameters

in	*queue*↩ _	Handle to the queue to lock. Must be a valid queue created with xQueueCreate().

Returns

ReturnOK if queue locked successfully, ReturnError if operation failed due to invalid queue handle or queue not found.

Warning

Locking a queue does NOT prevent receiving, peeking, or dropping messages—only xQueueSend() is blocked. Consumers can still drain the queue while it is locked.

Locks must be explicitly released with xQueueUnLockQueue(). There is no automatic timeout or unlock mechanism.

Note

Locking an already-locked queue is safe and has no effect.

xQueueSend() on a locked queue returns ReturnError immediately without blocking or queuing the message.

See also

xQueueUnLockQueue() - Unlock queue to resume sending

xQueueSend() - Send message (fails on locked queue)

xQueueReceive() - Receive message (works on locked queue)

4.2.4.47 xQueueMessagesWaiting() Return_t xQueueMessagesWaiting (const Queue_t * *queue_*, Base_t * *res_*)

Retrieves the count of messages currently waiting in a queue. This non-destructive query provides precise queue occupancy information, allowing tasks to make informed decisions about message processing, flow control, and resource allocation. The count represents messages successfully sent with xQueueSend() but not yet received with xQueueReceive().

Common use cases:

- **Batch processing**: Process multiple messages when threshold reached
- **Queue utilization**: Monitor queue load and congestion
- **Flow control**: Throttle senders based on queue depth
- **Diagnostics**: Report queue activity for debugging

Example 1: Batch message processing

```
 Queue_t *eventQueue;
#define BATCH_THRESHOLD 5
void processBatchEvents(void) {
  Base_t waiting;
  if (OK(xQueueMessagesWaiting(eventQueue, &waiting))) {
    if (waiting >= BATCH_THRESHOLD) {
      // Process messages in batch for efficiency for (Base_t i = 0; i <
waiting; i++) {
        QueueMessage_t *msg;
        if (OK(xQueueReceive(eventQueue, &msg))) {
          handleEvent(&msg);
          xMemFree((Addr_t *)msg.value);
        }
      }
    }
  }
}
```

Example 2: Queue utilization monitoring

```
 Queue_t *commandQueue;
void reportQueueStats(void) {
  Base_t waiting, limit;
  if (OK(xQueueMessagesWaiting(commandQueue, &waiting)) &&
      OK(xQueueGetLength(commandQueue, &limit))) {
    Base_t utilization = (waiting * 100) / limit;
    printf("Queue: %d/%d messages (%d%% full)\n", waiting, limit,
utilization);
    if (utilization > 90) {
      logWarning("Queue nearly full - consider increasing capacity");
    }
  }
}
```

Example 3: Producer flow control

```
 Queue_t *dataQueue;
#define HIGH_WATER_MARK 8
#define LOW_WATER_MARK 3
static Base_t producerThrottled = 0;
void produceData(Byte_t *data, Base_t size) {
  Base_t waiting;
  if (OK(xQueueMessagesWaiting(dataQueue, &waiting))) {
    // Implement hysteresis for flow control if (waiting >=
HIGH_WATER_MARK) {
      producerThrottled = 1;
      logInfo("Producer throttled - queue depth: %d", waiting);
    } else if (waiting <= LOW_WATER_MARK) {
      producerThrottled = 0;
    }
    if (!producerThrottled) {
      xQueueSend(dataQueue, size, data);
    }
  }
}
```

Parameters

in	*queue_*	Handle to the queue to query. Must be a valid queue created with xQueueCreate().
out	*res_*	Pointer to variable receiving the message count. On success, contains the number of messages currently waiting (0 to limit).

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid queue handle or queue not found.

Note

This is a snapshot query—in multitasking environments, the message count may change immediately after the query if other tasks send or receive messages.

The count includes all messages from the head to the tail of the queue. Messages accessed with xQueuePeek() are still counted as waiting.

See also

xQueueIsQueueEmpty() - Check if count is zero (no messages)

xQueueIsQueueFull() - Check if count equals limit (at capacity)

xQueueGetLength() - Get queue maximum capacity

xQueueReceive() - Remove and receive next message

xQueueSend() - Add message to queue

4.2.4.48 xQueuePeek()

```
Return_t xQueuePeek (
    const Queue_t * queue_,
    QueueMessage_t ** message_ )
```

Retrieves a copy of the oldest message from the queue without removing it, allowing inspection of queue contents without consuming messages. Unlike xQueueReceive(), which removes the message, xQueuePeek() leaves the queue unchanged—the message remains available for subsequent peek or receive operations. This enables conditional message processing and queue inspection.

The peeked message is allocated from the heap as a copy—the original message stays in the queue. The caller must free the peeked message copy with xMemFree() after use. Multiple peek calls return the same message until it is removed with xQueueReceive() or xQueueDropMessage().

Common use cases:

- **Conditional processing**: Inspect message before deciding to receive
- **Message filtering**: Check message type/priority before consuming
- **Queue monitoring**: Examine next message without affecting queue state
- **Priority handling**: Process high-priority messages first

Example 1: Priority message handling

```
 Queue_t *eventQueue;
#define MSG_PRIORITY_HIGH 1
#define MSG_PRIORITY_NORMAL 0
void processEvents(void) {
  QueueMessage_t *msg;
  // Peek at next message if (OK(xQueuePeek(eventQueue, &msg))) {
    Byte_t priority = msg.value[0];  // Assume first byte is priority
    if (priority == MSG_PRIORITY_HIGH) {
      // High priority - receive and process immediately xMemFree((Addr_t
*)msg.value);  // Free peek copy xQueueReceive(eventQueue,
&msg);
      handleHighPriority(&msg);
      xMemFree((Addr_t *)msg.value);
    } else {
      // Normal priority - skip for now xMemFree((Addr_t *)msg.value);  //
Free peek copy
    }
  }
}
```

Example 2: Message type filtering

```
 Queue_t *commandQueue;
Base_t hasCommandType(Byte_t type) {
  QueueMessage_t *msg;
  // Peek without consuming if (OK(xQueuePeek(commandQueue, &msg))) {
    Byte_t cmdType = msg.value[0];
    xMemFree((Addr_t *)msg.value);
    return (cmdType == type);
  }
  return 0;
}
```

Parameters

in	*queue_*	Handle to the queue to peek. Must be a valid queue created with xQueueCreate().
out	*message_*	Pointer to QueueMessage_t *structure receiving a copy of the message. Memory is allocated for message->value and must be freed with xMemFree() after use.

Returns

ReturnOK if message peeked successfully, ReturnError if peek failed due to empty queue, invalid queue handle, or memory allocation failure.

Warning

The peeked message is a heap-allocated COPY. You must free it with xMemFree() even though the original message remains in the queue.

Multiple peeks return the SAME message until it is removed. If you peek, then another task receives the message, subsequent peeks will return the next message in the queue.

Note

xQueuePeek() does not modify the queue state—message count and queue position remain unchanged.

See also

xQueueReceive() - Receive and remove message

xQueueDropMessage() - Remove message without retrieving

xQueueMessagesWaiting() - Check if messages available before peeking

xMemFree() - Free peeked message copy

4.2.4.49 xQueueReceive() Return_t xQueueReceive (Queue_t * *queue_*, QueueMessage_t ** *message_*)

Retrieves the oldest message from the specified queue and removes it, making space for new messages. This is the primary mechanism for tasks to receive data from other tasks in a producer-consumer pattern. The operation is non-blocking and returns immediately whether a message is available or not.

This function combines the operations of xQueuePeek() (examine message) and xQueueDropMessage() (remove message) into a single atomic operation, ensuring thread-safe message retrieval even with multiple consumers.

The received message is allocated from the heap and must be freed by the caller using xMemFree() after processing. This heap allocation is necessary because message size is variable.

Message Receiving Process:

1. Check if queue has messages (not empty) 2. Allocate QueueMessage_t ∗ structure from heap 3. Copy oldest message data into structure 4. Remove message from queue 5. Return message structure to caller 6. Caller processes message then frees with xMemFree()

Typical Consumer Pattern:

1. Check if messages are waiting (optional optimization) 2. Call xQueueReceive() to get message 3. Check return value - success means message received 4. Process message->message bytes (size is in message->size) 5. Free message structure with xMemFree()

Warning

The returned message structure is allocated from the heap and MUST be freed by the caller using xMemFree(). Failing to free messages will cause memory leaks and eventually exhaust available heap memory.

Note

If the queue is empty, xQueueReceive() returns ReturnError immediately without blocking. Check queue status with xQueueIsQueueEmpty() or xQueueMessagesWaiting() before receiving if needed.

The QueueMessage_t ∗structure contains:

- message->size: Number of bytes in the message
- message->message: Pointer to the message data bytes

Unlike xQueuePeek() which leaves the message in the queue, xQueueReceive() removes the message, allowing the next message to be retrieved on the next call.

Example Usage:

```
 Queue_t *commandQueue;
xQueueCreate(&commandQueue, 10);
// Consumer task: Process commands from queue void
commandProcessorTask(Task_t *task, TaskParm_t *parm) {
  QueueMessage_t **msg = NULL;
  // Try to receive a message if (OK(xQueueReceive(commandQueue, &msg))) {
    // Message received successfully
    // Process based on message size and content if (msg->size == 1) {
      // Single byte command uint8_t cmd = msg->message[0];
      handleCommand(cmd);
    } else if (msg->size == sizeof(SensorData_t)) {
      // Structure message SensorData_t *data = (SensorData_t
*)msg->message;
      processSensorData(data);
    }
```

```
    // IMPORTANT: Free the message structure xMemFree((Addr_t *)msg);
  } else {
    // No messages available - queue empty
  }
}
// Complete producer-consumer example Queue_t *dataQueue;
// Producer void producerTask(Task_t *task, TaskParm_t *parm) {
  uint8_t data[4] = {0x01, 0x02, 0x03, 0x04};
  xQueueSend(dataQueue, sizeof(data), data);
}
// Consumer void consumerTask(Task_t *task, TaskParm_t *parm) {
  QueueMessage_t **msg = NULL;
  // Check for messages first (optional optimization) Base_t numMessages;
  if (OK(xQueueMessagesWaiting(dataQueue, &numMessages))) {
    if (numMessages > 0) {
      // Messages available, receive one if (OK(xQueueReceive(dataQueue,
&msg))) {
        // Process msg->message[0] through msg->message[msg->size-1]
processData(msg->message, msg->size);
        xMemFree((Addr_t *)msg);
      }
    }
  }
}
// Process all pending messages Base_t isEmpty;
while (OK(xQueueIsQueueEmpty(dataQueue, &isEmpty)) && !isEmpty) {
  QueueMessage_t **msg = NULL;
  if (OK(xQueueReceive(dataQueue, &msg))) {
    processMessage(msg);
    xMemFree((Addr_t *)msg);
  }
}
```

Parameters

in	*queue_*	Handle of the queue to receive from. Must be a valid queue handle previously returned by xQueueCreate().
out	*message_*	Pointer to QueueMessage_t *pointer variable. On success, receives a pointer to the message structure containing the message data and size. Caller MUST free this with xMemFree().

Returns

ReturnOK if a message was successfully received and removed from the queue, ReturnError if the receive failed (queue empty, invalid queue handle, memory allocation failed, or null message pointer).

See also

xQueueSend() - Send a message to the queue (producer operation)

xQueuePeek() - Examine next message without removing it

xQueueDropMessage() - Remove message after peeking

xQueueIsQueueEmpty() - Check if queue has no messages

xQueueMessagesWaiting() - Get number of messages in queue

xQueueCreate() - Create a new message queue

xMemFree() - Free the message structure (REQUIRED!)

4.2.4.50 xQueueSend()

```
Return_t xQueueSend (
        Queue_t * queue_,
        const Base_t bytes_,
        const Byte_t * value_ )
```

Adds a message to the specified queue, copying the provided data into the queue's internal storage. This is the primary mechanism for tasks to send data to other tasks in a producer-consumer pattern. The operation is non-blocking and returns immediately whether successful or not.

Messages are stored in FIFO order and will be retrieved by xQueueReceive() in the same order they were sent. Each message is a byte array up to CONFIG_MESSAGE_VALUE_BYTES (default 8) bytes, allowing flexible message formats including structures, commands, sensor readings, or any other data that fits within the size limit.

Message Sending Process:

1. Check if queue has space (not full) 2. Copy message data into queue storage 3. Increment queue message count 4. Return success or failure immediately (non-blocking)

Common Message Patterns:

- **Commands**: Single byte command codes (e.g., 0x01 = START, 0x02 = STOP)
- **Sensor data**: Multi-byte readings packed into message
- **Events**: Event type and optional data
- **Pointers**: Address of larger data structure (use with caution)
- **Structures**: Small structs that fit within byte limit

Warning

Messages are limited to CONFIG_MESSAGE_VALUE_BYTES bytes (default 8). Attempting to send larger messages will fail. For larger data, consider sending a pointer to heap-allocated data or using multiple messages.

The message data is COPIED into the queue. The original buffer can be reused or freed immediately after xQueueSend() returns. The queue maintains its own copy.

Note

If the queue is full, xQueueSend() returns ReturnError immediately without blocking. Check queue status with xQueueIsQueueFull() before sending if needed, or handle send failures appropriately.

Messages are copied, not moved. This ensures the sender retains control of its data and prevents aliasing issues.

Example Usage:

```
 Queue_t *dataQueue;
xQueueCreate(&dataQueue, 10);
// Send a simple command byte uint8_t command = 0x42;
if (OK(xQueueSend(dataQueue, 1, &command))) {
  // Command queued successfully
}
// Send sensor reading (multi-byte) typedef struct {
  uint16_t temperature;
  uint16_t humidity;
} SensorReading_t;
SensorReading_t reading;
reading.temperature = 2350;  // 23.50°C reading.humidity = 6500;     //
65.00%
if (OK(xQueueSend(dataQueue, sizeof(SensorReading_t), (Byte_t *)&reading)))
{
  // Sensor data queued
} else {
  // Queue full - handle overflow handleQueueOverflow();
}
// Producer task example void sensorTask(Task_t *task, TaskParm_t *parm) {
  uint8_t sensorData[4];
  readSensors(sensorData);
  // Try to send, handle failure if (ERROR(xQueueSend(dataQueue,
sizeof(sensorData), sensorData))) {
    // Queue full - either drop data or wait dataSamplesDropped++;
  }
}
// Check queue space before sending Base_t isFull;
if (OK(xQueueIsQueueFull(dataQueue, &isFull)) && !isFull) {
  // Safe to send xQueueSend(dataQueue, msgSize, msgData);
}
```

Parameters

in	*queue_*	Handle of the queue to send to. Must be a valid queue handle previously returned by xQueueCreate().
in	*bytes_*	Size of the message in bytes. Must be greater than 0 and not exceed CONFIG_MESSAGE_VALUE_BYTES (default 8).
in	*value_*	Pointer to the message data to send. The data will be copied into the queue, so this buffer can be reused after the call.

Returns

ReturnOK if the message was successfully added to the queue, ReturnError if the send failed (queue full, invalid queue handle, invalid size, or null value pointer).

See also

xQueueReceive() - Receive a message from the queue (consumer operation)

xQueuePeek() - Examine next message without removing it

xQueueIsQueueFull() - Check if queue is at capacity

xQueueMessagesWaiting() - Get number of messages in queue

xQueueCreate() - Create a new message queue

xQueueLockQueue() - Lock queue during critical operations

CONFIG_MESSAGE_VALUE_BYTES - Maximum message size configuration

4.2.4.51 xQueueUnLockQueue() `Return_t xQueueUnLockQueue ( Queue_t * queue_ )`

Removes the locked state from a queue, allowing tasks to resume sending messages with xQueueSend(). After unlocking, the queue returns to normal operation where send operations succeed (if queue is not full). This function must be called to restore normal queue behavior after locking with xQueueLockQueue().

Unlocking affects only send operations—receive, peek, and drop operations were unaffected by the lock and continue to work normally after unlock.

Common use cases:

- **Resume normal operation**: Re-enable message sending after batch processing
- **End critical section**: Allow producers to continue after critical operations
- **Flow control**: Resume accepting messages after backpressure relieved

Example 1: Lock/unlock pattern for batch processing

```
 Queue_t *dataQueue;
void batchProcessor(void) {
  // Lock to get consistent snapshot xQueueLockQueue(dataQueue);
  Base_t count;
  if (OK(xQueueMessagesWaiting(dataQueue, &count))) {
    // Process exactly this many messages for (Base_t i = 0; i < count;
i++)
{
      QueueMessage_t *msg;
      if (OK(xQueueReceive(dataQueue, &msg))) {
```

```
        processBatchItem(&msg);
        xMemFree((Addr_t *)msg.value);
      }
    }
  }
  // Unlock to allow new messages xQueueUnLockQueue(dataQueue);
}
```

Example 2: Conditional unlock based on processing result

```
 Queue_t *eventQueue;
Base_t processCriticalEvents(void) {
  xQueueLockQueue(eventQueue);
  Base_t success = 1;
  QueueMessage_t *msg;
  while (OK(xQueueReceive(eventQueue, &msg)) && success) {
    if (ERROR(handleCriticalEvent(&msg))) {
      success = 0;  // Processing failed
    }
    xMemFree((Addr_t *)msg.value);
  }
  if (success) {
    // Success - resume normal operation xQueueUnLockQueue(eventQueue);
  } else {
    // Failure - leave locked for manual intervention logError("Critical
event processing failed - queue locked");
  }

  return success;
}
```

Parameters

in	*queue*↩ _	Handle to the queue to unlock. Must be a valid queue created with xQueueCreate().

Returns

ReturnOK if queue unlocked successfully, ReturnError if operation failed due to invalid queue handle or queue not found.

Warning

Unlocking an already-unlocked queue is safe and has no effect.

Note

Always pair xQueueLockQueue() with xQueueUnLockQueue() to avoid leaving queues permanently locked, which would prevent all future message sending.

If you delete a locked queue with xQueueDelete(), you do not need to unlock it first—the deletion frees all queue resources including lock state.

See also

xQueueLockQueue() - Lock queue to prevent sending

xQueueSend() - Send message (enabled after unlock)

xQueueReceive() - Receive message (always works)

4.2.4.52 xStreamBytesAvailable()

```
Return_t xStreamBytesAvailable (
        const StreamBuffer_t * stream_,
        HalfWord_t * bytes_ )
```

Returns the count of bytes currently available for retrieval in the specified stream buffer without removing them. This non-destructive query allows tasks to check data availability before committing to receive operations, enabling more sophisticated flow control and buffering strategies.

Unlike xStreamReceive() which retrieves and removes bytes from the stream, xStreamBytesAvailable() is a read-only query that leaves the stream contents unchanged. This makes it useful for making decisions about when to process data, whether to wait for more data, or how to handle buffer capacity.

Common use cases:

- **Threshold-based processing**: Wait until minimum amount of data available
- **Flow control**: Monitor buffer levels to regulate producer rate
- **Batch optimization**: Collect data until optimal batch size reached
- **Buffer monitoring**: Check capacity before sending more data
- **Conditional receive**: Decide whether to retrieve data based on quantity

Example 1: Threshold-based data processing

```
 StreamBuffer_t *dataStream;
#define MIN_DATA_SIZE 16
void processDataTask(Task_t *task, TaskParm_t *parm) {
  HalfWord_t available;
  // Check if enough data accumulated if
(OK(xStreamBytesAvailable(dataStream, &available))) {
    if (available >= MIN_DATA_SIZE) {
      // Enough data - retrieve and process HalfWord_t count;
      Byte_t *data;
      if (OK(xStreamReceive(dataStream, &count, &data))) {
        processBatch(data, count);
        xMemFree((Addr_t *)data);
      }
    }
    // Otherwise wait for more data
  }
}
```

Example 2: Producer flow control

```
 StreamBuffer_t *txStream;
#define HIGH_WATER_MARK 28
#define LOW_WATER_MARK 8
void sendDataTask(Task_t *task, TaskParm_t *parm) {
  static Base_t throttled = 0;
  HalfWord_t buffered;
  if (OK(xStreamBytesAvailable(txStream, &buffered))) {
    // Implement hysteresis for flow control if (buffered >=
HIGH_WATER_MARK) {
      throttled = 1;  // Stop sending
    } else if (buffered <= LOW_WATER_MARK) {
      throttled = 0;  // Resume sending
    }
    if (!throttled) {
      Byte_t data = getNextByte();
      xStreamSend(txStream, data);
    }
  }
}
```

Example 3: Capacity checking before bulk send

```
 StreamBuffer_t *protocolStream;
Return_t sendFrame(Byte_t *frame, HalfWord_t frameLen) {
  HalfWord_t available;
  HalfWord_t capacity;
  // Check current buffer usage if
(OK(xStreamBytesAvailable(protocolStream, &available))) {
    capacity = CONFIG_STREAM_BUFFER_BYTES - available;
    // Verify enough space for entire frame if (capacity >= frameLen) {
      // Send all bytes for (HalfWord_t i = 0; i < frameLen; i++) {
```

```
        if (ERROR(xStreamSend(protocolStream, frame[i]))) {
          return ReturnError;  // Unexpected failure
        }
      }
      return ReturnOK;
    } else {
      // Not enough space - return error or wait return ReturnError;
    }
  }
  return ReturnError;
}
```

Example 4: Monitoring with status reporting

```
 StreamBuffer_t *sensorStream;
void monitorTask(Task_t *task, TaskParm_t *parm) {
  static HalfWord_t maxObserved = 0;
  HalfWord_t current;
  if (OK(xStreamBytesAvailable(sensorStream, &current))) {
    // Track high-water mark if (current > maxObserved) {
      maxObserved = current;
    }
    // Calculate utilization percentage Byte_t utilization =
(Byte_t)((current * 100) / CONFIG_STREAM_BUFFER_BYTES);
    // Report if approaching capacity if (utilization > 90) {
      logWarning("Stream buffer near capacity", utilization);
    }
  }
}
```

Parameters

in	*stream*↩_	Handle to the stream buffer to query. Must be a valid stream created with xStreamCreate().
out	*bytes*↩_	Pointer to variable receiving the byte count. On success, contains the number of bytes currently waiting in the stream (0 to CONFIG_STREAM_BUFFER_BYTES).

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid stream handle or stream not found.

Warning

This function only reports the current state of the stream buffer. In multitasking environments, the byte count may change immediately after the query if other tasks send or receive data. Use appropriate synchronization if you need atomic check-and-receive operations.

Note

xStreamBytesAvailable() is a non-destructive query—it does not modify the stream contents or affect subsequent operations.

The returned byte count includes all bytes sent with xStreamSend() that have not yet been retrieved with xStreamReceive().

To calculate free space in the stream, subtract bytes_ from CONFIG_STREAM_BUFFER_BYTES: freeSpace = CONFIG_STREAM_BUFFER_BYTES - bytes_

See also

xStreamReceive() - Retrieve and remove bytes from stream

xStreamSend() - Send byte to stream

xStreamIsEmpty() - Check if stream has no waiting bytes

xStreamIsFull() - Check if stream is at capacity

xStreamReset() - Clear all bytes from stream

xStreamCreate() - Create a stream buffer

4.2.4.53 xStreamCreate() `Return_t xStreamCreate ( StreamBuffer_t ** stream_ )`

Creates a new stream buffer for byte-by-byte data transfer between tasks. Stream buffers provide a lightweight alternative to message queues when communicating variable-length byte sequences or continuous data streams. Unlike queues which handle discrete messages, streams accept individual bytes that accumulate in a circular buffer until consumed by a receiver.

Stream buffers are ideal for:

- **Serial/UART data**: Buffering incoming bytes from serial ports
- **Protocol parsing**: Accumulating bytes until a complete frame is ready
- **Sensor streaming**: Continuous data collection from sensors
- **Character-based I/O**: Terminal or text-based interfaces
- **Variable-length data**: When message size isn't known in advance

Key characteristics:

- Fixed capacity: CONFIG_STREAM_BUFFER_BYTES (default 32 bytes)
- FIFO ordering: Bytes are retrieved in the order they were sent
- Byte-level granularity: Send and receive one byte at a time
- No message boundaries: Unlike queues, streams don't preserve message structure
- Lightweight: Lower overhead than message queues for byte-oriented data

Stream Lifecycle:

1. Create stream with xStreamCreate() (allocates kernel resources) 2. Producer tasks send bytes with xStreamSend() 3. Consumer tasks receive bytes with xStreamReceive() 4. Optionally query status with xStreamBytesAvailable(), xStreamIsEmpty(), xStreamIsFull() 5. Optionally clear stream with xStreamReset() 6. Delete stream with xStreamDelete() when no longer needed

Example 1: UART receive buffer

```
 StreamBuffer_t *uartRxStream;
// Create stream during initialization if
(OK(xStreamCreate(&uartRxStream))) {
  // Stream ready for use
}
// ISR or receive task sends bytes as they arrive void uartISR(void) {
  Byte_t rxByte = UART_DATA_REG;
  xStreamSend(uartRxStream, rxByte);
}
// Processing task receives accumulated data void processUART(Task_t *task,
TaskParm_t *parm) {
  HalfWord_t count;
  Byte_t *data;
  if (OK(xStreamReceive(uartRxStream, &count, &data))) {
    if (count > 0) {
      processReceivedData(data, count);
      xMemFree((Addr_t *)data);  // Always free received data
    }
  }
}
```

Example 2: Protocol parser with framing

```
 StreamBuffer_t *protocolStream;
void setupProtocol(void) {
  xStreamCreate(&protocolStream);
}
```

```
// Producer accumulates bytes void receiveTask(Task_t *task, TaskParm_t
parm)
{
  Byte_t receivedByte = getByteFromSource();
  xStreamSend(protocolStream, receivedByte);
  // Check if frame delimiter received if (receivedByte ==
FRAME_END_MARKER) {
    xTaskNotifyGive(parserTask);  // Signal parser
  }
}
// Consumer processes complete frames void parserTask(Task_t *task,
TaskParm_t *
parm) {
  HalfWord_t frameSize;
  Byte_t *frame;
  xTaskWait(1);  // Wait for notification
  if (OK(xStreamReceive(protocolStream, &frameSize, &frame))) {
    parseProtocolFrame(frame, frameSize);
    xMemFree((Addr_t *)frame);
  }
}
```

Example 3: Sensor data streaming

```
 StreamBuffer_t *sensorStream;
void initSensors(void) {
  xStreamCreate(&sensorStream);
}
// High-frequency sensor sampling void sampleSensor(Task_t *task,
TaskParm_t *
parm) {
  Byte_t sample = readADC();
  Base_t isFull;
  if (OK(xStreamIsFull(sensorStream, &isFull)) && isFull) {
    // Stream full - handle overflow xStreamReset(sensorStream);  //
Discard old data
  }
  xStreamSend(sensorStream, sample);
}
// Lower-frequency processing void processSamples(Task_t *task, TaskParm_t
*
parm) {
  HalfWord_t sampleCount;
  Byte_t *samples;
  if (OK(xStreamReceive(sensorStream, &sampleCount, &samples))) {
    if (sampleCount >= MIN_SAMPLES_FOR_PROCESSING) {
      computeStatistics(samples, sampleCount);
    }
    xMemFree((Addr_t *)samples);
  }
}
```

Parameters

out	*stream*↩ _	Pointer to StreamBuffer_t ∗handle to be initialized. After successful creation, this handle is used in all subsequent stream operations. The handle remains valid until xStreamDelete() is called.

Returns

ReturnOK if stream created successfully, ReturnError if creation failed due to insufficient memory or invalid parameters.

Warning

Stream buffers have a fixed capacity defined by CONFIG_STREAM_BUFFER_BYTES (typically 32 bytes). Sending to a full stream will fail with ReturnError. Use xStreamIsFull() to check capacity before sending critical data.

The stream_ parameter must point to valid memory. Passing null will result in ReturnError.

Note

Stream buffers are allocated from kernel memory and persist until explicitly deleted with xStreamDelete(). Always delete streams when no longer needed to prevent memory leaks.

Unlike message queues, streams do not preserve message boundaries. If you send bytes "ABC" and "DEF" separately, the receiver sees "ABCDEF" as a continuous byte sequence. Implement your own framing if you need to distinguish between separate transmissions.

Streams operate on individual bytes (Byte_t/Byte_t). To send multi-byte values, send each byte separately or use a message queue instead.

See also

xStreamDelete() - Delete stream and free resources

xStreamSend() - Send byte to stream

xStreamReceive() - Receive all waiting bytes from stream

xStreamBytesAvailable() - Query number of bytes waiting in stream

xStreamReset() - Clear all bytes from stream

xStreamIsEmpty() - Check if stream has no waiting bytes

xStreamIsFull() - Check if stream is at capacity

xQueueCreate() - Alternative for message-oriented communication

4.2.4.54 xStreamDelete() `Return_t xStreamDelete ( const StreamBuffer_t * stream_ )`

Deletes a stream buffer created by xStreamCreate(), freeing all associated kernel memory and resources. After deletion, the stream handle becomes invalid and must not be used in any subsequent stream operations. Any bytes waiting in the stream at the time of deletion are discarded.

This function should be called when a stream buffer is no longer needed to prevent memory leaks in long-running embedded systems. Proper resource cleanup is essential for system reliability, especially in applications that dynamically create and destroy communication channels.

Deletion scenarios:

- **Application shutdown**: Clean up resources during de-initialization
- **Dynamic reconfiguration**: Remove old streams when changing communication topology
- **Error recovery**: Delete and recreate streams after communication failures
- **Resource management**: Free streams in resource-constrained systems

Example 1: Basic stream cleanup

```
 StreamBuffer_t *tempStream;
// Create stream for temporary operation if
(OK(xStreamCreate(&tempStream))) {
  // Use stream for data transfer xStreamSend(tempStream, 0x42);
  // ... perform operations ...
  // Clean up when done xStreamDelete(tempStream);
}
```

Example 2: Communication channel lifecycle

```
 StreamBuffer_t *channelStream = null;
void openChannel(void) {
  if (OK(xStreamCreate(&channelStream))) {
```

```
    // Channel ready for use
  }
}
void closeChannel(void) {
  if (channelStream != null) {
    xStreamDelete(channelStream);
    channelStream = null;  // Prevent use-after-delete
  }
}
```

Example 3: Error recovery with stream recreation

```
 StreamBuffer_t *dataStream;
void setupDataStream(void) {
  xStreamCreate(&dataStream);
}
void resetCommunication(void) {
  // Delete corrupted stream xStreamDelete(dataStream);
  // Wait briefly for cleanup delayMs(10);
  // Create fresh stream if (OK(xStreamCreate(&dataStream))) {
    // Stream reset successful
  }
}
```

Parameters

in	*stream*↩ _	Handle to the stream buffer to delete. Must be a valid handle obtained from a previous successful call to xStreamCreate(). After deletion, this handle becomes invalid.

Returns

ReturnOK if stream deleted successfully, ReturnError if deletion failed due to invalid handle or stream not found.

Warning

After calling xStreamDelete(), the stream handle becomes invalid and must not be used in any subsequent stream operations. Attempting to use a deleted stream will result in ReturnError.

If other tasks hold references to the stream buffer, ensure they stop using the stream before deletion. Deleting a stream while other tasks are actively sending or receiving data may cause those operations to fail with ReturnError.

Any bytes waiting in the stream buffer at the time of deletion are permanently lost. If you need to preserve data, call xStreamReceive() to retrieve all pending bytes before deleting the stream.

Note

It is good practice to set the stream handle to null after deletion to prevent accidental use of an invalid handle (use-after-delete bugs).

xStreamDelete() only affects the stream buffer itself. Any data previously received with xStreamReceive() and stored in heap memory remains allocated until explicitly freed with xMemFree().

See also

xStreamCreate() - Create a new stream buffer

xStreamReset() - Clear stream contents without deleting the stream

xStreamReceive() - Retrieve pending bytes before deletion

xMemFree() - Free memory allocated by xStreamReceive()

4.2.4.55 xStreamIsEmpty()

```
Return_t xStreamIsEmpty (
        const StreamBuffer_t * stream_,
        Base_t * res_ )
```

Queries whether the specified stream buffer is empty (contains zero bytes). This is a boolean status check that provides a simple, readable way to test for data availability without needing to interpret byte counts. An empty stream has no data waiting to be retrieved.

This function is logically equivalent to checking if xStreamBytesAvailable() returns 0, but offers more expressive code when you only need a boolean empty/not-empty status rather than the exact byte count.

Common use cases:

- **Conditional processing**: Skip receive operations when no data available
- **Stream state verification**: Confirm stream cleared after reset
- **Idle detection**: Determine if communication channel is idle
- **Polling optimization**: Avoid unnecessary memory allocations for empty receives

Example 1: Conditional receive

```
 StreamBuffer_t *dataStream;
void processTask(Task_t *task, TaskParm_t *parm) {
  Base_t isEmpty;
  // Check before attempting receive if (OK(xStreamIsEmpty(dataStream,
&isEmpty)) && !isEmpty) {
    // Data available - retrieve and process HalfWord_t count;
    Byte_t *data;
    if (OK(xStreamReceive(dataStream, &count, &data))) {
      processData(data, count);
      xMemFree((Addr_t *)data);
    }
  }
  // Otherwise skip processing this cycle
}
```

Example 2: Verify reset completion

```
 StreamBuffer_t *protocolStream;
void resetProtocol(void) {
  xStreamReset(protocolStream);
  // Verify stream actually empty Base_t isEmpty;
  if (OK(xStreamIsEmpty(protocolStream, &isEmpty))) {
    if (isEmpty) {
      // Reset confirmed - ready for new data protocolState =
PROTOCOL_IDLE;
    } else {
      // Unexpected - reset failed?
      handleError();
    }
  }
}
```

Example 3: Communication idle detection

```
 StreamBuffer_t *rxStream;
Timer_t *idleTimer;
void monitorActivity(Task_t *task, TaskParm_t *parm) {
  Base_t isEmpty;
  Base_t timerExpired;
  if (OK(xStreamIsEmpty(rxStream, &isEmpty)) && isEmpty) {
    // No data waiting - check how long idle if
(OK(xTimerHasTimerExpired(idleTimer, &timerExpired)) && timerExpired) {
      // Idle timeout - enter power-saving mode enterLowPowerMode();
    }
  } else {
    // Activity detected - reset idle timer xTimerReset(idleTimer);
  }
}
```

Parameters

in	*stream_*	Handle to the stream buffer to query. Must be a valid stream created with xStreamCreate().
out	*res_*	Pointer to variable receiving the empty status. Set to non-zero (true) if stream is empty (0 bytes waiting), zero (false) if stream contains data.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid stream handle or stream not found.

Note

This is a non-destructive query that does not modify stream contents.

For more detailed information about stream state, use xStreamBytesAvailable() to get the exact byte count instead of just empty/not-empty status.

See also

xStreamBytesAvailable() - Get exact byte count

xStreamIsFull() - Check if stream is at capacity

xStreamReceive() - Retrieve bytes from stream

xStreamReset() - Clear stream to empty state

xStreamCreate() - Create a stream buffer

4.2.4.56 xStreamIsFull()

```
Return_t xStreamIsFull (
    const StreamBuffer_t * stream_,
    Base_t * res_ )
```

Queries whether the specified stream buffer is full (contains the maximum number of bytes defined by CONFIG_STREAM_BUFFER_BYTES, typically 32). This boolean status check provides a simple way to test for buffer saturation before attempting send operations or to implement overflow handling strategies.

A full stream cannot accept additional bytes via xStreamSend() until space is freed by xStreamReceive() or xStreamReset(). This function is logically equivalent to checking if xStreamBytesAvailable() equals CONFIG_STREAM_BUFFER_BYTES, but offers more expressive code for capacity-related logic.

Common use cases:

- **Pre-send validation**: Check capacity before attempting xStreamSend()
- **Overflow prevention**: Detect full condition to trigger consumer notification
- **Flow control**: Throttle producers when buffer reaches capacity
- **Overflow strategies**: Decide whether to drop data, reset buffer, or block
- **Buffer health monitoring**: Track how often buffer reaches capacity

Example 1: Pre-send capacity check

```
 StreamBuffer_t *txStream;
Return_t sendByte(Byte_t data) {
  Base_t isFull;
  // Check capacity before sending if (OK(xStreamIsFull(txStream,
&isFull))) {
    if (isFull) {
      // Buffer full - return error or wait return ReturnError;
    }
    // Space available - send byte return xStreamSend(txStream, data);
  }
  return ReturnError;
}
```

Example 2: Overflow handling with notification

```
 StreamBuffer_t *dataStream;
Task_t *consumerTask;
void producerTask(Task_t *task, TaskParm_t *parm) {
  Byte_t data = generateData();
  Base_t isFull;
  // Check if buffer full if (OK(xStreamIsFull(dataStream, &isFull)) &&
isFull) {
    // Wake up consumer to drain buffer xTaskNotifyGive(consumerTask);
    // Wait briefly for consumer delayMs(5);
  }
  // Attempt send if (ERROR(xStreamSend(dataStream, data))) {
    // Still full - data lost logDataLoss();
  }
}
```

Example 3: Overflow strategy selection

```
 typedef enum {
  OVERFLOW_DROP_OLDEST, OVERFLOW_DROP_NEWEST, OVERFLOW_ERROR
} OverflowStrategy_t;
StreamBuffer_t *sensorStream;
OverflowStrategy_t strategy = OVERFLOW_DROP_OLDEST;
void recordSample(Byte_t sample) {
  Base_t isFull;
  if (OK(xStreamIsFull(sensorStream, &isFull)) && isFull) {
    switch (strategy) {
      case OVERFLOW_DROP_OLDEST:
        // Clear buffer and add new sample xStreamReset(sensorStream);
        xStreamSend(sensorStream, sample);
        break;
      case OVERFLOW_DROP_NEWEST:
        // Discard new sample, keep old data logDroppedSample();
        break;
      case OVERFLOW_ERROR:
        // Report error condition handleBufferOverflow();
        break;
    }
  } else {
    // Normal send xStreamSend(sensorStream, sample);
  }
}
```

Example 4: Buffer utilization monitoring

```
 StreamBuffer_t *commStream;
typedef struct {
  Word_t fullCount;
  Word_t totalChecks;
  Byte_t peakUtilization;
} BufferStats_t;
BufferStats_t stats = {0, 0, 0};
void monitorBuffer(Task_t *task, TaskParm_t *parm) {
  Base_t isFull;
  HalfWord_t available;
  stats.totalChecks++;
  if (OK(xStreamIsFull(commStream, &isFull)) && isFull) {
    stats.fullCount++;
  }
  // Track peak utilization if (OK(xStreamBytesAvailable(commStream,
&available))) {
    Byte_t utilization = (Byte_t)((available * 100) /
CONFIG_STREAM_BUFFER_BYTES);
    if (utilization > stats.peakUtilization) {
      stats.peakUtilization = utilization;
    }
  }
  // Report statistics periodically if (stats.totalChecks % 1000 == 0) {
    Byte_t fullPercent = (Byte_t)((stats.fullCount * 100) /
stats.totalChecks);
    reportStats(fullPercent, stats.peakUtilization);
  }
}
```

Parameters

in	*stream_*	Handle to the stream buffer to query. Must be a valid stream created with xStreamCreate().
out	*res_*	Pointer to variable receiving the full status. Set to non-zero (true) if stream is full (CONFIG_STREAM_BUFFER_BYTES bytes waiting), zero (false) if stream has available space.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid stream handle or stream not found.

Warning

A full stream will cause xStreamSend() to fail with ReturnError. Always check for full condition before critical send operations, or implement appropriate error handling for send failures.

Note

This is a non-destructive query that does not modify stream contents.

The buffer capacity is defined by CONFIG_STREAM_BUFFER_BYTES at compile time (default 32 bytes). To calculate free space: freeSpace = CONFIG_STREAM_BUFFER_BYTES - currentBytes

In multitasking environments, the full status may change immediately after the query if another task performs a receive operation. Use appropriate synchronization for atomic check-and-send operations if needed.

See also

xStreamBytesAvailable() - Get exact byte count and calculate free space

xStreamIsEmpty() - Check if stream has no waiting bytes

xStreamSend() - Send byte to stream (fails if full)

xStreamReset() - Clear stream to free all space

xStreamReceive() - Retrieve bytes to free space

xStreamCreate() - Create a stream buffer

4.2.4.57 xStreamReceive()

```
Return_t xStreamReceive (
    const StreamBuffer_t * stream_,
    HalfWord_t * bytes_,
    Byte_t ** data_ )
```

Retrieves all bytes currently waiting in the specified stream buffer and returns them in a newly allocated memory buffer. This is the consumer-side operation for stream-based inter-task communication. After retrieval, the bytes are removed from the stream, freeing space for new data.

xStreamReceive() allocates memory from the user heap to hold the retrieved bytes. The caller is responsible for freeing this memory with xMemFree() after processing the data. This memory management pattern ensures safe data transfer between tasks without buffer ownership conflicts.

Key characteristics:

- **Retrieves all waiting bytes**: Returns complete contents of stream in one call
- **Allocates memory**: Creates new buffer in user heap for received data
- **Clears stream**: Removes received bytes from stream buffer
- **Returns byte count**: Indicates number of bytes retrieved via bytes_ parameter
- **Non-blocking**: Returns immediately even if stream is empty (bytes_ = 0)
- **FIFO ordering**: Bytes returned in the order they were sent

Typical consumer workflow:

1. Call xStreamReceive() to get all waiting bytes 2. Check if any bytes were retrieved (bytes_ $>$ 0) 3. Process the received data 4. Free the allocated buffer with xMemFree() 5. Repeat as needed for continuous data flow

Example 1: UART receive processing

```
 StreamBuffer_t *uartRxStream;
// Producer (ISR or task) sends bytes to stream void uartISR(void) {
  if (UART_RX_READY) {
    Byte_t rxByte = UART_DATA_REG;
    xStreamSend(uartRxStream, rxByte);
  }
}
// Consumer task processes received data void processUARTTask(Task_t *task,
TaskParm_t *parm) {
  HalfWord_t byteCount;
  Byte_t *rxData;
  if (OK(xStreamReceive(uartRxStream, &byteCount, &rxData))) {
    if (byteCount > 0) {
      // Process received bytes for (HalfWord_t i = 0; i < byteCount; i++)
{
        processCharacter(rxData[i]);
      }
      // Always free allocated memory xMemFree((Addr_t *)rxData);
    }
  }
}
```

Example 2: Protocol frame parsing

```
 StreamBuffer_t *protocolStream;
void parseProtocolTask(Task_t *task, TaskParm_t *parm) {
  HalfWord_t frameSize;
  Byte_t *frame;
  if (OK(xStreamReceive(protocolStream, &frameSize, &frame))) {
    if (frameSize > 0) {
      // Look for start byte (0xAA) for (HalfWord_t i = 0; i < frameSize;
i++) {
        if (frame[i] == 0xAA && (i + 3) < frameSize) {
          Byte_t command = frame[i + 1];
          Byte_t length = frame[i + 2];
          // Verify complete frame available if ((i + 3 + length) <
frameSize) {
            Byte_t *payload = &frame[i + 3];
            processCommand(command, payload, length);
            i += 3 + length;  // Skip past this frame
          }
        }
      }
      xMemFree((Addr_t *)frame);
    }
  }
}
```

Example 3: Sensor data batch processing

```
 StreamBuffer_t *sensorStream;
#define MIN_SAMPLES 10
void analyzeSensorTask(Task_t *task, TaskParm_t *parm) {
  HalfWord_t sampleCount;
  Byte_t *samples;
  if (OK(xStreamReceive(sensorStream, &sampleCount, &samples))) {
    // Only process if we have enough samples if (sampleCount >=
MIN_SAMPLES) {
      // Calculate statistics Word_t sum = 0;
      Byte_t min = 255, max = 0;
      for (HalfWord_t i = 0; i < sampleCount; i++) {
```

```
        sum += samples[i];
        if (samples[i] < min) min = samples[i];
        if (samples[i] > max) max = samples[i];
      }
      Byte_t average = (Byte_t)(sum / sampleCount);
      reportStatistics(average, min, max);
    } else if (sampleCount > 0) {
      // Put samples back by re-sending them for (HalfWord_t i = 0; i <
sampleCount; i++) {
        xStreamSend(sensorStream, samples[i]);
      }
    }
    // Always free even if we didn't process if (sampleCount > 0) {
      xMemFree((Addr_t *)samples);
    }
  }
}
```

Example 4: Complete producer-consumer pattern

```
 StreamBuffer_t *dataStream;
Task_t *producerTask, consumerTask;
// Producer accumulates data void producer(Task_t *task, TaskParm_t *parm)
{
  Byte_t dataPoint = collectData();
  if (OK(xStreamSend(dataStream, dataPoint))) {
    // Check if buffer is getting full HalfWord_t available;
    if (OK(xStreamBytesAvailable(dataStream, &available))) {
      if (available >= 16) {
        // Signal consumer to process data xTaskNotifyGive(consumerTask);
      }
    }
  }
}
// Consumer processes batches void consumer(Task_t *task, TaskParm_t *parm)
{
  xTaskWait(1);  // Wait for notification from producer
  HalfWord_t count;
  Byte_t *data;
  if (OK(xStreamReceive(dataStream, &count, &data))) {
    if (count > 0) {
      processBatch(data, count);
      xMemFree((Addr_t *)data);
    }
  }
}
```

Parameters

in	*stream_*	Handle to the stream buffer to receive from. Must be a valid stream created with xStreamCreate().
out	*bytes_*	Pointer to variable receiving the number of bytes retrieved. Set to 0 if stream is empty. The caller should check this value before processing data_.
out	*data_*	Pointer to variable receiving address of newly allocated buffer containing the retrieved bytes. Memory is allocated from user heap and must be freed with xMemFree() after use. If no bytes are available, this may point to null or uninitialized memory.

Returns

ReturnOK if receive operation completed (even if 0 bytes received), ReturnError if operation failed due to invalid stream handle, memory allocation failure, or stream not found.

Warning

The memory allocated for data_ MUST be freed by the caller using xMemFree() after processing. Failure to free this memory will cause a memory leak. Always pair xStreamReceive() with xMemFree() in your code.

Always check bytes_ before accessing data_. If bytes_ is 0, the stream was empty and data_ should not be accessed.

xStreamReceive() retrieves ALL waiting bytes in a single call. If you need to process bytes one at a time or in smaller chunks, you must implement your own buffering or use the retrieved data array with indexing.

Receiving from an invalid or deleted stream handle will return ReturnError.

Note

xStreamReceive() is non-blocking. If the stream is empty, it returns immediately with bytes_ set to 0. This differs from some RTOS queue implementations that support blocking receives with timeouts.

After xStreamReceive() completes, the retrieved bytes are removed from the stream buffer, freeing space for new data from producers.

Memory allocation failures are rare but possible in heap-constrained systems. If xStreamReceive() returns ReturnError and bytes_ indicates data was available, suspect memory allocation failure and consider increasing heap size.

See also

xStreamSend() - Send bytes to stream (producer operation)

xMemFree() - Free memory allocated by xStreamReceive()

xStreamCreate() - Create a stream buffer

xStreamBytesAvailable() - Query number of bytes waiting without receiving

xStreamIsEmpty() - Check if stream has any waiting bytes

xStreamReset() - Clear stream without retrieving bytes

xQueueReceive() - Alternative for message-oriented communication

4.2.4.58 xStreamReset() `Return_t xStreamReset ( const StreamBuffer_t * stream_ )`

Resets the specified stream buffer to an empty state by discarding all waiting bytes. After reset, the stream behaves as if newly created— xStreamBytesAvailable() returns 0 and xStreamIsEmpty() returns true. This operation is useful for error recovery, protocol resyncs, or discarding stale data.

Unlike xStreamDelete() which destroys the stream entirely, xStreamReset() preserves the stream structure and handle while only clearing its contents. The stream remains fully functional and can immediately accept new data via xStreamSend().

Common scenarios for stream reset:

- **Error recovery**: Clear corrupted or incomplete protocol data
- **Protocol resync**: Discard partial frames when reestablishing sync
- **Buffer overflow handling**: Clear old data when capacity exceeded
- **Timeout handling**: Discard stale data after communication timeout
- **State machine reset**: Clear buffered data when returning to idle state
- **Channel cleanup**: Prepare stream for reuse without deallocating

Example 1: Protocol error recovery

```
 StreamBuffer_t *protocolStream;
void handleProtocolError(void) {
  // Error detected - discard partial/corrupted data
xStreamReset(protocolStream);
  // Send resync request to peer sendResyncCommand();
  // Stream now empty and ready for fresh data
}
```

Example 2: Timeout-based data discard

```
 StreamBuffer_t *rxStream;
Timer_t *rxTimeout;
void receiveTask(Task_t *task, TaskParm_t *parm) {
  Base_t timerExpired;
  // Check for receive timeout if (OK(xTimerHasTimerExpired(rxTimeout,
&timerExpired)) && timerExpired) {
    HalfWord_t staleBytes;
    // Check if incomplete data waiting if
(OK(xStreamBytesAvailable(rxStream, &staleBytes)) && staleBytes > 0) {
      // Data incomplete after timeout - discard it xStreamReset(rxStream);
      logWarning("Receive timeout - data discarded");
    }
    // Reset timeout for next receive xTimerReset(rxTimeout);
  }
}
```

Example 3: Overflow handling with reset

```
 StreamBuffer_t *sensorStream;
void sampleSensor(Task_t *task, TaskParm_t *parm) {
  Byte_t sample = readADC();
  Base_t isFull;
  // Check if buffer full if (OK(xStreamIsFull(sensorStream, &isFull)) &&
isFull) {
    // Option 1: Drop oldest data, keep newest xStreamReset(sensorStream);
    xStreamSend(sensorStream, sample);
    // Log overflow event overflowCount++;
  } else {
    // Normal send xStreamSend(sensorStream, sample);
  }
}
```

Example 4: State machine with stream cleanup

```
 typedef enum {
  STATE_IDLE, STATE_RECEIVING, STATE_PROCESSING
} CommState_t;
StreamBuffer_t *commStream;
CommState_t commState = STATE_IDLE;
void communicationTask(Task_t *task, TaskParm_t *parm) {
  switch (commState) {
    case STATE_IDLE:
      // Ensure stream clean when entering new transaction
xStreamReset(commStream);
      commState = STATE_RECEIVING;
      break;
    case STATE_RECEIVING:
      // Accumulate data...
      if (frameComplete()) {
        commState = STATE_PROCESSING;
      }
      break;
    case STATE_PROCESSING:
      // Process received frame processFrame();
      commState = STATE_IDLE;  // Will clear on next iteration break;
  }
}
```

Parameters

in	*stream*↩ _	Handle to the stream buffer to reset. Must be a valid stream created with xStreamCreate().

Returns

ReturnOK if stream reset successfully, ReturnError if reset failed due to invalid stream handle or stream not found.

Warning

All bytes waiting in the stream are permanently discarded. If you need to preserve data, call xStreamReceive() before calling xStreamReset().

In multitasking environments, ensure no other task is actively sending to or receiving from the stream during reset. Resetting a stream while another task is mid-operation may lead to unexpected behavior or data loss.

Note

xStreamReset() does not delete or invalidate the stream handle. After reset, the stream remains fully functional and ready for new send/receive operations.

This operation is typically faster than deleting and recreating a stream, making it preferable for error recovery scenarios where the stream structure should be reused.

After reset, xStreamBytesAvailable() returns 0, xStreamIsEmpty() returns true, and xStreamIsFull() returns false.

See also

xStreamCreate() - Create a stream buffer

xStreamDelete() - Delete stream entirely (vs. just clearing contents)

xStreamReceive() - Retrieve data before reset if needed

xStreamBytesAvailable() - Check byte count before reset

xStreamIsEmpty() - Verify stream empty after reset

4.2.4.59 xStreamSend()

```
Return_t xStreamSend (
    StreamBuffer_t * stream_,
    const Byte_t byte_ )
```

Sends one byte to the specified stream buffer, adding it to the end of the FIFO byte sequence. This is the producer-side operation for stream-based inter-task communication. Bytes are stored in the stream until consumed by a receiver task using xStreamReceive().

Stream buffers provide byte-level granularity for communication, making them ideal for character-oriented protocols, serial data buffering, and continuous data streaming. Unlike message queues, streams don't preserve message boundaries— bytes are retrieved in the exact order they were sent, but without any structure or framing.

Operational characteristics:

- **FIFO ordering**: Bytes are retrieved in the order they were sent
- **Fixed capacity**: Stream holds up to CONFIG_STREAM_BUFFER_BYTES (default 32)
- **Blocking behavior**: Returns ReturnError immediately if stream is full
- **Atomic operation**: Single byte send is atomic (safe from interrupts)
- **Low overhead**: Minimal processing compared to message queue operations

Common usage patterns:

- **Serial/UART buffering**: Send received bytes from ISR to stream for processing
- **Character output**: Build strings character-by-character for display
- **Protocol assembly**: Accumulate protocol bytes until complete frame ready
- **Sensor sampling**: Stream continuous ADC or sensor readings
- **Event logging**: Record byte-oriented event codes or timestamps

Example 1: UART transmit buffering

```
 StreamBuffer_t *uartTxStream;
void setupUART(void) {
  xStreamCreate(&uartTxStream);
}
// Application sends string to UART void sendString(const char *str) {
  while (*str) {
    if (ERROR(xStreamSend(uartTxStream, (Byte_t)*str))) {
      // Stream full - wait for transmitter to drain delayMs(1);
    } else {
      str++;
    }
  }
}
// UART task transmits buffered bytes void uartTransmitTask(Task_t *task,
TaskParm_t *parm) {
  HalfWord_t count;
  Byte_t *data;
  if (OK(xStreamReceive(uartTxStream, &count, &data))) {
    if (count > 0) {
      uartWriteBytes(data, count);
      xMemFree((Addr_t *)data);
    }
  }
}
```

Example 2: Protocol frame assembly

```
 StreamBuffer_t *protocolStream;
// Send protocol header, payload, and checksum Return_t
sendProtocolFrame(Byte_t command, Byte_t *payload, HalfWord_t len) {
  // Send start byte if (ERROR(xStreamSend(protocolStream, 0xAA))) {
    return ReturnError;
  }
  // Send command if (ERROR(xStreamSend(protocolStream, command))) {
    return ReturnError;
  }
  // Send length if (ERROR(xStreamSend(protocolStream, (Byte_t)len))) {
    return ReturnError;
  }
  // Send payload bytes for (HalfWord_t i = 0; i < len; i++) {
    if (ERROR(xStreamSend(protocolStream, payload[i]))) {
      return ReturnError;
    }
  }
  // Send checksum Byte_t checksum = calculateChecksum(command, payload,
len);
  return xStreamSend(protocolStream, checksum);
}
```

Example 3: Sensor data streaming with overflow handling

```
 StreamBuffer_t *sensorStream;
void sampleSensorTask(Task_t *task, TaskParm_t *parm) {
  Byte_t sample = readADC();
  // Try to send sample if (ERROR(xStreamSend(sensorStream, sample))) {
    // Stream full - check how to handle overflow Base_t isFull;
    if (OK(xStreamIsFull(sensorStream, &isFull)) && isFull) {
      // Option 1: Drop oldest data and reset xStreamReset(sensorStream);
      xStreamSend(sensorStream, sample);
      // Option 2: Notify error handler
      // logOverflowError();
    }
  }
}
```

Parameters

in	*stream_*	Handle to the stream buffer to send data to. Must be a valid stream created with xStreamCreate().
in	*byte_*	The byte value to send to the stream. Accepts any value from 0x00 to 0xFF.

Returns

ReturnOK if byte sent successfully, ReturnError if send failed due to stream full, invalid stream handle, or stream not found.

Warning

If the stream is full (contains CONFIG_STREAM_BUFFER_BYTES bytes), xStreamSend() returns ReturnError immediately without blocking. Check stream capacity with xStreamIsFull() or xStreamBytesAvailable() before sending critical data, or implement error handling for full conditions.

Sending to an invalid or deleted stream handle will return ReturnError. Always verify stream creation succeeded before calling xStreamSend().

Note

xStreamSend() operates on individual bytes only. To send multi-byte values (integers, floats, structures), send each byte separately in the appropriate order (considering endianness if necessary).

Stream buffers do not preserve message boundaries. Bytes sent in separate xStreamSend() calls are concatenated into a continuous sequence. If you need message framing, implement your own protocol with headers/delimiters.

For high-throughput applications, consider checking xStreamIsFull() before sending large amounts of data to avoid repeated ReturnError conditions.

See also

xStreamReceive() - Receive all waiting bytes from stream (consumer operation)

xStreamCreate() - Create a stream buffer

xStreamBytesAvailable() - Query number of bytes waiting in stream

xStreamIsFull() - Check if stream is at capacity

xStreamReset() - Clear all bytes from stream

xQueueSend() - Alternative for message-oriented communication

4.2.4.60 xSystemAssert()

```
Return_t xSystemAssert (
    const char * file_,
    const int line_ )
```

Raises a system assertion to handle critical error conditions that violate fundamental assumptions or invariants in the code. Assertions are defensive programming constructs that catch bugs during development and provide configurable responses in production builds.

The behavior when an assertion triggers is determined by CONFIG_SYSTEM_ASSERT_BEHAVIOR:

- **Halt**: Stop system execution completely (safest for critical systems)
- **Log**: Record assertion and continue (for debugging)
- **Callback**: Invoke custom handler function
- **Disabled**: Assertions compiled out entirely (production optimization)

It is strongly recommended to use the **AssertOnElse**() macro rather than calling xSystemAssert() directly. The macro automatically captures file name and line number, making debugging significantly easier.

Common use cases:

- **Precondition checking**: Validating function parameters and state
- **Postcondition verification**: Ensuring expected results after operations
- **Invariant enforcement**: Checking critical system state consistency
- **Null pointer detection**: Catching unexpected null dereferences
- **Range validation**: Detecting out-of-bounds array/buffer access
- **Resource validation**: Ensuring handles and resources are valid

Example 1: Parameter validation with **AssertOnElse**()

```
 void processData(Byte_t *buffer, Size_t size) {
  // Assert buffer is not null __AssertOnElse__(buffer != NULL, return);
  // Assert size is reasonable __AssertOnElse__(size > 0 && size <=
MAX_BUFFER_SIZE, return);
  // Proceed with processing - preconditions verified for (Size_t i = 0; i
<
size; i++) {
    processBuffer(buffer[i]);
  }
}
```

Example 2: State invariant checking

```
 typedef enum {
  STATE_IDLE, STATE_ACTIVE, STATE_ERROR
} SystemState_t;
SystemState_t systemState = STATE_IDLE;
void startOperation(void) {
  // Assert we're in correct state to start __AssertOnElse__(systemState ==
STATE_IDLE, return);
  systemState = STATE_ACTIVE;
  performOperation();
  // Assert valid ending state __AssertOnElse__(systemState == STATE_ACTIVE
|| systemState == STATE_ERROR, systemState = STATE_ERROR);
}
```

Example 3: Handle validation

```
 Return_t sendToQueue(Queue_t *queue, Byte_t *data, Size_t len) {
  // Assert queue handle is valid __AssertOnElse__(queue != NULL, return
ReturnError);
  // Assert data parameters are reasonable __AssertOnElse__(data != NULL,
return ReturnError);
  __AssertOnElse__(len > 0, return ReturnError);
  return xQueueSend(queue, len, data);
}
```

Example 4: Array bounds checking

```
#define SENSOR_COUNT 8 Byte_t sensorReadings[SENSOR_COUNT];
Byte_t getSensorReading(Base_t sensorId) {
  // Assert index is in valid range __AssertOnElse__(sensorId <
SENSOR_COUNT, return 0);
  return sensorReadings[sensorId];
}
```

Parameters

in	*file*↩ _	Source file name where assertion occurred. Automatically provided by **AssertOnElse**() macro using **FILE**.
in	*line*↩ _	Line number where assertion occurred. Automatically provided by **AssertOnElse**() macro using **LINE**.

Returns

ReturnOK if assertion handling completed (when configured to continue), ReturnError if assertion handling failed. Note: May not return if CONFIG_SYSTEM_ASSERT_BEHAVIOR halts the system.

Warning

Assertions are for catching programming errors, NOT for runtime error handling. Use normal error checking (if/return) for expected error conditions like invalid user input or communication failures.

If CONFIG_SYSTEM_ASSERT_BEHAVIOR is set to halt, calling this function will stop system execution permanently. The system must be reset to recover.

Assertions enabled in production builds add code size and execution overhead. Consider CONFIG_↩ ENABLE_SYSTEM_ASSERT carefully based on safety vs performance requirements.

Note

Use **AssertOnElse**(condition, action) macro instead of calling this function directly. The macro provides automatic file/line capture and better readability.

Assertions can be completely disabled by not defining CONFIG_ENABLE_SYSTEM_ASSERT, causing **AssertOnElse**() to compile to nothing for zero overhead in production.

The file_ and line_ parameters help identify exactly where the assertion failed, making debugging much faster than generic error messages.

Consider different assert behaviors for development vs production: development might halt immediately, while production might log and attempt graceful degradation.

See also

AssertOnElse() - Recommended macro for triggering assertions

CONFIG_SYSTEM_ASSERT_BEHAVIOR - Controls assertion response

CONFIG_ENABLE_SYSTEM_ASSERT - Enables/disables assertion system

xSystemHalt() - May be called by assertion handler

4.2.4.61 xSystemGetSystemInfo() `Return_t xSystemGetSystemInfo ( SystemInfo_t ** info_ )`

Obtains detailed information about the HeliOS system configuration and current runtime state. The function allocates and populates a SystemInfo_t

- structure containing the operating system name, version string, and task count. This information is useful for diagnostics, logging, runtime monitoring, and version verification.

The returned SystemInfo_t ∗structure contains:

- **productName**: String identifying the operating system (e.g., "HeliOS")
- **productVersion**: Version string in semantic versioning format (e.g., "2.1.0")
- **numberOfTasks**: Current count of tasks registered with the scheduler, including both running and suspended tasks

Memory for the SystemInfo_t *structure is allocated from the user heap using xMemAlloc(). The caller is responsible for freeing this memory using xMemFree() when the information is no longer needed. Failure to free the structure will result in a memory leak.

This function can be called at any time after xSystemInit() has completed successfully. It works both before and after the scheduler has started, making it useful for initialization-time validation as well as runtime monitoring.

The task count reflects the instantaneous number of tasks at the moment of the call. In a running system, this count may change as tasks are created or deleted by other tasks.

Common use cases:

- System diagnostics: Log version information at startup for debugging
- Version verification: Ensure correct OS version is deployed
- Runtime monitoring: Track task count during system operation
- Status reporting: Send system information over communication interfaces
- Compatibility checking: Verify application compatibility with OS version
- Debug output: Include system details in error reports or crash dumps

Example 1: Basic system information logging

```
 void logSystemInfo(void) {
  SystemInfo_t **sysInfo = NULL;
  if (OK(xSystemGetSystemInfo(&sysInfo))) {
    printf("System: %s v%s\n", sysInfo->productName,
sysInfo->productVersion);
    printf("Active tasks: %d\n", sysInfo->numberOfTasks);
    // Clean up allocated memory xMemFree(sysInfo);
  } else {
    printf("Failed to get system info\n");
  }
}
int main(void) {
  xSystemInit();
  logSystemInfo();  // Log at startup
  // Create tasks and start scheduler
  // ...
}
```

Example 2: Version compatibility check

```
#define REQUIRED_MAJOR_VERSION 2
#define REQUIRED_MINOR_VERSION 1
Return_t checkSystemVersion(void) {
  SystemInfo_t **sysInfo = NULL;
  if (ERROR(xSystemGetSystemInfo(&sysInfo))) {
    return ReturnError;
  }
  // Parse version string (assumes "X.Y.Z" format) int major, minor, patch;
  sscanf(sysInfo->productVersion, "%d.%d.%d", &major, &minor, &patch);
  Return_t result = ReturnOK;
  if (major < REQUIRED_MAJOR_VERSION ||
      (major == REQUIRED_MAJOR_VERSION && minor < REQUIRED_MINOR_VERSION))
{
    printf("ERROR: HeliOS version %s too old (need %d.%d+)\n",
sysInfo->productVersion, REQUIRED_MAJOR_VERSION, REQUIRED_MINOR_VERSION);
    result = ReturnError;
  }
  xMemFree(sysInfo);
  return result;
}
```

Example 3: Runtime task monitoring

```
 void monitoringTask(Task_t *task, TaskParm_t *parm) {
  static Base_t lastTaskCount = 0;
  SystemInfo_t **sysInfo = NULL;
  if (OK(xSystemGetSystemInfo(&sysInfo))) {
    if (sysInfo->numberOfTasks != lastTaskCount) {
      printf("Task count changed: %d -> %d\n", lastTaskCount,
sysInfo->numberOfTasks);
      lastTaskCount = sysInfo->numberOfTasks;
```

```
    }
    xMemFree(sysInfo);
  }
}
```

Example 4: Sending system info over UART

```
 void sendSystemInfoToHost(xDevice uart) {
  SystemInfo_t **sysInfo = NULL;
  if (OK(xSystemGetSystemInfo(&sysInfo))) {
    // Format as JSON for easy parsing by host char buffer[128];
    snprintf(buffer, sizeof(buffer),
             "{\"os\":\"%s\",\"version\":\"%s\",\"tasks\":%d}\n",
sysInfo->productName, sysInfo->productVersion, sysInfo->numberOfTasks);
    // Send over UART xDeviceSimpleWrite(uart, strlen(buffer),
(Byte_t*)buffer);
    xMemFree(sysInfo);
  }
}
```

Parameters

out	*info*↩_	Pointer to SystemInfo_t ∗pointer that will receive the allocated system information structure. Must not be NULL. On success, points to allocated structure that must be freed with xMemFree(). On failure, remains unchanged.

Returns

ReturnOK if system information was successfully retrieved and the structure allocated. ReturnError if the operation failed (e.g., info_ is NULL, memory allocation failed, system not initialized).

Warning

The caller MUST free the returned SystemInfo_t ∗structure using xMemFree(). Failure to do so will leak memory from the user heap.

Do NOT free the structure using standard free() or other allocators. Only xMemFree() is compatible with HeliOS memory management.

The strings within the SystemInfo_t ∗structure (productName, productVersion) are part of the allocated structure and should NOT be freed separately. xMemFree() on the structure handles all cleanup.

The numberOfTasks value is a snapshot at the moment of the call. It may change immediately afterward if tasks are created or deleted. Do not rely on this value remaining constant.

Note

This function allocates memory using xMemAlloc() from the user heap. Ensure adequate heap space is available, especially if called frequently.

The function can be called before the scheduler starts, making it useful for startup diagnostics and initialization-time verification.

Task count includes all tasks in any state (running, suspended, waiting). It does not distinguish between active and inactive tasks.

The version string format follows semantic versioning (major.minor.patch) but the exact format is determined by the HeliOS build configuration.

See also

SystemInfo_t ∗ - Structure containing system information

xMemFree() - Must be used to free the returned structure

xSystemInit() - Must be called before this function

xTaskGetNumberOfTasks() - Alternative way to get task count

xMemGetHeapStats() - For detailed memory usage information

4.2.4.62 xSystemHalt() Return_t xSystemHalt (void)

Triggers an immediate, unrecoverable system halt that stops all task execution, disables interrupts, and freezes the processor. This function is used for critical error conditions where continued operation would be unsafe or impossible. Once halted, the system requires a hardware reset to recover.

The halt operation performs the following sequence:

- Disables all interrupts globally to prevent further interrupt processing
- Stops the scheduler to prevent task switching
- Calls port-specific halt functions that may include watchdog disabling, peripheral shutdown, or entry into low-power sleep modes
- Enters an infinite loop or processor halt instruction
- May optionally flash an LED or output diagnostic information if configured at the port layer

This function is typically called in response to unrecoverable errors such as assertion failures, stack overflow detection, memory corruption, or critical hardware faults. It provides a safe shutdown mechanism that prevents erratic behavior or dangerous actions from a malfunctioning system.

The system halt is permanent and irreversible without physical intervention (reset button, power cycle, watchdog timer, or external reset signal). This makes it suitable for fail-safe scenarios where the system must stop completely rather than risk continued operation in an unknown state.

Common use cases:

- Assertion failure handling: Stop execution when invariants are violated
- Critical error response: Halt on unrecoverable errors like stack overflow
- Safety-critical shutdown: Stop dangerous operations immediately
- Debug breakpoints: Halt for debugging when critical conditions occur
- Watchdog trigger: Halt before external watchdog forces reset
- Memory corruption detection: Stop when heap or kernel memory is damaged

Example 1: Assertion failure handler

```
// Custom assertion behavior using halt void assertionHandler(const char
file, int line) {
  // Log assertion information if possible
  #ifdef DEBUG_UART printf("ASSERTION FAILED: %s:%d\n", file, line);
  #endif
  // Halt system - no recovery possible xSystemHalt();
  // Never reached
}
```

Example 2: Critical error handling

```
 void criticalErrorHandler(ErrorCode error) {
  // Flash error code on LED if available for (int i = 0; i < error; i++) {
    toggleErrorLED();
    delayMs(200);
  }
  // Critical error - cannot continue xSystemHalt();
}
void sensorTask(Task_t *task, TaskParm_t *parm) {
  Byte_t buffer[128];
  if (ERROR(xDeviceSimpleRead(sensorDevice, 128, buffer))) {
    // Sensor communication critical for safety
criticalErrorHandler(ERROR_SENSOR_COMM);
  }
  // Process sensor data
```

```
  // ...
}
```

Example 3: Stack overflow detection

```
#define STACK_CANARY 0xDEADBEEF
void checkStackIntegrity(void) {
  volatile uint32_t *stackCanary = (uint32_t*)STACK_CANARY_ADDRESS;
  if (*stackCanary != STACK_CANARY) {
    // Stack overflow detected!
    #ifdef DEBUG_OUTPUT printf("FATAL: Stack overflow detected\n");
    #endif
    xSystemHalt();  // Stop immediately
  }
}
void taskWithStackCheck(Task_t *task, TaskParm_t *parm) {
  checkStackIntegrity();
  // Perform task operations
  // ...
}
```

Example 4: Watchdog-aware halt

```
 void safeHalt(void) {
  // Disable watchdog before halting to prevent continuous resets
  #ifdef WATCHDOG_ENABLED disableWatchdog();
  #endif
  // Output diagnostic information
  #ifdef DEBUG_MODE SystemInfo_t **info = NULL;
  if (OK(xSystemGetSystemInfo(&info))) {
    printf("System halted. Tasks: %d\n", info->numberOfTasks);
    xMemFree(info);
  }
  #endif
  // Halt system xSystemHalt();
}
```

Returns

This function typically does NOT return. If it does return ReturnOK, the halt operation completed but the port layer implementation allowed return (unusual). ReturnError if halt could not be performed (rare - usually indicates port layer issue).

Warning

This function NEVER returns under normal circumstances. Any code following xSystemHalt() will not execute. Always treat this as a terminal function call.

Once halted, the system cannot be recovered without a hardware reset. All task state, memory contents, and device configurations are lost on reset.

Interrupts are disabled before halting. This means any pending or future interrupt requests will not be serviced. External hardware expecting interrupt acknowledgment may time out or enter error states.

On battery-powered systems, halting may not fully power down the device. Consider entering a low-power sleep mode instead if power conservation is important.

Do NOT use this function for normal shutdown or task termination. It is only for critical, unrecoverable error conditions. For normal operation control, use task management functions.

Note

Some port implementations may flash an LED or output a diagnostic pattern before entering the halt loop, which can aid in debugging.

The port layer implementation (xPortSystemHalt()) determines the exact behavior. Common implementations include infinite loops, WFI (Wait For Interrupt) instructions, or entering deep sleep modes.

In some debugging environments (JTAG, debugger attached), the halt may trigger a breakpoint that allows the developer to inspect system state.

If a watchdog timer is enabled and not disabled before halting, the watchdog will eventually reset the system, which may or may not be desired depending on the application.

See also

xSystemAssert() - May call this function when assertions are configured to halt

xSystemInit() - Must be called before system is operational

CONFIG_SYSTEM_ASSERT_BEHAVIOR - May configure assertions to halt

xPortSystemHalt() - Port-specific halt implementation (internal)

4.2.4.63 xSystemInit() `Return_t xSystemInit ( void )`

Performs initial system bootstrap and initialization of the HeliOS real-time kernel. This function MUST be the very first HeliOS function called in any application, as it establishes the fundamental runtime environment for all subsequent operations.

The initialization process performs several critical operations in sequence:

- Initializes the kernel memory management subsystem (both user heap and kernel memory pools)
- Establishes internal kernel data structures for task management, scheduling, and device tracking
- Calls port-specific initialization functions to configure hardware-dependent features (timers, interrupts, system clock)
- Sets up the system state machine to prepare for task creation and scheduler startup
- Validates configuration parameters defined at compile time

After successful initialization, the system is ready to accept task creation requests (xTaskCreate()) and device registrations (xDeviceConfig()). The scheduler itself is not started until xTaskStartScheduler() is called, allowing the application to set up all tasks and devices before cooperative multitasking begins.

Initialization is a one-time operation that cannot be reversed without a system reset. The function maintains internal state to detect and reject multiple initialization attempts, returning ReturnError if called more than once.

Common use cases:

- Standard application startup: Initialize HeliOS before any other system calls
- Robust initialization: Check return value to detect initialization failures
- Configuration validation: Verify system configuration before proceeding
- Resource preparation: Ensure kernel resources are ready before task creation
- Hardware setup: Trigger port-specific hardware initialization sequences
- Memory pool establishment: Set up heap and kernel memory allocators

Example 1: Basic application initialization

```
 int main(void) {
  // First thing: initialize HeliOS if (ERROR(xSystemInit())) {
    // Initialization failed - cannot proceed
    // On embedded systems, might enter error loop while (1) {
      // Flash error LED or output diagnostic
    }
  }
  // System ready - create tasks Task_t *taskHandle = NULL;
  xTaskCreate(&taskHandle, "MainTask", sensorTask, NULL);
  xTaskResume(taskHandle);
  // Start scheduler xTaskStartScheduler();
  // Never reached return 0;
}
```

Example 2: Initialization with system validation

```
#include <stdio.h>
Return_t initializeSystem(void) {
  Return_t res = xSystemInit();
  if (OK(res)) {
    // Get and validate system information SystemInfo_t **sysInfo = NULL;
    if (OK(xSystemGetSystemInfo(&sysInfo))) {
      printf("Initialized %s v%s\n", sysInfo->productName,
sysInfo->productVersion);
      xMemFree(sysInfo);
    }
  }
  return res;
}
int main(void) {
  if (ERROR(initializeSystem())) {
    return -1;
  }
  // Continue with application setup return 0;
}
```

Example 3: Memory configuration validation after init

```
 int main(void) {
  // Initialize system if (ERROR(xSystemInit())) {
    return -1;
  }
  // Verify memory configuration is adequate xMemStats *heapStats = NULL;
  if (OK(xMemGetHeapStats(&heapStats))) {
    if (heapStats->regionSizeInBytes < 4096) {
      // Warning: limited memory available
    }
    xMemFree(heapStats);
  }
  // Proceed with task creation
  // ...
  return 0;
}
```

Example 4: Multiple subsystem initialization

```
// Application-specific initialization sequence Return_t appInit(void) {
  // Step 1: Initialize HeliOS (MUST be first) if (ERROR(xSystemInit())) {
    return ReturnError;
  }
  // Step 2: Initialize device drivers xDevice uartDev = NULL;
  if (ERROR(xDeviceInitDevice("UART0   ", &uartDev, myUARTDriver))) {
    return ReturnError;
  }
  // Step 3: Create application tasks Task_t *commTask = NULL;
  if (ERROR(xTaskCreate(&commTask, "CommTask", commHandler, uartDev))) {
    return ReturnError;
  }
  xTaskResume(commTask);
  return ReturnOK;
}
int main(void) {
  if (OK(appInit())) {
    xTaskStartScheduler();  // Start cooperative multitasking
  }
  return 0;
}
```

Returns

ReturnOK if initialization completed successfully and the system is ready for task creation and device registration. ReturnError if initialization failed (e.g., already initialized, memory pool configuration invalid, port layer initialization failure).

Warning

This function MUST be called before any other HeliOS functions. Calling any HeliOS function before xSystemInit() results in undefined behavior and likely system crashes.

This function can only be called ONCE. Subsequent calls will return ReturnError. The system cannot be re-initialized without a hardware reset.

Initialization failure typically indicates a critical system problem (invalid configuration, hardware failure, etc.). If this function returns ReturnError, the application should not attempt to use any HeliOS features.

On embedded systems without an operating system, initialization failure often requires entering an infinite error loop or triggering a watchdog reset, as there is no graceful way to recover.

Note

This function does NOT start the scheduler. After initialization, tasks must be created with xTaskCreate() and the scheduler started with xTaskStartScheduler().

Port-specific initialization (xPortSystemInit()) is called internally and varies by platform. This may include timer setup, interrupt configuration, or other hardware-dependent operations.

The memory configuration (CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS) is validated during initialization. Invalid configurations will cause initialization to fail.

Memory allocations (xMemAlloc, xMemFree) can be performed after successful initialization but before the scheduler starts. This is useful for pre-allocating shared resources.

See also

xTaskCreate() - Create tasks after initialization

xTaskStartScheduler() - Start scheduler after task setup

xSystemGetSystemInfo() - Query system information post-init

xSystemHalt() - Halt system in case of critical errors

CONFIG_MEMORY_REGION_SIZE_IN_BLOCKS - Memory configuration

xPortSystemInit() - Port-specific initialization (internal)

4.2.4.64 xTaskChangePeriod()

```
Return_t xTaskChangePeriod (
    Task_t * task_,
    const Ticks_t period_ )
```

Changes the interval period that controls how frequently a task executes under the HeliOS scheduler. The period determines the minimum time (in system ticks) between successive executions of the task's callback function. This enables periodic task execution for time-driven operations like sensor sampling, LED blinking, or periodic communication.

Period semantics:

- **Period = 0**: Task runs every scheduler cycle (maximum frequency)
- **Period $>$ 0**: Task runs when elapsed ticks $>=$ period

The period is measured in system ticks, which are platform-dependent but typically represent 1 millisecond. A task with period 100 executes approximately every 100ms (10 Hz), assuming the scheduler runs at least that frequently.

Tasks created with xTaskCreate() default to period 0, meaning they run on every scheduler cycle. Use this function to set periodic behavior after creation. The period can be changed at any time - even while the scheduler is running - allowing dynamic adjustment of task execution frequency.

Common use cases:

- Periodic sampling: Set sensor tasks to sample at regular intervals
- LED patterns: Control blink rates by adjusting period
- Communication timing: Schedule periodic transmissions
- Adaptive frequency: Dynamically adjust task rate based on workload
- Power management: Reduce update frequency to save power

Parameters

in	*task_*	Task handle. Must be valid from xTaskCreate() or xTaskGetHandleByName().
in	*period_*	Execution period in ticks. 0 = every cycle, >0 = periodic.

Returns

ReturnOK if period was set. ReturnError if task_ invalid.

Warning

Period 0 causes task to run every scheduler cycle, consuming maximum CPU time. Use sparingly to avoid starving other tasks.

See also

xTaskGetPeriod() - Query current task period

xTaskCreate() - Tasks default to period 0

4.2.4.65 xTaskChangeWDPeriod()

```
Return_t xTaskChangeWDPeriod (
    Task_t * task_,
    const Ticks_t period_ )
```

Configures the per-task watchdog timer period that automatically suspends tasks exceeding their execution time budget. This safety feature prevents runaway tasks from monopolizing CPU time and starving other tasks in the system.

When a task's single execution exceeds the watchdog period, HeliOS automatically suspends it (transitions to TaskStateSuspended), preventing further execution until explicitly resumed. This protects system responsiveness and prevents infinite loops or excessive computation from blocking other tasks.

Watchdog semantics:

- Monitors single execution duration, not cumulative time
- Automatically suspends tasks that exceed period
- Requires CONFIG_TASK_WD_TIMER_ENABLE at compile time
- Per-task configuration (each task has independent watchdog)

Common use cases:

- Infinite loop protection: Catch tasks stuck in loops
- Execution time enforcement: Ensure tasks complete within budgets
- System stability: Prevent CPU monopolization
- Debug assistance: Identify performance problems

Parameters

in	*task_*	Task handle. Must be valid.
in	*period_*	Watchdog timeout in ticks. Task suspended if single execution exceeds this duration.

Returns

ReturnOK if watchdog period set. ReturnError if task_ invalid or CONFIG_TASK_WD_TIMER_ENABLE not defined.

Warning

Requires CONFIG_TASK_WD_TIMER_ENABLE defined at compile time. Without this, function has no effect.

Suspended tasks must be manually resumed with xTaskResume(). Watchdog does not auto-recover.

See also

xTaskGetWDPeriod() - Query watchdog period

xTaskResume() - Resume suspended task

CONFIG_TASK_WD_TIMER_ENABLE - Enables watchdog feature

4.2.4.66 xTaskCreate()

```
Return_t xTaskCreate (
    Task_t ** task_,
    const Byte_t * name_,
    void(*)(Task_t *task_, TaskParm_t *parm_) callback_,
    TaskParm_t * taskParameter_ )
```

Creates a new task and registers it with the HeliOS scheduler. Tasks are the fundamental unit of execution in HeliOS and represent functions that execute cooperatively under scheduler control. Each task has a name, callback function, optional parameters, and scheduling properties.

When created, tasks begin in TaskStateSuspended state and will not execute until xTaskResume() is called. Tasks can have an associated period that controls how frequently they are scheduled for execution.

Task Lifecycle:

1. Create task with xTaskCreate() (starts in TaskStateSuspended) 2. Optionally set task period with xTaskChangePeriod() 3. Resume task with xTaskResume() (transitions to TaskStateRunning) 4. Task executes when scheduler runs 5. Delete task with xTaskDelete() when no longer needed

Warning

This function must be called BEFORE xTaskStartScheduler() is invoked. It cannot be called from within a task while the scheduler is running. Creating or deleting tasks during scheduler execution will result in undefined behavior.

Note

Task names are used for identification and debugging. They must be exactly CONFIG_TASK_NAME_BYTES characters (default 8). Shorter names should be space-padded; longer names will be truncated.

The taskParameter_ allows passing data to the task. This is commonly used to pass pointers to configuration structures or shared data. The memory management of this parameter is the responsibility of the caller.

Example Usage:

```
 void blinkLED(Task_t *task, TaskParm_t *parm) {
  // Access parameter if needed int *ledPin = (int*)parm;
  // Task logic here toggleLED(*ledPin);
  // Task yields when function returns
}
int main(void) {
  Task_t *ledTask;
  static int pin = 13;  // Must persist beyond task creation
  // Create task (starts in suspended state) if (OK(xTaskCreate(&ledTask,
"LED      ", blinkLED, (TaskParm_t *)&pin))) {
    // Set task to run every 100ms xTaskChangePeriod(ledTask, 100);
    // Activate the task xTaskResume(ledTask);
    // Start scheduler xTaskStartScheduler();
  }
}
```

Parameters

out	*task_*	Pointer to Task_t *variable that will receive the task handle. This handle is used in subsequent operations on the task.
in	*name_*	Task name string, exactly CONFIG_TASK_NAME_BYTES bytes. Common pattern: "TaskName" (8 bytes with space padding).
in	*callback_*	Pointer to the task function that will be executed. Function signature: void func(Task_t * task, TaskParm_t *parm).
in	*task↩ Parameter_*	Optional parameter passed to the task function. Use NULL if no parameter is needed. Can be a pointer to any data type.

Returns

ReturnOK if task was successfully created, ReturnError if creation failed (e.g., out of memory, invalid parameters, or scheduler is already running).

See also

xTaskDelete() - Remove a task from the scheduler

xTaskResume() - Activate a task for scheduling

xTaskSuspend() - Deactivate a task

xTaskChangePeriod() - Set task execution frequency

xTaskStartScheduler() - Begin executing tasks

CONFIG_TASK_NAME_BYTES - Configuration for task name length

4.2.4.67 xTaskDelete() `Return_t xTaskDelete (`
`const Task_t * task_ )`

Removes a task from the HeliOS scheduler and frees all memory associated with the task. Once deleted, the task will no longer be scheduled for execution and its task handle becomes invalid.

This function performs cleanup including:

- Removing the task from the scheduler's task list
- Freeing internal task control structures
- Invalidating the task handle

Warning

This function must be called BEFORE xTaskStartScheduler() starts or AFTER the scheduler has been suspended with xTaskSuspendAll(). It cannot be called from within a running task while the scheduler is active. Attempting to delete a task during scheduler execution will result in undefined behavior.

After deleting a task, the task handle becomes invalid and must not be used in any subsequent HeliOS function calls. Using a deleted task handle will result in undefined behavior.

Note

If a task needs to be temporarily deactivated rather than permanently removed, use xTaskSuspend() instead. Suspended tasks can be resumed later without recreating them.

Memory allocated by the task (via xMemAlloc()) is NOT automatically freed when the task is deleted. The application is responsible for managing any dynamically allocated memory used by the task.

Example Usage:

```
 Task_t *temporaryTask;
// Create a task for one-time initialization if
(OK(xTaskCreate(&temporaryTask, "InitTask", initFunction, NULL))) {
  xTaskResume(temporaryTask);
  xTaskStartScheduler();
  // Later, after initialization is complete (scheduler suspended)
xTaskSuspendAll();
  // Delete the task as it's no longer needed if
(OK(xTaskDelete(temporaryTask))) {
    // Task successfully deleted
  }
  xTaskResumeAll();
  xTaskStartScheduler();
}
```

Parameters

in	*task_*	Handle of the task to delete. Must be a valid task handle previously returned by xTaskCreate().

Returns

ReturnOK if the task was successfully deleted, ReturnError if deletion failed (invalid task handle, scheduler is running, or system error).

See also

xTaskCreate() - Create a new task

xTaskSuspend() - Temporarily deactivate a task (without deleting it)

xTaskSuspendAll() - Suspend scheduler to allow task deletion

xTaskStartScheduler() - Start the scheduler

4.2.4.68 xTaskGetAllRunTimeStats()

```
Return_t xTaskGetAllRunTimeStats (
    TaskRunTimeStats_t ** stats_,
    Base_t * tasks_ )
```

Collects and returns runtime performance statistics for every task currently registered with the HeliOS scheduler. This function provides comprehensive visibility into task execution patterns, CPU utilization, and system workload distribution. The statistics are essential for performance analysis, optimization, and debugging.

The function allocates an array of TaskRunTimeStats_t ∗structures, one entry for each task in the system. Each structure contains detailed timing and execution metrics:

- **totalRunTime**: Total accumulated execution time in system ticks
- **lastRunTime**: Duration of the most recent execution in ticks
- **taskId**: Unique identifier for the task
- Additional port-specific timing information

The statistics array is dynamically allocated from the user heap and must be freed by the caller using xMemFree() to prevent memory leaks. The number of entries in the array is returned through the tasks_ parameter, allowing the caller to iterate through all task statistics.

Runtime statistics tracking must be enabled at compile time with CONFIG_ENABLE_RUNTIME_STATS for this function to return meaningful data. If runtime stats are disabled, the function may return empty or zero-valued statistics.

The statistics represent a snapshot of task execution at the moment of the call. In a running system, these values continuously change as tasks execute. For accurate profiling, consider calling this function periodically and analyzing trends over time.

Common use cases:

- Performance profiling: Identify CPU-intensive tasks consuming excessive time
- Load balancing: Analyze workload distribution across tasks
- Optimization targets: Find tasks that are candidates for optimization
- System monitoring: Track overall system execution patterns
- Debug analysis: Investigate scheduling anomalies or starvation issues
- Capacity planning: Determine if system can handle additional tasks

Example 1: Basic runtime statistics display

```
 void displayTaskStats(void) {
  TaskRunTimeStats_t **stats = NULL;
  Base_t taskCount = 0;
  if (OK(xTaskGetAllRunTimeStats(&stats, &taskCount))) {
    printf("Runtime Statistics for %d tasks:\n", taskCount);
    for (Base_t i = 0; i < taskCount; i++) {
      printf("  Task ID %d: Total=%lu ticks, Last=%lu ticks\n",
stats[i].taskId, stats[i].totalRunTime, stats[i].lastRunTime);
    }
    xMemFree(stats);
  } else {
    printf("Failed to get runtime statistics\n");
  }
}
```

Example 2: CPU utilization percentage calculation

```
 void showCPUUtilization(void) {
  TaskRunTimeStats_t **stats = NULL;
  Base_t taskCount = 0;
  if (OK(xTaskGetAllRunTimeStats(&stats, &taskCount))) {
    // Calculate total system runtime unsigned long totalTime = 0;
    for (Base_t i = 0; i < taskCount; i++) {
      totalTime += stats[i].totalRunTime;
    }
    // Display percentage utilization per task printf("CPU
Utilization:\n");
    for (Base_t i = 0; i < taskCount; i++) {
      float percent = (stats[i].totalRunTime * 100.0f) / totalTime;
      printf("  Task %d: %.2f%%\n", stats[i].taskId, percent);
    }

    xMemFree(stats);
  }
}
```

Example 3: Identifying overloaded tasks

```
#define CPU_THRESHOLD_PERCENT 30.0f
void detectOverloadedTasks(void) {
  TaskRunTimeStats_t **stats = NULL;
  Base_t taskCount = 0;
  if (ERROR(xTaskGetAllRunTimeStats(&stats, &taskCount))) {
    return;
  }
  // Calculate total runtime unsigned long totalTime = 0;
  for (Base_t i = 0; i < taskCount; i++) {
    totalTime += stats[i].totalRunTime;
  }
  // Identify tasks exceeding threshold printf("Tasks exceeding %0.f%%
CPU:\n", CPU_THRESHOLD_PERCENT);
  for (Base_t i = 0; i < taskCount; i++) {
    float percent = (stats[i].totalRunTime * 100.0f) / totalTime;
    if (percent > CPU_THRESHOLD_PERCENT) {
      printf("  WARNING: Task %d using %.2f%% CPU\n", stats[i].taskId,
percent);
    }
  }

  xMemFree(stats);
}
```

Example 4: Periodic monitoring with trend detection

```
 void monitoringTask(Task_t *task, TaskParm_t *parm) {
  static unsigned long lastTotalRuntime[32] = {0};
  TaskRunTimeStats_t **stats = NULL;
  Base_t taskCount = 0;
  if (OK(xTaskGetAllRunTimeStats(&stats, &taskCount))) {
    for (Base_t i = 0; i < taskCount && i < 32; i++) {
      unsigned long delta = stats[i].totalRunTime - lastTotalRuntime[i];
      if (delta > 10000) {  // More than 10000 ticks since last check
printf("Task %d execution increased significantly: +%lu ticks\n",
stats[i].taskId, delta);
      }
      lastTotalRuntime[i] = stats[i].totalRunTime;
    }
    xMemFree(stats);
  }
}
```

Parameters

out	*stats_*	Pointer to TaskRunTimeStats_t *pointer that will receive the allocated array of runtime statistics. One entry per task. Must not be NULL. On success, points to allocated array that must be freed with xMemFree(). On failure, remains unchanged.
out	*tasks_*	Pointer to Base_t variable that will receive the number of tasks (and array elements) in stats_. Must not be NULL. On success, contains the task count. On failure, remains unchanged.

Returns

ReturnOK if statistics were successfully retrieved and allocated. ReturnError if the operation failed (e.g., stats_ or tasks_ is NULL, memory allocation failed, no tasks exist, runtime stats not enabled).

Warning

The caller MUST free the returned array using xMemFree(). Failure to do so will leak memory from the user heap.

Runtime statistics tracking must be enabled with CONFIG_ENABLE_RUNTIME_STATS at compile time. If disabled, this function may return zero values or return an error.

The statistics are a snapshot in time. Values change continuously as tasks execute. Do not rely on absolute values remaining constant between calls.

Large systems with many tasks may allocate significant memory for the statistics array. Ensure adequate heap space is available, especially if calling frequently.

Note

The array is allocated using xMemAlloc() from the user heap. Each element corresponds to one task in the system.

The order of tasks in the array is not guaranteed and may change between calls. Use the taskId field to identify specific tasks.

Runtime values are typically in system tick units. The exact time resolution depends on the port configuration and system tick frequency.

For single-task statistics, use xTaskGetTaskRunTimeStats() to avoid allocating memory for all tasks.

See also

TaskRunTimeStats_t * - Structure containing runtime statistics

xTaskGetTaskRunTimeStats() - Get statistics for a single task

xMemFree() - Must be used to free the returned array

xTaskGetNumberOfTasks() - Get task count without statistics

CONFIG_ENABLE_RUNTIME_STATS - Must be enabled for statistics tracking

4.2.4.69 xTaskGetAllTaskInfo() `Return_t xTaskGetAllTaskInfo (`

```
TaskInfo_t ** info_,
Base_t * tasks_ )
```

Returns detailed information for all tasks in the system in a single efficient call. Allocates an array of TaskInfo_t ∗structures containing name, state, and runtime statistics for every task, providing a complete system snapshot for monitoring, diagnostics, or reporting.

This function allocates a single contiguous array from the user heap containing one TaskInfo_t ∗structure per task. The array is indexed by task ID (0 to N-1), making it easy to access information for any specific task. The caller MUST free this array with xMemFree() after use.

Each TaskInfo_t ∗in the array contains:

- **name**: Task name (CONFIG_TASK_NAME_BYTES bytes)
- **state**: Current state (Running, Suspended, or Waiting)
- **Runtime statistics**: Execution time, run counts, performance metrics

Common use cases:

- **System dashboards**: Display all task statuses and performance
- **Performance analysis**: Compare execution times across all tasks
- **System health checks**: Verify all expected tasks are present and running
- **Diagnostic dumps**: Capture complete system state for troubleshooting
- **Load balancing**: Analyze task distribution and CPU utilization

Example 1: Display complete system task status

```
void displaySystemStatus(void) {
  TaskInfo_t **allInfo = NULL;
  Base_t taskCount;
  if (OK(xTaskGetAllTaskInfo(&allInfo, &taskCount))) {
    printf("System Tasks (%u total):\n", taskCount);
    printf("%-10s %-12s %-10s %-15s\n",
           "ID", "Name", "State", "Runtime");
    printf("-------------------------------------------------\n");
    for (Base_t i = 0; i < taskCount; i++) {
      const char *stateStr =
        allInfo[i].state == TaskStateRunning ? "Running" :
        allInfo[i].state == TaskStateSuspended ? "Suspended" : "Waiting";
      printf("%-10u %-12s %-15lu\n", i, allInfo[i].name, stateStr,
allInfo[i].totalRunTime);
    }
    // Always free the array xMemFree((Addr_t *)allInfo);
  }
}
```

Example 2: Find tasks in specific states

```
void findSuspendedTasks(void) {
  TaskInfo_t **allInfo = NULL;
  Base_t taskCount;
  Base_t suspendedCount = 0;
  if (OK(xTaskGetAllTaskInfo(&allInfo, &taskCount))) {
    printf("Suspended tasks:\n");
    for (Base_t i = 0; i < taskCount; i++) {
      if (allInfo[i].state == TaskStateSuspended) {
        printf("  %s (ID: %u)\n", allInfo[i].name, i);
        suspendedCount++;
      }
    }
    if (suspendedCount == 0) {
      printf("  (none)\n");
    }
    xMemFree((Addr_t *)allInfo);
  }
```

```
}
```

Example 3: Analyze CPU utilization across tasks

```
 void analyzeCPUUsage(void) {
  TaskInfo_t **allInfo = NULL;
  Base_t taskCount;
  if (OK(xTaskGetAllTaskInfo(&allInfo, &taskCount))) {
    Word_t totalRuntime = 0;
    // Calculate total runtime for (Base_t i = 0; i < taskCount; i++) {
      totalRuntime += allInfo[i].totalRunTime;
    }
    if (totalRuntime > 0) {
      printf("CPU Usage by Task:\n");
      // Calculate and display percentage for each task for (Base_t i = 0;
i
< taskCount; i++) {
        Byte_t percentage = (Byte_t)((allInfo[i].totalRunTime * 100) /
                                     totalRuntime);
        printf("  %-12s: %3u%%\n", allInfo[i].name, percentage);
      }
    }
    xMemFree((Addr_t *)allInfo);
  }
}
```

Example 4: Periodic system health check

```
#define EXPECTED_TASKS 8
Return_t performHealthCheck(void) {
  TaskInfo_t **allInfo = NULL;
  Base_t taskCount;
  Base_t runningCount = 0;
  Base_t suspendedCount = 0;
  if (ERROR(xTaskGetAllTaskInfo(&allInfo, &taskCount))) {
    logError("Failed to get task info");
    return ReturnError;
  }
  // Verify expected task count if (taskCount != EXPECTED_TASKS) {
    logWarning("Task count mismatch: expected %u, got %u", EXPECTED_TASKS,
taskCount);
  }
  // Count tasks by state for (Base_t i = 0; i < taskCount; i++) {
    if (allInfo[i].state == TaskStateRunning) {
      runningCount++;
    } else if (allInfo[i].state == TaskStateSuspended) {
      suspendedCount++;
      logWarning("Task %s is suspended", allInfo[i].name);
    }
  }
  logInfo("Health check: %u running, %u suspended", runningCount,
suspendedCount);
  xMemFree((Addr_t *)allInfo);
  return ReturnOK;
}
```

Parameters

out	*info*↩ _	Pointer to TaskInfo_t ∗pointer that will receive the allocated array of task information structures. Array contains taskCount elements indexed by task ID. Caller MUST free with xMemFree() after use.
out	*tasks*↩ _	Pointer to variable that receives the number of tasks (and array elements) returned.

Returns

ReturnOK if information retrieved successfully, ReturnError if operation failed (memory allocation error or invalid parameters).

Warning

The caller MUST free the returned array using xMemFree(). Failing to do so will cause significant memory leaks since the array size is proportional to the number of tasks.

Do not access the array after freeing it. Set the pointer to NULL after calling xMemFree() to prevent accidental use.

The returned information is a snapshot at the time of the call. Task states and statistics change continuously, so the data may be stale immediately after return.

Note

This function is more efficient than calling xTaskGetTaskInfo() in a loop, as it allocates memory once and captures all task data atomically.

The array is indexed by task ID (0 to taskCount-1), providing direct access to any task's information.

For systems with many tasks, this function allocates significant memory. Ensure sufficient heap space is available before calling.

The array size is (sizeof(TaskInfo_t *) * taskCount) bytes. On memory-constrained systems, consider iterating through tasks individually with xTaskGetTaskInfo() instead.

See also

xTaskGetTaskInfo() - Get info for single task (uses less memory)

xTaskGetNumberOfTasks() - Get task count (for array sizing)

xTaskGetAllRunTimeStats() - Get runtime stats only (smaller structures)

xMemFree() - Free the allocated array

xTaskGetHandleById() - Access individual tasks by ID

4.2.4.70 xTaskGetHandleById()

```
Return_t xTaskGetHandleById (
    Task_t ** task_,
    const Base_t id_ )
```

Searches for a task by its unique numeric identifier and returns the corresponding task handle. Each task is automatically assigned a sequential ID starting from 0 when created, providing a lightweight alternative to name-based lookup. This is faster than xTaskGetHandleByName() and useful when working with task arrays or numeric references.

Task IDs are assigned sequentially in the order tasks are created: the first task gets ID 0, the second gets ID 1, and so on. IDs are never reused even after a task is deleted, so they remain unique throughout system lifetime.

Common use cases:

- **Task arrays**: Managing tasks indexed by ID for efficient lookup
- **Numeric configuration**: Task references stored as numbers in config
- **Performance-critical code**: Faster lookup than string-based search
- **Task iteration**: Enumerating all tasks by ID from 0 to N-1
- **Compact references**: Storing task references as small integers

Example 1: Iterate through all tasks

```
 void inspectAllTasks(void) {
  Base_t numTasks;
  Task_t *task;
  // Get total task count if (ERROR(xTaskGetNumberOfTasks(&numTasks))) {
    return;
  }
  printf("Inspecting %u tasks:\n", numTasks);
  // Iterate through each task by ID for (Base_t id = 0; id < numTasks;
id++) {
    if (OK(xTaskGetHandleById(&task, id))) {
      TaskInfo_t **info = NULL;
      if (OK(xTaskGetTaskInfo(task, &info))) {
        printf("  Task %u: %s (state: %d)\n", id, info->name, info->state);
        xMemFree((Addr_t *)info);
      }
    }
  }
}
```

Example 2: Task array management

```
#define MAX_WORKER_TASKS 8 Task_t *workerTasks[MAX_WORKER_TASKS];
Base_t workerTaskIds[MAX_WORKER_TASKS];
Base_t workerCount = 0;
void createWorkerTask(const char *name) {
  Task_t *newTask;
  Base_t taskId;
  // Create the task if (OK(xTaskCreate(&newTask, name, workerFunction,
NULL))) {
    // Get its ID if (OK(xTaskGetId(newTask, &taskId))) {
      workerTasks[workerCount] = newTask;
      workerTaskIds[workerCount] = taskId;
      workerCount++;
    }
  }
}
void notifyWorker(Base_t workerIndex) {
  Task_t *task;
  // Look up task by stored ID if (OK(xTaskGetHandleById(&task,
workerTaskIds[workerIndex]))) {
    xTaskNotifyGive(task);
  }
}
```

Example 3: Configuration-driven task control

```
// Configuration might specify task IDs to enable/disable typedef struct {
  Base_t taskId;
  Base_t enabled;
} TaskConfig_t;
void applyTaskConfiguration(TaskConfig_t *config, Base_t count) {
  for (Base_t i = 0; i < count; i++) {
    Task_t *task;
    if (OK(xTaskGetHandleById(&task, config[i].taskId))) {
      if (config[i].enabled) {
        xTaskResume(task);
        logInfo("Enabled task ID %u", config[i].taskId);
      } else {
        xTaskSuspend(task);
        logInfo("Disabled task ID %u", config[i].taskId);
      }
    }
  }
}
```

Example 4: Find task by ID with error handling

```
 Return_t processTaskById(Base_t taskId) {
  Task_t *task;
  TaskState_t state;
  // Verify task exists if (ERROR(xTaskGetHandleById(&task, taskId))) {
    logError("Task ID %u not found or deleted", taskId);
    return ReturnError;
  }
  // Check task is in valid state if (OK(xTaskGetTaskState(task, &state)))
{
    if (state == TaskStateRunning) {
      // Perform operation on running task return
performTaskOperation(task);
    } else {
      logWarning("Task ID %u not running (state: %d)", taskId, state);
    }
  }
  return ReturnError;
}
```

Parameters

out	*task_*	Pointer to Task_t *variable that will receive the task handle if found.
in	*id_*	Numeric task ID to search for. Valid IDs range from 0 to (number of tasks - 1).

Returns

ReturnOK if task found, ReturnError if task ID invalid, task deleted, or invalid parameters.

Warning

Task IDs are never reused after task deletion. If a task is deleted, its ID becomes permanently invalid even if new tasks are created.

Valid task IDs range from 0 to N-1 where N is the number of tasks ever created. However, some IDs in this range may be invalid if those tasks were deleted.

Note

This function is faster than xTaskGetHandleByName() because it uses direct numeric comparison instead of string comparison.

Task IDs are assigned sequentially based on creation order, starting from 0. The first task created has ID 0, second has ID 1, etc.

To enumerate all valid tasks, use xTaskGetNumberOfTasks() to get the count, then iterate through IDs checking for ReturnOK.

Task IDs are stable for the lifetime of the task but become invalid permanently after xTaskDelete() is called on that task.

See also

xTaskGetHandleByName() - Get task handle by name (slower but more readable)

xTaskGetId() - Get numeric ID from task handle

xTaskGetNumberOfTasks() - Get total task count for iteration

xTaskGetTaskInfo() - Get comprehensive task information

xTaskCreate() - Create task (assigns automatic ID)

4.2.4.71 xTaskGetHandleByName()

```
Return_t xTaskGetHandleByName (
    Task_t ** task_,
    const Byte_t * name_ )
```

Searches for a task by its string name and returns the corresponding task handle. This enables runtime task lookup when you know the task name but don't have a direct reference to its handle, useful for inter-task communication, dynamic task management, and debugging scenarios.

Task names are assigned during task creation via xTaskCreate() and must be exactly CONFIG_TASK_NAME_BYTES (default 8) bytes in length. Names shorter than this must be null-padded. The search is case-sensitive and performs an exact byte-for-byte comparison.

Common use cases:

- **Inter-task communication**: Finding a target task to send notifications
- **Dynamic task control**: Suspending/resuming tasks by name at runtime
- **Debugging and diagnostics**: Inspecting task state by name
- **Configuration-driven systems**: Task names from config files or commands
- **Task coordination**: One task finding and controlling related tasks

Example 1: Find and notify a task by name

```
Return_t notifyTaskByName(const char *taskName) {
  Task_t *targetTask;
  Byte_t paddedName[CONFIG_TASK_NAME_BYTES];
  // Prepare padded name (CONFIG_TASK_NAME_BYTES = 8) memset(paddedName, 0,
sizeof(paddedName));
  strncpy((char*)paddedName, taskName, sizeof(paddedName));
  // Find the task if (OK(xTaskGetHandleByName(&targetTask, paddedName))) {
    // Notify the task if (OK(xTaskNotifyGive(targetTask))) {
      return ReturnOK;
    }
  }
  logError("Failed to find or notify task: %s", taskName);
  return ReturnError;
}
```

Example 2: Suspend task by name from command handler

```
void handleCommand(const char *cmd) {
  if (strncmp(cmd, "suspend ", 8) == 0) {
    Task_t *task;
    Byte_t taskName[CONFIG_TASK_NAME_BYTES];
    // Extract and pad task name memset(taskName, 0, sizeof(taskName));
    strncpy((char*)taskName, cmd + 8, sizeof(taskName));
    // Find and suspend the task if (OK(xTaskGetHandleByName(&task,
taskName))) {
      if (OK(xTaskSuspend(task))) {
        printf("Task '%s' suspended\n", taskName);
      }
    } else {
      printf("Task '%s' not found\n", taskName);
    }
  }
}
```

Example 3: Check if specific tasks are running

```
Base_t areSystemTasksRunning(void) {
  const char *requiredTasks[] = {
    "Monitor",
    "Logger",
    "Network",
    "Storage"
  };
  Base_t numTasks = 4;
  for (Base_t i = 0; i < numTasks; i++) {
    Task_t *task;
    Byte_t paddedName[CONFIG_TASK_NAME_BYTES];
    TaskState_t state;
    // Prepare name memset(paddedName, 0, sizeof(paddedName));
    strncpy((char*)paddedName, requiredTasks[i], sizeof(paddedName));
    // Check if task exists and is running if
(ERROR(xTaskGetHandleByName(&task, paddedName))) {
      logError("Required task not found: %s", requiredTasks[i]);
      return 0;
    }
    if (OK(xTaskGetTaskState(task, &state))) {
      if (state != TaskStateRunning) {
        logWarning("Task not running: %s", requiredTasks[i]);
        return 0;
      }
    }
  }
  return 1;  // All required tasks running
}
```

Example 4: Build task name from pattern

```
Return_t getSensorTask(Base_t sensorId, Task_t **task) {
  Byte_t taskName[CONFIG_TASK_NAME_BYTES];
  // Build task name: "Sensor0", "Sensor1", etc.
  memset(taskName, 0, sizeof(taskName));
  snprintf((char*)taskName, sizeof(taskName), "Sensor%u", sensorId);
  // Find the sensor task if (OK(xTaskGetHandleByName(task, taskName))) {
```

```
    return ReturnOK;
  }
  logError("Sensor task %u not found", sensorId);
  return ReturnError;
}
```

Parameters

out	*task*↩ _	Pointer to Task_t ∗variable that will receive the task handle if found.
in	*name*↩ _	Pointer to task name buffer. Must be exactly CONFIG_TASK_NAME_BYTES bytes long (default 8). Names shorter than this must be null-padded to the full length.

Returns

ReturnOK if task found, ReturnError if task not found or invalid parameters.

Warning

Task names must be exactly CONFIG_TASK_NAME_BYTES bytes. Shorter names must be null-padded. Use memset() or strncpy() to ensure proper padding.

The search is case-sensitive. "MyTask" and "mytask" are different names.

If multiple tasks have identical names (which shouldn't happen but is technically possible), this function returns the first match found. Use unique task names to avoid ambiguity.

Note

Task names are assigned during xTaskCreate(). Ensure tasks are created with meaningful, unique names for reliable lookup.

This function searches through all tasks, which may take time if many tasks exist. Consider caching task handles if lookups are frequent.

Task names cannot be changed after creation. If you need dynamic task identification, consider using task IDs with xTaskGetHandleById() instead.

See also

xTaskGetHandleById() - Get task handle by numeric ID

xTaskCreate() - Create task with name

xTaskGetTaskInfo() - Get task information including name

xTaskGetId() - Get numeric ID from task handle

CONFIG_TASK_NAME_BYTES - Task name length configuration

4.2.4.72 xTaskGetId() `Return_t xTaskGetId (`

```
const Task_t * task_,
Base_t * id_ )
```

Obtains the unique integer ID assigned to a task when it was created by xTaskCreate(). Each task in HeliOS has a unique numeric identifier that remains constant throughout the task's lifetime. Task IDs are useful for compact task identification, array indexing, and logging where numeric values are more efficient than name strings.

Task IDs are automatically assigned by HeliOS during task creation and cannot be changed. The ID assignment is typically sequential, starting from 1 (or 0 depending on implementation), but the exact assignment mechanism should be treated as opaque. Task IDs are never reused during a single boot cycle, even if tasks are deleted.

Unlike task names (which are strings of fixed length CONFIG_TASK_NAME_BYTES), task IDs are simple integers (Base_t type) that require minimal storage and can be efficiently compared, logged, or used as array indices. This makes IDs ideal for performance-critical code or space-constrained logging.

Task IDs complement task names and handles:

- **Task Handle (Task_t ∗)**: Opaque pointer used for all task operations
- **Task Name (string)**: Human-readable identifier for debugging
- **Task ID (integer)**: Compact numeric identifier for efficient processing

The ID can be used with xTaskGetHandleById() to perform reverse lookup, converting from numeric ID back to task handle. This is useful when task IDs are stored in data structures or received over communication channels.

Common use cases:

- Compact logging: Store task IDs in logs instead of full names
- Array indexing: Use task IDs as indices into task-specific data arrays
- Efficient comparison: Compare integers instead of string names
- Cross-reference: Store task IDs in data structures for later lookup
- Performance tracking: Associate numeric IDs with performance metrics
- Communication protocols: Send task IDs over serial/network interfaces

Example 1: Basic task ID retrieval

```
void printTaskId(Task_t *task) {
  Base_t taskId;
  if (OK(xTaskGetId(task, &taskId))) {
    printf("Task ID: %d\n", taskId);
  } else {
    printf("ERROR: Invalid task handle\n");
  }
}
```

Example 2: Task-specific data array using IDs as indices

```
#define MAX_TASKS 16 typedef struct {
  unsigned long executionCount;
  unsigned long errorCount;
} TaskMetrics;
TaskMetrics taskMetrics[MAX_TASKS] = {0};
void recordTaskExecution(Task_t *task) {
  Base_t taskId;
  if (OK(xTaskGetId(task, &taskId)) && taskId < MAX_TASKS) {
    taskMetrics[taskId].executionCount++;
  }
}
void recordTaskError(Task_t *task) {
  Base_t taskId;
  if (OK(xTaskGetId(task, &taskId)) && taskId < MAX_TASKS) {
```

```
    taskMetrics[taskId].errorCount++;
  }
}
```

Example 3: Compact logging with task IDs

```
 typedef struct {
  Base_t taskId;
  unsigned long timestamp;
  Byte_t eventCode;
} LogEntry;
#define LOG_SIZE 128 LogEntry eventLog[LOG_SIZE];
int logIndex = 0;
void logEvent(Task_t *task, Byte_t eventCode) {
  Base_t taskId;
  if (OK(xTaskGetId(task, &taskId))) {
    eventLog[logIndex % LOG_SIZE].taskId = taskId;
    eventLog[logIndex % LOG_SIZE].timestamp = getSystemTime();
    eventLog[logIndex % LOG_SIZE].eventCode = eventCode;
    logIndex++;
  }
}
void printLog(void) {
  for (int i = 0; i < LOG_SIZE; i++) {
    printf("Task %d: Event 0x%02X at %lu\n", eventLog[i].taskId,
eventLog[i].eventCode, eventLog[i].timestamp);
  }
}
```

Example 4: ID-based task lookup and verification

```
 Return_t verifyTaskById(Base_t expectedId) {
  // Get handle by ID Task_t *task = NULL;
  if (ERROR(xTaskGetHandleById(&task, expectedId))) {
    printf("ERROR: No task with ID %d\n", expectedId);
    return ReturnError;
  }
  // Verify ID matches (round-trip test) Base_t retrievedId;
  if (OK(xTaskGetId(task, &retrievedId))) {
    if (retrievedId == expectedId) {
      printf("Task ID %d verified\n", expectedId);
      return ReturnOK;
    }
  }
  return ReturnError;
}
```

Parameters

in	*task_*	Task handle for the task to query. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*id_*	Pointer to Base_t variable that will receive the task's unique numeric identifier. Must not be NULL. On success, contains the task ID. On failure, remains unchanged.

Returns

ReturnOK if ID was successfully retrieved. ReturnError if the operation failed (e.g., task_ is NULL or invalid, id_ is NULL, task has been deleted).

Warning

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

Task IDs should not be assumed to be sequential or start from any specific value. Always treat IDs as opaque identifiers.

Task IDs are not reused within a single boot cycle, but may be reused after a system reset. Do not persist task IDs across reboots.

When using task IDs as array indices, always validate the ID is within array bounds before indexing. IDs may be larger than expected.

Note

Task IDs are assigned automatically during xTaskCreate() and cannot be changed. They remain constant for the task's lifetime.

Task IDs are lightweight integers (Base_t type) that require minimal storage compared to name strings or handles.

The ID is particularly useful in logging and telemetry where bandwidth or storage is limited.

For reverse lookup (ID to handle), use xTaskGetHandleById(). For name lookup, use xTaskGetHandleByName().

The Base_t type is typically a 16-bit or 32-bit integer depending on platform. Check your platform's type definitions.

See also

xTaskGetHandleById() - Get task handle from ID (reverse operation)

xTaskGetName() - Get task name string

xTaskGetTaskInfo() - Get comprehensive info including ID

xTaskCreate() - Assigns task ID during creation

4.2.4.73 xTaskGetName()

```
Return_t xTaskGetName (
    const Task_t * task_,
    Byte_t ** name_ )
```

Obtains the human-readable ASCII name assigned to a task when it was created with xTaskCreate(). Task names are essential for debugging, logging, and identifying tasks in system diagnostics. This function allocates and returns a copy of the task's name string.

Task names in HeliOS have a fixed length defined by CONFIG_TASK_NAME_BYTES (default is 8 bytes). When tasks are created, names shorter than this length should be space-padded, and names longer than this length are truncated. The returned name buffer is exactly CONFIG_TASK_NAME_BYTES in size.

The name buffer is allocated from the user heap using xMemAlloc() and must be freed by the caller using xMemFree() to prevent memory leaks. This is a common pattern in HeliOS for returning variable-length or dynamically allocated data to the caller.

Task names are particularly useful for:

- **Debug output**: Including readable task names in printf/logging statements
- **Task identification**: Finding specific tasks without hard-coding task IDs
- **Error reporting**: Identifying which task encountered an error
- **System monitoring**: Displaying human-readable task information in monitoring interfaces
- **Reverse lookup**: Verifying you have the correct task handle

The name string is a direct copy of what was provided to xTaskCreate(), so any padding or truncation that occurred during creation is preserved in the returned string.

Common use cases:

- Debug logging: Include task names in diagnostic output

- Error reporting: Identify tasks in error messages
- Task verification: Confirm you have the correct task handle
- System monitoring: Display task names in status reports
- Task enumeration: Build lists of task names for user interfaces
- Reverse lookup: Verify task handle corresponds to expected name

Example 1: Basic task name retrieval for debugging

```
void debugTask(Task_t *task) {
  Byte_t *name = NULL;
  if (OK(xTaskGetName(task, &name))) {
    printf("Task name: %.*s\n", CONFIG_TASK_NAME_BYTES, name);
    xMemFree(name);
  } else {
    printf("ERROR: Failed to get task name\n");
  }
}
```

Example 2: Error reporting with task name

```
void reportTaskError(Task_t *task, const char *errorMsg) {
  Byte_t *taskName = NULL;
  if (OK(xTaskGetName(task, &taskName))) {
    printf("ERROR in task '%.*s': %s\n", CONFIG_TASK_NAME_BYTES, taskName,
errorMsg);
    xMemFree(taskName);
  } else {
    printf("ERROR in unknown task: %s\n", errorMsg);
  }
}
```

Example 3: Verify task handle corresponds to expected name

```
Return_t verifyTaskHandle(Task_t *task, const char *expectedName) {
  Byte_t *actualName = NULL;
  if (ERROR(xTaskGetName(task, &actualName))) {
    return ReturnError;  // Invalid handle
  }
  // Compare names (using strncmp for safety) Return_t result = ReturnOK;
  if (strncmp((char*)actualName, expectedName, CONFIG_TASK_NAME_BYTES) !=
0) {
    printf("WARNING: Expected task '%s' but got '%.8s'\n", expectedName,
actualName);
    result = ReturnError;
  }
  xMemFree(actualName);
  return result;
}
```

Example 4: Build list of all task names

```
void listAllTaskNames(void) {
  Base_t taskCount = 0;
  if (ERROR(xTaskGetNumberOfTasks(&taskCount))) {
    return;
  }
  printf("System Tasks (%d total):\n", taskCount);
  // Get all task info TaskInfo_t **taskInfo = NULL;
  if (OK(xTaskGetAllTaskInfo(&taskInfo, &taskCount))) {
    for (Base_t i = 0; i < taskCount; i++) {
      printf("  %d. %.*s (ID: %d)\n", i + 1, CONFIG_TASK_NAME_BYTES,
taskInfo[i].name, taskInfo[i].id);
    }
    xMemFree(taskInfo);
  }
}
```

Parameters

in	*task*↩ _	Task handle for the task to query. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*name*↩ _	Pointer to Byte_t pointer that will receive the allocated name buffer. Must not be NULL. On success, points to allocated buffer of exactly CONFIG_TASK_NAME_BYTES size that must be freed with xMemFree(). On failure, remains unchanged.

Returns

ReturnOK if name was successfully retrieved and allocated. ReturnError if the operation failed (e.g., task_ is NULL or invalid, name_ is NULL, memory allocation failed, task has been deleted).

Warning

The caller MUST free the returned name buffer using xMemFree(). Failure to do so will leak memory from the user heap.

The returned name is exactly CONFIG_TASK_NAME_BYTES in length. It may NOT be null-terminated if the original name was exactly that length. Always use length-limited string functions (strncmp, snprintf with width) when working with task names.

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

Do not modify the task's name after creation. Task names are immutable in HeliOS. This function returns a copy - modifying the copy does not affect the task's actual name.

Note

The name buffer is allocated using xMemAlloc() from the user heap. Ensure adequate heap space is available.

Task names are set during xTaskCreate() and cannot be changed afterward. This function only retrieves the existing name.

The CONFIG_TASK_NAME_BYTES configuration (default 8) determines the exact size of all task names. This is a compile-time constant.

For efficient lookup by name without allocating memory, use xTaskGetHandleByName() which searches for tasks by name string.

The returned name may contain spaces if the original name was shorter than CONFIG_TASK_NAME_BYTES and was space-padded during creation.

See also

xTaskCreate() - Assigns the task name during creation

xTaskGetHandleByName() - Find task by name (reverse operation)

xMemFree() - Must be used to free the returned buffer

xTaskGetTaskInfo() - Get comprehensive info including name

CONFIG_TASK_NAME_BYTES - Defines task name length

4.2.4.74 xTaskGetNumberOfTasks() `Return_t xTaskGetNumberOfTasks ( Base_t * tasks_ )`

Returns the total count of all tasks currently managed by HeliOS, regardless of their state (running, suspended, or waiting). This count includes tasks in all states and is useful for system monitoring, task enumeration, and diagnostic reporting.

The returned count reflects the current number of active task control blocks in the system. This number changes dynamically as tasks are created with xTaskCreate() and deleted with xTaskDelete(). The count includes tasks in all states: running, waiting, and suspended.

Common use cases:

- **Task iteration**: Determining loop bounds for enumerating all tasks
- **System monitoring**: Tracking task count for capacity management
- **Diagnostics**: Reporting system resource utilization
- **Validation**: Verifying expected number of tasks are active
- **Dynamic arrays**: Allocating arrays sized to hold all task data

Example 1: Enumerate and display all tasks

```
 void listAllTasks(void) {
  Base_t taskCount;
  if (OK(xTaskGetNumberOfTasks(&taskCount))) {
    printf("System has %u tasks:\n", taskCount);
    for (Base_t id = 0; id < taskCount; id++) {
      Task_t *task;
      TaskInfo_t **info = NULL;
      if (OK(xTaskGetHandleById(&task, id))) {
        if (OK(xTaskGetTaskInfo(task, &info))) {
          printf("  %u: %s [%s]\n", id, info->name, info->state ==
TaskStateRunning ? "Running" :
                  info->state == TaskStateSuspended ? "Suspended" :
 "Waiting");
          xMemFree((Addr_t *)info);
        }
      }
    }
  }
}
```

Example 2: Monitor task count for system health

```
#define EXPECTED_TASK_COUNT 10
#define MAX_TASK_COUNT 15
void monitorSystemHealth(void) {
  Base_t taskCount;
  if (OK(xTaskGetNumberOfTasks(&taskCount))) {
    if (taskCount < EXPECTED_TASK_COUNT) {
      logWarning("Task count below expected: %u (expected %u)", taskCount,
EXPECTED_TASK_COUNT);
    } else if (taskCount > MAX_TASK_COUNT) {
      logError("Task count exceeds maximum: %u (max %u)", taskCount,
MAX_TASK_COUNT);
    } else {
      logInfo("Task count normal: %u tasks", taskCount);
    }
  }
}
```

Example 3: Allocate array for all task info

```
 Return_t analyzeAllTasks(void) {
  Base_t taskCount;
  TaskInfo_t **allInfo = NULL;
  // Get task count if (ERROR(xTaskGetNumberOfTasks(&taskCount))) {
    return ReturnError;
  }
  // Allocate array to hold all task info Size_t arraySize =
sizeof(TaskInfo_t *) * taskCount;
  if (ERROR(xMemAlloc((volatile Addr_t **)&allInfo, arraySize))) {
    return ReturnError;
  }
  // Collect info for each task for (Base_t i = 0; i < taskCount; i++) {
    Task_t *task;
    TaskInfo_t **info = NULL;
    if (OK(xTaskGetHandleById(&task, i))) {
      if (OK(xTaskGetTaskInfo(task, &info))) {
        allInfo[i] = *info;  // Copy to array xMemFree((Addr_t *)info);
      }
    }
  }
  // Analyze tasks... analyzeTaskArray(allInfo, taskCount);
  xMemFree((Addr_t *)allInfo);
  return ReturnOK;
}
```

Example 4: Wait for all worker tasks to be created

```
#define EXPECTED_WORKERS 5
Return_t waitForWorkersReady(void) {
  Base_t retries = 0;
  Base_t maxRetries = 100;
  while (retries < maxRetries) {
```

```
    Base_t taskCount;
    if (OK(xTaskGetNumberOfTasks(&taskCount))) {
      // Assuming 1 main task + EXPECTED_WORKERS if (taskCount >= (1 +
EXPECTED_WORKERS)) {
        logInfo("All worker tasks ready");
        return ReturnOK;
      }
    }
    // Wait a bit for tasks to be created xTaskDelayUntil(10);  // 10 ticks
retries++;
  }
  logError("Timeout waiting for worker tasks");
  return ReturnError;
}
```

Parameters

out	*tasks*↩ _	Pointer to variable that receives the total number of tasks currently in the system.

Returns

ReturnOK if count retrieved successfully, ReturnError if operation failed (invalid parameter).

Note

The count includes ALL tasks regardless of state: running, suspended, and waiting. It does not distinguish between active and inactive tasks.

The returned count is a snapshot at the time of the call. Other tasks may create or delete tasks immediately after this function returns, changing the count.

This function is very lightweight and can be called frequently for monitoring without performance concerns.

Task IDs range from 0 to N-1, but if tasks have been deleted, some IDs in this range may be invalid. Use xTaskGetHandleById() and check its return value when iterating.

See also

xTaskGetHandleById() - Get task handle for iteration (use with task count)

xTaskGetAllTaskInfo() - Get comprehensive info for all tasks at once

xTaskCreate() - Create task (increases task count)

xTaskDelete() - Delete task (decreases task count)

xTaskGetTaskInfo() - Get info about individual task

4.2.4.75 xTaskGetPeriod() `Return_t xTaskGetPeriod (`

```
    const Task_t * task_,
    Ticks_t * period_ )
```

Retrieves the interval period that controls how frequently the task executes. Returns the value previously set with xTaskChangePeriod() or the default value (0) if never explicitly set.

Period values:

- **0**: Task runs every scheduler cycle (maximum frequency)
- **>0**: Task runs when elapsed time >= period (in ticks)

Parameters

in	*task_*	Task handle. Must be valid.
out	*period_*	Receives current period in ticks.

Returns

ReturnOK if period retrieved. ReturnError if task_ or period_ invalid.

See also

xTaskChangePeriod() - Set task execution period

4.2.4.76 xTaskGetSchedulerState() `Return_t xTaskGetSchedulerState ( SchedulerState_t * state_ )`

Retrieves the scheduler's current operational state, which indicates whether the scheduler is actively running tasks or has been suspended. The scheduler state determines whether xTaskStartScheduler() will enter the scheduling loop or return immediately.

Scheduler states:

- **SchedulerStateRunning**: Scheduler is active and executing tasks
- **SchedulerStateSuspended**: Scheduler suspended via xTaskSuspendAll()

This function is useful for debugging, system monitoring, or conditional logic that depends on scheduler status. Tasks can query scheduler state to determine if they're running under active scheduling or during suspended periods.

Common use cases:

- Debug diagnostics: Log scheduler state during troubleshooting
- Conditional behavior: Execute different code paths based on scheduler state
- System monitoring: Track scheduler transitions
- State verification: Confirm scheduler is in expected state

Parameters

out	*state_*	Receives current scheduler state (SchedulerStateRunning or SchedulerStateSuspended).

Returns

ReturnOK if state retrieved. ReturnError if state_ is NULL.

See also

xTaskStartScheduler() - Start scheduler (behavior depends on state)

xTaskSuspendAll() - Suspend scheduler

xTaskResumeAll() - Resume scheduler

SchedulerState_t - Scheduler state enumeration

4.2.4.77 xTaskGetTaskInfo()

```
Return_t xTaskGetTaskInfo (
        const Task_t * task_,
        TaskInfo_t ** info_ )
```

Returns detailed information about a specific task including its name, state, and runtime statistics. This provides a complete snapshot of a task's identity and performance in a single call, allocating and returning a TaskInfo_t ∗structure that must be freed by the caller.

The returned TaskInfo_t ∗structure contains:

- **name**: Task name (CONFIG_TASK_NAME_BYTES bytes)
- **state**: Current task state (TaskStateRunning, TaskStateSuspended, TaskStateWaiting)
- **Runtime statistics**: Execution time, run count, and performance metrics

This function allocates memory from the user heap for the info structure and returns it to the caller. The caller MUST free this memory with xMemFree() after use to prevent memory leaks.

Common use cases:

- **Task monitoring**: Inspecting individual task state and performance
- **Debugging**: Examining specific task behavior during development
- **Health checks**: Verifying critical tasks are in expected states
- **Performance profiling**: Analyzing task execution patterns
- **Diagnostic logging**: Recording task status for troubleshooting

Example 1: Check task state and log status

```
void checkTaskHealth(Task_t *task, const char *expectedName) {
  TaskInfo_t **info = NULL;
  if (OK(xTaskGetTaskInfo(task, &info))) {
    logInfo("Task: %s", info->name);
    logInfo("  State: %s", info->state == TaskStateRunning ? "Running" :
            info->state == TaskStateSuspended ? "Suspended" : "Waiting");
    // Verify task is as expected if (strncmp((char*)info->name,
expectedName, CONFIG_TASK_NAME_BYTES) != 0) {
      logWarning("Task name mismatch!");
    }
    if (info->state != TaskStateRunning) {
      logWarning("Task not running!");
    }
    // Always free the allocated info structure xMemFree((Addr_t *)info);
  }
}
```

Example 2: Monitor task runtime statistics

```
 void profileTask(Task_t *task) {
  TaskInfo_t **info = NULL;
  if (OK(xTaskGetTaskInfo(task, &info))) {
    printf("Task Profile: %s\n", info->name);
    printf("  Total Runtime: %lu ticks\n", info->totalRunTime);
    printf("  Run Count: %lu\n", info->runCount);
    // Calculate average runtime per execution if (info->runCount > 0) {
      Word_t avgRuntime = info->totalRunTime / info->runCount;
      printf("  Avg Runtime: %lu ticks/run\n", avgRuntime);
    }
    xMemFree((Addr_t *)info);
  }
}
```

Example 3: Find and inspect task by name

```
 Return_t inspectTaskByName(const char *taskName) {
  Task_t *task;
  TaskInfo_t **info = NULL;
  Byte_t paddedName[CONFIG_TASK_NAME_BYTES];
  // Prepare padded name memset(paddedName, 0, sizeof(paddedName));
  strncpy((char*)paddedName, taskName, sizeof(paddedName));
  // Find task if (ERROR(xTaskGetHandleByName(&task, paddedName))) {
    logError("Task '%s' not found", taskName);
    return ReturnError;
  }
  // Get detailed info if (OK(xTaskGetTaskInfo(task, &info))) {
    printf("Task Information:\n");
    printf("  Name: %s\n", info->name);
    printf("  State: %d\n", info->state);
    printf("  Statistics available\n");
    xMemFree((Addr_t *)info);
    return ReturnOK;
  }
  return ReturnError;
}
```

Example 4: Periodic task health monitoring

```
 void monitorCriticalTasks(void) {
  Task_t *watchdogTask;
  Byte_t taskName[CONFIG_TASK_NAME_BYTES] = "Watchdog";
  if (OK(xTaskGetHandleByName(&watchdogTask, taskName))) {
    TaskInfo_t **info = NULL;
    if (OK(xTaskGetTaskInfo(watchdogTask, &info))) {
      // Check if watchdog is running if (info->state != TaskStateRunning)
{
        logCritical("Watchdog task not running! State: %d", info->state);
        // Attempt to resume xTaskResume(watchdogTask);
      }
      // Check if watchdog is executing regularly static Word_t
lastRunCount
= 0;
      if (info->runCount == lastRunCount) {
        logWarning("Watchdog task may be stuck");
      }
      lastRunCount = info->runCount;
      xMemFree((Addr_t *)info);
    }
  }
}
```

Parameters

in	*task*↩ _	Task handle to query. Must be a valid task handle from xTaskCreate(), xTaskGetHandleByName(), or xTaskGetHandleById().
out	*info*↩ _	Pointer to TaskInfo_t ∗pointer that will receive the allocated task information structure. Caller MUST free this with xMemFree() after use.

Returns

ReturnOK if information retrieved successfully, ReturnError if operation failed (invalid task handle, task deleted, or memory allocation failed).

Warning

The caller MUST free the returned info structure using xMemFree(). Failing to do so will cause memory leaks. The structure is allocated by this function specifically for the caller.

Do not access the info structure after freeing it with xMemFree(). Set the pointer to NULL after freeing to prevent accidental use.

The returned information is a snapshot at the time of the call. Task state and statistics may change immediately after this function returns.

Note

This function combines task identity (name, state) with runtime statistics in a single structure, making it more convenient than calling multiple separate functions.

For bulk operations on all tasks, use xTaskGetAllTaskInfo() instead, which is more efficient than calling this function in a loop.

The TaskInfo_t *structure size depends on configuration. The function allocates the exact size needed from the heap.

See also

xTaskGetAllTaskInfo() - Get info for all tasks at once

xTaskGetTaskRunTimeStats() - Get runtime stats only (without name/state)

xTaskGetTaskState() - Get just the task state

xTaskGetName() - Get just the task name

xMemFree() - Free the allocated info structure

xTaskGetHandleByName() - Find task by name before getting info

xTaskGetHandleById() - Find task by ID before getting info

4.2.4.78 xTaskGetTaskRunTimeStats()

```
Return_t xTaskGetTaskRunTimeStats (
    const Task_t * task_,
    TaskRunTimeStats_t ** stats_ )
```

Obtains detailed runtime performance statistics for a single task identified by its task handle. This function provides targeted performance data for individual task analysis without the overhead of retrieving statistics for all tasks in the system. It's ideal for monitoring specific critical tasks or profiling individual task behavior.

The function allocates and populates a TaskRunTimeStats_t *structure containing comprehensive timing metrics for the specified task:

- **totalRunTime**: Cumulative execution time since task creation, measured in system ticks
- **lastRunTime**: Duration of the most recent task execution in ticks
- **taskId**: Unique numeric identifier for the task
- Additional port-specific timing information that may include execution counts, context switch counts, or other performance metrics

The statistics structure is allocated from the user heap and must be freed by the caller using xMemFree() to prevent memory leaks. This allocation is significantly smaller than xTaskGetAllRunTimeStats() which allocates an array for all tasks.

Runtime statistics tracking must be enabled at compile time using CONFIG_ENABLE_RUNTIME_STATS. If disabled, this function may return zero-valued statistics or an error depending on the configuration.

The returned statistics are a point-in-time snapshot. For a running task, the totalRunTime value increases continuously as the task executes. The lastRunTime value reflects the duration of the most recent complete execution cycle.

Common use cases:

- Critical task monitoring: Track execution time of safety-critical or high-priority tasks
- Performance debugging: Profile specific tasks suspected of performance issues
- Periodic sampling: Monitor individual task behavior over time
- Watchdog implementation: Verify tasks are executing within expected time bounds
- Resource optimization: Identify if specific tasks need optimization
- SLA verification: Ensure tasks meet service level agreements for execution time

Example 1: Monitor specific critical task

```
 void checkSensorTaskPerformance(Task_t *sensorTask) {
  TaskRunTimeStats_t **stats = NULL;
  if (OK(xTaskGetTaskRunTimeStats(sensorTask, &stats))) {
    printf("Sensor Task (ID %d):\n", stats->taskId);
    printf("  Total runtime: %lu ticks\n", stats->totalRunTime);
    printf("  Last execution: %lu ticks\n", stats->lastRunTime);
    xMemFree(stats);
  } else {
    printf("ERROR: Failed to get sensor task stats\n");
  }
}
```

Example 2: Watchdog for task execution time

```
#define MAX_EXECUTION_TIME_TICKS 1000
Return_t validateTaskExecution(Task_t *task) {
  TaskRunTimeStats_t **stats = NULL;
  if (ERROR(xTaskGetTaskRunTimeStats(task, &stats))) {
    return ReturnError;
  }
  Return_t result = ReturnOK;
  if (stats->lastRunTime > MAX_EXECUTION_TIME_TICKS) {
    printf("WARNING: Task %d exceeded max execution time: %lu > %d\n",
stats->taskId, stats->lastRunTime, MAX_EXECUTION_TIME_TICKS);
    result = ReturnError;
  }
  xMemFree(stats);
  return result;
}
```

Example 3: Periodic task profiling

```
 void profileTask(Task_t *task, TaskParm_t *parm) {
  static unsigned long lastTotalRuntime = 0;
  static unsigned long callCount = 0;
  TaskRunTimeStats_t **stats = NULL;
  // Get stats for the monitored task (passed as parameter) Task_t *
monitoredTask = (Task_t *)parm;
  if (OK(xTaskGetTaskRunTimeStats(monitoredTask, &stats))) {
    unsigned long delta = stats->totalRunTime - lastTotalRuntime;
    callCount++;
    if (callCount % 100 == 0) {  // Report every 100 samples printf("Task
%d: Average execution per sample: %lu ticks\n", stats->taskId, delta /
100);
      lastTotalRuntime = stats->totalRunTime;
    }

    xMemFree(stats);
  }
}
```

Example 4: Compare task execution before and after optimization

```
 typedef struct {
   unsigned long totalBefore;
   unsigned long totalAfter;
} OptimizationResults;
void measureOptimizationImpact(Task_t *task, OptimizationResults *results)
{
   TaskRunTimeStats_t **stats = NULL;
   // Measure before optimization if (OK(xTaskGetTaskRunTimeStats(task,
&stats))) {
     results->totalBefore = stats->totalRunTime;
     xMemFree(stats);
   }
   // ... perform optimization ...
   // ... let task run for measurement period ...
   // Measure after optimization if (OK(xTaskGetTaskRunTimeStats(task,
&stats))) {
     results->totalAfter = stats->totalRunTime;
     xMemFree(stats);
     unsigned long improvement = results->totalBefore - results->totalAfter;
     float percent = (improvement * 100.0f) / results->totalBefore;
     printf("Optimization reduced runtime by %.2f%%\n", percent);
   }
}
```

Parameters

in	*task*↩ _	Task handle for the task to query. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*stats*↩ _	Pointer to TaskRunTimeStats_t ∗pointer that will receive the allocated statistics structure. Must not be NULL. On success, points to allocated structure that must be freed with xMemFree(). On failure, remains unchanged.

Returns

ReturnOK if statistics were successfully retrieved and allocated. ReturnError if the operation failed (e.g., task_ is NULL or invalid, stats_ is NULL, memory allocation failed, task no longer exists, runtime stats not enabled).

Warning

The caller MUST free the returned structure using xMemFree(). Failure to do so will leak memory from the user heap.

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError. Always verify task existence before calling.

Runtime statistics tracking must be enabled with CONFIG_ENABLE_RUNTIME_STATS at compile time. If disabled, this function may return zero values or an error.

The statistics are a snapshot. For a currently executing task, the values may be outdated immediately after return. Use for trend analysis rather than instant-precise measurements.

Note

This function is more efficient than xTaskGetAllRunTimeStats() when you only need information about a single task, as it allocates less memory and performs less processing.

The structure is allocated using xMemAlloc() from the user heap. Ensure adequate heap space is available.

Runtime values are in system tick units. Convert to real time using the system tick frequency (typically configured in port layer).

The taskId in the returned structure matches the ID returned by xTaskGetId() for the same task.

See also

TaskRunTimeStats_t * - Structure containing runtime statistics

xTaskGetAllRunTimeStats() - Get statistics for all tasks at once

xMemFree() - Must be used to free the returned structure

xTaskGetId() - Get task ID for comparison

CONFIG_ENABLE_RUNTIME_STATS - Must be enabled for statistics tracking

4.2.4.79 xTaskGetTaskState()

```
Return_t xTaskGetTaskState (
    const Task_t * task_,
    TaskState_t * state_ )
```

Retrieves the current state of a task, which indicates whether the task is actively running, suspended awaiting resume, or waiting for a notification. Understanding task state is essential for debugging scheduling behavior, implementing state machines, and monitoring system health.

HeliOS tasks exist in one of three states at any given time:

- **TaskStateRunning**: Task is actively scheduled and will execute when the scheduler selects it. This is the normal operational state for tasks that should be executing their callback functions periodically.
- **TaskStateSuspended**: Task has been suspended via xTaskSuspend() and will not execute until explicitly resumed with xTaskResume(). Newly created tasks start in this state.
- **TaskStateWaiting**: Task is blocked waiting for a direct-to-task notification. It will not execute until a notification is sent via xTaskNotify() or xTaskNotifyGive(). This state is used for event-driven task synchronization.

The state query is instantaneous and reflects the task state at the moment of the call. In a running system, states can change rapidly as tasks are suspended, resumed, and notified. This function does not affect the task's state - it is purely a query operation.

State information is useful for debugging race conditions, verifying that tasks transition properly through state machines, and ensuring that synchronization mechanisms are working correctly. It's also valuable for system health monitoring and diagnostic reporting.

Common use cases:

- State verification: Confirm task is in expected state before performing operations
- Debug output: Include task state in diagnostic messages
- Health monitoring: Detect tasks stuck in unexpected states
- State machine implementation: Query state to determine next action
- Synchronization debugging: Verify tasks are waiting as expected
- System visualization: Display task states in monitoring tools

Example 1: Verify task state before operation

```
Return_t safeTaskOperation(Task_t *task) {
  TaskState_t state;
  if (ERROR(xTaskGetTaskState(task, &state))) {
    return ReturnError;  // Invalid task handle
  }
  if (state != TaskStateRunning) {
    printf("WARNING: Task not in running state (state=%d)\n", state);
    return ReturnError;
  }
  // Task is running - safe to proceed return ReturnOK;
}
```

Example 2: Debug output with state information

```
void debugPrintTaskState(Task_t *task, const char *taskName) {
  TaskState_t state;
  if (OK(xTaskGetTaskState(task, &state))) {
    const char *stateName;
    switch (state) {
      case TaskStateRunning:
        stateName = "Running";
        break;
      case TaskStateSuspended:
        stateName = "Suspended";
        break;
      case TaskStateWaiting:
        stateName = "Waiting";
        break;
      default:
        stateName = "Unknown";
    }
    printf("%s: %s\n", taskName, stateName);
  } else {
    printf("%s: Error querying state\n", taskName);
  }
}
```

Example 3: Monitoring task for stuck state

```
void monitorTask(Task_t *task, TaskParm_t *parm) {
  static TaskState_t lastState = TaskStateRunning;
  static int sameStateCount = 0;
  Task_t *monitoredTask = (Task_t *)parm;
  TaskState_t currentState;
  if (OK(xTaskGetTaskState(monitoredTask, &currentState))) {
    if (currentState == lastState) {
      sameStateCount++;
      if (sameStateCount > 1000 && currentState == TaskStateWaiting) {
        printf("WARNING: Task stuck in Waiting state for %d checks\n",
sameStateCount);
      }
    } else {
      sameStateCount = 0;
      lastState = currentState;
    }
  }
}
```

Example 4: Conditional task resume based on state

```
Return_t ensureTaskRunning(Task_t *task) {
  TaskState_t state;
  if (ERROR(xTaskGetTaskState(task, &state))) {
    return ReturnError;
  }
  // If suspended, resume it if (state == TaskStateSuspended) {
    printf("Task was suspended - resuming\n");
    return xTaskResume(task);
  }
  // If waiting, send notification to unblock if (state ==
TaskStateWaiting) {
    printf("Task was waiting - sending notification\n");
    return xTaskNotifyGive(task);
  }
  // Already running return ReturnOK;
}
```

Parameters

in	*task*↩ _	Task handle for the task to query. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*state*↩ _	Pointer to TaskState_t variable that will receive the task's current state. Must not be NULL. On success, contains one of: TaskStateRunning, TaskStateSuspended, or TaskStateWaiting. On failure, remains unchanged.

Returns

ReturnOK if state was successfully retrieved. ReturnError if the operation failed (e.g., task_ is NULL or invalid, state_ is NULL, task has been deleted).

Warning

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

The returned state is a snapshot and may change immediately after return, especially in systems with multiple tasks or interrupt handlers that modify task states.

Do not assume state persistence across function calls. Always re-query state if making decisions based on it, as another task or interrupt may have changed it.

Repeatedly polling task state in a tight loop wastes CPU cycles. Use notifications or timers for event-driven task coordination instead.

Note

This function does not change the task's state - it only queries it. To change state, use xTaskSuspend(), xTaskResume(), or notification functions.

The state query is very lightweight and can be called frequently without significant performance impact.

Newly created tasks start in TaskStateSuspended state until xTaskResume() is called.

Tasks transition to TaskStateWaiting only when explicitly waiting for notifications via xTaskNotifyTake() or x↩ TaskNotifyWait().

See also

TaskState_t - Enumeration of possible task states

xTaskSuspend() - Transition task to suspended state

xTaskResume() - Transition task to running state

xTaskNotifyTake() - Wait for notification (transitions to waiting state)

xTaskNotifyGive() - Send notification (may exit waiting state)

xTaskGetTaskInfo() - Get comprehensive task information including state

4.2.4.80 xTaskGetWDPeriod()

```
Return_t xTaskGetWDPeriod (
    const Task_t * task_,
    Ticks_t * period_ )
```

Retrieves the watchdog timer period for a task. This is the maximum execution time allowed for a single task invocation before automatic suspension occurs.

Returns the value previously set with xTaskChangeWDPeriod(). Requires CONFIG_TASK_WD_TIMER_ENABLE defined at compile time.

Parameters

in	*task*↩ _	Task handle. Must be valid.
out	*period*↩ _	Receives watchdog period in ticks.

Returns

ReturnOK if period retrieved. ReturnError if task_ or period_ invalid, or CONFIG_TASK_WD_TIMER_ENABLE not defined.

Warning

Requires CONFIG_TASK_WD_TIMER_ENABLE. Returns error if not enabled.

See also

xTaskChangeWDPeriod() - Set watchdog timeout

CONFIG_TASK_WD_TIMER_ENABLE - Enables watchdog feature

4.2.4.81 xTaskNotificationIsWaiting()

```
Return_t xTaskNotificationIsWaiting (
    const Task_t * task_,
    Base_t * res_ )
```

Queries whether a task has a pending direct-to-task notification waiting to be consumed. Returns a boolean result indicating notification presence without actually consuming the notification. This non-destructive query is useful for polling notification status or implementing conditional logic based on notification availability.

HeliOS's direct-to-task notification system provides a lightweight mechanism for task-to-task signaling. Each task has a single notification slot that can hold one pending notification sent via xTaskNotifyGive(). This function checks whether that slot currently contains a pending notification without removing it or accessing its value.

The result is a simple boolean indicator (non-zero = notification waiting, zero = no notification). Unlike xTaskNotifyTake() which blocks or returns the notification value, this function only queries the notification state and always returns immediately.

Common scenarios for checking notification status:

- **Polling patterns**: Periodically check for notifications without blocking
- **Conditional processing**: Execute different code paths based on notification presence
- **Non-blocking checks**: Determine if notification is available before calling xTaskNotifyTake()
- **Diagnostic monitoring**: Track notification patterns for debugging
- **Priority handling**: Process notifications only when available

Common use cases:

- Poll for notifications: Check periodically without blocking
- Conditional logic: Branch based on notification availability
- Diagnostic monitoring: Log notification arrival patterns
- Performance optimization: Avoid blocking on xTaskNotifyTake() when no notification exists
- Multi-source handling: Check multiple tasks for pending notifications
- Timeout implementation: Combine with counters for custom timeout behavior

Example 1: Poll for notification without blocking

```
 void pollingTask(Task_t *task, TaskParm_t *parm) {
  Base_t hasNotification;
  // Check if notification is waiting if
(OK(xTaskNotificationIsWaiting(task, &hasNotification))) {
    if (hasNotification) {
      // Notification present - process it TaskNotification_t notification;
      if (OK(xTaskNotifyTake(task, &notification))) {
        processNotification(&notification);
      }
    } else {
      // No notification - do other work performBackgroundTasks();
    }
  }
}
```

Example 2: Conditional processing based on notification

```
 void smartTask(Task_t *task, TaskParm_t *parm) {
  Base_t hasWork;
  if (OK(xTaskNotificationIsWaiting(task, &hasWork)) && hasWork) {
    // High-priority: Process notification first TaskNotification_t *notif;
    xTaskNotifyTake(task, &notif);
    handleUrgentWork(&notif);
  } else {
    // Low-priority: Do regular housekeeping performMaintenanceTasks();
  }
}
```

Example 3: Multi-task notification monitoring

```
// Global task handles Task_t *sensorTask, commTask, storageTask;
void monitorNotifications(void) {
  Base_t pending;
  int totalPending = 0;
  if (OK(xTaskNotificationIsWaiting(sensorTask, &pending)) && pending) {
    printf("Sensor task has pending notification\n");
    totalPending++;
  }
  if (OK(xTaskNotificationIsWaiting(commTask, &pending)) && pending) {
    printf("Comm task has pending notification\n");
    totalPending++;
  }
  if (OK(xTaskNotificationIsWaiting(storageTask, &pending)) && pending) {
    printf("Storage task has pending notification\n");
    totalPending++;
  }
  printf("Total pending notifications: %d\n", totalPending);
}
```

Example 4: Custom timeout with notification check

```
#define NOTIFICATION_TIMEOUT 1000  // 1000 task iterations
void taskWithCustomTimeout(Task_t *task, TaskParm_t *parm) {
  static int waitCount = 0;
  Base_t hasNotification;
  if (OK(xTaskNotificationIsWaiting(task, &hasNotification))) {
    if (hasNotification) {
      // Notification arrived - process it TaskNotification_t *notif;
      xTaskNotifyTake(task, &notif);
      handleNotification(&notif);
      waitCount = 0;  // Reset timeout
    } else {
      // No notification yet - check timeout waitCount++;
      if (waitCount >= NOTIFICATION_TIMEOUT) {
        printf("Timeout: No notification received in %d cycles\n",
waitCount);
        handleTimeout();
        waitCount = 0;
      }
    }
  }
}
```

Parameters

in	*task*↩ _	Task handle for the task to query. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*res*↩ _	Pointer to Base_t variable that will receive the result. Must not be NULL. On success, set to non-zero if a notification is pending, or zero if no notification is pending. On failure, remains unchanged.

Returns

ReturnOK if the query was successful and res_ was updated. ReturnError if the operation failed (e.g., task_ is NULL or invalid, res_ is NULL, task has been deleted).

Warning

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

The result is a snapshot in time. The notification state may change immediately after this function returns if another task sends or clears a notification.

This function only checks for notification presence. It does NOT retrieve the notification value or remove the notification. Use xTaskNotifyTake() to actually consume the notification.

Do not use tight polling loops based solely on this function - this wastes CPU cycles. Consider using event-driven patterns or periodic checks within a task's normal execution cycle.

Note

This is a non-blocking, non-destructive query. The notification (if present) remains pending after this call.

The returned value is boolean: non-zero = notification waiting, zero = no notification. The exact non-zero value should not be relied upon.

This function is lightweight and can be called frequently without significant performance impact.

Each task has only one notification slot. Multiple notifications sent before consumption will overwrite each other - only the most recent is available.

See also

xTaskNotifyGive() - Send a notification to a task

xTaskNotifyTake() - Receive and consume a notification

xTaskNotifyStateClear() - Clear a pending notification without reading it

TaskNotification_t * - Structure containing notification data

4.2.4.82 xTaskNotifyGive()

```
Return_t xTaskNotifyGive (
    Task_t * task_,
    const Base_t bytes_,
    const Byte_t * value_ )
```

Sends a lightweight direct-to-task notification to the specified task, optionally including a small data payload of up to CONFIG_NOTIFICATION_VALUE_BYTES (default 8 bytes). This provides efficient task-to-task signaling without the overhead of message queues. If the task is waiting for a notification (TaskStateWaiting), this unblocks it and transitions it to TaskStateRunning.

HeliOS's notification mechanism is optimized for simple signaling and small data transfers between tasks. Each task has a single notification slot that stores one pending notification. If a notification is sent while one is already pending, the old notification is overwritten with the new one - there is no queuing.

The notification value is optional - you can send a zero-length notification (bytes_=0, value_=NULL) purely as a wake-up signal, or include up to CONFIG_NOTIFICATION_VALUE_BYTES of data. Common uses include sending status codes, event flags, sensor readings, or pointers to shared data structures.

Notification semantics:

- **Overwrite behavior**: New notifications overwrite pending ones
- **Lightweight**: No memory allocation, minimal overhead
- **Task wakeup**: Transitions waiting tasks to running state
- **Data payload**: Optional byte array up to CONFIG_NOTIFICATION_VALUE_BYTES

Common use cases:

- Event signaling: Notify tasks of events without data transfer
- Semaphore emulation: Use zero-length notifications as binary semaphores
- Status updates: Send small status codes or flags between tasks
- Data ready signals: Notify when shared data is ready for processing
- Synchronization: Coordinate task execution sequences
- Interrupt-to-task: Signal tasks from ISRs (if port supports it)

Example 1: Simple event notification (zero-length)

```
// Producer task signals consumer when data is ready Task_t *consumerTask =
NULL;  // Obtained during initialization
void producerTask(Task_t *task, TaskParm_t *parm) {
  // Produce data prepareData();
  // Signal consumer (no data payload needed) if
(OK(xTaskNotifyGive(consumerTask, 0, NULL))) {
    printf("Consumer notified\n");
  }
}
```

Example 2: Notification with status code

```
 typedef enum {
  STATUS_SUCCESS = 0, STATUS_WARNING = 1, STATUS_ERROR = 2
} StatusCode;
void sendStatus(Task_t *targetTask, StatusCode status) {
  Byte_t statusByte = (Byte_t)status;
  if (ERROR(xTaskNotifyGive(targetTask, 1, &statusByte))) {
    printf("ERROR: Failed to send status notification\n");
  }
}
```

Example 3: Send sensor reading

```
 void sensorTask(Task_t *task, TaskParm_t *parm) {
  Task_t **displayTask = (Task_t **)parm;
  // Read temperature sensor (16-bit value) uint16_t temperature =
readTemperatureSensor();
  // Send as notification (2 bytes) xTaskNotifyGive(*displayTask,
sizeof(temperature), (Byte_t*)&temperature);
}
```

Example 4: Producer-consumer pattern

```
#define BUFFER_SIZE 128 Byte_t sharedBuffer[BUFFER_SIZE];
Task_t *processorTask;
void dataCollectorTask(Task_t *task, TaskParm_t *parm) {
  // Fill buffer with data Base_t bytesCollected =
collectDataIntoBuffer(sharedBuffer, BUFFER_SIZE);
  if (bytesCollected > 0) {
    // Notify processor with byte count xTaskNotifyGive(processorTask,
sizeof(bytesCollected), (Byte_t*)&bytesCollected);
  }
}
void dataProcessorTask(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notif;
  if (OK(xTaskNotifyTake(task, &notif))) {
    // Extract byte count from notification Base_t byteCount =
*((Base_t*)notif.value);
    // Process that many bytes from shared buffer processData(sharedBuffer,
byteCount);
  }
}
```

Parameters

in	*task_*	Task handle for the task to notify. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
in	*bytes_*	Number of bytes in the notification value payload. Must be 0 to CONFIG_NOTIFICATION_VALUE_BYTES (default 8). Use 0 for simple event notifications without data.
in	*value_*	Pointer to byte array containing the notification payload, or NULL if bytes_=0. If non-NULL, must point to valid memory of at least bytes_ length. The data is copied - the caller retains ownership of this memory.

Returns

ReturnOK if notification was successfully sent. ReturnError if the operation failed (e.g., task_ is NULL or invalid, bytes_ exceeds CONFIG_NOTIFICATION_VALUE_BYTES, task has been deleted).

Warning

If a notification is already pending for the task, it will be overwritten by the new notification. There is no queuing - only the most recent notification is stored.

The bytes_ parameter must not exceed CONFIG_NOTIFICATION_VALUE_BYTES. Exceeding this limit results in ReturnError.

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

The notification value is copied into the task's notification slot. Changes to the source buffer after this call do not affect the notification.

Note

This function can be called from any task or (if port supports) interrupt service routine to signal other tasks.

If the target task is in TaskStateWaiting (blocked in xTaskNotifyTake()), this notification immediately unblocks it and transitions it to TaskStateRunning.

Zero-length notifications (bytes_=0, value_=NULL) are valid and useful for simple event signaling without data transfer.

The CONFIG_NOTIFICATION_VALUE_BYTES configuration (default 8) determines the maximum payload size. This is a compile-time constant.

See also

xTaskNotifyTake() - Receive and consume a notification

xTaskNotificationIsWaiting() - Check if notification is pending

xTaskNotifyStateClear() - Clear a notification without reading it

TaskNotification_t * - Structure containing notification data

CONFIG_NOTIFICATION_VALUE_BYTES - Maximum notification payload size

4.2.4.83 xTaskNotifyStateClear() `Return_t xTaskNotifyStateClear ( Task_t * task_ )`

Clears (discards) any pending direct-to-task notification that is waiting for the specified task. If a notification is waiting, it is removed and its value is discarded. The task's notification state transitions from "waiting" to "no notification pending". This function is useful for resetting notification state or discarding stale notifications.

HeliOS implements a lightweight direct-to-task notification mechanism that allows one task to signal another with an optional data payload. Each task has a single notification "slot" that can hold one pending notification at a time. When a notification is sent via xTaskNotifyGive(), it remains pending until consumed via xTaskNotifyTake() or cleared with this function.

Clearing notifications is useful in several scenarios:

- **State reset**: Clearing stale notifications before starting a new operation
- **Error recovery**: Discarding notifications that arrived during error conditions
- **Synchronization reset**: Clearing notifications to ensure clean starting state
- **Spurious notification handling**: Discarding unexpected notifications

If no notification is pending when this function is called, it has no effect and still returns ReturnOK. The operation is idempotent - calling it multiple times produces the same result as calling it once.

The cleared notification's value (if any) is discarded and cannot be retrieved. If you need the notification value, use xTaskNotifyTake() instead.

Common use cases:

- Initialization: Clear any stale notifications before task begins operation
- Error recovery: Discard notifications from failed operations
- State machine reset: Clear notifications when resetting to initial state
- Timeout handling: Clear notifications after waiting period expires
- Synchronization cleanup: Remove notifications during shutdown sequences
- Protocol reset: Clear notifications when restarting communication protocol

Example 1: Clear notification before starting operation

```
 void processData(Task_t *task, TaskParm_t *parm) {
  // Clear any stale notifications from previous iterations
xTaskNotifyStateClear(task);
  // Perform data processing doWork();
  // Now wait for fresh notification TaskNotification_t *notification;
  if (OK(xTaskNotifyTake(task, &notification))) {
    // Process notification value
  }
}
```

Example 2: Error recovery with notification cleanup

```
 Return_t recoverFromError(Task_t *task) {
  // Clear any notifications that arrived during error state if
(ERROR(xTaskNotifyStateClear(task))) {
    return ReturnError;
  }
  // Reset task state resetTaskState();
  // Task ready for fresh operation return ReturnOK;
}
```

Example 3: State machine reset

```
 typedef enum {
  STATE_IDLE, STATE_PROCESSING, STATE_ERROR
```

```
} TaskState;
TaskState currentState = STATE_IDLE;
void resetStateMachine(Task_t *task) {
  // Clear any pending notifications xTaskNotifyStateClear(task);
  // Reset state to idle currentState = STATE_IDLE;
  printf("State machine reset to IDLE\n");
}
```

Example 4: Timeout with notification discard

```
 void taskWithTimeout(Task_t *task, TaskParm_t *parm) {
  static int waitCounter = 0;
  TaskNotification_t *notification;
  Base_t isWaiting;
  // Check if notification is pending if
(OK(xTaskNotificationIsWaiting(task, &isWaiting)) && isWaiting) {
    // Notification available - process it if (OK(xTaskNotifyTake(task,
&notification))) {
      processNotification(&notification);
      waitCounter = 0;
    }
  } else {
    // No notification - increment timeout counter waitCounter++;
    if (waitCounter > 100) {
      printf("Timeout waiting for notification\n");
      // Clear any notifications and reset xTaskNotifyStateClear(task);
      waitCounter = 0;
    }
  }
}
```

Parameters

in	*task*↩ _	Task handle for the task whose notification state should be cleared. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.

Returns

ReturnOK if the notification state was successfully cleared (or if no notification was pending). ReturnError if the operation failed (e.g., task_ is NULL or invalid, task has been deleted).

Warning

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

Any pending notification value is permanently discarded and cannot be retrieved. Use xTaskNotifyTake() if you need to access the notification value before clearing it.

This function only clears the notification state. It does NOT change the task's execution state (running/suspended/waiting). A task in TaskStateWaiting remains in that state after clearing notifications.

Calling this on a task that is currently blocked in xTaskNotifyTake() does not unblock the task. It only clears pending notifications, not the waiting state itself.

Note

This function is idempotent - calling it multiple times has the same effect as calling it once. It always returns ReturnOK for valid tasks, whether or not a notification was actually pending.

Each task has only one notification slot. There is no queue - only the most recent notification is stored. Clearing removes this single pending notification.

This is a lightweight operation with minimal overhead. It can be called frequently without performance concerns.

To check if a notification is pending before clearing, use xTaskNotificationIsWaiting().

See also

xTaskNotifyGive() - Send a notification to a task

xTaskNotifyTake() - Receive and consume a notification

xTaskNotificationIsWaiting() - Check if notification is pending

TaskNotification_t * - Structure containing notification data

4.2.4.84 xTaskNotifyTake()

```
Return_t xTaskNotifyTake (
    Task_t * task_,
    TaskNotification_t ** notification_ )
```

Retrieves and consumes a pending direct-to-task notification sent via xTaskNotifyGive(). If a notification is waiting, it is removed from the task's notification slot and its value is returned. If no notification is pending, the function returns ReturnError immediately (non-blocking behavior in HeliOS cooperative scheduler).

The received notification is packaged in a TaskNotification_t *structure containing the notification value (byte array) and the number of bytes. This structure must be examined to extract the notification data sent by the notifying task.

Taking a notification is a destructive operation - the notification is removed from the task's notification slot after being retrieved. If no new notifications arrive, subsequent calls to this function will return ReturnError until another notification is sent.

In HeliOS's cooperative multitasking model, tasks are not pre-emptively interrupted. Therefore, this function does not block waiting for notifications - it either returns the pending notification immediately or returns ReturnError if none is available. Tasks typically poll for notifications during their execution cycle or use xTaskNotificationIsWaiting() to check availability first.

Notification lifecycle:

1. Sender calls xTaskNotifyGive() - notification becomes pending 2. Receiver calls xTaskNotifyTake() - notification is consumed and removed 3. If no notification pending, xTaskNotifyTake() returns ReturnError

Common use cases:

- Event-driven processing: Wait for and process notifications from other tasks
- Producer-consumer: Receive data-ready signals from producer tasks
- Status reception: Receive status codes or flags from other tasks
- Synchronization: Coordinate task execution with notification handshakes
- ISR-to-task: Receive signals from interrupt handlers (port-dependent)
- Command reception: Receive small command codes or parameters

Example 1: Simple notification reception

```
 void consumerTask(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notification;
  // Try to receive notification if (OK(xTaskNotifyTake(task,
&notification))) {
    printf("Received notification with %d bytes\n", notification.bytes);
    // Process notification
  } else {
    // No notification - do other work performBackgroundWork();
  }
}
```

Example 2: Extract status code from notification

```
 typedef enum {
  CMD_START = 1, CMD_STOP = 2, CMD_RESET = 3
} CommandCode;
void commandTask(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notif;
  if (OK(xTaskNotifyTake(task, &notif)) && notif.bytes >= 1) {
    CommandCode cmd = (CommandCode)notif.value[0];
    switch (cmd) {
      case CMD_START:
        startOperation();
        break;
      case CMD_STOP:
        stopOperation();
        break;
      case CMD_RESET:
        resetOperation();
        break;
    }
  }
}
```

Example 3: Producer-consumer with byte count

```
 extern Byte_t sharedBuffer[256];
void processorTask(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notif;
  if (OK(xTaskNotifyTake(task, &notif))) {
    // Notification contains number of valid bytes in shared buffer if
(notif.bytes == sizeof(Base_t)) {
      Base_t byteCount = *((Base_t*)notif.value);
      printf("Processing %d bytes from buffer\n", byteCount);
      // Process that many bytes for (Base_t i = 0; i < byteCount; i++) {
        processBytes(sharedBuffer[i]);
      }
    }
  }
}
```

Example 4: Multi-value notification with structured data

```
 typedef struct {
  uint8_t sensorId;
  uint16_t value;
  uint8_t status;
} __attribute__((packed)) SensorData;  // 4 bytes total
void monitorTask(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notif;
  if (OK(xTaskNotifyTake(task, &notif))) {
    if (notif.bytes == sizeof(SensorData)) {
      SensorData *data = (SensorData*)notif.value;
      printf("Sensor %d: value=%u, status=%u\n", data->sensorId,
data->value, data->status);
      // Take action based on sensor reading if (data->value > THRESHOLD) {
        triggerAlarm();
      }
    }
  }
}
```

Parameters

in	*task_*	Task handle for the task receiving the notification. Must be a valid task handle obtained from xTaskCreate() or xTaskGetHandleByName(). Must not be NULL or refer to a deleted task.
out	*notification_*	Pointer to TaskNotification_t *structure that will receive the notification data. Must not be NULL. On success, contains the notification value and byte count. On failure, remains unchanged.

Returns

ReturnOK if a notification was pending and has been retrieved. ReturnError if no notification was pending, or task_ is NULL/invalid, or notification_ is NULL, or task has been deleted.

Warning

This function is non-blocking. If no notification is pending, it returns ReturnError immediately. There is no blocking/waiting behavior in HeliOS's cooperative model.

The task handle must be valid. If the task has been deleted or the handle is corrupted, this function returns ReturnError.

Taking a notification is destructive - it removes the notification from the task's slot. You cannot retrieve the same notification twice.

The notification value is embedded in the TaskNotification_t * structure. Examine notification_.bytes to determine how many bytes of notification_.value are valid.

The notification value array has a maximum size of CONFIG_NOTIFICATION_VALUE_BYTES (default 8). Ensure your code handles notifications of varying lengths correctly.

Note

HeliOS uses cooperative multitasking - tasks are not preempted. This function returns immediately whether or not a notification is available.

Each task has only one notification slot. Multiple pending notifications are not queued - only the most recent notification is available.

Zero-length notifications (bytes=0) are valid. Check notification_.bytes to determine if a value payload is included.

The TaskNotification_t *structure is not dynamically allocated - it's filled in by this function. No memory management is required.

To check for notification availability before taking, use xTaskNotificationIsWaiting().

See also

xTaskNotifyGive() - Send a notification to a task

xTaskNotificationIsWaiting() - Check if notification is pending

xTaskNotifyStateClear() - Clear a notification without reading it

TaskNotification_t * - Structure containing notification data

CONFIG_NOTIFICATION_VALUE_BYTES - Maximum notification payload size

4.2.4.85 xTaskResetTimer() `Return_t xTaskResetTimer ( Task_t * task_ )`

Resets the task's internal elapsed time counter back to zero, effectively restarting the timing measurement for periodic task execution. This forces the task to restart its period measurement from the current moment, delaying the next scheduled execution by one full period.

Each task maintains an elapsed time counter that tracks how many ticks have passed since the task last executed. When this elapsed time meets or exceeds the task's configured period (set via xTaskChangePeriod()), the scheduler runs the task. Resetting the timer to zero extends the wait time before the next execution.

Use cases:

- Synchronize task execution: Reset timing after external events
- Delay execution: Push back next execution without changing period
- Phase alignment: Coordinate multiple tasks to execute in sequence
- Event response: Restart timing after processing an event

Example 1: Delay next execution

```
void sensorTask(Task_t *task, TaskParm_t *parm) {
  if (errorDetected()) {
    // Reset timer to delay next sample xTaskResetTimer(task);
    return;
  }
  readSensor();
}
```

Example 2: Synchronize on external event

```
void displayTask(Task_t *task, TaskParm_t *parm) {
  if (dataAvailable()) {
    updateDisplay();
    // Restart timing from now xTaskResetTimer(task);
  }
}
```

Parameters

in	*task*↩ _	Task handle. Must be valid.

Returns

ReturnOK if timer reset. ReturnError if task_ invalid.

Note

This does not change the task's period, only resets the elapsed time counter to zero.

For tasks with period 0 (run every cycle), this has no practical effect since they execute every scheduler cycle regardless.

See also

xTaskChangePeriod() - Set task execution period

xTaskGetPeriod() - Query current period

4.2.4.86 xTaskResume() `Return_t xTaskResume ( Task_t * task_ )`

Transitions a task to the TaskStateRunning state, making it eligible for execution by the cooperative scheduler. Tasks in the running state are scheduled according to their configured period and will execute continuously until suspended, deleted, or transitioned to waiting state.

When a task is created with xTaskCreate(), it begins in TaskStateSuspended and will not execute until this function is called. This allows tasks to be fully configured (period, parameters, etc.) before activation.

State Transition:

- TaskStateSuspended → TaskStateRunning: Task becomes active
- TaskStateWaiting → TaskStateRunning: Task resumes normal scheduling
- TaskStateRunning → TaskStateRunning: No effect (already running)

Scheduling Behavior: Once resumed, the task will be scheduled for execution based on its period setting. Tasks with period 0 execute on every scheduler cycle. Tasks with non-zero periods execute when their elapsed time exceeds the period.

Note

This function can be called before or during scheduler execution. It safely transitions the task state and the change takes effect on the next scheduling cycle.

A task in the running state will continue executing until explicitly suspended with xTaskSuspend(), transitioned to waiting with xTaskWait(), or deleted with xTaskDelete().

Example Usage:

```
 Task_t *sensorTask;
Task_t *displayTask;
// Create tasks (both start in suspended state) xTaskCreate(&sensorTask,
"Sensor  ", readSensor, NULL);
xTaskCreate(&displayTask, "Display ", updateDisplay, NULL);
// Configure task periods xTaskChangePeriod(sensorTask, 100);   // Run
every 100ms xTaskChangePeriod(displayTask, 500);  // Run every 500ms
// Activate tasks xTaskResume(sensorTask);   // Now scheduled for execution
xTaskResume(displayTask);  // Now scheduled for execution
// Start scheduler - both tasks will now execute xTaskStartScheduler();
// Can also resume tasks while scheduler is running void controlTask(Task_t
*
task, TaskParm_t *parm) {
  if (systemReady) {
    xTaskResume(sensorTask);  // Activate sensor readings
  }
}
```

Parameters

in	*task*↩ _	Handle of the task to resume. Must be a valid task handle previously returned by xTaskCreate().

Returns

ReturnOK if the task state was successfully changed to running, ReturnError if the operation failed (invalid task handle or system error).

See also

xTaskCreate() - Create a new task (starts in suspended state)

xTaskSuspend() - Deactivate a task

xTaskWait() - Place a task in event-waiting state

xTaskGetTaskState() - Query current task state

xTaskChangePeriod() - Set task execution frequency

TaskState_t - Task state enumeration

4.2.4.87 xTaskResumeAll() `Return_t xTaskResumeAll ( void )`

Sets the scheduler state to SchedulerStateRunning, allowing the scheduler to execute tasks when xTaskStartScheduler() is called. This function does not start the scheduler itself; it only changes the scheduler's state flag.

This function is used in two scenarios:

1. After calling xTaskSuspendAll() to re-enable scheduling 2. During system initialization to set the scheduler to running state before calling xTaskStartScheduler()

Typical Usage Pattern:

- Call xTaskSuspendAll() to stop scheduler and regain control
- Perform critical operations while scheduler is suspended
- Call xTaskResumeAll() to mark scheduler as ready
- Call xTaskStartScheduler() to resume task execution

Note

This function only changes the scheduler state flag. You must still call xTaskStartScheduler() to actually begin executing tasks.

If xTaskStartScheduler() is called while the scheduler state is SchedulerStateSuspended, it will return immediately without executing any tasks.

Example Usage:

```
// Suspend scheduler from within a task xTaskSuspendAll();
// Perform time-sensitive operations performCriticalOperation();
// Resume scheduler state xTaskResumeAll();
// Restart scheduler xTaskStartScheduler();
```

Returns

ReturnOK on success, ReturnError on failure.

See also

xTaskStartScheduler() - Start executing tasks

xTaskSuspendAll() - Suspend scheduler and return control

xTaskGetSchedulerState() - Query current scheduler state

SchedulerState_t - Scheduler state enumeration

4.2.4.88 xTaskStartScheduler() `Return_t xTaskStartScheduler ( void )`

Transfers control from application code to the HeliOS scheduler, which begins executing tasks in a cooperative multitasking fashion. This is typically the last function called in main() after all tasks, timers, and other objects have been created and configured.

The scheduler operates in a continuous loop, executing tasks based on their state and elapsed time since last execution. Tasks in TaskStateRunning or TaskStateWaiting (with pending events) will be scheduled according to their configured period.

Control Flow:

- If the scheduler state is SchedulerStateRunning, execution enters the scheduler loop and does not return unless xTaskSuspendAll() is called from within a running task
- If the scheduler state is SchedulerStateSuspended, this function returns immediately without scheduling any tasks
- To resume after suspension, call xTaskResumeAll() followed by xTaskStartScheduler() again

Note

HeliOS uses cooperative multitasking. Tasks must explicitly yield control by returning from their task function to allow other tasks to execute.

Warning

Do not call this function multiple times without first suspending the scheduler using xTaskSuspendAll() and resuming it with xTaskResumeAll().

Example Usage:

```
int main(void) {
  Task_t *myTask;
  // Create tasks xTaskCreate(&myTask, "Task1", myTaskFunction, NULL);
  xTaskResume(myTask);
  // Start scheduler (does not return) xTaskStartScheduler();
  return 0;
}
```

Returns

ReturnOK if the scheduler successfully starts or is already suspended. ReturnError if a critical error occurs preventing scheduler operation. Note that under normal operation with tasks running, this function does not return until xTaskSuspendAll() is called.

See also

xTaskResumeAll() - Resume scheduler after suspension

xTaskSuspendAll() - Suspend scheduler and return control

xTaskGetSchedulerState() - Query current scheduler state

xTaskCreate() - Create a new task

SchedulerState_t - Scheduler state enumeration

4.2.4.89 xTaskSuspend() `Return_t xTaskSuspend (`

`Task_t * task_ )`

Transitions a task to the TaskStateSuspended state, preventing it from being scheduled for execution. The task remains registered with the scheduler and retains all its configuration (period, parameters, etc.), but will not execute until reactivated with xTaskResume().

Suspending a task is useful for temporarily disabling functionality without the overhead of deleting and recreating the task. Unlike xTaskWait(), which waits for specific events, suspended tasks remain inactive indefinitely until explicitly resumed.

State Transition:

- TaskStateRunning → TaskStateSuspended: Task becomes inactive
- TaskStateWaiting → TaskStateSuspended: Task becomes inactive
- TaskStateSuspended → TaskStateSuspended: No effect (already suspended)

Use Cases:

- Temporarily disable a task based on system state or mode
- Reduce CPU usage by deactivating unused functionality
- Disable tasks during power-saving modes
- Pause task execution during system reconfiguration

Note

This function can be called before or during scheduler execution. When called on an active task, the task will not be scheduled on subsequent scheduler cycles until resumed.

Suspending a task does NOT free its resources or invalidate its handle. The task can be resumed at any time with xTaskResume(). To permanently remove a task, use xTaskDelete() instead.

This function (xTaskSuspend) operates on individual tasks. For suspending the entire scheduler, use xTaskSuspendAll() instead.

Example Usage:

```
 Task_t *bluetoothTask;
Task_t *wifiTask;
xTaskCreate(&bluetoothTask, "BT Task", bluetoothHandler, NULL);
xTaskCreate(&wifiTask, "WiFi    ", wifiHandler, NULL);
xTaskResume(bluetoothTask);
xTaskResume(wifiTask);
xTaskStartScheduler();
// In a control task, disable wireless when not needed void
powerManagementTask(Task_t *task, TaskParm_t *parm) {
  if (lowPowerMode) {
    // Suspend wireless tasks to save power xTaskSuspend(bluetoothTask);
    xTaskSuspend(wifiTask);
  } else {
    // Resume wireless tasks when needed xTaskResume(bluetoothTask);
    xTaskResume(wifiTask);
  }
}
```

Parameters

in	*task_*	Handle of the task to suspend. Must be a valid task handle previously returned by xTaskCreate().

Returns

ReturnOK if the task state was successfully changed to suspended, ReturnError if the operation failed (invalid task handle or system error).

See also

xTaskCreate() - Create a new task

xTaskResume() - Reactivate a suspended task

xTaskWait() - Place a task in event-waiting state

xTaskDelete() - Permanently remove a task

xTaskSuspendAll() - Suspend the entire scheduler

xTaskGetTaskState() - Query current task state

TaskState_t - Task state enumeration

4.2.4.90 xTaskSuspendAll() `Return_t xTaskSuspendAll ( void )`

Sets the scheduler state to SchedulerStateSuspended and causes xTaskStartScheduler() to exit its scheduling loop, returning control back to the point where xTaskStartScheduler() was called. This provides a mechanism to temporarily halt cooperative multitasking and regain direct control of program execution.

When called from within a running task, this function causes the scheduler loop to terminate after the current task completes its execution. Control returns to the line immediately following the original xTaskStartScheduler() call.

Use Cases:

- Temporarily halt all task execution for critical operations
- Enter a low-power mode that requires stopping the scheduler
- Transition to a different operating mode
- Debug or diagnostic operations that require full control

To resume task execution:

1. Call xTaskResumeAll() to set scheduler state to running 2. Call xTaskStartScheduler() to re-enter the scheduling loop

Warning

When the scheduler is suspended, no tasks will execute, including timer tasks and periodic operations. Ensure critical system functions are maintained during suspension.

Note

This function must be called from within a task context, not from main() or initialization code.

Example Usage:

```
 void myTaskFunction(Task_t *task, TaskParm_t *parm) {
  // Normal task operations...
  if (needToSuspendScheduler) {
    // Suspend scheduler and return control xTaskSuspendAll();
    // Execution returns to line after xTaskStartScheduler() in main()
  }
}
int main(void) {
  Task_t *task;
  xTaskCreate(&task, "MyTask", myTaskFunction, NULL);
  xTaskResume(task);
  xTaskStartScheduler(); // Will return here when xTaskSuspendAll() called
  // Code here executes after scheduler suspension
performCriticalOperation();
  // Resume if needed xTaskResumeAll();
  xTaskStartScheduler();
}
```

Returns

ReturnOK on success, ReturnError on failure.

See also

xTaskStartScheduler() - Start the scheduler

xTaskResumeAll() - Resume scheduler state

xTaskGetSchedulerState() - Query current scheduler state

SchedulerState_t - Scheduler state enumeration

4.2.4.91 xTaskWait() `Return_t xTaskWait ( Task_t * task_ )`

Transitions a task to the TaskStateWaiting state, where it will not be scheduled for execution until a specific event occurs. This enables efficient event-driven multitasking where tasks sleep until notified, reducing unnecessary CPU usage.

Unlike TaskStateSuspended (which requires explicit xTaskResume() to reactivate), tasks in TaskStateWaiting automatically become schedulable when their awaited event occurs. Once the event is consumed (via xTaskNotifyTake() or xTaskNotificationStateClear()), the task automatically returns to waiting state.

State Transition:

- TaskStateRunning → TaskStateWaiting: Task enters event-waiting mode
- TaskStateSuspended → TaskStateWaiting: Task enters event-waiting mode
- TaskStateWaiting → TaskStateWaiting: No effect (already waiting)

Supported Event Types:

1. Task Timers: Task wakes when its timer expires (via xTaskResetTimer())
2. Direct-to-Task Notifications: Task wakes when another task sends a notification (via xTaskNotifyGive())

Event-Driven Workflow:

1. Task calls xTaskWait() to enter waiting state 2. Task stops executing and does not consume CPU 3. Another task or timer triggers an event 4. Scheduler automatically schedules the waiting task 5. Task executes and processes the event 6. Task consumes the event (xTaskNotifyTake() or xTaskNotificationStateClear()) 7. Task automatically returns to waiting state

Note

Event-driven tasks are more efficient than polling-based tasks as they only execute when needed, reducing CPU usage and power consumption.

A task in waiting state will remain inactive until its event occurs. To unconditionally activate a waiting task, use xTaskResume().

The event mechanism is edge-triggered. If an event occurs before xTaskWait() is called, the task will execute once to process it.

Example Usage:

```
 Task_t *buttonTask;
Task_t *ledTask;
// Button task waits for button press notifications void
buttonHandler(Task_t *task, TaskParm_t *parm) {
  TaskNotification_t *notification;
  // Wait for button press event xTaskWait(task);
  // Check if notification is waiting if
(OK(xTaskNotificationIsWaiting(task, &notification))) {
    // Process button press if (OK(xTaskNotifyTake(task, &notification))) {
      handleButtonPress();
      // Task automatically returns to waiting state
    }
  }
}
// Interrupt handler or another task sends notification void
buttonISR(void) {
  // Wake up button task xTaskNotifyGive(buttonTask);
}
int main(void) {
  xTaskCreate(&buttonTask, "Button  ", buttonHandler, NULL);
  xTaskResume(buttonTask);  // Initially resume the task
xTaskStartScheduler();
}
```

Parameters

in	*task*↩ _	Handle of the task to place in waiting state. Must be a valid task handle previously returned by xTaskCreate().

Returns

ReturnOK if the task state was successfully changed to waiting, ReturnError if the operation failed (invalid task handle or system error).

See also

xTaskCreate() - Create a new task

xTaskResume() - Activate a task (overrides waiting state)

xTaskSuspend() - Deactivate a task

xTaskNotifyGive() - Send a notification to wake a waiting task

xTaskNotifyTake() - Consume a notification event

xTaskNotificationIsWaiting() - Check for pending notifications

xTaskNotificationStateClear() - Clear notification state

xTaskResetTimer() - Reset task timer for timer-based events

TaskState_t - Task state enumeration

4.2.4.92 xTimerChangePeriod()

```
Return_t xTimerChangePeriod (
        Timer_t * timer_,
        const Ticks_t period_ )
```

Modifies the expiration period of an application timer without requiring timer deletion and recreation. The new period takes effect immediately and will be used for all subsequent timer expirations. This operation allows dynamic adjustment of timer intervals based on runtime conditions, enabling adaptive timing behavior in response to system state or application needs.

When xTimerChangePeriod() is called, the timer's period is updated to the new value, but the timer's current state (active/stopped) and elapsed time are preserved. If the timer is running, it continues running with the new period. The timer will expire when the elapsed time reaches the new period value, which may be sooner or later than the original period.

Key characteristics:

- **Immediate effect**: New period applies to current and future timing cycles
- **State preservation**: Timer remains active or stopped as before the change
- **Elapsed time preserved**: Current elapsed time is not reset automatically
- **No recreation needed**: Avoids overhead of deleting and recreating timer
- **Non-blocking**: Returns immediately after period update

Common period change scenarios:

- **Adaptive sampling**: Adjust sensor polling rate based on value changes
- **Power management**: Slow down non-critical timers to conserve energy
- **Load balancing**: Distribute timer expirations to avoid processing spikes
- **Configuration updates**: Apply new timing settings from user preferences
- **Error recovery**: Increase retry intervals during communication failures

Example 1: Adaptive sensor polling

```
 Timer_t *sensorPollTimer;
Byte_t lastReading = 0;
void sensorTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(sensorPollTimer, &expired)) && expired) {
    Byte_t reading = readSensor();
    Byte_t change = abs(reading - lastReading);
    // Adapt polling rate based on change magnitude if (change > 20) {
      // Fast changes - poll rapidly xTimerChangePeriod(sensorPollTimer,
10);
    } else if (change > 5) {
      // Moderate changes - normal rate xTimerChangePeriod(sensorPollTimer,
50);
    } else {
      // Stable - slow down polling xTimerChangePeriod(sensorPollTimer,
200);
    }
    lastReading = reading;
    xTimerReset(sensorPollTimer);
  }
}
```

Example 2: Power-aware timing

```
 Timer_t *backgroundTimer;
typedef enum {
  POWER_MODE_ACTIVE, POWER_MODE_SLEEP, POWER_MODE_DEEP_SLEEP
} PowerMode_t;
void setPowerMode(PowerMode_t mode) {
  switch (mode) {
```

```
    case POWER_MODE_ACTIVE:
      xTimerChangePeriod(backgroundTimer, 50);   // Fast updates break;
    case POWER_MODE_SLEEP:
      xTimerChangePeriod(backgroundTimer, 500);  // Slower updates break;
    case POWER_MODE_DEEP_SLEEP:
      xTimerChangePeriod(backgroundTimer, 5000); // Minimal updates break;
  }
}
```

Example 3: Communication retry with exponential backoff

```
 Timer_t *retryTimer;
Ticks_t currentRetryInterval = 100;
void attemptConnection(void) {
  if (connectionSuccessful()) {
    // Reset to initial interval on success currentRetryInterval = 100;
    xTimerChangePeriod(retryTimer, currentRetryInterval);
    xTimerStop(retryTimer);
  } else {
    // Exponential backoff on failure (max 10 seconds) if
(currentRetryInterval < 10000) {
      currentRetryInterval *= 2;
      xTimerChangePeriod(retryTimer, currentRetryInterval);
    }
    xTimerReset(retryTimer);
  }
}
```

Example 4: User-configurable timer

```
 Timer_t *userTimer;
void applyUserSettings(Ticks_t newInterval) {
  // Validate interval range if (newInterval < 10) {
    newInterval = 10;     // Minimum 10 ticks
  } else if (newInterval > 60000) {
    newInterval = 60000;  // Maximum 60 seconds
  }
  // Apply new interval if (OK(xTimerChangePeriod(userTimer, newInterval)))
{
    // Reset to start timing with new period xTimerReset(userTimer);
    saveSettings(newInterval);
  }
}
```

Parameters

in	*timer*↩_	Handle to the timer whose period should be changed. Must be a valid timer created with xTimerCreate().
in	*period*↩_	The new timer period in system ticks. Must be greater than zero. The timer will expire when elapsed time equals this value.

Returns

ReturnOK if period changed successfully, ReturnError if operation failed due to invalid timer handle, timer not found, or invalid period value.

Warning

Changing the period does NOT automatically reset the elapsed time. If a timer has already accumulated elapsed time, it may expire immediately if the new period is less than the current elapsed time. Call xTimerReset() after changing the period if you want to start timing from zero with the new period.

Reducing the period on a running timer may cause immediate expiration if the elapsed time already exceeds the new period. Always check or reset the timer after reducing the period.

Note

xTimerChangePeriod() can be called whether the timer is running or stopped. The new period takes effect in both cases.

The timer's active state is preserved—if it was running, it continues running; if stopped, it remains stopped.

Changing to period value 0 is invalid and will return ReturnError. To disable a timer, use xTimerStop() instead.

See also

xTimerGetPeriod() - Query current timer period

xTimerReset() - Reset elapsed time after changing period

xTimerCreate() - Create timer with initial period

xTimerStart() - Start timer with its current period

xTimerStop() - Stop timer to prevent expiration

xTimerHasTimerExpired() - Check if timer has expired

4.2.4.93 xTimerCreate()

```
Return_t xTimerCreate (
    Timer_t ** timer_,
    const Ticks_t period_ )
```

Creates a software timer that can be used for measuring time intervals, implementing delays, or triggering periodic actions. Software timers are independent of task timers and provide general-purpose timing facilities for application use.

Software timers in HeliOS are NOT the same as task timers used for event-driven multitasking. Application timers are lightweight timing objects that can be:

- Started and stopped on demand
- Reset to restart counting from zero
- Queried to check if the period has elapsed
- Modified to change their period dynamically

Timer Operation: Timers count system ticks from when they are started. When the elapsed ticks exceed the configured period, the timer is considered "expired." Applications check timer expiration using xTimerHasTimerExpired() and can reset the timer to begin a new timing cycle.

Common Use Cases:

- Implementing timeout detection
- Creating delays without blocking
- Measuring elapsed time between events
- Implementing periodic operations (e.g., blinking LED)
- Rate limiting operations
- Watchdog functionality

Note

Software timers are passive timing objects. They do not automatically trigger callbacks or events when they expire. Applications must actively check timer status using xTimerHasTimerExpired().

Timer resolution is determined by the system tick rate, which depends on how frequently xTaskStartScheduler() executes (typically tied to hardware timer interrupts or main loop frequency).

Multiple timers can be created for different timing needs. Each timer maintains independent state and period configuration.

Example Usage:

```
 Timer_t *blinkTimer;
Timer_t *timeoutTimer;
// Create a timer with 1000 tick period (e.g., 1 second if 1ms ticks) if
(OK(xTimerCreate(&blinkTimer, 1000))) {
  // Start the timer xTimerStart(blinkTimer);
  // In your task or main loop:
  Base_t expired;
  if (OK(xTimerHasTimerExpired(blinkTimer, &expired)) && expired) {
    // Timer has expired - toggle LED toggleLED();
    // Reset for next cycle xTimerReset(blinkTimer);
  }
}
// Create a timeout timer if (OK(xTimerCreate(&timeoutTimer, 5000))) {  //
5 second timeout xTimerStart(timeoutTimer);
  // Wait for operation with timeout while (!operationComplete) {
    Base_t timedOut;
    if (OK(xTimerHasTimerExpired(timeoutTimer, &timedOut)) && timedOut) {
      // Timeout occurred handleTimeout();
      break;
    }
    // Continue waiting...
  }
  // Clean up xTimerDelete(timeoutTimer);
}
```

Parameters

out	*timer*↩ _	Pointer to Timer_t *variable that will receive the timer handle. This handle is used in subsequent timer operations.
in	*period*↩ _	Timer period in system ticks. The timer expires when elapsed ticks exceed this value. Must be greater than zero.

Returns

ReturnOK if the timer was successfully created, ReturnError if creation failed (out of memory, invalid parameters, or system error).

See also

xTimerDelete() - Delete a timer and free its resources

xTimerStart() - Start a timer counting

xTimerStop() - Stop a timer

xTimerReset() - Reset a timer to zero

xTimerHasTimerExpired() - Check if timer has expired

xTimerChangePeriod() - Change timer period

xTimerGetPeriod() - Get current timer period

xTimerIsTimerActive() - Check if timer is running

4.2.4.94 xTimerDelete() `Return_t xTimerDelete ( const Timer_t * timer_ )`

Permanently removes an application timer created with xTimerCreate() and releases all associated kernel resources. After deletion, the timer handle becomes invalid and must not be used in any subsequent timer operations. This operation is typically performed during cleanup, reconfiguration, or when a timer is no longer needed by the application.

xTimerDelete() immediately removes the timer from the kernel timer list regardless of its current state (stopped, running, or expired). If the timer was active, it is stopped automatically before deletion. No further timer events will occur, and all internal timer state is freed.

Key characteristics:

- **Immediate deletion**: Timer removed regardless of active/stopped state
- **Resource cleanup**: All kernel memory associated with timer is freed
- **Handle invalidation**: Timer handle cannot be reused after deletion
- **Idempotent operation**: Safe to call even if timer was never started
- **No blocking**: Returns immediately after cleanup completes

Typical deletion scenarios:

- **Application cleanup**: Remove timers during task or module shutdown
- **Dynamic timer management**: Delete and recreate timers with different periods
- **Resource reclamation**: Free timers no longer needed to reduce memory usage
- **Error recovery**: Clean up timers after configuration or initialization failures

Example 1: Timer lifecycle management

```
 Timer_t *watchdogTimer;
// Create and use timer if (OK(xTimerCreate(&watchdogTimer, 1000))) {
  xTimerStart(watchdogTimer);
  // ... use timer for some period ...
  // Clean up when done xTimerStop(watchdogTimer);
  xTimerDelete(watchdogTimer);
  // watchdogTimer handle is now invalid
}
```

Example 2: Dynamic timer reconfiguration

```
 Timer_t *intervalTimer;
void setMonitoringInterval(Ticks_t newInterval) {
  // Delete existing timer if present if (intervalTimer != null) {
    xTimerDelete(intervalTimer);
  }
  // Create new timer with updated period if
(OK(xTimerCreate(&intervalTimer, newInterval))) {
    xTimerStart(intervalTimer);
  }
}
```

Example 3: Cleanup multiple timers

```
 Timer_t *timers[MAX_SENSORS];
Base_t timerCount = 0;
void initSensorTimers(void) {
  for (Base_t i = 0; i < MAX_SENSORS; i++) {
    if (OK(xTimerCreate(&timers[i], 100 * (i + 1)))) {
      xTimerStart(timers[i]);
      timerCount++;
    }
  }
}
void shutdownSensorTimers(void) {
  for (Base_t i = 0; i < timerCount; i++) {
```

```
    xTimerDelete(timers[i]);
  }
  timerCount = 0;
}
```

Example 4: Conditional timer cleanup

```
 Timer_t *periodicTimer;
Base_t timerActive = 0;
void disablePeriodicTask(void) {
  if (timerActive) {
    xTimerDelete(periodicTimer);
    timerActive = 0;
  }
}
void enablePeriodicTask(Ticks_t period) {
  // Clean up old timer if exists disablePeriodicTask();
  // Create new timer if (OK(xTimerCreate(&periodicTimer, period))) {
    xTimerStart(periodicTimer);
    timerActive = 1;
  }
}
```

Parameters

in	*timer*↩_	Handle to the timer to delete. Must be a valid timer created with xTimerCreate(). After deletion, this handle becomes invalid.

Returns

ReturnOK if timer deleted successfully, ReturnError if deletion failed due to invalid timer handle or timer not found.

Warning

After xTimerDelete() returns successfully, the timer handle is invalid and must not be used in any subsequent operations. Using a deleted timer handle will result in ReturnError.

Deleting a timer that is referenced by multiple tasks can lead to errors if those tasks attempt to use the timer after deletion. Coordinate timer deletion across all tasks using the timer.

Note

xTimerDelete() automatically stops the timer before deletion if it was running. You do not need to explicitly call xTimerStop() first.

Unlike task and queue deletion, timer deletion does not require waiting for operations to complete. The timer is removed immediately.

Deletion is the only way to reclaim kernel memory allocated for a timer. Simply stopping a timer does not free its resources.

See also

xTimerCreate() - Create an application timer

xTimerStop() - Stop a running timer without deleting it

xTimerStart() - Start or restart a timer

xTimerReset() - Reset timer elapsed time to zero

xTimerIsTimerActive() - Check if timer is currently running

xTaskDelete() - Delete a task (similar resource cleanup pattern)

4.2.4.95 xTimerGetPeriod()

```
Return_t xTimerGetPeriod (
        const Timer_t * timer_,
        Ticks_t * period_ )
```

Retrieves the configured expiration period for an application timer in system ticks. This non-destructive query allows tasks to inspect timer configuration without affecting timer state or operation. The returned period value reflects the most recent setting from xTimerCreate() or xTimerChangePeriod().

xTimerGetPeriod() provides read-only access to the timer's period configuration. This is useful for validation, diagnostics, dynamic adjustments based on current settings, or coordination between multiple tasks that share timer resources. The query does not modify the timer state, elapsed time, or active/stopped status.

Common use cases:

- **Configuration validation**: Verify timer period matches expected values
- **Dynamic adjustments**: Calculate new periods based on current setting
- **Diagnostics and logging**: Report timer configuration for debugging
- **Coordinated timing**: Synchronize related timers based on period ratios
- **Settings persistence**: Save current timer configuration to non-volatile memory

Example 1: Validate timer configuration

```
 Timer_t *watchdogTimer;
void initWatchdog(void) {
  Ticks_t expectedPeriod = 5000;  // 5 seconds
  if (OK(xTimerCreate(&watchdogTimer, expectedPeriod))) {
    // Verify timer created with correct period Ticks_t actualPeriod;
    if (OK(xTimerGetPeriod(watchdogTimer, &actualPeriod))) {
      if (actualPeriod == expectedPeriod) {
        xTimerStart(watchdogTimer);
      } else {
        // Configuration mismatch - handle error logError("Watchdog period
mismatch");
      }
    }
  }
}
```

Example 2: Adaptive period adjustment

```
 Timer_t *pollTimer;
void speedUpPolling(void) {
  Ticks_t currentPeriod;
  if (OK(xTimerGetPeriod(pollTimer, &currentPeriod))) {
    // Halve the period (double the rate), but enforce minimum Ticks_t
newPeriod = currentPeriod / 2;
    if (newPeriod < 10) {
      newPeriod = 10;  // Minimum 10 ticks
    }
    xTimerChangePeriod(pollTimer, newPeriod);
    xTimerReset(pollTimer);
  }
}
void slowDownPolling(void) {
  Ticks_t currentPeriod;
  if (OK(xTimerGetPeriod(pollTimer, &currentPeriod))) {
    // Double the period (halve the rate), but enforce maximum Ticks_t
newPeriod = currentPeriod * 2;
    if (newPeriod > 10000) {
      newPeriod = 10000;  // Maximum 10 seconds
    }
    xTimerChangePeriod(pollTimer, newPeriod);
    xTimerReset(pollTimer);
  }
}
```

Example 3: Coordinated timer relationships

```
 Timer_t *fastTimer;
Timer_t *slowTimer;
void setupCoordinatedTimers(void) {
  // Fast timer runs at base rate xTimerCreate(&fastTimer, 100);
  xTimerStart(fastTimer);
```

```
  // Slow timer should run at 10x the fast timer period Ticks_t fastPeriod;
  if (OK(xTimerGetPeriod(fastTimer, &fastPeriod))) {
    xTimerCreate(&slowTimer, fastPeriod * 10);
    xTimerStart(slowTimer);
  }
}
```

Example 4: System diagnostics and reporting

```
 Timer_t *timers[MAX_TIMERS];
Base_t timerCount = 0;
void reportTimerStatus(void) {
  for (Base_t i = 0; i < timerCount; i++) {
    Ticks_t period;
    Base_t isActive;
    Base_t hasExpired;
    if (OK(xTimerGetPeriod(timers[i], &period))) {
      xTimerIsTimerActive(timers[i], &isActive);
      xTimerHasTimerExpired(timers[i], &hasExpired);
      // Log timer status printf("Timer %d: period=%lu, active=%d,
expired=%d\n", i, period, isActive, hasExpired);
    }
  }
}
```

Parameters

in	*timer*↩_	Handle to the timer to query. Must be a valid timer created with xTimerCreate().
out	*period*↩_	Pointer to variable receiving the timer period in system ticks. On success, contains the current period value.

Returns

ReturnOK if period retrieved successfully, ReturnError if query failed due to invalid timer handle or timer not found.

Warning

This function returns the configured period, not the elapsed time or remaining time. To determine how close a timer is to expiration, you need to track elapsed time or check expiration status with xTimerHasTimerExpired().

Note

xTimerGetPeriod() is a non-destructive query that does not modify timer state, elapsed time, or active/stopped status.

The returned period reflects the most recent value set by either xTimerCreate() (initial period) or xTimerChangePeriod() (updated period).

HeliOS does not provide direct access to elapsed time. To track progress toward expiration, implement your own elapsed time tracking using the scheduler's tick count.

See also

xTimerChangePeriod() - Modify timer period

xTimerCreate() - Create timer with initial period

xTimerIsTimerActive() - Check if timer is running

xTimerHasTimerExpired() - Check if timer has expired

xTimerReset() - Reset elapsed time to zero

xTimerStart() - Start timer with its current period

4.2.4.96 xTimerHasTimerExpired() `Return_t xTimerHasTimerExpired (`

```
const Timer_t * timer_,
Base_t * res_ )
```

Queries whether an application timer's elapsed time has reached or exceeded its configured period, indicating that the timer has expired. This query is the primary mechanism for detecting timer events in polling-based timing patterns. Once expired, the timer remains in the expired state until reset with xTimerReset() or stopped with xTimerStop().

A timer can only expire if it is active (started with xTimerStart()). Inactive timers never expire, even if sufficient time has passed. The expired state is "sticky"—once a timer expires, it remains expired until explicitly cleared, allowing multiple tasks to observe the expiration event without race conditions.

Typical expiration handling workflow:

1. Check if timer has expired with xTimerHasTimerExpired() 2. If expired, perform the timed action 3. Reset the timer with xTimerReset() to clear expiration and restart timing 4. Repeat the cycle

Key characteristics:

- **Sticky expiration**: Remains expired until xTimerReset() or xTimerStop()
- **Requires active timer**: Only active timers can expire
- **Non-destructive query**: Does not clear expiration or modify state
- **Event detection**: Primary method for implementing periodic actions

Example 1: Periodic sensor polling

```
 Timer_t *pollTimer;
void initSensor(void) {
  xTimerCreate(&pollTimer, 100);  // Poll every 100 ticks
xTimerStart(pollTimer);
}
void sensorTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(pollTimer, &expired)) && expired) {
    // Timer expired - read sensor Byte_t sensorValue = readSensor();
    processSensorData(sensorValue);
    // Reset for next period xTimerReset(pollTimer);
  }
}
```

Example 2: Watchdog timeout detection

```
 Timer_t *watchdogTimer;
void initWatchdog(void) {
  xTimerCreate(&watchdogTimer, 5000);  // 5 second timeout
xTimerStart(watchdogTimer);
}
void watchdogTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(watchdogTimer, &expired)) && expired) {
    // Watchdog expired - system not responding logError("Watchdog timeout
- system hung");
    performSystemReset();
  }
}

void petWatchdog(void) {
  // Called periodically by application to prevent timeout
xTimerReset(watchdogTimer);
}
```

Example 3: Multi-rate execution with multiple timers

```
 Timer_t *fastTimer, mediumTimer, slowTimer;
void initTimers(void) {
  xTimerCreate(&fastTimer, 10);    // Every 10 ticks
xTimerCreate(&mediumTimer, 100); // Every 100 ticks
xTimerCreate(&slowTimer, 1000);  // Every 1000 ticks
  xTimerStart(fastTimer);
```

```
  xTimerStart(mediumTimer);
  xTimerStart(slowTimer);
}
void controlTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  // Fast update - 10 ticks if (OK(xTimerHasTimerExpired(fastTimer,
&expired)) && expired) {
    updateFastController();
    xTimerReset(fastTimer);
  }
  // Medium update - 100 ticks if (OK(xTimerHasTimerExpired(mediumTimer,
&expired)) && expired) {
    updateMediumController();
    xTimerReset(mediumTimer);
  }
  // Slow update - 1000 ticks if (OK(xTimerHasTimerExpired(slowTimer,
&expired)) && expired) {
    updateSlowController();
    xTimerReset(slowTimer);
  }
}
```

Example 4: Timeout for communication protocol

```
 Timer_t *responseTimer;
typedef enum {
  STATE_IDLE, STATE_WAITING_RESPONSE, STATE_COMPLETE
} CommState_t;
CommState_t commState = STATE_IDLE;
void sendRequest(void) {
  transmitData();
  commState = STATE_WAITING_RESPONSE;
  // Start timeout timer xTimerReset(responseTimer);
  xTimerStart(responseTimer);
}
void commTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (commState == STATE_WAITING_RESPONSE) {
    if (OK(xTimerHasTimerExpired(responseTimer, &expired)) && expired) {
      // Timeout - no response received logError("Communication timeout");
      xTimerStop(responseTimer);
      commState = STATE_IDLE;
      retryRequest();
    }
  }
}
void onResponseReceived(void) {
  // Stop timer on successful response xTimerStop(responseTimer);
  commState = STATE_COMPLETE;
  processResponse();
}
```

Parameters

in	*timer*↩ _	Handle to the timer to query. Must be a valid timer created with xTimerCreate().
out	*res*↩ _	Pointer to variable receiving the expiration status. Set to non-zero (true) if timer has expired, zero (false) if timer has not expired or is not active.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid timer handle or timer not found.

Warning

xTimerHasTimerExpired() does NOT automatically reset the timer. Once expired, the timer remains expired until you explicitly call xTimerReset() or xTimerStop(). Failing to reset an expired timer will cause it to appear expired on every subsequent check.

Inactive (stopped) timers never expire. Always verify the timer is active with xTimerStart() before expecting expiration events.

Note

The expiration state is "sticky"—it persists until cleared. This allows multiple tasks to safely check the same timer expiration without missing the event.

Expiration occurs when elapsed time >= period. At the moment the elapsed time equals the period, the timer transitions to expired state.

Unlike some RTOS timers that provide callback functions on expiration, HeliOS timers use polling with xTimerHasTimerExpired(). Tasks must explicitly check for expiration in their execution loops.

See also

xTimerReset() - Clear expiration and restart timing

xTimerStart() - Activate timer to enable expiration

xTimerStop() - Deactivate timer and clear expiration

xTimerIsTimerActive() - Check if timer is running

xTimerCreate() - Create timer with period

xTimerChangePeriod() - Modify timer period

4.2.4.97 xTimerIsTimerActive()

```
Return_t xTimerIsTimerActive (
    const Timer_t * timer_,
    Base_t * res_ )
```

Queries whether an application timer is in the active (running) state. A timer is considered active if it has been started with xTimerStart() and has not been stopped with xTimerStop(). Active timers accumulate elapsed time on each scheduler tick and will eventually expire when elapsed time reaches the configured period.

This non-destructive query allows tasks to check timer state before performing operations, coordinate timing between tasks, or implement conditional logic based on whether timing is currently in progress. The query does not affect timer operation or elapsed time accumulation.

Timer state transitions:

- **Inactive (stopped)**: Initial state after creation, or after xTimerStop()
- **Active (running)**: After xTimerStart(), accumulating elapsed time
- **Active and expired**: Timer remains active until xTimerReset() or xTimerStop()

Key characteristics:

- **Non-destructive query**: Does not modify timer state or elapsed time
- **State only**: Returns active/stopped status, not expiration status
- **Instant snapshot**: Reflects current state at time of call

Example 1: Conditional timer start

```
 Timer_t *processingTimer;
void startProcessing(void) {
  Base_t isActive;
  // Only start if not already running if
(OK(xTimerIsTimerActive(processingTimer, &isActive))) {
    if (!isActive) {
      xTimerReset(processingTimer);
      xTimerStart(processingTimer);
      beginProcessing();
    } else {
      logWarning("Processing already in progress");
    }
  }
}
```

Example 2: Watchdog monitoring

```
 Timer_t *watchdogTimer;
void checkWatchdog(void) {
  Base_t isActive;
  Base_t hasExpired;
  if (OK(xTimerIsTimerActive(watchdogTimer, &isActive))) {
    if (!isActive) {
      // Watchdog not running - critical error handleWatchdogFailure();
    } else if (OK(xTimerHasTimerExpired(watchdogTimer, &hasExpired)) &&
hasExpired) {
      // Watchdog expired - system hung handleWatchdogTimeout();
    } else {
      // Watchdog running normally
    }
  }
}
void petWatchdog(void) {
  Base_t isActive;
  if (OK(xTimerIsTimerActive(watchdogTimer, &isActive)) && isActive) {
    xTimerReset(watchdogTimer);  // Reset only if active
  }
}
```

Example 3: Performance measurement

```
 Timer_t *perfTimer;
void startMeasurement(void) {
  Base_t isActive;
  if (OK(xTimerIsTimerActive(perfTimer, &isActive)) && isActive) {
    logWarning("Measurement already in progress");
    return;
  }
  xTimerReset(perfTimer);
  xTimerStart(perfTimer);
}
void endMeasurement(void) {
  Base_t isActive;
  if (OK(xTimerIsTimerActive(perfTimer, &isActive)) && isActive) {
    xTimerStop(perfTimer);
    // Analyze performance metrics
  } else {
    logError("No measurement in progress");
  }
}
```

Example 4: System diagnostics

```
 Timer_t *systemTimers[MAX_TIMERS];
Base_t timerCount = 0;
void reportSystemStatus(void) {
  Base_t activeCount = 0;
  Base_t expiredCount = 0;
  for (Base_t i = 0; i < timerCount; i++) {
    Base_t isActive, hasExpired;
    if (OK(xTimerIsTimerActive(systemTimers[i], &isActive)) && isActive) {
      activeCount++;
      if (OK(xTimerHasTimerExpired(systemTimers[i], &hasExpired)) &&
hasExpired) {
        expiredCount++;
      }
    }
  }
  printf("Timers: %d total, %d active, %d expired\n", timerCount,
activeCount, expiredCount);
}
```

Parameters

in	*timer_*	Handle to the timer to query. Must be a valid timer created with xTimerCreate().
out	*res_*	Pointer to variable receiving the active state. Set to non-zero (true) if timer is active/running, zero (false) if timer is stopped.

Returns

ReturnOK if query succeeded, ReturnError if query failed due to invalid timer handle or timer not found.

Warning

Active state does not indicate whether the timer has expired. A timer can be both active and expired simultaneously. Use xTimerHasTimerExpired() to check expiration status.

Note

xTimerIsTimerActive() returns the state at the moment of the call. In multitasking environments, the state may change immediately after if another task starts or stops the timer.

A timer that has expired remains active until explicitly stopped with xTimerStop() or reset with xTimerReset(). Expiration does not automatically stop the timer.

Newly created timers are inactive by default. They must be explicitly started with xTimerStart() to begin accumulating elapsed time.

See also

xTimerStart() - Start timer (make it active)

xTimerStop() - Stop timer (make it inactive)

xTimerHasTimerExpired() - Check if active timer has expired

xTimerReset() - Reset elapsed time while keeping timer active

xTimerCreate() - Create timer in inactive state

4.2.4.98 xTimerReset() `Return_t xTimerReset ( Timer_t * timer_ )`

Clears an application timer's accumulated elapsed time, returning it to the state immediately after creation or start. This operation is the primary method for acknowledging timer expiration and beginning a new timing cycle. The timer's configured period and active/stopped state are preserved—only the elapsed time is reset to zero.

xTimerReset() is typically called after detecting timer expiration with xTimerHasTimerExpired() to clear the expired state and restart the timing cycle for periodic operations. The reset does not affect whether the timer is active or stopped; an active timer continues running from zero elapsed time, while a stopped timer remains stopped with zero elapsed time.

Key characteristics:

- **Clears elapsed time**: Resets accumulated time to zero

- **Clears expiration**: Timer no longer reports as expired after reset
- **Preserves state**: Active/stopped state unchanged
- **Preserves period**: Configured timer period unchanged
- **Immediate effect**: Next expiration occurs one full period after reset

Common reset scenarios:

- **Periodic operations**: Reset after each expiration to start next cycle
- **Watchdog petting**: Reset watchdog timer to prevent timeout
- **Event acknowledgment**: Clear timer after handling timed event
- **Synchronization**: Align timer phases with system events
- **Period changes**: Reset after calling xTimerChangePeriod() to start fresh

Example 1: Periodic sensor polling pattern

```
 Timer_t *sensorTimer;
void initSensor(void) {
  xTimerCreate(&sensorTimer, 100);  // 100 tick period
xTimerStart(sensorTimer);
}
void sensorTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(sensorTimer, &expired)) && expired) {
    // Read and process sensor Byte_t value = readSensor();
    processSensorValue(value);
    // Reset for next cycle xTimerReset(sensorTimer);
  }
}
```

Example 2: Watchdog timer implementation

```
 Timer_t *watchdogTimer;
void initWatchdog(void) {
  xTimerCreate(&watchdogTimer, 5000);  // 5 second timeout
xTimerStart(watchdogTimer);
}
void petWatchdog(void) {
  // Called periodically by application to prevent timeout
xTimerReset(watchdogTimer);
}
void watchdogTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(watchdogTimer, &expired)) && expired) {
    // System hung - watchdog not reset in time logCritical("Watchdog
timeout");
    performSystemReset();
  }
}
```

Example 3: Debouncing button input

```
 Timer_t *debounceTimer;
Base_t buttonPressed = 0;
void onButtonPress(void) {
  if (!buttonPressed) {
    // First press - start debounce timer buttonPressed = 1;
    xTimerReset(debounceTimer);
    xTimerStart(debounceTimer);
    handleButtonPress();
  }
}
void buttonTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (buttonPressed) {
    if (OK(xTimerHasTimerExpired(debounceTimer, &expired)) && expired) {
      // Debounce period elapsed - ready for next press buttonPressed = 0;
      xTimerStop(debounceTimer);
    }
  }
}
```

Example 4: Synchronized multi-timer operations

```
 Timer_t *timer1, timer2, timer3;
void synchronizeTimers(void) {
  // Reset all timers simultaneously to align phases xTimerReset(timer1);
  xTimerReset(timer2);
  xTimerReset(timer3);
  // All timers now start from zero elapsed time
}
void onSystemEvent(void) {
  // Synchronize timer phases on important system event
synchronizeTimers();
}
```

Parameters

in	*timer*↩ _	Handle to the timer to reset. Must be a valid timer created with xTimerCreate().

Returns

ReturnOK if timer reset successfully, ReturnError if operation failed due to invalid timer handle or timer not found.

Warning

xTimerReset() only clears elapsed time—it does NOT start a stopped timer. If the timer is not active, call xTimerStart() before or after xTimerReset() to begin timing.

After reset, the timer will not expire until a full period has elapsed. If you reset a timer that has partially elapsed, you lose that partial progress and must wait the full period again.

Note

xTimerReset() is typically called after detecting expiration to acknowledge the event and start a new timing cycle.

For periodic operations, the pattern is: check expired → perform action → reset timer. This ensures you don't miss expiration events.

Resetting does not change the timer's period. To change the period, use xTimerChangePeriod() and then optionally xTimerReset().

Calling xTimerReset() on an already-reset or newly-created timer is safe and has no adverse effects.

See also

xTimerHasTimerExpired() - Check if timer has expired before resetting

xTimerStart() - Activate timer after reset

xTimerStop() - Stop timer (also clears expiration)

xTimerChangePeriod() - Change period (usually followed by reset)

xTimerCreate() - Create timer (starts with zero elapsed time)

xTimerIsTimerActive() - Check if timer is running

4.2.4.99 xTimerStart() `Return_t xTimerStart ( Timer_t * timer_ )`

Activates an application timer, transitioning it from the stopped (inactive) state to the running (active) state. Once started, the timer begins accumulating elapsed time on each scheduler tick and will eventually expire when elapsed time reaches the configured period. This operation is required to enable timer expiration—only active timers can expire.

xTimerStart() preserves the timer's current elapsed time. If you start a timer that was previously stopped with xTimerStop(), it resumes from the elapsed time it had accumulated before stopping. To start timing from zero, call xTimerReset() before or after xTimerStart(). For newly created timers, elapsed time is already zero, so xTimerReset() is not necessary.

Key characteristics:

- **Activates timer**: Enables elapsed time accumulation
- **Preserves elapsed time**: Continues from current elapsed time
- **Enables expiration**: Only active timers can expire
- **Idempotent**: Starting an already-running timer has no effect
- **Non-blocking**: Returns immediately after state change

Typical startup sequences:

- **New timer, start from zero**: xTimerCreate() → xTimerStart()
- **New timer, explicit reset**: xTimerCreate() → xTimerReset() → xTimerStart()
- **Resume stopped timer**: xTimerStart() (continues from stopped elapsed time)
- **Restart from zero**: xTimerReset() → xTimerStart()

Example 1: Basic timer startup

```
 Timer_t *periodicTimer;
void initTimer(void) {
  // Create timer with 100 tick period if (OK(xTimerCreate(&periodicTimer,
100))) {
    // Start timer - begins timing from zero xTimerStart(periodicTimer);
  }
}
void periodicTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(periodicTimer, &expired)) && expired) {
    performPeriodicAction();
    xTimerReset(periodicTimer);  // Reset for next cycle
  }
}
```

Example 2: Conditional start based on system state

```
 Timer_t *monitorTimer;
void enableMonitoring(void) {
  Base_t isActive;
  if (OK(xTimerIsTimerActive(monitorTimer, &isActive)) && !isActive) {
    // Timer not running - start it xTimerReset(monitorTimer);  // Start
from zero xTimerStart(monitorTimer);
    logInfo("Monitoring enabled");
  }
}
void disableMonitoring(void) {
  Base_t isActive;
  if (OK(xTimerIsTimerActive(monitorTimer, &isActive)) && isActive) {
    xTimerStop(monitorTimer);
    logInfo("Monitoring disabled");
  }
}
```

Example 3: Pause and resume pattern

```
 Timer_t *processTimer;
void pauseProcessing(void) {
  // Stop timer - preserves elapsed time xTimerStop(processTimer);
  logInfo("Processing paused");
}
void resumeProcessing(void) {
  // Start timer - continues from paused elapsed time
xTimerStart(processTimer);
  logInfo("Processing resumed");
}
void resetAndRestart(void) {
  // Restart from zero elapsed time xTimerReset(processTimer);
  xTimerStart(processTimer);
  logInfo("Processing restarted from zero");
}
```

Example 4: Multiple timer coordination

```
 Timer_t *timer1, timer2, timer3;
void startAllTimers(void) {
  // Reset all to zero xTimerReset(timer1);
  xTimerReset(timer2);
  xTimerReset(timer3);
  // Start all simultaneously xTimerStart(timer1);
  xTimerStart(timer2);
  xTimerStart(timer3);
  logInfo("All timers started synchronously");
}
void stopAllTimers(void) {
  xTimerStop(timer1);
  xTimerStop(timer2);
  xTimerStop(timer3);
  logInfo("All timers stopped");
}
```

Parameters

in	*timer*↩ _	Handle to the timer to start. Must be a valid timer created with xTimerCreate().

Returns

ReturnOK if timer started successfully, ReturnError if operation failed due to invalid timer handle or timer not found.

Warning

xTimerStart() does NOT reset elapsed time. If you stopped a timer partway through its period and then start it again, it will continue from where it left off. Call xTimerReset() before xTimerStart() if you want to begin timing from zero.

Only active (started) timers can expire. Calling xTimerHasTimerExpired() on a stopped timer will always return false, even if sufficient time has passed.

Note

xTimerStart() is idempotent—calling it multiple times on an already-running timer has no effect and is not an error.

For newly created timers, xTimerReset() is not necessary before xTimerStart() because elapsed time is already zero.

The timer begins accumulating elapsed time immediately on the next scheduler tick after xTimerStart() is called.

See also

xTimerStop() - Stop timer (opposite operation)

xTimerReset() - Reset elapsed time to zero

xTimerCreate() - Create timer (starts in stopped state)

xTimerIsTimerActive() - Check if timer is running

xTimerHasTimerExpired() - Check for expiration (requires active timer)

xTimerChangePeriod() - Change timer period

4.2.4.100 xTimerStop() `Return_t xTimerStop ( Timer_t * timer_ )`

Deactivates an application timer, transitioning it from the running (active) state to the stopped (inactive) state. Once stopped, the timer ceases accumulating elapsed time and cannot expire, even if sufficient scheduler ticks pass. The timer's current elapsed time is preserved, allowing for pause-and-resume scenarios if needed.

xTimerStop() is the primary method for temporarily disabling timing without destroying the timer. Unlike xTimerDelete(), which permanently removes the timer, xTimerStop() keeps the timer structure intact and allows restarting with xTimerStart(). If the timer was expired when stopped, the expiration state is cleared—xTimerHas↩ TimerExpired() will return false after stopping.

Key characteristics:

- **Deactivates timer**: Halts elapsed time accumulation
- **Preserves elapsed time**: Current elapsed time is maintained
- **Clears expiration**: Expired state is reset to non-expired
- **Prevents expiration**: Stopped timers cannot expire
- **Idempotent**: Stopping an already-stopped timer has no effect
- **Non-blocking**: Returns immediately after state change

Common stop scenarios:

- **Conditional timing**: Stop timer when condition is met
- **Pause-resume patterns**: Stop to pause, xTimerStart() to resume
- **Event completion**: Stop timer after timed event finishes early
- **Power management**: Stop non-essential timers to reduce overhead
- **Timeout cancellation**: Stop timer when expected response arrives

Example 1: Communication timeout with early completion

```
 Timer_t *timeoutTimer;
void sendRequest(void) {
  transmitData();
  // Start timeout timer xTimerReset(timeoutTimer);
  xTimerStart(timeoutTimer);
}
void onResponseReceived(void) {
  // Response arrived - cancel timeout xTimerStop(timeoutTimer);
  processResponse();
}
void commTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(timeoutTimer, &expired)) && expired) {
    // Timeout - no response received xTimerStop(timeoutTimer);
    handleTimeout();
  }
}
```

Example 2: Conditional timer management

```
 Timer_t *activityTimer;
void onActivityDetected(void) {
  Base_t isActive;
  // Start timer on activity if (OK(xTimerIsTimerActive(activityTimer,
&isActive)) && !isActive) {
    xTimerReset(activityTimer);
    xTimerStart(activityTimer);
  }
}
void onIdleCondition(void) {
  Base_t isActive;
  // Stop timer when system goes idle if
(OK(xTimerIsTimerActive(activityTimer, &isActive)) && isActive) {
    xTimerStop(activityTimer);
  }
}
```

Example 3: Pause and resume timing

```
 Timer_t *processTimer;
void pauseProcessing(void) {
  // Stop timer - preserves elapsed time for resume
xTimerStop(processTimer);
  suspendProcessing();
}
void resumeProcessing(void) {
  // Start timer - continues from paused elapsed time
xTimerStart(processTimer);
  continueProcessing();
}
void cancelProcessing(void) {
  // Stop and reset - clears elapsed time xTimerStop(processTimer);
  xTimerReset(processTimer);
  cleanupProcessing();
}
```

Example 4: Power-aware timer management

```
 Timer_t *backgroundTimers[MAX_TIMERS];
Base_t timerCount = 0;
void enterLowPowerMode(void) {
  // Stop all non-critical timers for (Base_t i = 0; i < timerCount; i++) {
    xTimerStop(backgroundTimers[i]);
  }
  enableSleepMode();
}
void exitLowPowerMode(void) {
  // Resume all timers disableSleepMode();
  for (Base_t i = 0; i < timerCount; i++) {
    xTimerStart(backgroundTimers[i]);
  }
}
```

Example 5: One-shot timer pattern

```
 Timer_t *oneShotTimer;
void startOneShotTimer(Ticks_t delay) {
  // Configure and start one-shot timer xTimerChangePeriod(oneShotTimer,
delay);
  xTimerReset(oneShotTimer);
  xTimerStart(oneShotTimer);
}
void timerTask(Task_t *task, TaskParm_t *parm) {
  Base_t expired;
  if (OK(xTimerHasTimerExpired(oneShotTimer, &expired)) && expired) {
    // Timer expired - stop to prevent re-triggering
```

```
xTimerStop(oneShotTimer);
    performOneShotAction();
  }
}
```

Parameters

in	*timer*↩ _	Handle to the timer to stop. Must be a valid timer created with xTimerCreate().

Returns

ReturnOK if timer stopped successfully, ReturnError if operation failed due to invalid timer handle or timer not found.

Warning

xTimerStop() preserves the current elapsed time. If you later call xTimerStart() without calling xTimerReset(), the timer will continue from its stopped elapsed time, not from zero. Call xTimerReset() after xTimerStop() if you want to clear elapsed time.

Stopped timers cannot expire. xTimerHasTimerExpired() will return false for stopped timers regardless of elapsed time.

Note

xTimerStop() clears the expiration flag. Even if the timer was expired when stopped, xTimerHasTimerExpired() will return false after the stop.

xTimerStop() is idempotent—calling it multiple times on an already-stopped timer has no effect and is not an error.

Stopping a timer does not free its resources. The timer remains allocated and can be restarted with xTimerStart(). To free timer resources, use xTimerDelete().

Newly created timers are already in the stopped state. Calling xTimerStop() on a newly created timer has no effect but is not an error.

See also

xTimerStart() - Start timer (opposite operation)

xTimerReset() - Reset elapsed time (often called after stop)

xTimerDelete() - Permanently remove timer and free resources

xTimerIsTimerActive() - Check if timer is running

xTimerHasTimerExpired() - Check for expiration (always false for stopped timers)

xTimerCreate() - Create timer (starts in stopped state)

Index